

Provable Security of Symmetric-key Cryptographic Schemes

Ashwin Jha

under the supervision of

Mridul Nandi



Applied Statistics Unit
Indian Statistical Institute, Kolkata
203 B. T. Road, Kolkata 700108
India

October, 2019

*A thesis submitted in partial fulfillment of the thesis requirements
for the degree of*
Doctor of Philosophy
in
Computer Science
by
Ashwin Jha
to the
Indian Statistical Institute, Kolkata
on
October 10, 2019

Examination Committee:

Dr. Mridul Nandi, Supervisor
Indian Statistical Institute, Kolkata

Prof. Utpal Garain, Convener, Ph.D. & D.Sc. Committee
Indian Statistical Institute, Kolkata

Prof. Abhijit Das, External Examiner
Indian Institute of Technology, Kharagpur

Dedicated to my mother.

Abstract

In this thesis, we provide quantitative and/or qualitative improvements in the provable security of several symmetric-key schemes, encompassing major information security goals, viz. data authentication, encryption, and authenticated encryption.

AUTHENTICATION AND INTEGRITY: Among authentication schemes, we analyze the CBC-MAC family and counter-based MACs (XMACC, XMACR, PCS, LightMAC etc.), referred as the XMAC family. First, we revisit the security proofs for CBC-MAC and EMAC, and identify a critical flaw in the state-of-the-art results. We revise the security proofs and obtain significantly better bounds in case of EMAC, ECBC and FCBC. Second, we study the security of CBC-MAC family, when the underlying primitive is pseudorandom function (PRF), and derive tight security bounds for EMAC, ECBC, FCBC, XCBC and TMAC. Third, we study the counter-based input encoding used in XMAC family. We present a generalized view on counter-based encoding and propose some efficient alternatives to the classical fixed length counter. Further, based on the generalized view, we identify some necessary and sufficient conditions, which result in simplified security arguments. As a side-result we also prove second preimage security for HAIFA-based hash function (using Davies-Meyer compression function) in the ideal cipher model.

ENCRYPTION: Among encryption schemes, we study the problem of constructing beyond-the-birthday bound (BBB) secure online encryption schemes using tweakable block ciphers (TBCs). First, we construct a birthday bound distinguisher for POEx, which invalidates the BBB security claims of POEx. Second, we propose a BBB secure online cipher, called XTC, and prove that it is optimally secure. As a by-product we suggest a generic distinguisher for a class of TBC-based online ciphers that encompasses both POEx and XTC. On a related topic, we study the problem of constructing BBB secure TBCs from block ciphers. Specifically, we derive a tight security bound for cascaded LRW2 under the assumption that the underlying hash functions are 3-wise almost XOR universal.

AUTHENTICATED ENCRYPTION: Finally, among authenticated encryption schemes, we study a generalization of the OCB family, called GOCB, with an aim to achieve efficient random read access. We introduce a relaxed notion of universal hash functions, called locally imperfect XOR universal (LIXU), and prove that GOCB is secure under this relaxed notion of universality. Further, we instantiate GOCB with AES round function based LIXU hash functions. These instantiations achieve significantly better random read access than OCB3.

Acknowledgements

This thesis is not only the culmination of my personal endeavor, but guidance, inspiration and assistance of many people. I am deeply grateful to my supervisor for mentorship and encouragement throughout my doctoral studies. I also thank my co-authors, colleagues, and the faculty and staff at my institution for helpful discussions and for providing an excellent research environment. I am grateful to the organizations that hosted my research visits and supported this work. Finally, I thank my family and friends for their constant encouragement.

Note: *Acknowledgements abridged for privacy.*

Contents

Abstract	ii
Acknowledgements	iii
Contents	iv
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Symmetric-key Security Goals and Primitives	2
1.2 Provable Security in Symmetric-key Settings	5
1.3 Thesis Outline and Our Contributions	7
2 Preliminaries	16
2.1 Notational Setup	16
2.2 Security Game, Adversary and Advantage	17
2.3 (Tweakable) Block Cipher and (XOR) Universal Hash	20
2.4 Message Authentication Codes	24
2.5 Online Encryption Schemes	29
2.6 Authenticated Encryption with Associated Data	30
I Analysis of Message Authentication Codes	33
3 Pseudorandom Permutation based CBC-MAC Family	34
3.1 PRF Analysis of CBC-MAC and EMAC	35
3.2 Revisiting Structure Graph	40
3.3 Characterization of Accident-1 Structure Graphs	47
3.4 Revisiting $\text{outCP}_{q,\ell,\sigma}^{\text{any}}$ and $\text{fullCP}_{q,\ell,\sigma}^{\text{pf}}$ Bounds	50
3.5 Revised Security Analysis of EMAC	57
4 Pseudorandom Function based CBC-MAC Family	62
4.1 PRF Analysis of PRF-based CBC-MAC Family	62
4.2 Proof of Theorem 4.1.4 [Upper Bound Theorem]	74

4.3	Proof of Theorem 4.1.5 [Lower Bound Theorem]	75
4.4	Proofs of Results on Structure Graph	81
5	Counter-based Input Encoding in MACs	84
5.1	Inefficiency of Classical Counters	85
5.2	A New Look at Counters	87
5.3	Counter-as-Encoding Constructions	97
5.4	Comparison and Empirical Result	105
II	Analysis of Online and Tweakable Ciphers	111
6	Tweakable Pseudorandom Permutation based Online Ciphers	112
6.1	A Design Strategy for TBC-based BBB Online Cipher	112
6.2	Revisiting POEx	115
6.3	XTC: Rate-1 (Almost) Optimally Secure Online Cipher	120
7	Security of Cascaded LRW2	129
7.1	A Note on Patarin's Mirror Theory	130
7.2	Revisiting Mennink's Improved Bound on CLRW2	131
7.3	The Alternating Collisions and Events Lemmata	137
7.4	Mirror Theory for Tweakable Permutations	140
7.5	Improved Security Bound of CLRW2	141
7.6	Proof of Mirror Theory for Tweakable Permutations	156
III	Analysis of Authenticated Encryptions	164
8	Random Read Access in OCB	165
8.1	Revisiting Small-Domain AXU Hash Functions	165
8.2	AES-based Small-Domain AXU Hash Functions	169
8.3	Generic view of OCB	173
8.4	Security Proofs	183
8.5	Software Performance	194
A	Graphs and Probability Theory	197
A.1	Directed Edge-Labeled Graphs	197
A.2	Some Basic Results in Probability Theory	199
	Bibliography	200

List of Figures

3.1.1 Evaluation of CBC function.	35
3.2.1 Block structure graph for a 5-block message.	42
3.2.2 Structure graph corresponding to a block-vertex structure graph.	42
3.2.3 Structure graphs with accident 1 and true collision 2.	46
3.3.1 Free graph of three messages.	47
3.3.2 Accident-1 structure graphs for single message.	50
3.4.1 Accident-1 structure graphs satisfying 0coll event.	53
4.1.1 INS graph corresponding to a BINS graph.	72
4.3.1 Unicycles and union of paths.	77
5.0.1 The XMAC family.	85
5.2.1 Rate plot for CFFs.	97
5.3.1 The CtHAIFA hash function.	103
5.4.1 CPB plot for book-keeping operations of the three CFF candidates.	107
5.4.2 CPB plot for CtMAC1 based on the three CFF candidates.	108
5.4.3 CPB plot for CtMAC2 st based on the three CFF candidates.	109
5.4.4 CPB plot for CtHAIFA based on the three CFF candidates.	110
6.2.1 Schematic of POEx.	116
6.3.1 Schematic of XTC.	124
6.3.2 Main steps in the security proof of XTC.	125
7.2.1 The cascaded LRW2 construction.	132
7.2.2 A transcript graph with q edges.	134
7.2.3 Transcript subgraphs with 4 edges.	135
7.5.1 All possible types of components in a good transcript graph.	145
7.6.1 Switchings used in the proof of Lemma 7.6.3.	159
8.3.1 Schematic of GOCB.	176

List of Tables

1.3.1 Summary of security bounds for the CBC-MAC family.	10
1.3.2 Summary of security bounds for PRF-based CBC-MAC family.	11
5.2.1 Rate comparison between the three CFFs.	97
5.4.1 Baseline AES-128 CPB value in Sandy Bridge.	105
5.4.2 Comparison between the book-keeping costs of the three CFFs.	107
5.4.3 Comparison between the CPB values of CtMAC1 candidates.	108
5.4.4 Comparison between the CPB values of CtMAC2 st candidates.	109
5.4.5 Comparison between the CPB values of CtHAIFA candidates.	109
8.1.1 Summary of some small-domain AXU hash functions.	166
8.5.1 Baseline AES-128 CPB value in Skylake.	195
8.5.2 CPB values for sequential implementation.	195
8.5.3 Summary of IPI values.	195
8.5.4 CPB values for random read access.	196

“It is not knowledge, but the act of learning, not possession but the act of getting there, which grants the greatest enjoyment.”

– Carl Friedrich Gauss

Chapter 1

Introduction

This thesis is an exploration in the field of cryptography. The word cryptography comes from two greek words, *kryptós* meaning “secret” and *graphein* meaning “to write”. In its most simplest form, it is the study and practice of techniques for concealing meaningful information by camouflaging it with some secret randomness. Since the dawn of civilization, humans have felt a need for secrecy in communication, be it military, diplomatic or otherwise. Indeed, the earliest known encrypted text occurred some 4000 years ago in the hieroglyphic inscriptions of ancient Egyptian civilization. In recent past, the two world wars followed by the subsequent cold war, and the Internet revolution have acted as catalysts in the rapid development of this field. In [113], Kahn gives an extensive account on the history and development of cryptography.

In circa 1948–49, Claude Shannon wrote two landmark papers. The first paper [179] laid the foundations of information theory, while the second paper, titled *Communication Theory of Secrecy Systems* [180], is the first concrete mathematical treatment of the field of cryptography. Up until mid-1970s, cryptography was quite one dimensional, concerned with just confidentiality or privacy in communication. In 1976, a seminal paper titled *New Directions in Cryptography* [60] by Diffie and Hellman identified the requirement of integrity, authenticity and non-repudiation in communication. Modern cryptography has incorporated all these elements along with the traditional need for confidentiality. Apart from identifying new security goals, Diffie and Hellman also invented a new paradigm in cryptography, the so-called *public-key cryptography* [60].

Modern cryptographic schemes are classified into two major classes: *symmetric-key* (or secret-key) and *asymmetric-key* (or public-key). In a symmetric-key scheme, a common (secret) key is shared between the two (or more) parties for secure communication. In an asymmetric-key scheme, each party holds a different (set of) key(s), but the keys are related in some precise mathematical sense. In general, symmetric-key schemes are

computationally much faster than asymmetric-key schemes, but they have an obvious prerequisite – *both parties should possess a common secret key*. This is not easy in a purely symmetric-key setting, because it requires face-to-face interaction or a trusted courier or a secure channel. In contrast, this is much easier in asymmetric-key setting, where a secure key exchange protocol can establish a common secret key. In fact, most of the modern communication protocols employ a hybrid of asymmetric and symmetric schemes, where the computationally intensive asymmetric-key scheme is used to fulfill the initial prerequisites, and the actual communication takes place using symmetric-key schemes.

In this thesis, we focus on purely symmetric-key setting, i.e., we assume that the key distribution prerequisite is satisfied.

1.1 Symmetric-key Security Goals and Primitives

Symmetric-key cryptography forms the back bone of contemporary information security needs. These schemes are employed in defense communications, banking operations, and, probably the most important of them all, network protocols such as SSH [93], TLS [129], WPA2 [181] etc. Any symmetric-key scheme is expected to guarantee either confidentiality or authenticity (and integrity), or both. We briefly discuss these goals, and the symmetric-key primitives that achieve these goals. Let us fix two parties Alice and Bob communicating over an insecure channel using a common secret key K .

1.1.1 Data Authenticity

Data authenticity demands that when Alice sends some information to Bob, then Bob should be able to verify that Alice was the sender. In symmetric-key settings, a related but different goal of data integrity is subsumed within data authenticity.

Message authentication codes or MACs are symmetric-key primitives, which ensure data integrity and authenticity. Informally, a MAC scheme $\mathfrak{M}$ is a tuple of two algorithms, the tag-generation algorithm $\mathfrak{M}^+$ and the tag-verification algorithm $\mathfrak{M}^-$. The working principle of a basic MAC scheme $\mathfrak{M}$ instantiated with secret key K is quite simple. Whenever Alice wants to send a message M to Bob, she sends (M, T) , where the *tag* T is the output of tag-generation algorithm, i.e., $T = \mathfrak{M}^+(K, M)$. When Bob receives a message-tag pair (M', T') , he runs $\mathfrak{M}^-(K, M', T')$, which internally checks the equality $T' =? \mathfrak{M}^+(K, M')$. See section 2.4 of chapter 2 for a formal definition of MAC.

There are several ways to construct MAC schemes. In this thesis, we will mainly study MAC schemes based on block ciphers. CBC-MAC family [15, 21, 34, 65, 97, 121, 189] and PMAC family [35, 171, 190, 191] are two most popular families of MACs. The CBC-MAC family is sequential whereas PMAC is parallelizable, though it requires more memory. Both CBC-MAC and PMAC are length dependent, i.e., their security reduces as the message length increases [19, 74]. Counter-based MACs, such as XMACC and XMACR [16], PCS [23], LightMAC [126], and LightMAC+[143], avoid such security degradation by using counter-based encodings.

1.1.2 Data Confidentiality or Privacy

Data confidentiality demands that when Alice sends some information to Bob, then Bob and only Bob should be able to read that information, and others should not get any knowledge about the information.

In symmetric-key setting, *encryption schemes* such as CTR and OFB [153] strive for confidentiality of data. An encryption scheme $\mathcal{E}$ is a tuple of two algorithms, the encryption algorithm $\mathcal{E}^+$, and the decryption algorithm $\mathcal{E}^-$. The working principle of a basic encryption scheme $\mathcal{E}$ instantiated with secret key K is as follows. Whenever Alice wants to send a message M to Bob, she sends C , where the *ciphertext* C is the output of encryption algorithm, i.e., $C = \mathcal{E}^+(K, M)$. When Bob receives the ciphertext C , he runs $\mathcal{E}^-(K, C)$, to get back the decrypted text M . Obviously, $\mathcal{E}^+$ should be a bijective function and $\mathcal{E}^-$ should be its inverse, otherwise Bob cannot decrypt with certainty. Further, if $\mathcal{E}$ is secure, then no one else can decrypt the ciphertext without the knowledge of key K . Some popular examples of encryption schemes include, CTR [153], CBC encryption [153], OCB encryption [119, 171, 174], CMC [88], EME [89], HCTR [184], and HEH [176].

1.1.2.1 Online Encryption Schemes

In low-memory devices and continuous data streaming platforms with high throughput demands, it is often desirable to produce encrypted data in online fashion, i.e., the encryption of the current data block should only depend on the previous data blocks and the current data block. For instance, [71, 72] noticed that several network APIs are, in practice, stream-oriented, for example OpenSSL `EVP_DecryptUpdate` interface. In such scenarios, the online property is desirable. Paraphrasing the informal definition by Rogaway and Zhang [173], an encryption scheme is said to satisfy the online property if (1) it can be realized by an algorithm that, for any input, read its input blocks one at a time in order and computes the corresponding output blocks one at a time in

order, and (2) it uses only a constant-bound amount of memory and/or latency. The introductory definition by Bellare et al. [18] satisfied (1), which was later strengthened by Boldyreva and Taesombut [39] to satisfy (2). See section 2.5 of chapter 2 for a formal definition of online encryption schemes.

Apart from their practical importance, online ciphers are also used to construct other symmetric-key primitives, most prominently they are used in authenticated encryption schemes to dilute the effect of nonce misuses [1, 3, 40, 56]. Recently, Andreeva et al. [7] used online ciphers to build deterministic encryption schemes. In the same paper they also showed that online ciphers are equivalent to tweakable block ciphers (block ciphers with an extra public input called *tweak*) with variable length tweaks.

Some popular online encryption modes include, HCBC1 and HCBC2 [18], HPCBC [39], MHCBC and MCBC [146], TC1, TC2 and TC3 [173], OleF [29], and POEx [71, 72]. In this thesis we will study online encryption schemes and their relationship with tweakable block ciphers.

1.1.3 Data Authenticity and Confidentiality

Suppose now that Alice wants to send some confidential information to Bob, and Bob wants to verify that Alice was the sender. This requires both authenticity and confidentiality.

Authenticated encryption (AE) or authenticated cipher schemes are symmetric-key primitives which achieve both confidentiality and authenticity. An authenticated encryption scheme $\mathfrak{A}$ is a tuple of two algorithms, the encryption algorithm $\mathfrak{A}^+$ and the decryption algorithm $\mathfrak{A}^-$. The working principle of any AE scheme $\mathfrak{A}$ instantiated with secret key K is similar to a MAC scheme. Whenever Alice wants to send a message M to Bob, she sends (C, T) , where the *ciphertext-tag* pair (C, T) is the output of encryption algorithm, i.e., $(C, T) = \mathfrak{A}^+(K, M)$. When Bob receives a ciphertext-tag pair (C', T') , he runs $\mathfrak{A}^-(K, C', T')$, which (in most cases) internally decrypts the ciphertext to some plaintext M' and then checks the equality $\mathfrak{A}^+(K, M') =? (C', T')$. See chapter 2 section 2.6 for formal definition of AE.

Usually, AE schemes also take an auxiliary input called the *associated data* (AD) or header that may contain some information about the sender. In these cases, the modified AE scheme is called AE with associated data functionality or AEAD. A secure AEAD scheme should ensure authenticity of both AD and message, and privacy of only message (as AD is usually some publicly known information). OCB family [119, 171, 174], GCM [130], CCM [154], COLM [5] etc. are some popular AEAD schemes.

1.2 Provable Security in Symmetric-key Settings

In [13], Bellare notes that the idea of provable security was introduced by Goldwasser and Micali [79] in context of asymmetric encryption, but it soon spread to other tasks, particularly pseudorandomness [37, 76].

The most fundamental step in proving the security of any scheme is to establish the security goal (privacy or authenticity or both) and a formal adversarial model—a security game that defines the meaning of security with regard to the security goal. Given the goal and the model, a (symmetric-key) cryptographic scheme is deemed to be provably secure, if one can formally argue that it is secure given the hardness of some underlying computational problem. For example AES-PMAC [35] is a secure MAC (see chapter 2) if AES [152] is a secure pseudorandom permutation or PRP¹ (see chapter 2). Here MAC gives a precise definition of the security goal (unforgeability of tag values) and adversarial model (access to tag-generation and tag-verification algorithms as black boxes), and PRP assumption on AES is the hardness assumption.

1.2.1 Information-Theoretic Security

In general, a symmetric-key scheme is made up of two components:

- A *concrete primitive*, such as the block cipher AES [152], that operates on short and fixed length inputs; and
- A suitable *mode of operation*, such as GCM [130], that is used to extend the domain of applicability from short and fixed lengths to long and variable lengths.

For example, AES-GCM is a symmetric-key scheme that combines the AES block cipher (the concrete primitive) and GCM authenticated encryption mode (the mode of operation) to achieve both authenticity and confidentiality.

The usual method for proving the computational² security of a symmetric-key scheme involves two steps:

1. Replacing the underlying concrete primitive with a suitable ideal object. For example, AES is replaced with a uniform random permutation Π ; This step uses the computational indistinguishability³ of the underlying primitive with respect to

¹Computationally indistinguishable from a uniform random permutation.

²Any algorithm must halt in polynomial (of input length) time.

³For example, a PRP is computationally indistinguishable from a uniform random permutation.

the ideal object. This is, in general, heuristic, and mostly depends upon the confidence on a particular primitive. For example, AES is considered to be a good PRP, as it is well-analyzed over a longer period of time.

2. Proving the security of the mode of operation using this ideal primitive. For example, Π -GCM (GCM instantiated with Π) is shown to be a secure AE. This second step, in most of the cases, proves information-theoretic (the adversary is allowed unbounded computational time) indistinguishability of the mode of operation. In other cases, the original security game can be reduced to some variant indistinguishability game.

In this thesis, we will be mainly concerned with the second step. The security is, in general, parameterized in terms of adversarial resources. For example, throughout this thesis we use q , ℓ , σ , and t to denote the number of queries, length of the longest query, the total length of all queries, and the computational time, respectively.

1.2.2 Directions in Provable Security

A provable security result has two main components: i) the *underlying hardness assumption/security precondition*; (ii) the *security bound under this assumption*. We use the following terminologies to refer to these facets:

- *Qualitative*: what is the underlying security precondition? This could be some number-theoretic problem like “factoring problem”, or the pseudorandomness of some family of function like “AES is PRP”, or the universal property of some hash function, etc.
- *Quantitative*: how much security (data and time complexity)? The quantitative value of some security bound depends on the relationship between the security parameters (i.e. q , ℓ , σ) and the input/output size of the primitive, for example the block size n and key size κ of the block cipher. This is represented by the notion of *advantage* of an adversary against the scheme (see chapter 2).

Naturally, any result on the provable security of some scheme means improvement in either its qualitative aspect, i.e. a relaxation in the security notions of underlying primitives, or its quantitative aspect, i.e. an improved security bound, or both. Consequently, all the results in this thesis improve over the existing results in quantitative and/or qualitative aspects.

1.3 Thesis Outline and Our Contributions

In this thesis, we aim to provide qualitative and/or quantitative improvements in the security of several symmetric-key schemes encompassing authentication, encryption and authenticated encryption. We start off with establishing the relevant notations and conventions in chapter 2. In the same chapter we also give a formal description of symmetric-key primitives and their security. Our contributions are given in chapter 3 onwards. Broadly our contributions can be grouped into three parts: a) analysis of message authentication codes (in chapters 3-5); b) analysis of online and tweakable ciphers (in chapters 6 and 7); and c) analysis of authenticated ciphers (in chapter 8). In each of the subsections 1.3.1-1.3.3, we first give a detailed summary of related works, followed by our contributions. Finally, subsection 1.3.4 lists down all the research papers on which this thesis is based.

1.3.1 Analysis of Message Authentication Codes

We analyze two family of MAC schemes, namely, the CBC-MAC family comprising of CBC-MAC [65], EMAC [15, 21], ECBC, FCBC and XCBC [34], TMAC [121] and OMAC [97], and the XMAC family comprising of XMACC and XMACR [16], PCS [23], and LightMAC [126].

The security of MAC constructions has seen constant research interest. Among block cipher based constructions, CBC-MAC [15], PMAC [35] and their variants [34, 97, 126] are the most popular.

ANALYSIS OF CBC-MAC: The first concrete result on CBC-MAC was given by Bellare et al. [15]. They showed a bound of $2q^2\ell^2/2^n$ for fixed length queries, which was further improved to $q^2\ell^2/2^n$ by Maurer [128]. Later, Bernstein [27] simplified the proof for fixed-length CBC-MAC. Petrank and Rackoff [162] extended the proof in [15] to prefix-free⁴ queries and a similar extension on Bernstein's proof was done by Rackoff and Gorbunov [80]. Both bounds are about $O(q^2\ell^2/2^n)$. The most recent bound on CBC-MAC is due to Bellare et al. [19] who improved (in terms of ℓ) the bound to $12q^2\ell/2^n + 64q^2\ell^4/2^{2n}$. Another way of improvement is to show a bound of the form $O(\sigma q/2^n)$ (see [149] for more details).

ANALYSIS OF EMAC: In [15], Bellare et al. also suggested some variants of CBC-MAC to handle variable length messages. In particular, they mentioned a construction, where the output of CBC-MAC is further encrypted by an independently keyed block

⁴None of the message is a prefix of another message.

cipher. This construction known as EMAC was first developed during the RACE project [21]. Petrank and Rackoff [162] proved that DMAC (same as EMAC) has security bound $2.5q^2\ell^2/2^n$. Bellare et al. [19] improved the bound to $O(q^2d'(\ell)/2^n)$, where $d'(\ell) = \ell^{o(1)}$ denotes the maximum number-of-divisors function. The latest bound on EMAC is due to Pietrzak [163] that claims $O(q^2/2^n)$ bound for $\ell < 2^{n/8}$.

ANALYSIS OF OTHER MEMBERS OF THE CBC-MAC FAMILY: Both CBC-MAC and EMAC require input lengths in multiple of the underlying block size n . Black and Rogaway [34] introduced three refinements to EMAC, viz., ECBC, FCBC and XCBC to allow use of variable block length strings. They showed that ECBC and FCBC have security bound $2.5\sigma^2/2^n$ and the bound on XCBC is $3.75\sigma^2/2^n$. Jaulmes et al. [102] gave a randomized version of EMAC which they called RMAC and proved that the construction resists birthday attacks. However the proof seems to be incorrect (as suggested in [19]). Other excellent variants of CBC-MAC are TMAC [121], OMAC [97] and GCBC [147]. A variant of OMAC, namely OMAC1 is equivalent to CMAC which became an NIST recommendation [155] in 2005. Another design approach is PMAC [35] by Black and Rogaway that processes the input message in parallel. In [35], Black and Rogaway proved $O(\sigma^2/2^n)$ bound for PMAC. In [98, 140, 148, 151] the improved bounds for XCBC, TMAC, PMAC and OMAC are shown in the form of $O(q^2\ell/2^n)$, $O(\sigma^2/2^n)$ and $O(\sigma q/2^n)$. Apart from these specific constructions, Jutla [112] suggested a general class of DAG-based constructions.

ANALYSIS OF PRF-BASED CBC-MAC FAMILY: There has been little study about the security of CBC-MAC family instantiated by pseudorandom functions or PRFs⁵ (see chapter 2). In fact, we came across only three prior works. First work is a study of iterated MAC constructions by Preneel and van Oorschot [167], which gives tight bounds for compressing PRFs. But, the result is not applicable to CBC-MAC family which requires just length-preserving primitives. Second one is the diploma thesis of Berke [22], where he showed that CBC-MAC restricted to prefix-free messages can be attacked with advantage $\Omega(q^2\ell^2/2^n)$. This is in fact a non trivial result which shows the gap between PRP vs PRF instantiation, because for PRP instantiation of CBC-MAC the advantage is bounded by $O(q^2\ell/2^n)$, whereas for PRF instantiation of CBC-MAC the attack shows that the PRF bound in [15] is tight. The third work is Guo et al's [86] collision distinguisher for iterated random function with advantage $\Omega(q^2\ell/2^n)$. As it turns out, this attack can be easily translated to attack against PRF-based EMAC, ECBC, FCBC, XCBC and TMAC. However, the bound holds when $q^2\ell^2 \leq 2^n$. Subject to this condition, $\Omega(q^2\ell/2^n)$ can be some constant only when $q = \Omega(2^{n/2})$. This lower bound on q is actually no better than the trivial birthday attack with advantage $\Omega(q^2/2^n)$. All of the above mentioned works concentrated on the lower bounds on the PRF advantage. In context

⁵Computationally indistinguishable from a uniform random function.

of upper bounds, we note that any PRP-based security bound can be transformed to PRF-based security bound with a loss of $O(\sigma^2/2^n)$ security using the PRP-PRF switching lemma [14, 47, 183]. However this generic transformation is known to be rather loose, and may not necessarily capture the exact security of PRF-based instantiations.

ANALYSIS OF XMAC FAMILY: All the above schemes have length dependent security bounds, i.e., the security decreases as the length increases. In [16], Bellare et al. proposed two counter-based MACs called XMACC and XMACR. Bellare et al. showed that XMACC and XMACR achieve length independent security bounds of $O(q/2^n)$ and $O(q^2/2^n)$ for large range of message length (up to two power the counter size). Bernstein utilized a similar counter-based encoding to obtain $O(q^2/2^n)$ bound for protected counter sum or PCS [23] assuming that the underlying primitive is PRF. Luykx et al. [126] gave the block cipher version of PCS, called LightMAC, and showed the security bound of $O(q^2/2^n)$. We call the group of all these counter-based schemes, the XMAC family. Although, the XMAC family achieves ℓ -free security bound, it loses on efficiency, since the encoded message is generally much longer than the original message. This is due to the encoding of the counter bits. In general, reducing the counter size as much as possible is desirable for efficiency, but it is not known how one can reduce the counter size, and whether the modified encoding will be secure or not.

1.3.1.1 Improved Security of CBC-MAC Family

In chapter 3, we identify and fix a critical flaw in the state-of-the-art results [19, 163] on the PRF security of CBC-MAC and EMAC. This results in a small change in the existing security bound for CBC-MAC, whereas the existing security bounds for EMAC degrades significantly. Consequently, we provide tight security bound for EMAC while $\ell < 2^{n/4}$. In concrete terms, for CBC-MAC we prove $O(\sigma q/2^n)$ security while $\ell < 2^{n/3}$, and for EMAC we prove $O(q^2/2^n)$ security while $\ell < 2^{n/4}$. Our bound for EMAC also implies tight security for ECBC and FCBC while $\ell < 2^{n/4}$. Table 1.3.1 summarizes the state-of-the-art results and our improvements on CBC-MAC family.

PUBLICATION HISTORY: Chapter 3 is based on our paper [104], accepted in *Journal of Mathematical Cryptology*.

1.3.1.2 Exact Security of PRF-based CBC-MAC Family

In chapter 4, we study the security of CBC-MAC family when the underlying primitive is a length-preserving PRF. We obtain a tight security bound for EMAC, ECBC, FCBC, XCBC, and TMAC. Further, our lower bound attack also applies to OMAC. In concrete

Table 1.3.1: Summary of security (PRF advantage) bounds for the CBC-MAC family. Here q , ℓ , and σ denote the number of queries, maximum permissible message length, and sum of message lengths of all q queries, respectively.

Scheme	State-of-the-art		Chapter 3	
	Bound	Restriction	Bound	Restriction
CBC-MAC [65]	$O(q^2\ell/2^n)$ [19] ¹	$\ell = o(2^{n/3})$	$O(\sigma q/2^n)$	$\ell = o(2^{n/3})$
EMAC [15, 21]	$O(q^2/2^n)$ [163] ¹	$\ell = o(2^{n/8})$	$O(q^2/2^n)$	$\ell = o(2^{n/4})$
	$O(q^2 d'(\ell)/2^n)$ [19] ¹	$\ell = o(2^{n/3})$		
ECBC [34]	$O(q^2 d'(\ell)/2^n)$ [19] ¹	$\ell = o(2^{n/3})$	$O(q^2/2^n)$	$\ell = o(2^{n/4})$
FCBC [34]	$O(q^2 d'(\ell)/2^n)$ [19] ^{1,2}	$\ell = o(2^{n/3})$	$O(q^2/2^n)$	$\ell = o(2^{n/4})$
XCBC [34]	$O(q^2\ell/2^n)$ [140] ³	$\ell = o(2^{n/3})$	-	-
	$O(\sigma^2/2^n)$ [98] ³	-		
TMAC [121]	$O(q^2\ell/2^n)$ [140] ³	$\ell = o(2^{n/3})$	-	-
	$O(\sigma^2/2^n)$ [98] ³	-		
OMAC [97]	$O(\sigma q/2^n)$ [148]	$\ell = o(2^{n/3})$	-	-

¹ Proof is flawed. See Chapter 3 for details.

² Although, FCBC is not mentioned in [19], the EMAC bound is applicable to FCBC as well.

³ σ^2 and $q^2\ell$ are incomparable, as the exact relation depends on the query length distribution.

terms, for EMAC, ECBC, FCBC, XCBC, and TMAC we prove $O(\sigma q/2^n)$ security while $\ell < 2^{n/3}$, and for EMAC, ECBC, FCBC, XCBC, TMAC, and OMAC we present an adversary that breaks the security with $\Omega(q^2\ell/2^n)$ advantage. Table 1.3.2 summarizes the state-of-the-art and our results on PRF-based CBC-MAC family.

PUBLICATION HISTORY: Chapter 4 is based on our paper [109], accepted in *IACR Transactions on Symmetric Cryptology*.

1.3.1.3 Formalizing Counter-based Encoding

In chapter 5, we study the counter-based encoding used in the XMAC family (XMACC, XMACR [16], PCS [23], LightMAC [126]). We present a unified notion for counters, called the *counter function family*. Based on this generalization we identify some necessary and sufficient conditions on counters which give (possibly) simple proof of security for various counter-based cryptographic schemes, including the XMAC family. We observe that these conditions are trivially true for the classical counters. We also identify and study two variants of the classical counter which satisfy the security conditions.

Table 1.3.2: Summary of security (PRF advantage) bounds for PRF-based CBC-MAC family. Here q , ℓ , and σ denote the number of queries, maximum permissible message length, and sum of message lengths of all q queries, respectively.

Scheme	State-of-the-art		Chapter 4	
	Upper Bound	Lower Bound	Upper Bound	Lower Bound
CBC-MAC	$O(q^2 \ell^2 / 2^n)$ [15]	$\Omega(q^2 \ell^2 / 2^n)$ [22]	-	-
EMAC	$O(\sigma^2 / 2^n)$ ¹	$\Omega(q^2 \ell / 2^n)$ [86] ²	$O(\sigma q / 2^n)$	$\Omega(q^2 \ell / 2^n)$ ³
ECBC	$O(\sigma^2 / 2^n)$ ¹	$\Omega(q^2 \ell / 2^n)$ [86] ²	$O(\sigma q / 2^n)$	$\Omega(q^2 \ell / 2^n)$ ³
FCBC	$O(\sigma^2 / 2^n)$ ¹	$\Omega(q^2 \ell / 2^n)$ [86] ²	$O(\sigma q / 2^n)$	$\Omega(q^2 \ell / 2^n)$ ³
XCBC	$O(\sigma^2 / 2^n)$ ¹	$\Omega(q^2 \ell / 2^n)$ [86] ²	$O(\sigma q / 2^n)$	$\Omega(q^2 \ell / 2^n)$ ³
TMAC	$O(\sigma^2 / 2^n)$ ¹	$\Omega(q^2 \ell / 2^n)$ [86] ²	$O(\sigma q / 2^n)$	$\Omega(q^2 \ell / 2^n)$ ³
OMAC	$O(\sigma^2 / 2^n)$ ¹	-	-	$\Omega(q^2 \ell / 2^n)$ ³

¹ Using PRP to PRF switching.

² The bound achieves constant probability for $q = \Omega(2^{n/2})$, and hence no better than folklore attack.

³ The bound holds while $\ell < 2^{n/3}$ for sufficiently large n . In fact, we can choose any $q, \ell \in \Theta(2^{n/3})$.

The first variant has message length dependent counter size, whereas the second variant, based on prefix codes, has message length independent counter size. Furthermore, these variants provide better performance for shorter messages. For instance, when the message size is 2^{19} bits, AES-LightMAC with 64-bit (classical) counter takes 1.51 cycles per byte (CPB), whereas it takes 0.81 CPB and 0.89 CPB for the first and second variant, respectively. We benchmark the software performance of these variants against the classical counter by implementing them in MACs and hash function.

PUBLICATION HISTORY: Chapter 5 is based on our paper [63], accepted in *IEEE Transactions on Computers*.

1.3.2 Analysis of Online and Tweakable Ciphers

We study a generic method of constructing beyond-the-birthday bound (BBB) secure online encryption schemes using tweakable block ciphers, and methods of constructing a tweakable block cipher using block ciphers.

BBB SECURE ONLINE CIPHERS: In [131], Mennink showed that it is (almost) impossible to get BBB secure tweakable block ciphers using block ciphers. A recent work of Andreeva et al. [7] shows that online ciphers are equivalent to arbitrary tweak length

(ATL) tweakable block ciphers (TBCs). By combining these two results one can easily see that building a BBB secure online cipher using block ciphers is also difficult. On the other hand, the proof in [7] is constructive, i.e., it might be possible to get highly secure online ciphers using ATL TBCs (such as XTX [139]). POEx by Forler et al. [71] is loosely based on this approach, and has been shown to have $O(\sigma^2/2^{n+\tau})$ security bound, where τ is the tweak size.

SECURITY OF BLOCK CIPHER-BASED TBCs: In light of [131], it would be interesting to analyze the security of existing TBCs based on block ciphers. Cascaded LRW2 or CLRW2, by Landecker et al. [123], is a popular block cipher to TBC converter. Landecker et al. [123], showed that CLRW2 has roughly $q^3/2^{2n}$ security. Recently, Mennink showed an attack on CLRW2 with advantage $q^4/2^{3n}$. In the same paper, he also showed that CLRW2 based on 4-wise independent AXU (see chapter 2) has $\gamma q/2^n$ security, where γ denotes the maximum number of queries with same tweak. However, Mennink's proof depends on the validity of a restricted version of the generalized Mirror Theory [160, 161], which has several unverified gaps [54, 58].

1.3.2.1 Online Ciphers using Tweakable Block Ciphers

In chapter 6, we study the consequences of instantiating the ATL TBC to online cipher converter of [7] with XTX by Minematsu and Iwata [139]. We first observe that XTX-based instantiations has security in the order of $\sigma^2\epsilon$, where ϵ is the pAXU bound (see chapter 2) of the underlying hash function. We show that there are genuine practical issues which render it almost impossible to get full security using this approach. We then observe that POEx by Forler et al. [71, 72] is actually an implicit example of this approach. We show a flaw in the analysis of POEx which results in a birthday bound attack and invalidates the BBB security claim. We take a slightly different approach than the one just mentioned and propose XTC which achieves security of $O(\max\{n\sigma/2^n, \sigma^2/2^{(n+\tau)}\})$ where τ is the tweak size. While doing so we present an impossibility result for $\tau > n$ which can be of independent interest.

PUBLICATION HISTORY: Chapter 6 is based on our paper [105], accepted in *Cryptography and Communications*.

1.3.2.2 Cascaded LRW2

In chapter 7, we revisit the recent result on Cascaded LRW2 or CLRW2 by Mennink [132]. In that paper, Mennink discussed some non-trivial bottlenecks in proving tight security bound, i.e. security up to $2^{3n/4}$ queries. Subsequently, he proved security up

to $2^{3n/4}$ queries for a variant of CLRW2 using 4-wise independent AXU assumption and the restriction that each tweak value occurs at most $2^{n/4}$ times. Moreover, his proof relies on a version of mirror theory which is yet to be publicly verified. We resolve the bottlenecks in Mennink's approach and prove that CLRW2 is a secure tweakable block cipher up to roughly $2^{3n/4}$ queries, with only one assumption that the underlying hash function family is 3-wise independent AXU. To do so, we develop two new tools: first, we give a probabilistic result that provides improved bound on the joint probability of some special collision events; second, we present a variant of Patarin's mirror theory in tweakable permutation settings with a self-contained and concrete proof. Both these results are of generic nature, and can be of independent interests.

PUBLICATION HISTORY: An abridged version of chapter 7 was under review in *Journal of Cryptology* at the time of submission of this thesis. Subsequently, the results of this work have been improved significantly from what has been described in chapter 7. Particularly, we have been able to drop the 3-wise independent AXU assumption and show tight security for the original CLRW2. This improved and updated result has been accepted in *Journal of Cryptology* [107].

1.3.3 Analysis of Authenticated Encryptions

We explore the scope of random read access and out-of-order decryption in OCB [119, 171, 174].

RANDOM READ ACCESS AND OUT-OF-ORDER DECRYPTION: *A block cipher based encryption mode of operation is said to have random read access feature, if it allows efficient decryption of any arbitrary encrypted block of message.* This property could be beneficial in secure storage of read only data on cloud servers. The encrypted storage helps in avoiding unauthorized access, and the authentication tag helps in maintaining the integrity of the stored data. For example, say some enterprise application stores day to day system log files on some cloud server. Here the day timestamp may act as the nonce value. Now suppose on a later day the application faces some issue which warrants some portion of a particular day's log file. In this case the system administrator should be able to repeatedly read arbitrary blocks from the decrypted log, with as little overhead as possible, by simply mentioning the relevant day timestamp and the offset in the log file.

Yet another use-case for random read access is in disk encryption where access to an arbitrary block of some disk sector is highly beneficial. A popular and standardized disk encryption scheme, called XTS by Rogaway [171], is quite similar to OCB encryption.

A related feature is *out-of-sequence decryption* where the mode of operation allows for the decryption of the encrypted block data stream in an arbitrary order. In other words, the original ordering of the ciphertext blocks is not necessary for the decryption phase. This is quite useful in secure data transmission over connection-less network protocols such as User Datagram Protocol (UDP) [165], which allows for out-of-order delivery of data packets. If the underlying encryption scheme offers efficient out-of-sequence decryption then the data packets can be decrypted in on-the-fly manner; otherwise the received packets have to be reordered first which adds to the network protocol overhead. Yet another application area is real time data streaming protocols such as Secure Real-time Transport Protocol (SRTP) [10, 111, 124] and SRTP Control Protocol (SRTCP) [178], where on-the-fly decryption of received data packets is necessary to maintain the continuity of audio or visual data. In general, a scheme with random read access feature also allows out-of-sequence decryption.

1.3.3.1 Random Read Access in OCB

In chapter 8, we first observe that the current versions of OCB are inefficient in random read access owing to the ineptness of the underlying mask (or offsets) generating function (MGF). We propose new candidates for MGF based on AES round function, which are efficient in direct computation and provide comparable performance in the usual setting. Our schemes are not the obvious choices for MGF in conventional sense as they do not have optimal almost XOR universal (AXU) bound. In existing OCB designs the MGFs are required to have 2^{-n} , i.e. optimal, AXU bound in order to upper bound the distinguishing advantage to $O(\sigma^2/2^n)$. We find this specific requirement too restrictive. We abstract the OCB design, termed as GOCB, to look into the universal notion required from the underlying MGF. We propose a relaxed notion of AXU, called locally imperfect XOR universal (LIXU) hash, which can be of independent interest. Using LIXU as the underlying MGF we recover reasonable security bounds for our schemes.

PUBLICATION HISTORY: Chapter 8 is based on our paper [110], accepted in *IEEE Transactions on Information Theory*.

1.3.4 List of Research Papers

This thesis is based on following research papers:

1. Ashwin Jha and Mridul Nandi: *Revisiting Structure Graphs: Applications to CBC-MAC and EMAC*. In J. Mathematical Cryptology, 10(3–4):157–180, 2016.

2. Ashwin Jha, Avradip Mandal and Mridul Nandi: *On the Exact Security of Message Authentication Using Pseudorandom Functions*. In IACR Trans. Symmetric Cryptology, 2017(1):427–448, 2017.
3. Avijit Dutta, Ashwin Jha and Mridul Nandi: *A New Look at Counters: Don't Run Like Marathon in a Hundred Meter Race*. In IEEE Trans. Computers, 66(11):1851–1864, 2017.
4. Ashwin Jha and Mridul Nandi: *On Rate-1 and Beyond-the-Birthday Bound Secure Online Ciphers using Tweakable Block Ciphers*. In Cryptography and Communications, 10(5):731–753, 2018.
5. Ashwin Jha and Mridul Nandi: *Tight Security of Cascaded LRW2*. In J. Cryptology, 2020.
6. Ashwin Jha, Cuauhtemoc Mancillas-López, Mridul Nandi and Sourav Sen Gupta: *On Random Read Access in OCB*. In IEEE Trans. Information Theory, 65(12):8325–8344, 2019.

Note. The ordering of the authors' name, in the publication, follows the **Hardy-Littlewood principle** i.e., it is determined by the alphabetical ordering of the last names of the authors.

Chapter 2

Preliminaries

2.1 Notational Setup

We write the set of all positive integers as $\mathbb{N}$ and $\mathbb{N}_0 := \mathbb{N} \cup \{0\}$. For $n \geq r \in \mathbb{N}_0$, we define the falling factorial $(n)_r := n!/(n-r)! = n(n-1) \cdots (n-r+1)$. For $a < b \in \mathbb{N}_0$, $[a \dots b]$ denotes the set $\{a, a+1, \dots, b\}$. For $a = 0$ and $a = 1$, we simply write it as (b) and $[b]$, respectively. We sometimes use the short hand notation $\exists^*$ to represent the phrase “there exists distinct”. For any finite set $\mathcal{X}$, $\mathbf{X} \leftarrow \$ \mathcal{X}$ represents the uniform at random sampling of $\mathbf{X}$ from $\mathcal{X}$.

STRINGS AND TUPLES: For $n \in \mathbb{N}$, $\{0, 1\}^n$ denotes the set of all n -bit strings, and $\{0, 1\}^+ := \cup_{i=1}^{\infty} \{0, 1\}^i$. We write ε to denote the empty string and $\{0, 1\}^* := \{0, 1\}^+ \cup \{\varepsilon\}$. For $x \in \{0, 1\}^*$, $|x|$ denotes the *bit-length* (or simply length) of x , and $|\varepsilon| = 0$ by convention. For $x, y \in \{0, 1\}^*$, $z = x\|y$ denotes the concatenation of x and y with $|z| = |x| + |y|$. Thus, by definition $x\|\varepsilon = \varepsilon\|x = x$. For $z = x\|y \in \{0, 1\}^*$, x and y are called the *prefix* and *suffix*, respectively, of z . For any $r, s \in \mathbb{N}$, $\langle s \rangle_r$ denotes the r -bit unsigned representation of integer s . For any $x \in \{0, 1\}^*$ and $r < |x|$, $\text{msb}_r(x)$ and $\text{lsb}_r(x)$ denote the substrings $x_1\| \cdots \|x_r$ and $x_{|x|-r+1}\| \cdots \|x_{|x|}$, respectively.

For a finite set $\mathcal{X} \subset \{0, 1\}^*$ and $q \in \mathbb{N}$, $x^q \in \mathcal{X}^q$ denotes the q -tuple (or sequence) $(x_i)_{i \in [q]} := (x_1, x_2, \dots, x_q)$, where $x_i \in \mathcal{X}$ for all $i \in [q]$. By an abuse of notation, we also use x^q to denote the multiset $\{x_i : i \in [q]\}$ and write $\mu(x^q, x')$ to denote the multiplicity of $x' \in x^q$. We sometimes write $\widehat{x^q}$ to denote the set $\{x_i : i \in [q]\}$. We write $(\mathcal{X})_q \subset \mathcal{X}^q$ as the set of all q -tuples with distinct elements from $\mathcal{X}$, i.e., for $x^q \in (\mathcal{X})_q$, $x^q = \widehat{x^q}$ and $|(\mathcal{X})_q| = (|\mathcal{X}|)_q$. For any tuple $x^q \in \mathcal{X}^q$, and for any function $f : \mathcal{X} \rightarrow \mathcal{Y}$, $f(x^q)$ denotes the q -tuple $(f(x_1), \dots, f(x_q)) \in \mathcal{Y}^q$. For a set $\mathcal{I} \subseteq [q]$ and a q -tuple x^q , $x^{\mathcal{I}}$ denotes the

tuple $(x_i)_{i \in \mathcal{I}}$. By extending the notation, we write $x^{\mathcal{I}} = (x_\alpha)_{\alpha \in \mathcal{I}}$ for any abstract index set $\mathcal{I}$. Thus, x^q is just a shorthand notation for $x^{[q]}$.

For $n, r, s \in \mathbb{N}$, let $x^r \in (\{0, 1\}^n)^r$, and $y^s \in (\{0, 1\}^n)^s$. We define the longest common prefix of x^r and y^s , denoted $\text{lcp}_n(x^r, y^s)$, as

$$\text{lcp}_n(x^r, y^s) := \begin{cases} 0 & \text{if } x_1 \neq y_1, \\ i & \text{if } (x_1, \dots, x_i) = (y_1, \dots, y_i) \text{ and } x_{i+1} \neq y_{i+1}, \end{cases}$$

and the longest common suffix of x^r and y^s , denoted $\text{lcs}_n(x^r, y^s)$, as

$$\text{lcs}_n(x^r, y^s) := \begin{cases} 0 & \text{if } x_r \neq y_s, \\ i & \text{if } (x_{r-i+1}, \dots, x_r) = (y_{s-i+1}, \dots, y_s) \text{ and } x_{r-i} \neq y_{s-i}, \end{cases}$$

It is easy to see that $\text{lcp}_n(x^r, y^s), \text{lcs}_n(x^r, y^s) \leq \min\{r, s\}$. The subscript n is dropped whenever it is clear from the context.

COMPATIBILITY OF TUPLES: For a pair of tuples $x^q, y^q \in \mathcal{X}^q$, (x^q, y^q) denotes the 2-ary q -tuple $((x_1, y_1), \dots, (x_q, y_q))$. An n -ary q -tuple is defined analogously.

1. A 2-ary q -tuple (x^q, y^q) is called *function compatible*, denoted $x^q \leftrightarrow y^q$, if $x_i = y_i \implies y_i = x_i$.
2. A 2-ary q -tuple (x^q, y^q) is called *permutation compatible*, denoted $x^q \rightsquigarrow y^q$, if $x_i = y_j \iff y_i = x_j$.
3. A 3-ary tuple (t^q, x^q, y^q) is called *tweakable permutation compatible*, denoted by $(t^q, x^q) \rightsquigarrow (t^q, y^q)$, if $(t_i, x_i) = (t_j, x_j) \iff (t_i, y_i) = (t_j, y_j)$.

2.2 Security Game, Adversary and Advantage

In (symmetric-key) cryptology the security of a given scheme is argued via various security games. A *security game* is nothing but an interactive game involving two parties: an *adversary* $\mathcal{A}$ that tries to break some scheme with respect to some security notion, and an oracle $\mathcal{O}$ that either represents the ideal object or the real scheme.

ADVERSARY AND ORACLE: An adversary $\mathcal{A}$ is an interactive Turing machine (or an algorithm)¹ that interacts with a given set of oracles that appear as black boxes to $\mathcal{A}$. An oracle $\mathcal{O}$ is nothing but an interface to a set of functions. For an oracle $\mathcal{O}$, $\mathcal{A}^{\mathcal{O}}$ denotes

¹We deliberately withheld the adjective “efficient”, as we will allow computationally unbounded adversaries in some cases.

the output of $\mathcal{A}$ after its interaction with $\mathcal{O}$. Based on $\mathcal{A}$'s response, the security game results in either success or failure.

DISTINGUISHING GAME: Looking ahead momentarily, in the context of this thesis, we mostly consider security game as a standard distinguishing game with an optional set of additional restrictions, chosen to reflect the desired security goal(s). For instance, some security games give bidirectional access to the underlying function of an oracle $\mathcal{O}$ and its inverse. This is denoted by $\mathcal{O}^\pm$. In a *distinguishing game*, the adversary or *distinguisher* $\mathcal{A}$ tries to distinguish between two sets of oracle(s), say $\mathcal{O}_0$ and $\mathcal{O}_1$. It interacts with the oracle(s) at hand (either $\mathcal{O}_0$ or $\mathcal{O}_1$) and then outputs a single bit response. We say that $\mathcal{A}$ wins the distinguishing game if $\mathcal{A}^{\mathcal{O}_b} = b$ for $b \in \{0, 1\}$. Formally, the distinguishing advantage of $\mathcal{A}$ is defined as

$$\mathbf{Adv}_{\mathcal{O}_1; \mathcal{O}_0}(\mathcal{A}) := \left| \Pr [\mathcal{A}^{\mathcal{O}_1} = 1] - \Pr [\mathcal{A}^{\mathcal{O}_0} = 1] \right| \quad (2.1)$$

CONVENTIONS: $\mathcal{O}_0$ conventionally represents an ideal primitive, while $\mathcal{O}_1$ represents either an actual construction or a mode of operation built of some other ideal primitives. Typically, the goal of the function represented by $\mathcal{O}_1$ is to emulate the ideal primitive represented by $\mathcal{O}_0$. We use the standard terms *ideal oracle* and *real oracle* for $\mathcal{O}_0$ and $\mathcal{O}_1$, respectively. When we talk of distinguishing advantage with a specific distinguishing game G in mind, we include g in the superscript, i.e., $\mathbf{Adv}_{\mathcal{O}_1; \mathcal{O}_0}^g(\mathcal{A})$. Usually, the ideal oracle $\mathcal{O}_0$ is completely identified by G , and hence dropped from the subscript.

In this thesis, we assume that the adversary is non-trivial, i.e. it never makes a duplicate query, and it never makes a query for which the response is already known due to some previous query. Typically, any adversary's resources will be measured by a subset of the following parameters:

- q , denotes the upper bound on the number of queries made by the adversary.
- ℓ , denotes the upper bound on the length of any query.
- σ , denotes the upper bound on the aggregate total length of all queries.
- t , denotes the upper bound on the computation time allowed to the adversary.

The actual parameters would depend upon the security game. We write $\mathbb{A}(\text{res})$ to denote the set of all adversaries whose resources are bounded by the tuple of parameters res .

2.2.1 The Expectation Method and Coefficient H-Technique

Let $\mathcal{A}$ be a computationally unbounded and deterministic² distinguisher. We denote the query-response tuple of $\mathcal{A}$'s interaction with its oracle by a transcript ω . This may also include any additional information the oracle chooses to reveal to the distinguisher at the end of the query-response phase of the game. We denote by Θ_1 (res. Θ_0) the random transcript variable when $\mathcal{A}$ interacts with $\mathcal{O}_1$ (res. $\mathcal{O}_0$). The probability of realizing a given transcript ω in the distinguishing game with an oracle $\mathcal{O}$ is known as the *interpolation probability* of ω with respect to $\mathcal{O}$. Note that, for a transcript to be realized, two things need to happen:

- The distinguisher needs to make the queries listed in the transcript;
- The oracle needs to make the corresponding responses.

Of these, the former is deterministic; the latter, probabilistic. Thus, when we talk of interpolation probability, we are only concerned with the oracle responses, with the assumption that the distinguisher's queries are consistent with the transcript. For any other distinguisher, the interpolation probability is trivially 0. A transcript ω is said to be *attainable* if $\Pr[\Theta_0 = \omega] > 0$. The expectation method (stated below) is quite useful in obtaining upper bounds on the distinguishing advantage in many cases [85, 90, 91].

Lemma 2.2.1 (Expectation Method [90]). *Let Ω be the set of all realizable transcripts. For some $\epsilon_{\text{bad}} > 0$ and a non-negative function $\epsilon_{\text{ratio}} : \Omega \rightarrow [0, \infty)$, suppose there is a set $\Omega_{\text{bad}} \subseteq \Omega$ satisfying the following:*

- $\Pr[\Theta_0 \in \Omega_{\text{bad}}] \leq \epsilon_{\text{bad}}$;
- For any $\omega \notin \Omega_{\text{bad}}$, $\frac{\Pr[\Theta_1 = \omega]}{\Pr[\Theta_0 = \omega]} \geq 1 - \epsilon_{\text{ratio}}(\omega)$.

Then for any deterministic distinguisher $\mathcal{A}$ that distinguishes $\mathcal{O}_1$ from $\mathcal{O}_0$, we have the following bound on its distinguishing advantage:

$$\text{Adv}_{\mathcal{O}_1, \mathcal{O}_0}(\mathcal{A}) \leq \epsilon_{\text{bad}} + \text{Ex}[\epsilon_{\text{ratio}}(\Theta_0)],$$

where $\text{Ex}[X]$ denotes the expectation of random variable X .

Proof. A proof of the technique is given in [90], and we reproduce it here for the sake of completeness. As $\mathcal{A}$ is deterministic, its advantage is at most the statistical distance

²One can always replace a probabilistic adversary with a deterministic one (with at least the same advantage) given unbounded computation time.

between Θ_0 and Θ_1 (see [48, 90, 106, 144, 159]). Thus, we have,

$$\begin{aligned}
\text{Adv}_{\mathcal{O}_1; \mathcal{O}_0}(\mathcal{A}) &= \left| \Pr[\mathcal{A}^{\mathcal{O}_1} = 1] - \Pr[\mathcal{A}^{\mathcal{O}_0} = 1] \right| \\
&\leq \sum_{\omega \in \Omega} \max \left\{ 0, \Pr[\Theta_0 = \omega] - \Pr[\Theta_1 = \omega] \right\} \\
&\leq \sum_{\omega \in \Omega_{\text{bad}}} \Pr[\Theta_0 = \omega] + \sum_{\omega \in \Omega \setminus \Omega_{\text{bad}}} \Pr[\Theta_0 = \omega] \cdot \max \left\{ 0, 1 - \frac{\Pr[\Theta_1 = \omega]}{\Pr[\Theta_0 = \omega]} \right\} \\
&\leq \epsilon_{\text{bad}} + \mathbb{E}[\epsilon_{\text{ratio}}(\Theta_0)].
\end{aligned}$$

□

The coefficient-H technique due to Patarin [158, 159] is a simple corollary of this result where ϵ_{ratio} is a constant function.

Corollary 2.2.2 (Coefficient-H Technique [158, 159]). *Let Ω be the set of all realizable transcripts. For some $\epsilon_{\text{bad}}, \epsilon_{\text{ratio}} > 0$, suppose there is a set $\Omega_{\text{bad}} \subseteq \Omega$ satisfying the following:*

- $\Pr[\Theta_0 \in \Omega_{\text{bad}}] \leq \epsilon_{\text{bad}};$
- For any $\omega \notin \Omega_{\text{bad}}, \frac{\Pr[\Theta_1 = \omega]}{\Pr[\Theta_0 = \omega]} \geq 1 - \epsilon_{\text{ratio}}.$

Then for any deterministic distinguisher $\mathcal{A}$ that distinguishes $\mathcal{O}_1$ from $\mathcal{O}_0$, we have the following bound on its distinguishing advantage:

$$\text{Adv}_{\mathcal{O}_1; \mathcal{O}_0}(\mathcal{A}) \leq \epsilon_{\text{bad}} + \epsilon_{\text{ratio}}.$$

In context of the coefficient-H technique, we note that in an independent work [23] Bernstein rediscovered a similar result, called the *interpolation theorem*. This was later strengthened by Nandi [144] as the *strong interpolation theorem*.

2.3 (Tweakable) Block Cipher and (XOR) Universal Hash

For $\mathcal{X}, \mathcal{Y} \subset \{0, 1\}^*$, we write $\text{Perm}(\mathcal{X})$ to denote the set of all permutations over $\mathcal{X}$, and $\text{Func}(\mathcal{X}, \mathcal{Y})$ to denote the set of all functions from $\mathcal{X}$ to $\mathcal{Y}$. For $\mathcal{K}, \mathcal{X} \subset \{0, 1\}^*$, $\text{BPerm}(\mathcal{K}, \mathcal{X})$ denotes the set of all families of permutations $\pi_k := \pi(k, \cdot) \in \text{Perm}(\mathcal{X})$, indexed by $k \in \mathcal{K}$. We sometime extend this notation for some $\mathcal{T} \subset \{0, 1\}^*$, whereby $\text{BPerm}(\mathcal{K}, \mathcal{T}, \mathcal{X})$ denotes the set of all families of permutations $\pi_{(k,t)}$, indexed by $(k, t) \in \mathcal{K} \times \mathcal{T}$. For $\mathcal{K}, \mathcal{X}, \mathcal{Y} \subset \{0, 1\}^*$, $\text{BFunc}(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ denotes the set of all families of functions $\gamma_k := \gamma(k, \cdot) \in \text{Func}(\mathcal{X}, \mathcal{Y})$, indexed by $k \in \mathcal{K}$. We will drop the parametrization of these sets (of functions/permutations), whenever the parameters are clear from the context.

2.3.1 (Tweakable) Block Cipher

A *block cipher* $E_{-n/\kappa}$, with key size κ and block size n is a family of permutations $E \in \text{BPerm}(\{0, 1\}^\kappa, \{0, 1\}^n)$. For $k \in \{0, 1\}^\kappa$, we denote $E_k(\cdot) := E(k, \cdot)$, and $E_k^{-1}(\cdot) := E^{-1}(k, \cdot)$. A *tweakable block cipher* $\tilde{E}_{-n/\tau/\kappa}$, with key size κ , tweak size τ and block size n is a family of permutations $\tilde{E} \in \text{BPerm}(\{0, 1\}^\kappa, \{0, 1\}^\tau, \{0, 1\}^n)$. For $k \in \{0, 1\}^\kappa$ and $t \in \{0, 1\}^\tau$, we write $\tilde{E}_k^t(\cdot) = \tilde{E}_k(t, \cdot) := \tilde{E}(k, t, \cdot)$, and $\tilde{E}_k^{-t}(\cdot) = \tilde{E}_k^{-1}(t, \cdot) := \tilde{E}^{-1}(k, t, \cdot)$.

Symmetric-key literature is filled with a plethora of (tweakable) block cipher candidates. Some popular block ciphers include, AES-128/128 [52, 152], PRESENT-64/128 [38], Simon-128/192 and Speck-96/144 [11], GIFT-64/128 [8] etc. Deoxys-BC-128/128/128 and Kiasu-BC-128/64/128 [103], Skinny-128/128/128 and Mantis-64/64/128 [12] etc., are some examples of tweakable block ciphers.

Throughout the thesis, we fix $\kappa, \tau, n \in \mathbb{N}$ as the key size, tweak size and block size, respectively, of the given (tweakable) block cipher. We also write $\mathbb{B} = \{0, 1\}^n$ and $\mathbb{T} = \{0, 1\}^\tau$. Accordingly, the elements of $\mathbb{B}, \mathbb{T}, \{0, 1\}^\kappa$ are referred as *blocks*, *tweaks*, and *keys*, respectively. For some $k \geq 0$, a tuple $x^k \in \mathbb{B}^k$ is called *block tuple* (or *block sequence*). Similarly, $x^k \in \mathbb{T}^k$ is called *tweak sequence*. Sometimes, we also write $|x|_n = \lceil |x|/n \rceil$ to denote the *block-length* of x .

Let $\eta : \mathbb{N} \rightarrow \mathbb{N}$ be a function such that for all $r \in \mathbb{N}$, $r \xrightarrow{\eta} s$, where $s \gg 2^r$. Throughout, we use $\eta(n)$ to denote a sufficiently large input length. Accordingly, we almost always write η to mean $\eta(n)$. For any finite set $\mathcal{S} \subset \{0, 1\}^*$, we write $\mathcal{S}^{\eta^*}$ to denote the set $\cup_{i=0}^{\eta} \mathcal{S}^i$. The set $\mathcal{S}^{\eta^+}$ is defined analogously.

We define the padding function $\text{pad} : \{0, 1\}^{\eta^*} \rightarrow (\mathbb{B}^n)^{\eta^+}$ by the following mapping

$$\forall M \in \{0, 1\}^{\eta^*}, \bar{M} = \text{pad}(M) := \begin{cases} M \parallel 10^{n|M|_n - |M| - 1} & \text{if } n \nmid |M|, \\ M & \text{otherwise.} \end{cases}$$

2.3.1.1 (Tweakable) Strong Pseudorandom Permutation

In the Strong Pseudorandom Permutation or SPRP security game, the distinguisher $\mathcal{A}$ tries to distinguish a block cipher $E_{-n/\kappa}$ instantiated with a key $K \leftarrow \$ \{0, 1\}^\kappa$ from a uniform random permutation $\Pi \leftarrow \$ \text{Perm}(\mathbb{B})$. The *SPRP advantage* of $\mathcal{A}$ against E is defined as

$$\text{Adv}_E^{\text{sprp}}(\mathcal{A}) := \text{Adv}_{E^\pm; \Pi^\pm}(\mathcal{A}) \quad (2.2)$$

For $\epsilon \in [0, 1]$, a block cipher family E is called (q, t, ϵ) -SPRP, if we have

$$\text{Adv}_E^{\text{sprp}}(q, t) := \max_{\mathcal{A} \in \mathbb{A}(q, t)} \text{Adv}_E^{\text{sprp}}(\mathcal{A}) \leq \epsilon.$$

Note that, the distinguisher is given bidirectional access to the oracles. When the distinguisher is restricted to forward-only queries, the security game is called PRP. $\text{Adv}_E^{\text{prp}}(\mathcal{A})$ and $\text{Adv}_E^{\text{prp}}(q, t, \epsilon)$ are defined analogously as above with appropriate restrictions on oracle access.

In the Tweakable SPRP or TSPRP security game, the distinguisher $\mathcal{A}$ tries to distinguish a tweakable block cipher $\tilde{E}$ with $n/\tau/\kappa$ instantiated with a key $K \leftarrow \$ \{0, 1\}^\kappa$ from a uniform tweakable random permutation $\tilde{\Pi} \leftarrow \$ \text{BPerm}(\mathbb{T}, \mathbb{B})$. The *TSPRP advantage* of distinguisher $\mathcal{A}$ against $\tilde{E}$ is defined as

$$\text{Adv}_{\tilde{E}}^{\text{tsprp}}(\mathcal{A}) := \text{Adv}_{\tilde{E}^\pm; \tilde{\Pi}^\pm}(\mathcal{A}). \quad (2.3)$$

For $\epsilon \in [0, 1]$, a tweakable block cipher family $\tilde{E}$ is called (q, t, ϵ) -TSPRP, if we have

$$\text{Adv}_{\tilde{E}}^{\text{tsprp}}(q, t) := \max_{\mathcal{A} \in \mathbb{A}(q, t)} \text{Adv}_{\tilde{E}}^{\text{tsprp}}(\mathcal{A}) \leq \epsilon,$$

where q and t are defined analogous to SPRP game. When the distinguisher is restricted to forward-only queries, the security game is called TPRP. $\text{Adv}_{\tilde{E}}^{\text{tprp}}(\mathcal{A})$ and $\text{Adv}_{\tilde{E}}^{\text{tprp}}(q, t, \epsilon)$ are defined analogously as above with appropriate restrictions on oracle access.

Throughout the thesis, we use Π and $\tilde{\Pi}$ to denote uniform random permutation and uniform tweakable random permutation, respectively, over suitable sets which will be clear from the context.

2.3.2 (XOR) Universal Hash Functions

For some $k \in \mathbb{N}$, $\mathcal{K}, \mathcal{X} \subset \{0, 1\}^{\eta(k)^*}$, and $\mathcal{Y} = \{0, 1\}^k$, a $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ -hash function H is a family of functions $H \in \text{BFunc}(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ defined over its *domain or message space* $\mathcal{X}$, *digest or hash space* $\mathcal{Y}$ and indexed by the *key space* $\mathcal{K}$.

There are several popular notions of universality for hash functions in literature [43, 57, 117, 132, 139, 171, 176, 185]. We discuss some of them which will be used later on in this thesis.

For the sake of simplicity, we let $\mathcal{H}$ denote the multiset $\{H_i : i \in \mathcal{K}\}$, and $H = H_K$ where $K \leftarrow \$ \mathcal{K}$. Thus, $H \leftarrow \$ \mathcal{H}$.

Definition 2.3.1 (Almost XOR universal hash function [117, 170]). A $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ -hash function H is called ϵ -almost XOR universal or AXU for some $\epsilon \in [0, 1]$, if for any $x^2 \in (\mathcal{X})_2$ and any $\delta \in \mathcal{Y}$, the differential probability is at most ϵ . In other words, we have

$$\Pr_{H \leftarrow \mathcal{H}} [H(x_1) \oplus H(x_2) = \delta] \leq \epsilon. \quad (2.4)$$

When $\delta = 0$, the 0-differential event is equivalent to a collision in hash value. This special case is handled by universal hash functions.

Definition 2.3.2 (Almost universal hash function [43]). A $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ -hash family H is called ϵ -almost universal or AU for some $\epsilon \in [0, 1]$, if for any $x^2 \in (\mathcal{X})_2$, the collision probability is at most ϵ . In other words, we have

$$\Pr_{H \leftarrow \mathcal{H}} [H(x_1) = H(x_2)] \leq \epsilon. \quad (2.5)$$

The notion of AXU hash functions was first introduced by Krawczyk [117] and later by Rogaway [170], as a generalization of the original universal hash definition by Carter and Wegman [43, 185]. Multi-linear hash [87, 177], PDP hash [36, 87, 118, 188], CBC [62] and PHash [35] are some popular examples of A(X)U hash functions.

In [132], Mennink gave a slightly generalized notion of AXU, called ℓ -wise independent AXU or AXU_ℓ , for $\ell \geq 2$.

Definition 2.3.3 (ℓ -wise Independent AXU hash function [132]). A $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ -hash family H is called ϵ - AXU_ℓ for some $\epsilon \in [0, 1]$, if for any $j \in [\ell] \setminus \{1\}$, any $x^j \in (\mathcal{X})_j$, and any $\delta^{j-1} \in \mathcal{Y}^{j-1}$ we have

$$\Pr_{H \leftarrow \mathcal{H}} [H(x_1) \oplus H(x_2) = \delta_1, \dots, H(x_1) \oplus H(x_j) = \delta_{j-1}] \leq \epsilon^{j-1}. \quad (2.6)$$

In [132], Mennink suggested some ways to construct AXU_ℓ hash function family over a small domain using finite field arithmetic. Note that, ℓ -wise independent hash functions [185] are also AXU_ℓ .

Minematsu and Iwata [139] introduced the notion of ϵ -partial-almost-XOR-universal or pAXU hash functions to capture simultaneous collision and differential events on two disjoint substrings of any hash value.

Definition 2.3.4 (ϵ -partial-almost-XOR-universal hash function [139]). A $(\mathcal{K}, \mathcal{X}, \mathcal{Y}_1 \times \mathcal{Y}_2)$ -hash function H is called ϵ -partial-almost-XOR-universal (or pAXU) if for all $x^2 \in (\mathcal{X})_2$ and $\delta \in \mathcal{Y}_2$, we have

$$\Pr_{H \leftarrow \mathcal{H}} [H(x) \oplus H(x') = (0, \delta)] \leq \epsilon. \quad (2.7)$$

2.3.2.1 Combiners of Hash Functions

We consider two types of combiners of A(X)U hash functions.

- *Concatenated Hash* — Let H_i be a $(\mathcal{K}_i, \mathcal{X}, \mathcal{Y}_i)$ hash family for $i \in [2]$. The concatenated hash $H_1 \| H_2$ is a $(\mathcal{K}_1 \times \mathcal{K}_2, \mathcal{X}, \mathcal{Y}_1 \times \mathcal{Y}_2)$ hash family defined by the following mapping:

$$(K_1, K_2, x) \mapsto H_1(K_1, x) \| H_2(K_2, x).$$

- *Sum Hash* — Let H_i be a $(\mathcal{K}_i, \mathcal{X}_i, \mathcal{Y})$ hash family for $i \in [2]$. The sum hash $H_1 \oplus H_2$ is a $(\mathcal{K}_1 \times \mathcal{K}_2, \mathcal{X}_1 \times \mathcal{X}_2, \mathcal{Y})$ hash family defined by the following mapping:

$$((K_1, K_2), (x_1, x_2)) \mapsto H_1(K_1, x_1) \oplus H_2(K_2, x_2).$$

Some easily verifiable properties of (p)A(X)U hash functions are accumulated in proposition 2.3.5.

Proposition 2.3.5. *Let H_i be a $(\mathcal{K}_i, \mathcal{X}_i, \mathcal{Y}_i)$ hash family for $i \in [2]$, and H_1 and H_2 are keyed independently. Then, we have*

1. *If H_1 is an ϵ -AXU hash, then it is an ϵ -AU hash.*
2. *For $\mathcal{X}_1 = \mathcal{X}_2$, consider the concatenated hash $H_1 \| H_2$.*
 - (a) *If H_i is ϵ_i -AXU hash for $i \in [2]$, then $H_1 \| H_2$ is an $\epsilon_1 \epsilon_2$ -AXU hash.*
 - (b) *If H_1 is ϵ_1 -AU hash and H_2 is ϵ_2 -AXU hash, then $H_1 \| H_2$ is an $\epsilon_1 \epsilon_2$ -pAXU hash.*
3. *For $\mathcal{Y}_1 = \mathcal{Y}_2$, consider the sum hash $H_1 \oplus H_2$. If H_i is ϵ_i -AXU hash for $i \in [2]$, then $H_1 \oplus H_2$ is $\max\{\epsilon_1, \epsilon_2\}$ -AXU hash.*
4. *For $k \leq m \in \mathbb{N}$ and $\mathcal{Y}_1 = \{0, 1\}^m$, if H_1 is an ϵ_1 -AXU, then the truncated hash $\text{msb}_k \circ H_1$ is an $\epsilon_1 2^{m-k}$ -AU hash function.*

2.4 Message Authentication Codes

A *message authentication code* or MAC is a tuple of algorithms $\mathfrak{M} = (\mathfrak{M}^+, \mathfrak{M}^-)$, defined over the key space $\mathcal{K}$, nonce space $\mathcal{N}$, message space $\mathcal{M}$, and tag space $\mathcal{T}$, where:

$$\mathfrak{M}^+ : \mathcal{K} \times \mathcal{N} \times \mathcal{M} \rightarrow \mathcal{T} \quad \mathfrak{M}^- : \mathcal{K} \times \mathcal{N} \times \mathcal{M} \times \mathcal{T} \rightarrow \{\top, \perp\}.$$

Here $\mathfrak{M}^+$ and $\mathfrak{M}^-$ are called the tag-generation and tag-verification algorithms, respectively, and $\top$ and $\perp$ denote the authentication success and failure symbols, respectively. Further, it is required that $\mathfrak{M}^-(K, N, \mathfrak{M}^+(K, N, M)) = \top$ for any $(K, N, M) \in \mathcal{K} \times \mathcal{N} \times \mathcal{M}$. For all key $K \in \mathcal{K}$, we write $\mathfrak{M}_K^+(\cdot)$ and $\mathfrak{M}_K^-(\cdot)$ to denote $\mathfrak{M}^+(K, \cdot)$ and $\mathfrak{M}^-(K, \cdot)$, respectively. In this thesis, we have $\mathcal{K}, \mathcal{T} \subseteq \mathbb{B}^{\eta^+}$, $\mathcal{N} \in \mathbb{B}^{\eta^*}$ and $\mathcal{M} \in \{0, 1\}^{\eta^*}$.

MACs can be classified into two classes based on the cardinality of $\mathcal{N}$:

1. **Deterministic MACs:** When the nonce space is empty, i.e., $\mathcal{N} = \emptyset$, the MAC scheme is classified as a deterministic MAC. Some popular deterministic MACs include, CBC-MAC [65], EMAC [15, 21], ECBC, FCBC and XCBC [34], TMAC [121], OMAC [97, 155], GCBC1 and GCBC2 [147], PCS [23], PMAC [35], LightMAC [126], SUM-ECBC [189], PMAC+[190], LightMAC+[143] etc.
2. **Non-deterministic MACs:** When the nonce space is non-empty, i.e., $\mathcal{N} \neq \emptyset$, the MAC scheme is classified as a non-deterministic MAC. Now, depending upon the nature of the nonce value in tag-generation algorithm, we may have two sub-classifications:
 - (a) **Stateful MACs:** In stateful MACs, each invocation of the tag-generation algorithm requires a distinct nonce value, i.e., each nonce value is used only once in tag-generation calls. In other words, the tag-generation algorithm is stateful. The uniqueness of nonce value is, in general, necessary for security, though there are some schemes which allow some amount of nonce repetitions. Some popular nonce-based MACs include, Wegman-Carter MACs [185], XMACC [16], WMAC [33], EWCDM [49], DWCDM [58], nEHtM [64] etc.
 - (b) **Probabilistic MACs:** In probabilistic MACs, the nonce value is treated as random *salt* in the tag-generation algorithm, i.e., the nonce value is sampled uniformly at random in each tag-generation call. In other words, the tag-generation algorithm is probabilistic. Some popular probabilistic MACs include, XMACR [16], MACRX [17], RMAC [102], FRMAC [101], EHtM [136], RWMAC [136] etc.

In this thesis, we will mainly focus on deterministic MACs.

2.4.1 MAC Security Definition

In MAC security game, a non-trivial adversary $\mathcal{A}$ has oracle access to $\mathfrak{M}_K^+$ and $\mathfrak{M}_K^-$ for $K \leftarrow \$ \mathcal{K}$. It can make “MAC” queries to $\mathfrak{M}_K^+$ and “verification” queries to $\mathfrak{M}_K^-$ in arbitrary order. We say that $\mathcal{A}$ forges if any of its verification queries to $\mathfrak{M}_K^-$ returns $\top$.

Suppose $\mathcal{A}$ halts with response 1 if it was successful in forgery, and 0 otherwise. The *MAC advantage* of $\mathcal{A}$ against $\mathfrak{M}$ is defined as

$$\mathbf{Adv}_{\mathfrak{M}}^{\text{mac}}(\mathcal{A}) := \Pr_{K \leftarrow \mathfrak{K}} \left[\mathcal{A}(\mathfrak{M}^+, \mathfrak{M}^-) = 1 \right]. \quad (2.8)$$

Note that, the non-triviality of $\mathcal{A}$ means that $\mathcal{A}$ is not allowed to ask a verification query (N, M, T) if it has previously made a MAC query (N, M) . Further, for non-deterministic MACs the adversary abides by the following restrictions:

- For stateful MACs, the adversary cannot repeat the nonce value in any MAC query, although it is allowed to repeat nonces in verification queries.
- For probabilistic MACs, the salt value is chosen uniformly at random in all MAC queries. In verification queries the adversary is allowed to pick the salt value freely.

In MAC game, we use the following notations to parametrize adversary's resources: q_m and q_v denote the number of queries to $\mathfrak{M}_K^+$ and $\mathfrak{M}_K^-$, respectively; ℓ_m and ℓ_v denote the maximum permissible message length in tag-generation and verification queries, respectively; σ_m and σ_v denote the maximum permissible aggregate total length across all tag-generation and verification queries, respectively. We also write $q = q_m + q_v$, $\ell = \ell_m + \ell_v$, and $\sigma = \sigma_m + \sigma_v$ to denote the combined query resources. For $\epsilon \in [0, 1]$, $\mathfrak{M}$ is called a $(q, \ell, \sigma, t, \epsilon)$ -MAC if and only if,

$$\mathbf{Adv}_{\mathfrak{M}}^{\text{mac}}(q, \ell, \sigma, t) := \max_{\mathcal{A} \in \mathbb{A}(q, \ell, \sigma, t)} \mathbf{Adv}_{\mathfrak{M}}^{\text{mac}}(\mathcal{A}) \leq \epsilon.$$

2.4.2 Some MAC Design Paradigms

We explain two popular MAC design paradigms which will be frequently used in this thesis. First, we discuss an important symmetric-key primitive called pseudorandom function or PRF [76] which are quite useful in constructing MAC schemes.

2.4.2.1 Pseudorandom Function

A keyed function F is a family of functions $F \in \text{BFunc}(\mathcal{K}, \mathcal{X}, \mathcal{Y})$. In the Pseudorandom Function or PRF security game, the distinguisher $\mathcal{A}$ tries to distinguish a keyed function F instantiated with a key $K \leftarrow \mathfrak{K}$ from a uniform random function $\Gamma \leftarrow \mathfrak{Func}(\mathcal{X}, \mathcal{Y})$. The *PRF advantage* of $\mathcal{A}$ against F is defined as

$$\mathbf{Adv}_F^{\text{prf}}(\mathcal{A}) := \mathbf{Adv}_{F_K; \Gamma}(\mathcal{A}). \quad (2.9)$$

For $\epsilon \in [0, 1]$, F is called a (q, t, ϵ) -PRF if we have

$$\mathbf{Adv}_F^{\text{prf}}(q, t) := \max_{\mathcal{A} \in \mathbb{A}(q, t)} \mathbf{Adv}_F^{\text{prf}}(\mathcal{A}) \leq \epsilon.$$

Throughout the thesis, we use Γ to denote a uniform random function over suitable sets which will be clear from the context.

The following proposition is a well-known result [15, 76, 77] which shows that the PRF security of the tag-generation algorithm of any deterministic MAC scheme is sufficient for MAC security of that scheme. In other words, any PRF construction can be viewed as a secure deterministic MAC scheme. Indeed, most of the provable security results on deterministic MAC schemes show PRF security, which implies MAC security.

Proposition 2.4.1 (PRF $\implies$ MAC). *For any deterministic MAC scheme $\mathfrak{M}$, if $\mathfrak{M}^+$ is a $(q_m, \ell_m, \sigma_m, t, \epsilon)$ -PRF then $\mathfrak{M}$ is $(q, \ell, \sigma, t, \epsilon + q_v/|\mathcal{T}|)$ -MAC.*

We skip the proof of this proposition as it is rather straightforward and available in [15].

2.4.2.2 Hash-then-PRF

For finite sets $\mathcal{K}_1, \mathcal{K}_2, \mathcal{M}, \mathcal{X}, \mathcal{Y} \subset \{0, 1\}^{\eta^*}$, let $H \in \text{BFunc}(\mathcal{K}_1, \mathcal{M}, \mathcal{X})$ be an ϵ -AU hash function, and $F \in \text{BFunc}(\mathcal{K}_2, \mathcal{X}, \mathcal{Y})$ be a keyed function. The composition function $F \circ H$ is known as *Hash-then-PRF* construction. It is probably the easiest and most popular way of constructing a PRF over variable-length inputs. Many PRF constructions, including, EMAC [15, 21], ECBC and FCBC [34], PMAC [35], and LightMAC [126], follow this paradigm. The following result due to Shoup [183] quantifies the PRF security of $F \circ H$.

Proposition 2.4.2 (Hash-then-PRF [183]). *Let H and F be defined as above. Then, we have*

$$\mathbf{Adv}_{F \circ H}^{\text{prf}}(q, \ell, \sigma, t) \leq \mathbf{Adv}_F^{\text{prf}}(q, t') + \binom{q}{2} \epsilon, \quad (2.10)$$

where $t' = t + qO(T_H)$ and T_H denotes the time complexity of computing H .

A proof of this proposition is available in many places, including [183] and [106]. In real-life applications for $\mathcal{Y} = \mathcal{X} = \mathbb{B}$, we often use a PRP E (some popular block cipher $E_{-n/\kappa}$) in place of F . Accordingly, we call the composition “Hash-then-PRP”. The following equation on the PRF security of Hash-then-PRP is a direct consequence of Proposition 2.4.2 and the PRP-PRF switching lemma [14, 47, 183].

Proposition 2.4.3 (Hash-then-PRP). *Let H and E be defined as above. Then, we have*

$$\mathbf{Adv}_{E \circ H}^{\text{prf}}(q, \ell, \sigma, t) \leq \mathbf{Adv}_E^{\text{prp}}(q, t') + \binom{q}{2} \epsilon + \binom{q}{2} \frac{1}{2^n}, \quad (2.11)$$

where $t' = t + qO(T_H)$ and T_H denotes the time complexity of computing H .

Since PRF implies deterministic MAC, Hash-then-PRF and Hash-then-PRP are natural paradigms for constructing deterministic MACs.

2.4.2.3 Hash-then-Mask

For finite sets $\mathcal{K}_1, \mathcal{K}_2, \mathcal{M}, \mathcal{N} \subset \{0, 1\}^*$, let $H \in \text{BFunc}(\mathcal{K}_1, \mathcal{M}, \mathbb{B})$ be an ϵ -AXU hash function, and $F \in \text{BFunc}(\mathcal{K}_2, \mathcal{N}, \mathbb{B})$ be a keyed function. The function $F \oplus H$, defined by the mapping

$$(K, L, N, M) \mapsto F_K(N) \oplus H_L(M)$$

for all $(K, L, N, M) \in \mathcal{K}_1 \times \mathcal{K}_2 \times \mathcal{N} \times \mathcal{M}$, is known as the *Hash-then-Mask* construction. Here $\mathcal{N}$ and $\mathcal{M}$ denote the nonce/salt and message space, respectively. Many non-deterministic MACs, including Wegman-Carter MAC [185], XMACC and XMACR [16], follow this paradigm. Hash-then-Mask was proposed by Wegman and Carter [185], whence it is often referred as the Wegman-Carter authenticator. The following result due to Wegman and Carter [185] quantifies the MAC security of $F \oplus H$.

Proposition 2.4.4 (Wegman-Carter [185]). *Let H and F be defined as above. Then,*

1. *for stateful or nonce-based MACs, we have*

$$\mathbf{Adv}_{F \oplus H}^{\text{mac}}(q, \ell, \sigma, t) \leq \mathbf{Adv}_F^{\text{prf}}(q, t') + q_v \epsilon, \quad (2.12)$$

2. *for probabilistic MACs, we have*

$$\mathbf{Adv}_{F \oplus H}^{\text{mac}}(q, \ell, \sigma, t) \leq \mathbf{Adv}_F^{\text{prf}}(q, t') + \binom{q_m}{2} \frac{1}{2^n} + q_v \epsilon, \quad (2.13)$$

where $t' = t + qO(T_H)$ and T_H denotes the time complexity of computing H .

A proof of this proposition is available in multiple places, including [185], [25] and [49]. Shoup [182] gave a variant of Hash-then-Mask, in which the mask is computed using a PRP $E \in \text{BPerm}(\mathcal{K}_2, \mathcal{N})$ for $\mathcal{N} = \mathbb{B}$, instead of a PRF. We call the resultant scheme “Wegman-Carter-Shoup” authenticator. The poly1305 MAC by Bernstein [24] is directly based on Wegman-Carter-Shoup. The following proposition, due to Bernstein [25], quantifies the MAC security of $E \oplus H$.

Proposition 2.4.5 (Wegman-Carter-Shoup). *Let H and E be defined as above. Then,*

1. *for stateful or nonce-based MACs, we have*

$$\mathbf{Adv}_{E \oplus H}^{\text{mac}}(q, \ell, \sigma, t) \leq \mathbf{Adv}_E^{\text{prp}}(q, t') + q_v \epsilon \left(1 - \frac{q_m}{2^n}\right)^{-\frac{q_m+1}{2}}, \quad (2.14)$$

2. *for probabilistic MACs, we have*

$$\mathbf{Adv}_{E \oplus H}^{\text{mac}}(q, \ell, \sigma, t) \leq \mathbf{Adv}_E^{\text{prp}}(q, t') + q_v \epsilon \left(1 - \frac{q_m}{2^n}\right)^{-\frac{q_m+1}{2}} + \binom{q_m}{2} \frac{1}{2^n}, \quad (2.15)$$

where $t' = t + qO(T_H)$ and T_H denotes the time complexity of computing H .

We remark that Bernstein [25] only proved the first part of Proposition 2.4.5. The second part can be easily derived from the first one by including an additional term to account for the probability of salt collision among any pair of distinct tag-generation queries.

2.5 Online Encryption Schemes

An online encryption scheme or *online cipher*, is a tuple of algorithms $\mathfrak{D} = (\mathfrak{D}^+, \mathfrak{D}^-)$, defined over the key space $\mathcal{K}$ and message space $\mathbb{B}^{\eta^*}$, where:

$$\mathfrak{D}^+ : \mathcal{K} \times \mathbb{B}^{\eta^*} \rightarrow \mathbb{B}^{\eta^*} \quad \mathfrak{D}^- : \mathcal{K} \times \mathbb{B}^{\eta^*} \rightarrow \mathbb{B}^{\eta^*},$$

and $\mathfrak{D}_K^+(\cdot) = \mathfrak{D}^+(K, \cdot)$ is a length-preserving permutation over $\mathbb{B}^{\eta^*}$ satisfying the *online property*: $x \in \mathbb{B}^{\eta^*}$ is a prefix of $y \in \mathbb{B}^{\eta^*}$ if and only if $\mathfrak{D}_K^+(x)$ is a prefix of $\mathfrak{D}_K^+(y)$. $\mathfrak{D}_K^-$ is defined as the inverse of $\mathfrak{D}_K^+$, i.e., for all $x \in \mathbb{B}^{\eta^*}$, $\mathfrak{D}_K^-(\mathfrak{D}_K^+(x)) = x$.

In their foundational paper Bellare et al. [18] proposed CBC like constructions with online property, viz. HCBC1 and HCBC2, both of which employed one call to block cipher and one call to an almost XOR universal (AXU) hash function. HPCBC, proposed by Boldyreva and Taesombut [39], was a variant of HCBC2 that prepends the encryption of a random IV in order to fit a stronger notion. Nandi [145, 146] proposed simpler proofs for HCBC1 and HCBC2, and gave two improved schemes called MHCBC and MCBC. Among these MCBC has the feature of replacing the call to a hash function by a second call to the block cipher.

In [173] Rogaway and Zhang proposed three schemes, namely TC1, TC2, and TC3, based on tweakable block ciphers. These schemes exploited the tweak input of TBCs to eliminate the additional calls to hash function/block cipher. In an independent work

Fleischmann et al. [70] presented a scheme called MCOE-G which is similar to TC3, albeit with a more practical handling of arbitrary length inputs.

For a variant of online property called diblock-online, Bhaumik and Nandi [29] gave an inverse-free construction called OleF, that achieved *diblock-online SPRP* security. Apart from these, several online ciphers were also proposed within full fledged authenticated encryption schemes [1, 3, 4, 40, 56].

Recently, Forler et al. [71] proposed an online cipher POEx, based on a tweakable block cipher and pAXU hash function, that was claimed to have beyond-the-birthday bound³ (BBB) security. Later, this claim was refuted (see chapter 6). Consequently, in a journal version of the same work Forler et al. proposed a modified definition [72] for POEx that provides BBB security under a strong assumption on the underlying hash function.

2.5.1 OSRP Security Definition

We let $\text{Operm}(\mathbb{B}^{\eta^*})$ to be the set of all online permutations over $\mathbb{B}^{\eta^*}$, and $\vec{\Pi} \leftarrow \$ \text{Operm}(\mathbb{B}^{\eta^*})$. In the Online Strong Pseudorandom Permutation or OSRP security game, the distinguisher $\mathcal{A}$ tries to distinguish an online cipher $\mathfrak{D}$ instantiated with a key $K \leftarrow \$ \mathcal{K}$ from an online uniform random permutation $\vec{\Pi}$. The *OSRP advantage* of $\mathcal{A}$ against $\mathfrak{D}$ is defined as

$$\text{Adv}_{\mathfrak{D}}^{\text{osrp}}(\mathcal{A}) := \text{Adv}_{\mathfrak{D}; \vec{\Pi}}(\mathcal{A}). \quad (2.16)$$

For $\epsilon \in [0, 1]$, $\mathfrak{D}$ is called a $(q, \ell, \sigma, t, \epsilon)$ -OSRP if we have

$$\text{Adv}_{\mathfrak{D}}^{\text{osrp}}(q, \ell, \sigma, t) := \max_{\mathcal{A} \in \mathbb{A}(q, \ell, \sigma, t)} \text{Adv}_{\mathfrak{D}}^{\text{osrp}}(\mathcal{A}) \leq \epsilon.$$

Throughout the thesis, we use $\vec{\Pi}$ to denote a uniform online random permutation over some suitable set(s) which will be clear from the context.

2.6 Authenticated Encryption with Associated Data

An *authenticated encryption scheme with associated data* functionality, or AEAD in short, is a tuple of algorithms $\mathfrak{A} = (\mathfrak{A}^+, \mathfrak{A}^-)$, defined over the *key space* $\mathcal{K}$, *nonce space* $\mathcal{N}$, *associated data space* $\mathcal{A}$, *message space* $\mathcal{M}$, *ciphertext space* $\mathcal{C}$, and *tag space* $\mathcal{T}$, where:

$$\mathfrak{A}^+ : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M} \rightarrow \mathcal{C} \times \mathcal{T} \quad \text{and} \quad \mathfrak{A}^- : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{C} \times \mathcal{T} \rightarrow \mathcal{M} \cup \{\perp\}.$$

³Secure up to $2^{x\eta}$ queries for some $x \in (0.5, 1]$.

Here, $\mathfrak{A}^+$ and $\mathfrak{A}^-$ are called the encryption and decryption algorithms, respectively, of $\mathfrak{A}$, and $\perp$ denotes the error symbol used to indicate authentication failure. Further, it is required that $\mathfrak{A}^-(K, N, A, \mathfrak{A}^+(K, N, A, M)) = M$ for any $(K, N, A, M) \in \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M}$. For all key $K \in \mathcal{K}$, we write $\mathfrak{A}_K^+(\cdot)$ and $\mathfrak{A}_K^-(\cdot)$ to denote $\mathfrak{A}^+(K, \cdot)$ and $\mathfrak{A}^-(K, \cdot)$, respectively.

In this thesis, we have $\mathcal{K}, \mathcal{T} \subseteq \mathbb{B}^{\eta^+}$, $\mathcal{N}, \mathcal{A}, \mathcal{M} \in \{0, 1\}^{\eta^*}$ and $\mathcal{C} = \mathcal{M}$, so we use $\mathcal{M}$ instead of $\mathcal{C}$ wherever necessary. In addition, in most popular authenticated encryption schemes (including the OCB family), the map $\text{proj}_{\mathcal{M}} \circ \text{Enc}(K, N, A, \cdot)$ for fixed K, N , and A is a length-preserving permutation, where $\text{proj}_{\mathcal{M}} : \mathcal{M} \times \mathcal{T} \rightarrow \mathcal{M}$ is the projection on $\mathcal{M}$, and $\mathcal{T} = \{0, 1\}^{kn}$ for some fixed $k \in \mathbb{N}$.

AEAD schemes can be classified based on the cardinality of $\mathcal{N}$, much in the same vein as MAC schemes. There are some good examples of deterministic AEAD ($\mathcal{N} = \emptyset$) schemes based on the SIV paradigm by Rogaway and Shrimpton [172]. GCM-SIV [84], GCM-SIV2 and its generalized variant GCM-SIVr [99], and the lightweight scheme SUNDIAE [9] are some of the notable SIV-based deterministic AEAD schemes. A major issue with SIV-based constructions is efficiency. These schemes are inherently two-pass, i.e., the message input is read twice, which makes them slow (and somewhat less popular) as compared to single-pass nonce-based ($\mathcal{N} \neq \emptyset$) AEAD schemes. Some of the popular nonce-based AEAD schemes include CCM [186], GCM [130], OCB3 [119], COLM [5], Ascon [61], Beetle [44] etc.

In this thesis, we will concentrate on nonce-based AEAD. Particularly, we focus on the provable security of OCB family [119, 171, 174, 174].

2.6.1 NAEAD Security Definition

In the nonce-respecting AEAD or NAEAD security game, the adversary $\mathcal{A}$ tries to distinguish between $(\mathfrak{A}_K^+, \mathfrak{A}_K^-)$ and $(\Gamma, \perp)$, where

$$\Gamma \leftarrow \$ \{f \in \text{Func}[\mathcal{N} \times \mathcal{A} \times \mathcal{M}, \mathcal{M} \times \mathcal{T}] : \forall (N, A, M) \in \mathcal{N} \times \mathcal{A} \times \mathcal{M}, |f(N, A, M)| = |M| + kn\},$$

and $\perp : \mathcal{N} \times \mathcal{A} \times \mathcal{M} \times \mathcal{T} \rightarrow \{\perp\}$, respectively. The distinguishing adversary operates under the restriction — no two encryption queries can have the same nonce. The *NAEAD advantage* of any adversary $\mathcal{A}$ against $\mathfrak{A}$ is defined as

$$\text{Adv}_{\mathfrak{A}}^{\text{naead}}(\mathcal{A}) := \text{Adv}_{\mathfrak{A}^{\pm}; (\Gamma, \perp)}(\mathcal{A}). \quad (2.17)$$

In NAEAD game, we use the following notations to parametrize adversary's resources: q_e and q_d denote the number of queries to $\mathfrak{A}_K^+$ and $\mathfrak{A}_K^-$, respectively; ℓ_e and ℓ_d denote

the maximum permissible input (associated data and message/ciphertext) lengths in encryption and decryption queries, respectively; σ_e and σ_d denote the sum of input lengths across all encryption and decryption queries, respectively. We also write $q = q_e + q_d$, $\ell = \ell_e + \ell_d$ and $\sigma = \sigma_e + \sigma_d$ to denote the combined query resources. For $\epsilon \in [0, 1]$, $\mathfrak{A}$ is called a $(q, \ell, \sigma, t, \epsilon)$ -NAEAD if and only if,

$$\mathbf{Adv}_{\mathfrak{A}}^{\text{naead}}(q, \ell, \sigma, t) := \max_{\mathcal{A} \in \mathbb{A}(q, \ell, \sigma, t)} \mathbf{Adv}_{\mathfrak{A}}^{\text{naead}}(\mathcal{A}) \leq \epsilon.$$

Note that, security under this formulation covers the two standard security goals of authenticated encryption:

1. *Privacy*: Security against an adversary who tries to distinguish the AE or AEAD construction from an ideal random function Γ using encryption queries, and
2. *Integrity*: Security against an adversary who tries to make a successful forging attempt on the scheme using both encryption and decryption queries.

Part I

Analysis of Message Authentication Codes

Chapter 3

Pseudorandom Permutation based CBC-MAC Family

This chapter studies the security of one of the most popular family of message authentication codes, the CBC-MAC family. CBC-MAC is a block cipher based MAC construction which is based on the CBC mode of operation [153] invented by Ehrtman et al. [65]. The CBC-MAC was an international standard [95], which was proven to be secure for fixed length messages in [15, 27], and for prefix-free¹ messages in [80, 162]. The fixed length constraint is not desired in practice. One way to circumvent this is to use the length of message as the first block in CBC-MAC computation. But this requires prior knowledge of the message length. A more reasonable and popular approach is to encrypt the CBC-MAC output with an independent keyed permutation. This later approach called EMAC was first proposed during the RACE project [21]. Petrank and Rackoff [162] proved that EMAC is secure for all messages in its message space.

Although the EMAC construction is tolerant to variable length messages it has a domain limited to $\mathbb{B}^{\eta^+}$. Later works by Black and Rogaway [34], and Iwata et al. [97, 121] propose variants of CBC-MAC that theoretically² work for any messages in $\{0, 1\}^{\eta^*}$. We refer the readers to section 1.3.1 of chapter 1 for a brief overview on the historical progress in the design and analysis of CBC-MAC and its family. In this chapter, we will mainly concentrate on CBC-MAC and EMAC. However, we present some interesting implications for ECBC and FCBC towards the end of the chapter.

¹No two messages are prefix of each other.

²There could be some limitations on the length of messages due to security bounds.

3.1 PRF Analysis of CBC-MAC and EMAC

Recall that proposition 2.4.1 shows that PRF security of deterministic MACs is sufficient for MAC security. So, we concentrate on the PRF security of CBC-MAC and EMAC.

3.1.1 Permutation-based CBC-MAC and EMAC

In this chapter, we always consider the set $\text{Perm}(\mathbb{B})$, and hence skip the parametrization in Perm . The CBC (cipher block chaining) function (see Figure 3.1.1) with a permutation $\pi \in \text{Perm}$, viewed as a key of the construction, takes as input a message $M \in \mathbb{B}^m$ with m blocks and outputs $\text{CBC}_\pi(M) := \mathbf{v}_m^\pi$. This is inductively computed as follows: $\mathbf{v}_0^\pi(M) = 0^n$ and for all $i \in [m]$, we have

$$\begin{aligned} \mathbf{u}_i^\pi(M) &:= \mathbf{v}_{i-1}^\pi(M) \oplus M_i, \\ \mathbf{v}_i^\pi(M) &:= \pi(\mathbf{u}_i^\pi(M)). \end{aligned} \quad (3.1)$$

We call $\mathbf{u}^\pi(M) := (\mathbf{u}_i^\pi(M))_{i \in [m]}$ and $\mathbf{v}^\pi(M) := (\mathbf{v}_i^\pi(M))_{i \in [m]}$, the *intermediate input* and *intermediate output* vectors, respectively, associated to π and M . Note that, the intermediate input vector $\mathbf{u}^\pi(M)$ is uniquely determined by $\mathbf{v}^\pi(M)$ (and does not depend on the permutation π). We can write down this association generically as a function $\text{out2in}_M : \mathbb{B}^{m+1} \rightarrow \mathbb{B}^m$ mapping any block vector y to another block vector x where $x_i = y_{i-1} \oplus M_i$ for all $i \in [m]$. So for all permutations $\pi \in \text{Perm}$, we have $\text{out2in}_M(\mathbf{v}^\pi(M)) = \mathbf{u}^\pi(M)$.

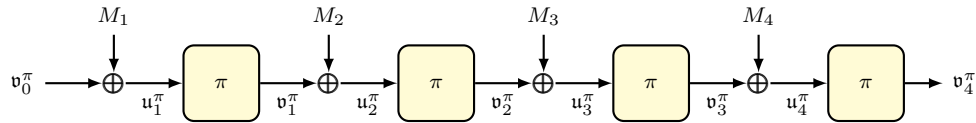


Figure 3.1.1: Evaluation of CBC function over a 4-block message M . Note that, we skipped M from the notations for intermediate input and output vectors for economical reasons.

Given the definition of CBC_π , one can easily define CBC-MAC and EMAC as follows:

1. CBC-MAC, based on a block cipher E instantiated with a key $K \in \mathcal{K}$, is defined as, $\text{CBC-MAC}_{E_K}(M) := \text{CBC}_{E_K}(M)$ for all $M \in \mathbb{B}^{\eta^*}$.
2. EMAC, based on a block cipher E instantiated twice with keys $K_1, K_2 \in \mathcal{K}$, is defined as, $\text{EMAC}_{E_{K_1}, E_{K_2}}(M) := E_{K_2}(\text{CBC}_{E_{K_1}}(M))$ for all $M \in \mathbb{B}^{\eta^*}$.

The first step in the analysis reduces the PRF analysis of CBC-MAC_E and $\text{EMAC}_{E,E}$ to the PRF analysis of CBC-MAC_Π and $\text{EMAC}_{\Pi_1, \Pi_2}$, respectively, where Π, Π_1 and Π_2 are

uniform random permutations over $\mathbb{B}$ and Π_1 is statistically independent of Π_2 . This step uses a standard hybrid argument that incurs additional cost of at most $\text{Adv}_E^{\text{prp}}(\sigma, t)$ and $2\text{Adv}_E^{\text{prp}}(\sigma, t)$ for CBC-MAC and EMAC, respectively. Formally, we have

$$\text{Adv}_{\text{CBC-MAC}_E}^{\text{prf}}(q, \ell, \sigma, t) \leq \text{Adv}_E^{\text{prp}}(\sigma, t) + \text{Adv}_{\text{CBC-MAC}_\Pi}^{\text{prf}}(q, \ell, \sigma) \quad (3.2)$$

$$\text{Adv}_{\text{EMAC}_{E,E}}^{\text{prf}}(q, \ell, \sigma, t) \leq 2\text{Adv}_E^{\text{prp}}(\sigma, t) + \text{Adv}_{\text{EMAC}_{\Pi_1, \Pi_2}}^{\text{prf}}(q, \ell, \sigma) \quad (3.3)$$

3.1.2 PRF Analysis of $\text{EMAC}_{\Pi_1, \Pi_2}$

Let M and M' be two distinct messages having m and m' many blocks, respectively, and $\pi \in \text{Perm}$. Let $0\text{coll}^\pi(M, M')$ denote the event $\text{CBC}_\pi(M) = \mathbf{v}_m^\pi(M) = \mathbf{v}_{m'}^\pi(M') = \text{CBC}_\pi(M')$. We call $0\text{coll}^\pi(M, M')$ the *output collision event* for a pair of messages M and M' . One can similarly define the *input collision event* $1\text{coll}^\pi(M, M')$ as the event $\mathbf{u}_m^\pi(M) = \mathbf{u}_{m'}^\pi(M')$. It is clear to see that $1\text{coll}^\pi(M, M')$ is equivalent to $0\text{coll}^\pi(M, M')$, as $\mathbf{u}_m^\pi(M) = \mathbf{u}_{m'}^\pi(M')$ if and only if $\mathbf{v}_m^\pi(M) = \mathbf{v}_{m'}^\pi(M')$. So henceforth we mainly focus on the event $0\text{coll}^\pi(M, M')$, which is also referred as collision event.

By extending the notation, we similarly define the collision event for a tuple of $q \geq 2$ distinct messages $M^q = (M_1, \dots, M_q)$, as

$$0\text{coll}^\pi(M^q) = \bigcup_{i \neq j \in [q]} 0\text{coll}^\pi(M_i, M_j). \quad (3.4)$$

We define *collision probability* as $\text{outCP}(M^q) = \Pr [0\text{coll}^{\Pi_1}(M^q)]$.

Let $\text{outCP}_{q, \ell, \sigma}^{\text{atk}} = \max_{M^q} \text{outCP}(M^q)$ where the maximum is taken over all q -tuples of distinct messages M^q having at most ℓ blocks each, and the total length over all q queries is at most σ . Further, the message tuple satisfies the input constraint atk , which could be one of the following:

1. eq, messages must have equal length.
2. pf, no message is a prefix to others.
3. any, messages can be chosen arbitrarily within the specified length.

If no constraint is mentioned, assume any for collision events. Looking ahead momentarily, we will use pf for full collision event (defined in the next subsection), and hence in case of full collision we can always assume pf constraint.

Following [19, 34], we view EMAC as an instance of Hash-then-PRP paradigm [183] (see chapter 2 section 2.4.2.2). This enables us to reduce the problem of bounding the

PRF advantage of EMAC to bounding the collision probability. The following Eq. (3.5) is a direct application of proposition 2.4.3.

$$\text{Adv}_{\text{EMAC}_{\Pi_1, \Pi_2}}^{\text{prf}}(q, \ell, \sigma) \leq \text{outCP}_{q, \ell, \sigma}^{\text{any}} + \frac{q(q-1)}{2^{n+1}}. \quad (3.5)$$

Note that, $\text{outCP}_{q, \ell, \sigma}^{\text{any}} \leq \binom{q}{2} \text{outCP}_{2, \ell}^{\text{any}}$ as the collision for q messages is the union of collision events for each of the $\binom{q}{2}$ pairs of messages. Bellare et al. [19] proved that

$$\text{outCP}_{2, \ell}^{\text{any}} \leq \frac{2d'(\ell)}{2^n} + \frac{64\ell^4}{2^{2n}}. \quad (3.6)$$

where $d'(\ell) = \max_{\ell' \leq \ell} d(\ell')$ and $d(\ell')$ is the the number of divisors of ℓ' . In [187], Wigert showed that $d'(\ell) = \ell^{1/\Theta(\ln \ln \ell)} = \ell^{o(1)}$. Using this bound of collision probability for a pair of messages, we see that the PRF advantage of EMAC is about $O(d'(\ell)q^2/2^n)$ for $\ell < 2^{n/4}$. Later Pietrzak [163] provided an improved analysis of EMAC and proved that the PRF advantage of EMAC is about $O(q^2/2^n)$ for $\ell < \min\{q^{1/2}, 2^{n/8}\}$. We revisit this improved analysis later in section 3.5.

We remark that for equal length messages Dodis et al. claim that $\text{outCP}_{2, \ell}^{\text{eq}} = 2^{-n} + (d(\ell))^2 \cdot \ell \cdot 2^{-2n} + \ell^6 \cdot 2^{-3n}$ (see [62, Lemma 3]), which is tight (if true).

3.1.3 PRF Analysis of CBC-MAC $_{\Pi}$

Let M and M' be two distinct messages having m and m' many blocks, respectively, and $\pi \in \text{Perm}$. Let $\text{Fcoll}^{\pi}(M, M')$ denote the event that

$$u_{m'}^{\pi}(M') \in u^{\pi}(M) \cup \{u_j^{\pi}(M') : j \in [m' - 1]\}.$$

We call this the *full collision event* for a pair of messages M and M' . In other words, if the event $\text{Fcoll}^{\pi}(M, M') \cup \text{Fcoll}^{\pi}(M', M)$ does not hold then $u_m^{\pi}(M)$ and $u_{m'}^{\pi}(M')$ are “fresh” (did not appear before). Intuitively, for a uniform random permutation Π if this event does not hold then the CBC output should behave “almost” randomly. This intuition is one of the crucial steps in the PRF analysis of CBC-MAC.

Remark 3.1.1. We remark that in the original paper [19], the full collision event is defined through the intermediate outputs instead of inputs. Since we only consider permutation based CBC-MAC, equalities among inputs imply identical equalities among outputs.

By extending the notation, we define the full collision event for a tuple of $q \geq 2$ distinct messages $M^q = (M_1, \dots, M_q)$ as

$$\text{Fcoll}^\pi(M^q) = \bigcup_{i \neq j \in [q]} \text{Fcoll}^\pi(M_i, M_j).$$

We define *full collision probability* as $\text{fullCP}(M^q) = \Pr [\text{Fcoll}^\Pi(M^q)]$.

Analogous to $\text{outCP}_{q,\ell,\sigma}^{\text{atk}}$, we have $\text{fullCP}_{q,\ell,\sigma}^{\text{atk}} = \max_{M^q} \text{fullCP}(M^q)$. Note that, $\text{fullCP}_{q,\ell,\sigma}^{\text{any}} = 1$ as one can always choose two messages M and M' such that M is a prefix of M' , whence the full collision holds trivially. So it only makes sense to study fullCP for input constraints, eq and pf. This is fine as CBC-MAC is trivially insecure when the adversary can make prefix queries (using the classical length extension attack). In [19], Bellare et al. proved the following result

$$\text{Adv}_{\text{CBC-MAC}_\Pi}^{\text{prf}}(q, \ell, \sigma) \leq q^2(\text{fullCP}_{2,\ell}^{\text{pf}} + 4\ell/2^n). \quad (3.7)$$

We state and prove another relation between PRF advantage of CBC-MAC and full collision probability among q prefix-free messages. The above Eq. (3.7) would be a straightforward application of the following result.

Proposition 3.1.2. *For prefix-free queries, we have*

$$\text{Adv}_{\text{CBC-MAC}_\Pi}^{\text{prf}}(q, \ell, \sigma) \leq \text{fullCP}_{q,\ell,\sigma}^{\text{pf}} + \frac{2\sigma q}{2^n} + \frac{q^2}{2^{n+1}}.$$

Proof. We use the coefficient-H technique to prove this proposition. Let Ω denote the set of all transcripts realizable via a uniform random function Γ . A typical transcript $\omega \in \Omega$ is of the form (M^q, T^q) , where $M^q \in (\mathbb{B}^{\eta^+})_q$ is prefix-free and $T^q \in \mathbb{B}^q$.

We say that a transcript $\omega \in \Omega$ is bad, denoted $\omega \in \Omega_{\text{bad}}$, if $T^q \neq \widehat{T}^q$, i.e., there exist $i \neq j \in [q]$ such that $T_i = T_j$. We have

$$\Pr [\Theta_0 \in \Omega_{\text{bad}}] \leq \binom{q}{2} \frac{1}{2^n},$$

as there are at most $\binom{q}{2}$ pairs (i, j) and for each such pair the equality $T_i = T_j$ holds with at most 2^{-n} probability.

Now, fix a transcript $\omega = (M^q, T^q) \in \Omega \setminus \Omega_{\text{bad}}$. Then, we must have $T^q \in (\mathbb{B})_q$. For $i \in [q]$, let m_i denote the length of M_i . Then $m_i \leq \ell$ and $\sum_{j \in [q]} m_j \leq \sigma$. In the ideal world, we have

$$\Pr [\Theta_0 = \omega] = \Pr [\Gamma(M^q) = T^q] = \frac{1}{2^{nq}}. \quad (3.8)$$

In the real world, a permutation $\pi \in \text{Perm}$ is called bad if and only if

1. $\text{Fcoll}^\pi(M^q)$ holds, or
2. $\mathbf{v}_i^\pi(M_r) = T_{r'}, \text{ for some } r, r' \in [q], i \in [m_r]$.

Let $\text{badPerm} = \{\pi \in \text{Perm} : \pi \text{ is bad}\}$. All other permutations $\pi \notin \text{badPerm}$ are called good. We define an equivalence relation $\sim$ on Perm as $\pi \sim \pi'$ if $\mathbf{u}^\pi(M_i) = \mathbf{u}^{\pi'}(M_i)$ for all $i \in [q]$. It is clearly an equivalence relation and a good permutation can only be related with another good permutation and good permutations are unrelated with bad permutations. An equivalence class consisting of good permutations is referred as good equivalence class. Let $\mathcal{C}$ be a good equivalence class. Let s denote the cardinality of $\cup_{r \in [q]} \widehat{\mathbf{u}}^\pi(M_r)$, where $\pi \in \mathcal{C}$. Recall that $\widehat{\mathbf{u}}^\pi(M_r)$ denotes the set of distinct elements in $\mathbf{u}^\pi(M_r)$. Note that, s is the same for all $\pi \in \mathcal{C}$. Then, $|\mathcal{C}| = (2^n - s)!$, as the outputs of exactly s inputs of $\pi \in \mathcal{C}$ are determined due to CBC computation on M^q . Since the T_i 's are not intermediate outputs, we have

$$|\{\pi \in \mathcal{C} : \text{CBC}_\pi(M^q) = T^q\}| = (2^n - s - q)!,$$

since q additional restrictions on input-output are being added. So for any class of good permutations $\mathcal{C}$,

$$\Pr[\text{CBC}_\Pi(M^q) = T^q \mid \Pi \in \mathcal{C}] = \frac{(2^n - s - q)!}{(2^n - s)!} \geq 2^{-nq}. \quad (3.9)$$

Thus,

$$\begin{aligned} \Pr[\Theta_1 = \omega] &= \Pr[\text{CBC}_\Pi(M^q) = T^q] \\ &\stackrel{1}{\geq} \sum_{\mathcal{C} \text{ is good}} \Pr[\Pi \in \mathcal{C}] \times \Pr[\text{CBC}_\Pi(M^q) = T^q \mid \Pi \in \mathcal{C}] \\ &\stackrel{2}{\geq} \Pr[\Pi \in \text{Perm} \setminus \text{badPerm}] \times 2^{-nq} \\ &\stackrel{3}{\geq} \left(1 - \Pr[\Pi \in \text{badPerm}]\right) \times \Pr[\Theta_0 = \omega] \\ &\stackrel{4}{\geq} \left(1 - \text{fullCP}_{q,\ell,\sigma}^{\text{pf}} - \frac{\sigma q}{2^{n-1}}\right) \times \Pr[\Theta_0 = \omega]. \end{aligned}$$

where 1 to 2 follows from Eq. (3.9); 2 to 3 follows from Eq. (3.8); and 3 to 4 follows from the two conditions for $\pi \in \text{badPerm}$. The first condition leads to the term $\text{fullCP}_{q,\ell,\sigma}^\Pi$. The second condition says that we sample at most $\sigma - q$ outputs of a random permutation and one of them belongs to the set $\widehat{T}^q$. This can happen with probability at most $\sigma q / (2^n - \sigma + q)$ which is further less than $\sigma q / 2^{n-1}$ provided $\sigma < 2^{n-1}$. If $\sigma \geq 2^{n-1}$, then the above bound holds trivially.

The result follows from coefficient-H technique (Corollary 2.2.2). $\square$

The bound in Proposition 3.1.2 is potentially a better bound than the original in Eq. (3.7) as it uses the total number of blocks σ , which could be much less than ℓq in some scenarios.³

Note that, $\text{fullCP}_{q,\ell}^{\text{pf}} \leq q(q-1)\text{fullCP}_{2,\ell}^{\text{pf}}$ by considering all ordered pairs (M_i, M_j) . This proves the original claim from [19] as stated in Eq. (3.7). In [19], it is proved that

$$\text{fullCP}_{2,\ell}^{\text{pf}} \leq \frac{8\ell}{2^n} + \frac{64\ell^4}{2^{2n}}. \quad (3.10)$$

In section 3.2, we show a critical flaw in [19, Lemma 10]. This invalidates the proofs of Eq. (3.6) and Eq. (3.10) given in [19]. Further, it also invalidates the security analysis of EMAC in [163]. Consequently, we revise the bounds for $\text{outCP}_{q,\ell,\sigma}^{\text{any}}$ and $\text{fullCP}_{q,\ell,\sigma}^{\text{pf}}$ in section 3.4. Moreover, our revised bound of $\text{fullCP}_{q,\ell,\sigma}^{\text{pf}}$ would be in the order $\sigma q/2^n$ instead of $q^2\ell/2^n$ (whenever $\ell \leq 2^{n/3}$). We also revise the analysis of [163] and obtain a degraded bound in section 3.5. In the same section, we provide a simplified security analysis for EMAC that achieves significantly improved bounds for message lengths less than $2^{n/4}$.

3.2 Revisiting Structure Graph

In the previous section, we have seen how the PRF advantage of CBC-MAC or EMAC is essentially reduced to the bound on probability of some collision events on internal inputs or outputs of the underlying permutation. Thus, it would be useful to have an object which deals with the intermediate inputs and outputs. Bellare et al. [19] introduced one such object, called structure graph, and used it to bound the (full) collision probabilities in [19]. In this section we revisit the structure graph formalism and show that one of the main claims, namely [19, Lemma 10] about structure graphs is false.

Let us fix a tuple of messages M^q for the rest of this chapter, where $M_i \in \mathbb{B}^{m_i}$ and $m_i \leq \ell$ for all $i \in [q]$, and $\sum_{i \in [q]} m_i = m \leq \sigma$. In addition, we may put additional constraint on these messages, like prefix-free property.

3.2.1 Intermediate Inputs and Outputs

INDEX SET: We first collect all intermediate inputs and outputs which are obtained through the computation of $\text{CBC}_\pi(M_r)$ for all r . These intermediate values will be

³Consider a scenario where most of the messages are of few kilobytes, but the maximum permissible message length is in Gigabytes.

defined as a sequence over a two-dimensional index set. Each index is a pair where the first element of the pair corresponds to the message number and the second element is the block number of that message. More formally, we define the *index set*

$$\mathcal{I} = \{(r, i) : r \in [q], i \in [m_r]\},$$

and the *dictionary order* $\prec$ on it as follows: $(r, i) \prec (r, i)$, and $(r, i) \prec (r', i')$ if $r < r'$ or $r = r'$ and $i < i'$. Let x be a sequence over this index set. For any $r \in [q]$, the subsequence $(x_{(r,1)}, \dots, x_{(r,m_r)})$ is denoted by x_{r*} . Sometimes we also consider the index set $\mathcal{I}_0 = \mathcal{I} \cup \{(r, 0) : r \in [q]\}$, and the natural extension of the order $\prec$ on $\mathcal{I}_0$.

SEQUENCES FOR INTERMEDIATE INPUTS AND OUTPUTS: We denote the *sequences of intermediate outputs* and *inputs* over the index set $\mathcal{I}_0$ and $\mathcal{I}$ as $\mathbf{v}^\pi(M^q)$ and $\mathbf{u}^\pi(M^q)$, respectively, where $\forall r \in [q]$, $\mathbf{v}_{r*}^\pi(M^q) := \mathbf{v}^\pi(M_r)$, and $\mathbf{u}_{r*}^\pi(M^q) := \mathbf{u}^\pi(M_r)$.

For a single message M , we have previously seen that the intermediate input sequence is uniquely determined by the intermediate output sequence, and we denoted the association by a function out2in_M . The same is true for q messages and we extend this definition as follows: Given any block sequence y over the index set $\mathcal{I}_0$, we define $\text{out2in}(y)$ as a block sequence x over the index set $\mathcal{I}$, where $x_{r*} = \text{out2in}_{M_r}(y_{r*})$ for all $r \in [q]$. Thus, for any π , we have $\text{out2in}_{M^q}(\mathbf{v}^\pi(M^q)) = \mathbf{u}^\pi(M^q)$. Onwards, we selectively skip the parametrization of $\mathbf{v}$ and $\mathbf{u}$ whenever the parameters are clear from the context.

3.2.2 Structure Graphs and Block-Vertex Structure Graphs

A block-vertex structure graph is a graph theoretic representation of intermediate output sequence $\mathbf{v}^\pi$. The *block-vertex structure graph* $\text{BStruct}^\pi = (\mathcal{V}^\pi, \mathcal{E}^\pi)$ for a permutation π is defined by the labeled vertex set $\mathcal{V}^\pi := \widehat{\mathbf{v}}^\pi$, and the set of labeled edges

$$\mathcal{E}^\pi := \bigcup_{r=1}^q \{(\mathbf{v}_{(r,i-1)}^\pi, \mathbf{v}_{(r,i)}^\pi, M_{(r,i)}) : i \in [m_r]\}.$$

Clearly, BStruct^π is a union of M_r -walks (see section A.1.2 of appendix A) for all $r \in [q]$, and vertex $0^n \in \mathcal{V}$ has positive out-degree. Note that,

$$u \xrightarrow{A} v \Rightarrow \pi(u \oplus A) = v. \quad (3.11)$$

So, for every $v \in \mathcal{V}$, all outward edges (similarly for inward edges) have distinct edge labels. Using this property, it is easy to see that *the walks are unique* and we denote them by W_{M_i} or simply W_i whenever the message tuple is understood. See Figure 3.2.1 for a possible block structure graph corresponding to a single message. It is clear to see that

block structure graph is just another view in which the input and output vectors are stored in a directed graph. While storing the intermediate sequences as a set of labeled

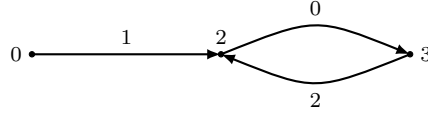


Figure 3.2.1: Let $M_1 = (1, 0, 2, 0, 2)$ and $\pi(1) = 2, \pi(2) = 3$. For any such π , we have $\mathbf{v}^\pi = (0, 2, 3, 2, 3, 2)$ and $\mathbf{u}^\pi = (1, 2, 1, 2, 1)$. However, the graph consists of just three vertices $\mathcal{V}^\pi = \{0, 2, 3\}$ and edge set $\mathcal{E}^\pi = \{(0, 2, 1), (2, 3, 0), (3, 2, 2)\}$. We see that the intermediate input and output sequences actually can be reconstructed from this labeled structure graph. The walk corresponding to the message M_1 will uniquely identify the output vector as $\mathbf{v}^\pi = (0, 2, 3, 2, 3, 2)$ and the input vector $\mathbf{u}^\pi = (1, 2, 1, 2, 1)$ can be constructed using the relation between input, output and message.

edges, we may loose the order as well as the repetition of the elements. Interestingly, we see that we can uniquely reconstruct the intermediate sequences from such an edge-labeled graph by using uniqueness of M_i -walk. More precisely, $\mathbf{v}_{(r,i)}^\pi = W_{(r,i)}$, where $W_{(r,i)}$ denotes the i -th vertex on the walk W_r .

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a labeled directed graph and $f : \mathcal{V} \rightarrow \mathcal{V}'$ be an injective function. Let $\tilde{\mathcal{V}} = \{v' \in \mathcal{V}' : \exists v \in \mathcal{V}, f(v) = v'\}$. Then, f is a bijection from $\mathcal{V}$ to $\tilde{\mathcal{V}}$. One can define a labeled directed graph $\tilde{\mathcal{G}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}})$ isomorphic to $\mathcal{G}$ for which f is an isomorphism. More precisely, $(u, v, a) \in \mathcal{E}$ if and only if $(f(u), f(v), a) \in \tilde{\mathcal{E}}$. We call the graph $\tilde{\mathcal{G}}$ so obtained, the f -transformed $\mathcal{G}$, denoted $\tilde{\mathcal{G}} = \alpha(\mathcal{G})$.

For a block-vertex structure graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, we define the *minimum index mapping* $\alpha : \mathcal{V} \rightarrow \mathcal{I}_0$ as $\alpha_v = \alpha(v) \mapsto (r, i)$ for all $v \in \mathcal{V}$, where (r, i) is the minimum index such that $W_{(r,i)} = v$. Clearly, α is an injective mapping, with some range set $\tilde{\mathcal{V}}$.

Definition 3.2.1 (Structure Graph). The structure graph $\text{Struct}^\pi = (\tilde{\mathcal{V}}^\pi, \tilde{\mathcal{E}}^\pi)$ associated to π is the α -transformed BStruct^π , i.e. $\text{Struct}^\pi = \alpha(\text{BStruct}^\pi)$.

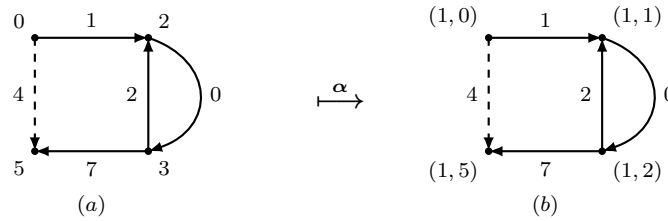


Figure 3.2.2: Structure graph corresponding to a block-vertex structure graph.

Example 3.1. Let $M_1 = (1, 0, 2, 0, 7)$ and $M_2 = (4)$ be two messages and $\pi(1) = 2; \pi(2) = 3; \pi(4) = 5$ for some $\pi \in \text{Perm}$. Then, we have $\mathbf{v}^\pi(M_1) = (0, 2, 3, 2, 3, 5)$ and $\mathbf{v}^\pi(M_2) = (0, 5)$. The corresponding block-vertex structure graph BStruct^π , shown in Figure 3.2.2(a), has vertex set $\mathcal{V}^\pi = \{0, 2, 3, 5\}$ and edge set

$$\mathcal{E}^\pi = \{(0, 2, 1), (0, 5, 4), (2, 3, 0), (3, 2, 2), (3, 5, 7)\}.$$

The corresponding structure graph Struct^π , illustrated in Figure 3.2.2(b), has vertex set $\tilde{\mathcal{V}}^\pi = \{(1, 0), (1, 1), (1, 2), (1, 5)\}$ and edges set

$$\tilde{\mathcal{E}}^\pi = \{((1, 0), (1, 1), 1), ((1, 1), (1, 2), 0), ((1, 2), (1, 1), 2), ((1, 2), (1, 5), 7), ((1, 0), (1, 5), 4)\}.$$

Let $\tilde{W}_r$ denote the M_r -walk in a structure graph $\tilde{\mathcal{G}}$ (corresponding to some block-vertex structure graph $\mathcal{G}$). It is easy to see that a structure graph is again a union of M_r -walks $\tilde{W}_r$ starting from $(1, 0)$. Note that, as per the convention used here and in the preceding discussion $\tilde{W}_r = \alpha(W_r)$, where W_r is the corresponding M_r -walk in $\mathcal{G}$. A structure graph is called a *zero-output graph* if $(1, 0)$ has positive in-degree, otherwise we call it *non-zero output graph*. To express it mathematically, we define a binary function lszero such that for each zero-output graph $\tilde{\mathcal{G}}$, $\text{lszero}(\tilde{\mathcal{G}}) = 1$, otherwise it maps to 0.

To reconstruct a block-vertex structure graph realizing $\tilde{\mathcal{G}}$ we have to find labels from $\mathbb{B}$ for all the vertices in a “consistent manner” and we call such a labeling *valid*. The mapping must be injective as distinct vertices in a block vertex structure graph map to distinct vertices in the corresponding structure graph.

Definition 3.2.2 (valid block labeling). An injective function $\beta : \tilde{\mathcal{V}} \rightarrow \mathbb{B}$ is called a *valid block labeling* for a structure graph $\tilde{\mathcal{G}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}})$ if the graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a block-vertex structure graph where

1. $\mathcal{V} := \beta(\tilde{\mathcal{V}}) = \{\beta_v := \beta(v) : v \in \tilde{\mathcal{V}}\}$, and
2. $\mathcal{E}$ is the edge set after relabeling v by β_v , i.e. $\mathcal{E} = \{(\beta_u, \beta_v, x) : (u, v, x) \in \tilde{\mathcal{E}}\}$.

NECESSARY CONDITIONS FOR A VALID BLOCK LABELING: Not all injective functions from $\tilde{\mathcal{V}}$ to $\mathbb{B}$ are valid block labeling. An obvious class of counter-example functions are those which map $(1, 0) \in \tilde{\mathcal{V}}$ to a non-zero value in $\mathbb{B}$. We now enumerate other necessary conditions for valid block labeling. First of all, by definition, β_i should all be distinct as the valid block label is injective. In addition to this, whenever $e_1 := (u, w, a), e_2 := (v, w, b) \in \tilde{\mathcal{E}}$ we must have $\beta_u \oplus a = \beta_v \oplus b$ as these are equal to $\pi^{-1}(z)$ for some $\pi \in \text{Perm}$.

An *input-collision* or simply a *collision* in a structure graph $\tilde{\mathcal{G}}$ is defined by such a triple $\delta = (u, v; w)$. The set $\{u, v\}$ is called the *source of the collision* whereas w is called the *head of the collision*. We also say the edges e_1 and e_2 are colliding edges. Thus, an input-collision $\delta = (u, v; w)$ induces a linear restriction $L_\delta : \beta_u \oplus \beta_v = c_\delta$ where $c_\delta = a \oplus b \in \mathbb{B}$. A valid block label must satisfy such linear restrictions for all collisions δ . Let $\Delta(\tilde{\mathcal{G}})$ denote the set of all collisions in $\tilde{\mathcal{G}}$. Let $\text{rank}(\tilde{\mathcal{G}})$ denote the rank of the system of linear equations $\{L_\delta : \delta \in \Delta(\tilde{\mathcal{G}})\}$.

Now, we define an important parameter, called *accident*, useful in characterizing structure graphs. It is defined depending on whether the graph is zero-output graph or not.

Definition 3.2.3 (Accident of a structure graph). The (number of) accident of a structure graph $\tilde{\mathcal{G}}$ is defined as, $\text{Acc}(\tilde{\mathcal{G}}) := \text{rank}(\tilde{\mathcal{G}}) + \text{lszero}(\tilde{\mathcal{G}})$.

Thus, the accident of a non-zero structure graph $\tilde{\mathcal{G}}$ is simply $\text{rank}(\tilde{\mathcal{G}})$, whereas the accident of a zero-output graph is $\text{rank}(\tilde{\mathcal{G}}) + 1$.

Lemma 3.2.4. *If there is a vertex v with in-degree d then $\text{Acc}(\tilde{\mathcal{G}}) \geq d - 1$. Moreover, if the graph is a zero-output graph then $\text{Acc}(\tilde{\mathcal{G}}) \geq d$.*

Proof. Let $u_1, \dots, u_d$ be all the predecessors of v . Let us define an input-collision $\delta_{i,j} := (u_i, u_j; v)$ (ignoring the edge labels). It is now easy to see that $L_{\delta_{i,j}} = L_{\delta_{1,i}} \oplus L_{\delta_{1,j}}$ for all $i, j \in [d] \setminus \{1\}$. Moreover, the $L_{\delta_{1,i}}$'s are linearly independent. This, immediately proves the first part. The second part is just an extension of the first part using the definition of accident. $\square$

Remark 3.2.5. Another simple but useful observation is as follows: if a structure graph $\tilde{\mathcal{G}}$ has at least two collisions with distinct sources, then $\text{rank}(\tilde{\mathcal{G}}) \geq 2$.

Let $\tilde{\mathcal{G}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}})$ be a structure graph with rank r and $|\tilde{\mathcal{V}}| = s + 1$. Then fixing some $s - r$ choices of β_i values will uniquely determine the rest⁴, and so the number of valid block labeling is at most $(2^n)_{s-r}$. Any valid choice of β induces a block-vertex structure graph $\mathcal{S} = (\mathcal{V}, \mathcal{E})$ such that $\tilde{\mathcal{G}} = \tilde{\mathcal{S}} = \alpha(\mathcal{S})$. Note that, $s + \text{lszero}(\mathcal{S})$ is the number of vertices $v \in \mathcal{V}$ with positive in-degree. So exactly $(2^n - s - \text{lszero}(\mathcal{S}))!$ permutations can result in the block-vertex structure graph $\mathcal{S}$. Therefore,

$$\Pr [\text{BStruct}^\Pi = \mathcal{S}] = \frac{(2^n - (s + \text{lszero}(\mathcal{S})))!}{2^n!} = \frac{1}{(2^n)_{s + \text{lszero}(\mathcal{S})}},$$

whence, we get

$$\begin{aligned} \Pr [\text{Struct}^\Pi = \tilde{\mathcal{G}}] &= \sum_{\mathcal{S}: \tilde{\mathcal{S}} = \tilde{\mathcal{G}}} \Pr [\text{BStruct}^\Pi = \mathcal{S}] \\ &= \sum_{\mathcal{S}: \tilde{\mathcal{S}} = \tilde{\mathcal{G}}} \frac{1}{(2^n)_{s + \text{lszero}(\mathcal{S})}} \\ &\stackrel{2}{\leq} \frac{(2^n)_{s-r}}{(2^n)_{s+a-r}}. \end{aligned} \tag{3.12}$$

⁴After assigning values for any $s - r$ variables, the system of linear equations will have a unique solution.

In 1 the sum is taken over all block-vertex structure graphs $\mathcal{S}$ such that the induced structure graph $\tilde{\mathcal{S}} = \tilde{\mathcal{G}}$. From 1 to 2, there are at most $(2^n)_{s-r}$ choices for $\mathcal{S}$, and for each such choice $\text{lszero}(\mathcal{S})$ is equal to $a - r$ (i.e. lszero is invariant for fixed $\tilde{\mathcal{G}}$), where a denotes the accident of $\tilde{\mathcal{G}}$.

The following result bounds the probability of realizing a structure graph with accident a for the tuple of messages M^q . The proof is just an extension of Eq. (3.12) using the condition that $s < m$.

Lemma 3.2.6. *For any structure graph $\tilde{\mathcal{G}}$ with accident a , we have*

$$\Pr_{\Pi} [\text{Struct}^{\Pi} = \tilde{\mathcal{G}}] \leq \frac{1}{(2^n - m)^a}.$$

Now we state another important result which bounds the number of structure graphs with accident a . We skip the proof of this result as it is readily available in [19, 163].

Lemma 3.2.7. *The number of structures graphs associated to $M^q = (M_1, \dots, M_q)$ with accident a is at most $\binom{m}{2}^a$. In particular, there exists exactly one structure graph with accident 0.*

Corollary 3.2.8. *For $a \in \mathbb{N}$ and $m < 2^{n-1}$, we have*

$$\Pr_{\Pi} [\text{Acc}(\text{Struct}^{\Pi}) \geq a] \leq \left(\frac{m^2}{2^n} \right)^a.$$

Proof. The result follows from straightforward algebraic simplification after applying Lemma 3.2.6 and Lemma 3.2.7. $\square$

3.2.3 True Collision and an Observation on [19, Lemma 10]

Let $\tilde{\mathcal{G}}$ be a structure graph and $\tilde{W}_i$ the M_i -walk. Suppose we reconstruct the graph $\tilde{\mathcal{G}}$ again by traversing all the walks $\tilde{W}_i$ for $i \in [q]$ in ascending order. While we walk along $\tilde{W}_i$ for all i we count how many times we reach an existing vertex, which increases its current in-degree. The total count is defined to be the number of *true collisions* of the graph.

Mathematically, one can define it as follows: For a vertex $v \in \tilde{\mathcal{V}} \setminus \{(1, 0)\}$, we define the number of true collisions at v by $\text{TC}(v) := \text{deg}_{\text{in}}(v) - 1$ and $\text{TC}(1, 0) = |\text{deg}_{\text{in}}(1, 0)|$, where $\text{deg}_{\text{in}}(u)$ denotes the in-degree of u (for details see section A.1.1 of appendix A). The (number of) true collisions of $\tilde{\mathcal{G}}$ is the sum $\text{TC}(\tilde{\mathcal{G}}) := \sum_{v \in \tilde{\mathcal{V}}} \text{TC}(v)$. By lemma 3.2.4, we know that $\text{Acc}(\tilde{\mathcal{G}}) \geq \text{TC}(v)$ for all $v \in \tilde{\mathcal{V}}$. From the definition of the accident it is also obvious that $\text{Acc}(\tilde{\mathcal{G}}) \leq \text{TC}(\tilde{\mathcal{G}})$.

LEMMA 10 OF [19]: Like [19, 163], we will predominantly work with accident 1 structure graphs. To identify all structure graphs with accident 1 it would be good if we can have some relationship between true collisions and accidents. Lemma 10 of [19] was meant for this. It says that when $q = 2$, $\text{Acc}(\tilde{\mathcal{G}}) = 1 \Rightarrow \text{TC}(\tilde{\mathcal{G}}) = 1$. We show that this lemma is not true, via two counter-examples described below.

1. For the first example, illustrated in Figure 3.2.3(a), let

$$M_1 = (M_{(1,1)}, M_{(1,2)}, M_{(1,3)}, M_{(1,2)}, M_{(1,4)}) \text{ and } M_2 = (M_{(2,1)})$$

be two messages such that $M_{(2,1)} := M_{(1,1)} \oplus M_{(1,3)} \oplus M_{(1,4)}$. Here, we have two input-collisions $\delta_1 := ((1, 0), (1, 2); (1, 1))$ and $\delta_2 := ((1, 0), (1, 2); (1, 5))$. The two linear equations L_{δ_1} and L_{δ_2} corresponding to the two input-collisions are same as $\beta_{(1,0)} \oplus \beta_{(1,2)} = M_{(1,1)} \oplus M_{(1,3)}$ and so the rank (which is also the accident in this case) is 1. However, true collision is two (at $(1, 1)$ and $(1, 5)$). This contradicts [19, Lemma 10].

2. For the second example, illustrated in Figure 3.2.3(b), let

$$M_1 = (M_{(1,1)}, M_{(1,2)}, M_{(1,3)}) \text{ and } M_2 = (M_{(2,1)})$$

be two messages such that $M_{(2,1)} := M_{(1,1)} \oplus M_{(1,2)} \oplus M_{(1,3)}$. Here, we have two input-collisions $\delta_1 := ((1, 0), (1, 1); (1, 1))$ and $\delta_2 := ((1, 0), (1, 1); (1, 3))$. The two linear equations L_{δ_1} and L_{δ_2} corresponding to the two input-collisions are same as $\beta_{(1,0)} \oplus \beta_{(1,1)} = M_{(1,1)} \oplus M_{(1,2)}$ and so the rank and accident is 1. Again, this contradicts [19, Lemma 10].

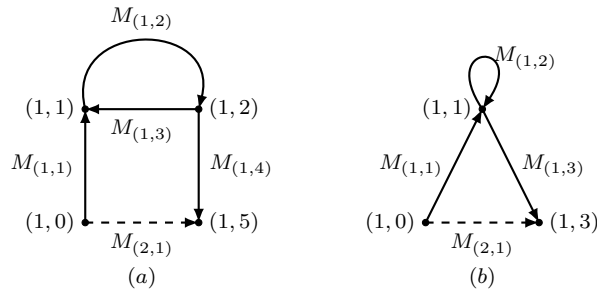


Figure 3.2.3: Two examples of structure graphs with accident 1 and true collision 2.

This lemma was a crucial step in characterizing all accident 1 structure graphs in [19, 163]. As this is shown to be wrong, it is important to revisit and rectify the results in [19, 163] as much as possible.

3.3 Characterization of Accident-1 Structure Graphs

In this section, we characterize all structure graphs with accident 0 or 1. To do so we characterize single message structure graphs which is much easier. Later in the section, we characterize all structure graphs for a pair of messages satisfying some event. From here onwards, we will not deal with block-vertex structure graph. So, for simplicity we use $\mathcal{G}$ (instead of $\tilde{\mathcal{G}}$) to represent a structure graph and W_r (instead of $\tilde{W}_r$) to represent the M_r -walk in the structure graph.

Let $\text{Struct}(M^q)$ denote the set of all structure graphs for the tuple of messages M^q . Let $\text{Struct}_a(M^q) = \{\mathcal{G} \in \text{Struct}(M^q) : \text{Acc}(\mathcal{G}) = a\}$, the set of all structure graphs associated to M^q with accident a . As before, we will drop the parameter M^q , whenever it is clear from the context. We are interested in $\text{Struct}_0(M^q)$ and $\text{Struct}_1(M^q)$, the sets of all structure graphs with accident 0 and 1, respectively. Lemma 3.2.7 says that the number of graphs with accident 1 is at most $\binom{m}{2}$, where $m = \sum_i m_i$ and $M_i \in \mathbb{B}^{m_i}$. The number of structure graphs with accident 0 is at most one. In the following we actually identify this structure graph.

FREE GRAPH: As there is no accident every non-zero vertex has in-degree 1 and $(1, 0)$ has in-degree 0 (i.e., non-zero output graph). Being a structure graph, $\mathcal{G}$ is union of M_i -walks W_i . An M_i -walk starting from $(1, 0)$ with no vertex having in-degree 2 must be a path. So $\mathcal{G}$ is a union of M_i -paths W_i . Now, for any $i \neq j$, let $p = \text{lcp}(M_i, M_j)$. Then, $W_i^{[p]} = W_j^{[p]}$, where $W_i^{[p]}$ denotes the sub-walk of W_i consisting of the first p vertices, and $W_{(i,p+1)} \neq W_{(j,p+1)}$ (if these are defined). It is also easy to see that $W_i^{[p]}$, $W_i^{[p+1 \dots m_i]}$, and $W_j^{[p+1 \dots m_j]}$ are disjoint paths. Thus, any two paths W_i and W_j are identical up to the length of the largest common prefix of M_i and M_j and afterwards they remain disjoint. We call this unique graph the *free graph* of M^q . A free graph for three messages is illustrated in Figure 3.3.1.

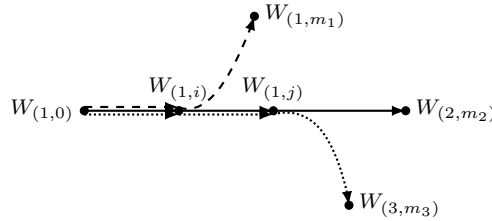


Figure 3.3.1: A free graph of three messages. The dashed, solid and dotted lines denote the three walks corresponding to the three messages.

3.3.1 Accident-one Structure Graphs for a Single Message

Now, we consider accident 1 structure graphs for a single message $M \in \mathbb{B}^m$. Note that, any such structure graph must be a walk W of length m .

In this subsection, we exclusively work with single message, hence the index set $\mathcal{I} = \{(1, i) : i \in (m)\}$ is uniquely identified by the set $(m]$. So, we skip the first coordinate in this subsection. We say a node W_i is *fresh* in the walk if $W_i \neq W_j$ for all $j \neq i$.

3.3.1.1 Case A: Zero-output Graphs

As 0 (this denotes $(1, 0)$) has positive in-degree there cannot be any more collision pairs otherwise the accident would be at least two. Let c be the minimum positive integer such that $W_c = 0$, so we have a cycle $(W_0, W_1, \dots, W_c)$. Let A be its label. Suppose $M = A^i \| B$, where i is the maximum positive integer for which we can write M in this form. So A is not a prefix of B . Let $s = \text{lcp}(A, B)$. Thus, $W_{ic+j} = W_j$ for all $j \in (s]$.

1. If B is a prefix of A then the structure graph is a cycle of size c ending at W_s . It is illustrated in Figure 3.3.2(a) (the $*$ is empty).
2. If B is not a prefix of A then $W_{ic+s} = W_s$ and $W_{ic+s+1} \neq W_{s+1}$. Further, $W_{ic+s+1} \neq W_j$ for all $j \in (c]$ since otherwise we get a collision. In fact, it can be shown that all subsequent nodes are fresh. Suppose not, then let $j > ic + s + 1$ be the first such integer for which $W_j = W_k$ for some $k < j$, hence we obtain a collision. So, the structure graph is an edge disjoint union of a cycle of size c and a path starting from s , as illustrated in Figure 3.3.2(a) (the $*$ is non-empty). The length of the cycle is c , whereas the length of the path is $m - ic - s$. We also call this graph ρ' graph. The tail (path from 0 to the cycle) of the ρ' walk is empty.

3.3.1.2 Case B: Non-zero Output Graphs

As 0 has in-degree 0, there is a collision $\delta = (u_0, v_0; w)$. In fact, all other collisions must have same source as that of δ (otherwise the number of accidents increases).

Let (i_0, j_0) be the smallest positive distinct integers such that $W_{i_0} = W_{j_0}$.⁵ As 0 has in-degree 0 so $1 \leq i_0 < j_0$, $W_{i_0-1} = u_0$, and $W_{j_0-1} = v_0$. Let A and B be the labels of $W^{(i_0]}$, and $W^{[i_0 \dots j_0]}$, respectively, and $j_0 - i_0 = c$. Then, $A \| B$ is a prefix of M . Let t be the largest positive integer such that $M = A \| B^t \| C$. So, B is not a prefix of C . If C is a

⁵ i_0 and j_0 can be fixed one by one. First fix i_0 to be the smallest positive integer such that $W_{i_0} = W_j$, for some $j \in [i_0 + 1 \dots m]$. Now, fix the smallest positive integer j_0 such that $W_{j_0} = W_{i_0}$.

prefix of B then we can have structure graphs as illustrated in Figure 3.3.2(d) or 3.3.2(f) (the end point lies inside the cycle). Suppose C is not a prefix and let $s = \text{lcp}(B; C)$.

Claim 3.3.1. The sub-walk left after traversing W till $A\|B^t\|C^{[s]}$ is a path and disjoint from the rest (illustrated in Figure 3.3.2(c)).

Proof. Suppose $\exists v \neq W_{i_0+tc+s} \in W^{[i_0+tc+s\dots m]} \cap W^{[i_0+tc+s]}$. Let $r = i_0 + tc + s$. We distinguish the following two cases:

Case B.1 — $W_{r+1} \in W^{[r]}$: If $s \neq c-1$, then we have a new collision $\delta' = (W_{i-1}, W_r; W_i)$ independent of δ which increases the accident to 2. If $s = c-1$, then $i \neq i_0 + kc$ for all $k \in [t]$. This can be argued as $\beta_{i_0-1} \oplus A_{i_0} = \beta_{j_0-1} \oplus B_c = \beta_{j_0-1} \oplus B_{s+1} \neq \beta_{j_0-1} \oplus C_{s+1}$ (as $B_{s+1} \neq C_{s+1}$). Now, the only way to make δ' dependent on δ is to have $W_{i-1} = W_{i_0-1}$. This implies a collision at W_j where $j \in [i_0-1]$, as the walk must come back to W_{i_0-1} at the $(i-1)$ -th step. This again gives a new accident as the source of collision is different.

Case B.2 — $W_{r+1} \notin W^{[r]}$ and $W_j = W_i$ for some $i \in [r]$ and $j \in [r+2\dots m]$: This gives a new collision $\delta' = (W_{j-1}, W_{i-1}; W_i)$ which is independent of δ , whence it increases the accident.

Hence, both case B.1 and B.2 lead to contradiction. Thus, we have $W^{[r+1\dots m]} \cap W^{[r]} = \emptyset$. Suppose $W^{[r\dots m]}$ is not a path. Therefore, $\exists i, j \in [r\dots m]$ such that $(W_i, W_j; W_{i+1})$ is a collision. Clearly, this collision is independent from δ (as $W^{[r+1\dots m]} \cap W^{[r]} = \emptyset$), and hence gives a new accident.

Observe that $s = c-1$ is a special case. In addition to this condition, suppose we have an edge $e := (W_{i_0-1}, W_{r+1})$. This creates a collision $\delta' = (W_{i_0-1}, W_r; W_{r+1})$ which is dependent on δ . Obviously, the edge e cannot occur in graph associated to a single message, as that will imply $\deg_{\text{in}}(W_j) \geq 2$ for some $j \in [0..i_0-1]$ which gives a new accident. But for two messages this is realizable, as exploited in the counter-examples given in Figure 3.2.3. We illustrate the structure graphs corresponding to this special case in Figure 3.3.2(b) and 3.3.2(e). $\square$

We summarize the above discussion in the following lemma.

Lemma 3.3.2. *For $m \geq 1, M \in \mathbb{B}^m$, the graphs in figure 3.3.2 exhaust all possible forms for any $\mathcal{G} \in \text{Struct}_1(M)$.*

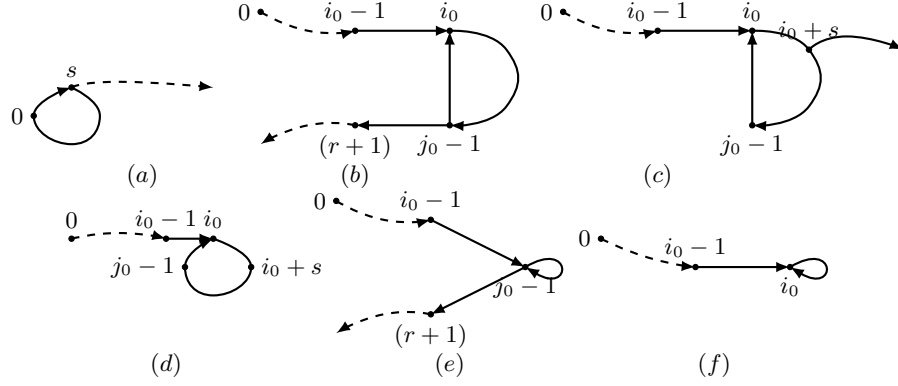


Figure 3.3.2: Characterizing all accident-one structure graphs realizable by a single message. The dashed lines in these illustrations represent optional subwalks. The vertex W_i is represented by i , for notational simplicity.

3.4 Revisiting $\text{outCP}_{q,\ell,\sigma}^{\text{any}}$ and $\text{fullCP}_{q,\ell,\sigma}^{\text{pf}}$ Bounds

In this section, we revise the upper bounds on outCP and fullCP, and consequently the PRF advantages of CBC-MAC and EMAC as given in [19]. We start off with a discussion that establishes the role of structure graphs in the PRF security analysis of CBC-MAC and EMAC. We have already seen that bounding PRF advantages of CBC-MAC and EMAC is reduced to bounding full collision probability $\text{fullCP}_{q,\ell,\sigma}^{\text{pf}}$ and collision probability $\text{outCP}_{q,\ell,\sigma}^{\text{any}}$, respectively. So, it would be sufficient to bound these probabilities. For this we first prove a general claim stated in Proposition 3.4.1.

STRUCTURE GRAPH EVENTS: Let E be an event defined on the intermediate output sequence $\mathbf{v}^\pi(M^q)$ for some permutation π . We say that the event E is *defined by a structure graph* if there is an event E' defined on the structure graph Struct^π such that E holds if and only if E' holds. We call such an event a *structure graph event*. Moreover, we say that E is *non-free* if it does not hold for free graphs (the structure graph with accident 0). The collision event for distinct messages as well as the full collision event for prefix-free messages are examples of non-free structure graph events. We write $\text{Struct}_a[E]$ to denote the set of all structure graphs with a accidents and satisfying a non-free event E .

Proposition 3.4.1. *Let E be a non-free structure graph event for the message tuple M^q . Then,*

$$\Pr_{\Pi}[E] \leq \frac{|\text{Struct}_1[E]|}{2^n - m} + \frac{m^4}{2^{2n}}.$$

Proof. For any structure graph event E ,

$$\Pr_{\Pi}[E] = \sum_{a \geq 0} \Pr \left[\text{Struct}^{\Pi} \in \text{Struct}_a[E] \right]$$

$$\begin{aligned}
&= \sum_{\mathcal{G} \in \text{Struct}_1[\mathbb{E}]} \Pr \left[\text{Struct}^\Pi = \mathcal{G} \right] + \sum_{a \geq 2} \Pr \left[\text{Struct}^\Pi \in \text{Struct}_a \right] \\
&\leq \frac{|\text{Struct}_1[\mathbb{E}]|}{2^n - m} + \frac{m^4}{2^{2n}},
\end{aligned}$$

where the last inequality follows from Lemma 3.2.6 and Corollary 3.2.8. $\square$

3.4.1 Revision of $\text{outCP}_{q,\ell,\sigma}^{\text{any}}$ Bound

Suppose $M_1 \in \mathbb{B}^{m_1}$, $M_2 \in \mathbb{B}^{m_2}$, and $0 \leq m_1 \leq m_2$. Also, we assume that $M_{(1,m_1)} \neq M_{(2,m_2)}$, since otherwise we can remove the largest common suffix which does not change the collision probability. Note that, the first message M_1 can be empty and in this case the collision event means $\mathbf{v}_{m_2}^\Pi(M_2) = 0^n$. This is a structure graph event because $(1,0)$ is a vertex of the structure graph. Due to Proposition 3.4.1, we only need to bound the number of structure graphs with accident 1 satisfying the 0coll event for the pair of messages. More precisely, we have to bound the size of the set $\text{Struct}_1(M_1, M_2)[0\text{coll}]$.

3.4.1.1 Case A: M_1 is Empty

In this case we have

$$\text{Struct}_1(M_1, M_2)[0\text{coll}] = \text{Struct}_1(M_2)[W_{(2,m_2)} = (1,0)].$$

We make the following claim, which is essentially [19, Lemma 14] in our notations.

Claim 3.4.2. $|\text{Struct}_1(M_2)[W_{(2,m_2)} = (1,0)]| \leq d(m_2)$, where d denotes the number of divisors function.

Proof. Let i be the smallest positive integer such that $W_{(2,i)} = 0$. Let A be the label of the cycle $W_2^{(i)}$. If $M_2 = A^k$ for some $k \in \mathbb{N}$, then $\text{Struct}_1(M_2)[W_{(2,i)} = (1,0)]$ contains exactly one structure graph. Note that, k must divide m_2 and hence the number of possible choices of k is at most $d(m_2)$, the number of divisors of m_2 . Suppose $M_2 = A^k \| B$ for some non-empty B where k is the largest such integer. If B is a prefix of A then $W_{(2,m_2)} = (1,0)$ only if $B = A$ which contradicts the maximality of k . So now assume that $\text{lcp}(A, B) = s$ where s is strictly less than the block length of B . Then, we must have $A_{s+1} \neq B_{s+1}$, whence $W_{(2,ki+s+1)} \neq W_{2,s+1}$. But, $W_{2,m_2} = (1,0)$, whence the walk must collide with the cycle $W_2^{(i)}$ at some point beyond $W_{2,s+1}$. However, as we have a zero-output structure graph with accident 1, we can not have any more collisions. Thus we can have at most $d(m_2)$ accident 1 graphs satisfying the event $W_{(2,m_2)} = (1,0)$. $\square$

3.4.1.2 Case B: M_1 is Non-empty

In this case, we must have a collision ($u := W_{(1,m_1-1)}, v := W_{(2,m_2-1)}; z := W_{(2,m_2)}$), as the labels of the last edges for walks W_1 and W_2 are different. Any other collision must have the same source set $\{u, v\}$ (otherwise, the number of accidents increases). Moreover, $(1, 0)$ can not have positive in-degree. Now, we consider different sub-cases:

Case B.1 — Both W_1 and W_2 are paths: The union of $W_1^{(m_1-1]}$ and $W_2^{(m_2-1]}$ is a free graph (as $W_{(1,m_1-1)}$ and $W_{(2,m_2-1)}$ can not appear before in the graph and so no collision among the path can occur). This gives only one choice of the graph as shown in the Figure 3.4.1(a). So the number of choices is bounded by at most 1. This is also proved as a part of [19, Lemma 15].

Case B.2 — W_2 is not a path: We have already characterized all possibilities of W_2 (in Figure 3.3.2). There exist some integers t and c such that $W_2^{(t]}$ is a path with $W_{(2,t-1)} = u$ and $W_{(2,t)} = w$, $W_2^{[t...t+c]}$ is a cycle of length c such that $W_{(2,t+c-1)} = v$. Recall that, $W_{(1,m_1-1)} = u$ and $W_{(2,m_2-1)} = v$.

Claim 3.4.3. $W_1^{(m_1-1]} = W_2^{(t-1]}$ and so $m_1 = t$.

Proof. Let s be the length of the largest common prefix of $M_1^{[t-1]}$ and $M_2^{[t-1]}$. If $s < t - 1$ then in the walk W_1 there is no way to reach u without coming back to the walk $W_2^{(t-1]}$. Coming back is not possible as it leads to a collision with a different generator set. Similarly, we can disprove that $s = t - 1$ and $m_1 > t$. Thus, we have $m_1 = t$ and $W_1^{(m_1-1]} = W_2^{(t-1]}$. $\square$

Now, based on $p = \text{lcp}(M_1, M_2)$, we may have two sub-cases.

Case B.2.(a) — $W_{(1,p)} = w$: In this case M_1 is a prefix of M_2 , i.e. $t = p$. The structure graph corresponding to this is illustrated in Figure 3.4.1(b). The number of such structure graphs is again at most $d(m_2 - m_1)$ (similar to the previous case where M_1 is the empty message). This is also shown in [19, Lemma 13].

Case B.2.(b) — $W_{(1,p)} \neq w$: This is the missing case in [19]. In this case, $W_{1,p}$ should be a fresh vertex otherwise we get a collision with different source set. The structure graph corresponding to this case is shown in Figure 3.4.1(c). Let $M_1 = A||a$ where $A = M_1^{[t-1]}$ and $a = M_{(1,t)}$. Note that, $t - 1 = p$ is the length of the largest common prefix of M_1 and M_2 . Then,

$$M_2 = A||b|(B||e)^{c-1}||B||f, \text{ where } f = M_{(2,m_2)}, b = M_{(2,t)}, e = a \oplus b \oplus f.$$

The choice of B is variable. But, it must satisfy the above restrictions for some $c > 1$. In fact, B is uniquely determined by c (as it occurs exactly c many times, and e and f are fixed). Now, c must divide $m_2 - m_1$ and hence the number of choices of c is at most $d(m_2 - m_1) - 1$.

This completes the characterization of all accident-1 structure graphs satisfying 0coll event and we get explicit upper bounds on the number of such graphs for all cases. Note that, cases B.2.(a) and B.2.(b) cannot hold simultaneously. But, Cases B.2.(b) and B.1 can hold simultaneously, which makes the total count of these two cases at most $d(m_2 - m_1)$. Since the ordering of messages does not matter in 0coll we are done.

Lemma 3.4.4. *For $m_1 \leq m_2 \in \mathbb{N}$, $M_1 \in \mathbb{B}^{m_1}$, $M_2 \in \mathbb{B}^{m_2}$, and $M_1 \neq M_2$, we have*

1. *If $\text{lcp}(M_1, M_2) = m_1$, then any $\mathcal{G} \in \text{Struct}_1(M_1, M_2)[0\text{coll}]$ is of the form illustrated in Figure 3.4.1(b) and the number of such graphs is at most $d'(m_2)$ (taking the maximum over all possible $d(\cdot)$ values).*
2. *If $\text{lcp}(M_1, M_2) = m_1 - 1$, then any $\mathcal{G} \in \text{Struct}_1(M_1, M_2)[0\text{coll}]$ is of the form illustrated in Figure 3.4.1(c) and the number of such graphs is at most $d'(m_2)$ (taking the maximum over all possible $d(\cdot)$ values).*
3. *In all other cases, any $\mathcal{G} \in \text{Struct}_1(M_1, M_2)[0\text{coll}]$ is of the form illustrated in Figure 3.4.1(a) and the number of such graphs is at most 1.*

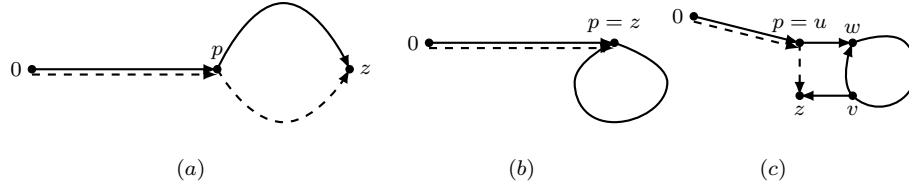


Figure 3.4.1: Characterizing all accident-one structure graphs realizable by two messages which satisfy the 0coll event. Dashed lines represent W_1 and solid lines represent W_2 , and * denote some arbitrary string.

Lemma 3.4.4 and Proposition 3.4.1 straightaway bound $\text{outCP}(M_i, M_j) \leq d'(\ell)/2^n + (m_i + m_j)^4/2^{2n}$ for any $i < j \in [q]$. Further, there exist a message tuple M^q such that $\text{outCP}_{q,\ell,\sigma}^{\text{any}} \leq \sum_{i < j \in [q]} \text{outCP}(M_i, M_j)$, which gives

Corollary 3.4.5. $\text{outCP}_{q,\ell,\sigma}^{\text{any}} \leq \frac{q^2 d'(\ell)}{2^n} + \frac{8\sigma q \ell^3}{2^{2n}}.$

Corollary 3.4.5 in combination with Eq. (3.3) and (3.5) gives the revised PRF security advantage for EMAC,

$$\text{Adv}_{\text{EMAC}_{E,E}}^{\text{prf}}(q, \ell, \sigma, t) \leq 2\text{Adv}_E^{\text{prp}}(\sigma, t) + \frac{q^2 d'(\ell)}{2^n} + \frac{8\sigma q \ell^3}{2^{2n}}. \quad (3.13)$$

3.4.2 Revision of $\text{fullCP}_{q,\ell,\sigma}^{\text{pf}}$ Bound

Since Fcoll is a non-free structure graph event, we have, using Proposition 3.4.1,

$$\text{fullCP}(M_1, M_2) \leq \frac{|\text{Struct}_1(M_1, M_2)[\text{Fcoll}]|}{2^n - m_1 - m_2} + \frac{(m_1 + m_2)^4}{2^{2n}}, \quad (3.14)$$

where M_1 and M_2 satisfy the pf (prefix-free) constraint. Thus, it would be again sufficient to bound the number of structure graphs for two messages with accident 1 which satisfy full collision event. Bellare et al. [19] proved that $|\text{Struct}_1[\text{Fcoll}]| \leq 4 \max\{m_1, m_2\}$. While bounding the $|\text{Struct}_1[\text{Fcoll}]|$, they proved a strong result [19, Lemma 19] that will be useful in our analysis also. We reproduce it here in our notations.

Lemma 3.4.6. *For $b \in \{1, 2\}$ and any $i \in (m_b]$, we have*

$$|\text{Struct}_1(M_1, M_2)[W_{(b,i)} \in W_b^{(i-1) \cup [i+1 \dots m_b]}]| \leq m_b.$$

We skip the proof of lemma 3.4.6 as it is available in [19]. We revise the upper bound on $|\text{Struct}_1(M_1, M_2)[\text{Fcoll}]|$ to $3(m_1 + m_2)$, and consequently the new bound on fullCP is as follows

Lemma 3.4.7. $\text{fullCP}(M_1, M_2) \leq \frac{3(m_1+m_2)}{2^n - m_1 - m_2} + \frac{(m_1+m_2)^4}{2^{2n}}.$

Proof. We need to bound the number of structure graphs for a pair of prefix-free messages $M_1 \in \mathbb{B}^{m_1}$ and $M_2 \in \mathbb{B}^{m_2}$ which satisfy the Fcoll event and have at most accident 1. Note that, the event implies that the structure graphs must have at least accident 1 as the messages are prefix-free. The event Fcoll can be written as $W_{(2,m_2)} \in W_2^{(m_2-1)} \cup W_1^{[m_1]}$.

3.4.2.1 Case A: $W_{(2,m_2)} \in W_2^{(m_2-1)}$

This case can be bounded to at most m_2 , by direct application of Lemma 3.4.6.

3.4.2.2 Case B: $W_{(2,m_2)} \in W_1^{[m_1]}$

Suppose $\text{Fcoll}(M_1, M_2)$ happens due to $W_{(2,m_2)} = W_{(1,r)}$ for an arbitrary $r \in [m_1 - 1]$. Then $\text{Fcoll}(M_1, M_2)$ is equivalent to $\text{Ocoll}(M_1^{[r]}, M_2)$. Let $s := \text{lcs}(M_1^{[r]}, M_2)$. Then $M_{(1,r-s)} \neq M_{(2,m_2-s)}$. Let $N_1 = M_1^{[r-s]}$ and $N_2 = M_2^{[m_2-s]}$. From lemma 3.4.4, we know

that $\tilde{\mathcal{G}} \in \text{Struct}_1(N_1, N_2)[0\text{coll}]$ must be one of (a), (b) or (c) in Figure 3.4.1. Keep in mind that $\tilde{\mathcal{G}}$ is a subgraph of some $\mathcal{G} \in \text{Struct}_1(M_1, M_2)[\text{Fcoll}]$.

Case B.1 — $\tilde{\mathcal{G}}$ is of the form Figure 3.4.1(a): In this case $\tilde{W}_1$ and $\tilde{W}_2$ are paths. For a fixed r the only possible collision is at $(\tilde{W}_{(1,r-s-1)}, \tilde{W}_{(2,m_2-s-1)}; \tilde{W}_{(1,r-s)})$, and hence the number of such graphs is at most 1. There are at most m_1 possible values for r . So, the number of choices for $\mathcal{G} \in \text{Struct}_1(M_1, M_2)[\text{Fcoll}]$ is at most m_1 .

Case B.2 — $\tilde{\mathcal{G}}$ is not of the form Figure 3.4.1(a): In this case, at least one of $\tilde{W}_1$ or $\tilde{W}_2$ is not a path. Without loss of generality, assume $\tilde{W}_1$ is not a path. Let $\tilde{p} = \text{lcp}(N_1, N_2)$. We know that N_1 and N_2 are prefixes of M_1 and M_2 , respectively. Thus $M_1^{[\tilde{p}]} = M_2^{[\tilde{p}]}$. Now, we must have a collision $(u, v; w)$ in $\tilde{W}_1$. From Lemma 3.4.4, we know that the graph can be either (b) or (c) in Figure 3.4.1 depending on whether $w = \tilde{W}_{(1,\tilde{p})}$ or $w = \tilde{W}_{(1,\tilde{p}+1)}$. Next, we make two claims which will enable us to bound these two cases. The proofs for these two claims are given later in the section.

Claim 3.4.8. If $\tilde{\mathcal{G}}$ is of the form Figure 3.4.1(b), then $W_{(1,\text{lcp}(M_1,M_2))}$ is not fresh in W_1 .

Claim 3.4.9. If $\tilde{\mathcal{G}}$ is of the form Figure 3.4.1(c), then $W_{(1,\text{lcp}(M_1,M_2)+1)}$ is not fresh in W_1 .

Recall that in a walk W a vertex W_i is not fresh if $\exists j \neq i$ such that $W_j = W_i$. By Claim 3.4.8, we know that $W_{(1,\text{lcp}(M_1,M_2))}$ is not fresh when $\tilde{\mathcal{G}}$ is as in Figure 3.4.1(b). Similarly, by Claim 3.4.9, we know that $W_{(1,\text{lcp}(M_1,M_2)+1)}$ is not fresh when $\tilde{\mathcal{G}}$ is as in Figure 3.4.1(c). So using Lemma 3.4.6, we bound the number of such graphs $\mathcal{G}$ to at most $2m_1$ (at most m_1 graphs for each of the two cases) when $\tilde{W}_1$ is not a path. Similarly we have at most $2m_2$ choices when $\tilde{W}_2$ is not a path. Therefore the total number of choices in case B.2 is at most $2(m_1 + m_2)$. Combining cases A, B.1 and B.2 we have at most $3(m_1 + m_2)$ choices. The result follows. $\square$

3.4.2.3 Proof of Claim 3.4.8

If $\tilde{\mathcal{G}}$ is like Figure 3.4.1(b), we must have $w = \tilde{W}_{(1,\tilde{p})}$. Let p be the smallest integer such that $\tilde{W}_{(1,p)} = \tilde{W}_{(1,\tilde{p})}$. Let P and Q be the labels of $\tilde{W}_1^{[\tilde{p}]}$ and $\tilde{W}_1^{[\tilde{p} \dots p]}$, respectively, and $c = p - \tilde{p}$. Then, $N_1 = P\|Q^i$ and $N_2 = P$ for some $i > 0$. As N_1 and N_2 are formed by removing the largest common suffix from $M_1^{[r]}$ and M_2 , respectively, therefore $M_1^{[r]} = P\|Q^{i'}\|R$ and $M_2 = P\|Q^j\|R$, where $i' \geq i$ and $j \geq 0$ are the largest such integers. Since $M_1^{[r]}$ and M_2 are prefix-free, we must have $i' > j$. Now, $M_1 = M_1^{[r]}\|S = P\|Q^{i'}\|R\|S$, where $|S| \geq 0$. From now on, we will work on the walk W_1 of graph $\mathcal{G}$ (instead of $\tilde{W}_1$ which is a subwalk of W_1) corresponding to M_1 . If R is a prefix of Q then M_2 must be a prefix of M_1 , which contradicts the prefix-free constraint. So R is not a prefix of Q . If Q is a prefix of R then it contradicts the maximality of i' and j . So Q is not a

prefix of R . Let $s = \text{lcp}(Q, R)$. Then, $Q_{s+1} \neq R_{s+1}$. Thus, $M_1^{[\tilde{p}+jc+s]} = M_2^{[\tilde{p}+jc+s]}$ and $M_{(1,\tilde{p}+jc+s+1)} \neq M_{(2,\tilde{p}+jc+s+1)}$. So, $\text{lcp}(M_1, M_2) = \tilde{p} + jc + s$. Further, since $j < i'$, we must have $W_{(1,\tilde{p}+i'c+s)} = W_{(1,\tilde{p}+jc+s)}$. Thus, $W_{(1,\text{lcp}(M_1, M_2))}$ is not fresh. Since, we started off with an arbitrary r , we conclude that $W_{(1,\text{lcp}(M_1, M_2))}$ is not fresh whenever $\tilde{\mathcal{G}}$ is of the form Figure 3.4.1(b). $\square$

3.4.2.4 Proof for Claim 3.4.9

If $\tilde{\mathcal{G}}$ is like Figure 3.4.1(c), we must have $w = \tilde{W}_1[\tilde{p} + 1]$. As noted earlier, in the revision of the outCP bound, this case was missing in the proof in [19]. Using a similar line of argument as in the proof of Claim 3.4.8, we can conclude that irrespective of the value of r , the cycle goes through $W_{(1,\text{lcp}(M_1, M_2)+1)}$ twice. Thus, $W_{(1,\text{lcp}(M_1, M_2)+1)}$ is not fresh. $\square$

Note that, our approach in Case B.2 above is a bit subtle. We used lemma 3.4.4 to identify a fundamental property (cycle goes through either $\text{lcp}(M_1, M_2)$ or $\text{lcp}(M_1, M_2) + 1$ twice), and then exploited this property to bound the counting. A straightforward approach of summing the number of graphs in Figure 3.4.1(b) and 3.4.1(c) over all values of r will give a worse bound of $m_b d'(m_b)$ for $b \in \{1, 2\}$. To get a tighter bound of m_b we needed this subtlety. Now, we extend the bound for $\text{fullCP}^{\text{pf}}(M_1, M_2)$ to $\text{fullCP}_{q,\ell,\sigma}^{\text{pf}}$, in order to get the revised PRF bound for CBC-MAC.

$$\begin{aligned}
 \text{fullCP}_{q,\ell,\sigma}^{\text{pf}} &\leq \sum_{i \neq j \in [q]} \text{fullCP}^{\text{pf}}(M_i; M_j) \\
 &\leq \sum_{i \neq j \in [q]} \left(\frac{3(m_i + m_j)}{2^n - m_1 - m_2} + \frac{(m_i + m_j)^4}{2^{2n}} \right) \\
 &\leq \sum_{i \neq j \in [q]} \left(\frac{6(m_i + m_j)}{2^n} + \frac{(m_i + m_j)^4}{2^{2n}} \right) \\
 &\leq \frac{6\sigma q}{2^n} + \frac{8\sigma q \ell^3}{2^{2n}}
 \end{aligned} \tag{3.15}$$

Here we have computed the bound in terms of q, ℓ and σ . Another approach (as used in [19]) is to bound the value using q and ℓ only, in which case the bound will be

$$\text{fullCP}_{q,\ell,\sigma}^{\text{pf}} \leq \frac{12q^2 \ell}{2^n} + \frac{16q^2 \ell^4}{2^{2n}}$$

Using Proposition 3.1.2, and Eq. 3.2 and 3.15, we get the following result.

Theorem 3.4.10. *For prefix-free messages, we have*

$$\mathbf{Adv}_{\text{CBC-MAC}_E}^{\text{prf}}(q, \ell, \sigma, t) \leq \mathbf{Adv}_E^{\text{prp}}(\sigma, t) + \frac{8\sigma q}{2^n} + \frac{8\sigma q \ell^3}{2^{2n}} + \frac{q^2}{2^{n+1}}. \quad (3.16)$$

This gives a bound of $O(\frac{\sigma q}{2^n})$ for $\ell < 2^{n/3}$. As noted earlier, this is a better bound whenever the average message length is much smaller than the length of the longest message.

3.5 Revised Security Analysis of EMAC

In this section, we revisit the improved PRF analysis of EMAC due to Pietrzak [163]. We first identify and rectify the flaw in the proof and obtain a degraded security bounds. Later, we obtain a better bound (in terms of ℓ) using a much simpler proof.

3.5.1 Flaw and Revision of PRF Advantage of EMAC [163]

For brevity, in this section we will use asymptotic bounds and avoid exact constant factors. The proposed bound for $\text{EMAC}_{\Pi_1, \Pi_2}$ as stated in [163] is

$$\mathbf{Adv}_{\text{EMAC}_{\Pi_1, \Pi_2}}^{\text{prf}}(q, \ell, \sigma) = O\left(\frac{q^2}{2^n} \left(1 + \frac{\ell^8}{2^n}\right)\right),$$

provided $\ell^2 \leq q$. Thus, the bound becomes $O(q^2/2^n)$ when $\ell \leq \min\{q^{1/2}, 2^{n/8}\}$.

PIETRZAK'S APPROACH: To show the above result, we need to bound the collision probability $\text{outCP}_{q, \ell}$. One possible approach is to group the q message into $O(q/\ell^2)$ sets, each set consisting of about ℓ^2 messages. Now, the collision event for q messages implies that a collision occurred in the union of two sets. Since, 0coll is a non-free event, Proposition 3.4.1 gives

$$\text{outCP}_{k, \ell} = O\left(\frac{|\text{Struct}_1(M^k)[\text{0coll}]|}{2^n}\right) + O\left(\frac{k^4 \ell^4}{2^{2n}}\right),$$

for some $k \leq q$. Applying this with $k = \ell^2$ (i.e. messages within two sets), we have

$$\text{outCP}_{q, \ell} = O\left(\frac{q^2}{\ell^4}\right) \times \text{outCP}_{2\ell^2, \ell} = O\left(\frac{q^2 N}{\ell^4 2^n}\right) + O\left(\frac{q^2 \ell^8}{2^{2n}}\right)$$

where N denotes the number of accident-one structure graphs satisfying 0coll for ℓ^2 messages with maximum length ℓ . The $O(q^2/\ell^4)$ term is due to the number of ways in

which we can choose two sets. In [163, Lemma 4], Pietrzak claimed that $N = O(\ell^4)$. More precisely, he showed that

$$N = \ell^4 \max_{\text{lcp}(M_1, M_2) < m_1} |\text{Struct}_1(M, M')[0\text{coll}]| + \ell^4. \quad (3.17)$$

where the maximum is taken over all (M_1, M_2) such that $m_1 < m_2$. To get a bound in $O(\ell^4)$ Pietrzak made the following claim.

Claim 1 of [163]: If $\text{lcp}(M_1, M_2) \neq m_1$, then $|\text{Struct}_1(M_1, M_2)[0\text{coll}]| = 1$.

If this claim happens to be true then $N = O(\ell^4)$. However, we have seen before that there exists M_1, M_2 such that $\text{lcp}(M_1, M_2) = m_1 - 1$ (see Figure 3.4.1(c)) and $|\text{Struct}_1(M, M')[0\text{coll}]| = d'(\ell)$. Thus,

$$\max_{\text{lcp}(M_1, M_2) < m_1} |\text{Struct}_1(M, M')[0\text{coll}]| = O(d'(\ell)).$$

If we plug in this, the modified bound is $N = O(\ell^4(d'(\ell)))$, whence the correct bound for collision probability becomes $O(q^2 d'(\ell)/2^n)$ which is not better than the bound in [19]. In fact, this is worse as the maximum length is restricted to $2^{n/8}$ as compared to a more relaxed bound of $2^{n/3}$ in [19].

3.5.2 Simple Proof of PRF Security for EMAC

We have seen in the last subsection that the influence of the flaw from [19, Lemma 10] is more serious in context of the tight bound analysis of EMAC given in [163]. One possible approach to fix the proof of [163], is by bounding N in a different way. For example, we can consider two cases: $\text{lcp}(M_1, M_2) = m_1$, and $\text{lcp}(M_1, M_2) = m_1 - 1$. For other pairs of messages which do not satisfy any of the two cases, the number of structure graphs can be shown to be one. However, we have to show that the the number of graphs is still at most ℓ^4 (see the second term of Eq. (3.17)).

We take a different and simpler approach. Instead of grouping the messages, we directly bound the number of structure graphs for a slightly different choices of permutations. We will ignore all those permutations (i.e. bad permutations) which induces one of the following:

1. For some pair of messages M_i and M_j the number of accident is two or more.
2. For some message M_i , the accident is one.

Let $\mathcal{G}$ be a structure graph associated to a q -tuple of messages. Recall that $\mathcal{G}$ is a union of q walks W_i . We write $\mathcal{G}_i$ and $\mathcal{G}_{i,j}$ to represent W_i and $W_i \cup W_j$. Note that, these are again structure graphs associated to M_i and (M_i, M_j) respectively. In this notation, all structure graphs generated by a good permutation must have two properties: for any $i \neq j \in [q]$, $\text{Acc}(\mathcal{G}_i) = 0$ and $\text{Acc}(\mathcal{G}_{i,j}) \leq 1$. We write Bad to denote the event that the structure graph is bad.

Lemma 3.5.1. *For any q -tuple of messages M^q and $\ell < 2^{n-2}$, we have*

$$\text{outCP}_{q,\ell}(M^q) \leq \frac{q^2}{2^n} + \frac{q\ell^2}{2^n} + \frac{8q^2\ell^4}{2^{2n}}.$$

PROOF OVERVIEW — We want to bound the probability of realizing a structure graph for a pair of messages that satisfies 0coll . The main idea is to simplify the analysis by handling graphs in $\text{Struct}(M^q)[\text{Bad}]$ separately. Lets characterize the nature of any structure graph for a pair of messages that satisfies $\neg\text{Bad} \wedge 0\text{coll}$. The number of accidents in the graph is at most 1, the individual walks corresponding to each message must be paths, and the two paths must share common first and last vertices. Now, the number of such graphs can be at most 1 (see Figure 3.4.1), leading to a probability bound of $q^2/2^n$ (due to $O(q^2)$ pairs of messages).

Proof. We first bound the probability that $\text{Struct}^\Pi(M^q) \in \text{Struct}(M^q)[\text{Bad}]$. For a bad graph (1) there exists i and j such that the accident for the pair of message M_i and M_j is at least 2, or (2) there exists i , such that the accident for M_i is at least one. The first event can happen with probability $8q^2\ell^4/2^{2n}$ using Corollary 3.2.8. Similarly, the second event can happen with $q\ell^2/2^n$. Now, we bound the probability that $\text{Struct}^\Pi(M^q) \in \text{Struct}(M^q)[0\text{coll} \wedge \neg\text{Bad}]$. Note that, the collision event implies that there exists i and j such that collision event holds for the message M_i and M_j . Now $\neg\text{Bad}$ implies that accident of $\mathcal{G}_{i,j}$ is one whereas accident of $\mathcal{G}_i$ and $\mathcal{G}_j$ are zero. In section 3.3, we have characterized all structure graphs for a pair of messages with accident one satisfying collision. Among all possibilities only one structure graph satisfies $\neg\text{Bad}$. Hence, there is exactly one structure graph. This implies that

$$\Pr \left[\text{Struct}^\Pi(M_i, M_j) \in \text{Struct}(M_i, M_j)[0\text{coll} \wedge \neg\text{Bad}] \right] \leq \frac{2}{2^n},$$

where we use the assumption that $m_i + m_j \leq 2\ell < 2^{n-1}$. By summing over all possible i and j , we have

$$\Pr \left[\text{Struct}^\Pi(M^q) \in \text{Struct}(M^q)[0\text{coll} \wedge \neg\text{Bad}] \right] \leq \frac{q^2}{2^n}.$$

Now, we summarize the above discussion as follows

$$\begin{aligned}
\text{outCP}_{q,\ell}(M^q) &\leq \Pr \left[\text{Struct}^\Pi(M^q) \in \text{Struct}(M^q)[0\text{coll} \wedge \neg \text{Bad}] \right] \\
&\quad + \Pr \left[\text{Struct}^\Pi(M^q) \in \text{Struct}^\Pi(M^q)[0\text{coll} \wedge \text{Bad}] \right] \\
&\leq \sum_{i < j} \frac{2|\text{Struct}(M_i, M_j)[0\text{coll} \wedge \neg \text{Bad}]|}{2^n} + \frac{q\ell^2}{2^n} + \frac{8q^2\ell^4}{2^{2n}} \\
&\leq \frac{q^2}{2^n} + \frac{q\ell^2}{2^n} + \frac{8q^2\ell^4}{2^{2n}}.
\end{aligned} \tag{3.18}$$

This completes the proof. $\square$

Using Eq. (3.3) and (3.5), and Lemma 3.5.1, we get the following result on the PRF security of EMAC.

Theorem 3.5.2. *For $\ell < 2^{n-2}$, we have*

$$\text{Adv}_{\text{EMAC}_{E,E}}^{\text{prf}}(q, \ell, \sigma, t) \leq 2\text{Adv}_E^{\text{prp}}(\sigma, t) + \frac{1.5q^2}{2^n} + \frac{q\ell^2}{2^n} + \frac{8q^2\ell^4}{2^{2n}}.$$

Thus, for $\ell \leq \min\{q^{1/2}, 2^{n/4}\}$, we have

$$\text{Adv}_{\text{EMAC}_{E,E}}^{\text{prf}}(q, \ell, \sigma, t) \leq 2\text{Adv}_E^{\text{prp}}(\sigma, t) + O\left(\frac{q^2}{2^n}\right).$$

Clearly, our result is a marked improvement over both [19] and [163]. The result in Theorem 3.5.2 is tight for $\ell \leq \min\{q^{1/2}, 2^{n/4}\}$, as one can obtain collisions in the output of EMAC in roughly $2^{n/2}$ queries, which leads to a easy length extension distinguishing event. The condition $q > \ell^2$ can be dropped if we assume that $\ell \leq 2^{n/4-k}$ for some small k such that 2^{-k} is negligible. More precisely, if $\ell \leq 2^{n/4-k}$, then the PRF advantage of EMAC is $O(q^2/2^n) + O(1/2^k)$.

3.5.3 Implications on the PRF Security of ECBC and FCBC

For any $M \in \{0, 1\}^{\eta^*}$, recall that $\overline{M} = \text{pad}(M) = M \parallel 10^{n|M|_n - |M| - 1}$ when $n \nmid |M|$, and $\overline{M} = \text{pad}(M) = M$ otherwise. In [34], Black and Rogaway gave three variants for CBC-MAC, namely ECBC, FCBC, and XCBC, to handle arbitrary message lengths. Here, we describe ECBC and FCBC.

1. ECBC, based on a block cipher E instantiated thrice with keys $K_1, K_2, K_3 \in \mathcal{K}$, is defined as,

$$\text{ECBC}_{E_{K_1}, E_{K_2}, E_{K_3}}(M) := \begin{cases} E_{K_2}(\text{CBC}_{E_{K_1}}(\overline{M})) & \text{if } |\overline{M}| = |M|, \\ E_{K_3}(\text{CBC}_{E_{K_1}}(\overline{M})) & \text{otherwise.} \end{cases}$$

2. FCBC, based on a block cipher E instantiated thrice with keys $K_1, K_2, K_3 \in \mathcal{K}$, is defined as,

$$\text{FCBC}_{E_{K_1}, E_{K_2}, E_{K_3}}(M) := \begin{cases} E_{K_2}(\text{CBC}_{E_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m) & \text{if } |\overline{M}| = |M|, \\ E_{K_3}(\text{CBC}_{E_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m) & \text{otherwise,} \end{cases}$$

$$\text{where } m = \left\lceil \frac{|M|}{n} \right\rceil.$$

It should be clear that the two keys K_2 and K_3 are chiefly used to separate the processing of block sequences (key K_2 is used for finalization) from non-block sequences (key K_3 is used for finalization). If we instantiate the block cipher E with $K^3 \leftarrow \$ \mathcal{K}^3$, then clearly the finalization of block and non-block sequences is independent from each other. This observation coupled with the fact that Ocoll implies Icoll gives the following corollary on the security of ECBC and FCBC

Corollary 3.5.3. *For $\ell < 2^{n-2}$, we have*

1. $\text{Adv}_{\text{ECBC}_{E,E,E}}^{\text{prf}}(q, \ell, \sigma, t) \leq 3\text{Adv}_E^{\text{prp}}(\sigma, t) + \frac{1.5q^2}{2^n} + \frac{q\ell^2}{2^n} + \frac{8q^2\ell^4}{2^{2n}}.$
2. $\text{Adv}_{\text{FCBC}_{E,E,E}}^{\text{prf}}(q, \ell, \sigma, t) \leq 3\text{Adv}_E^{\text{prp}}(\sigma, t) + \frac{1.5q^2}{2^n} + \frac{q\ell^2}{2^n} + \frac{8q^2\ell^4}{2^{2n}}.$

The proof is quite straightforward given Theorem 3.5.2 and the above mentioned observations. We remark that the bounds in Corollary 3.5.3 are significant improvements over the applicable state-of-the-art bounds in [19, 163].

Chapter 4

Pseudorandom Function based CBC-MAC Family

In this chapter, we continue our investigations on the security of the CBC-MAC family. In all the previous works the underlying primitive of the CBC-MAC family was assumed to be a block cipher or a keyed family of random permutations. However, the CBC mode of operation can also be used to construct a MAC when the underlying primitive is a length-preserving pseudorandom function (PRF) or a keyed family of length-preserving random functions. In fact, the very first security proof of CBC-MAC in [15] essentially models the block cipher as a PRF. We believe, when it comes to CBC-MAC the choice of block ciphers or pseudorandom permutations (PRPs) over PRFs as the underlying primitive is primarily for historical reasons, i.e. fast, secure, standardized block cipher implementations are readily available compared to pseudorandom functions.

Given a lack of understanding of the exact security of PRF-based CBC-MAC constructions, the primary goal of this chapter is to understand the gap between PRP vs PRF instantiation of CBC-MAC and its derivative constructions.

4.1 PRF Analysis of PRF-based CBC-MAC Family

As before, we will be interested in the PRF security of CBC-MAC family. Most of the notations and terminologies used in this chapter are analogous (replace $\pi \in \text{Perm}$ with $\gamma \in \text{Func}$) to the ones defined in chapter 3. So, sometimes we will directly use these notations and terminologies without first explicitly defining them.

4.1.1 Function-based CBC-MAC Family

In this chapter, we always use length-preserving functions γ over the set $\mathbb{B}$, i.e., $\gamma \in \text{Func}(\mathbb{B}, \mathbb{B})$, and hence skip the parametrization in Func . The CBC function with a function $\gamma \in \text{Func}$, viewed as a key of the construction, is defined analogous to CBC_π of section 3.1.1. In other words, CBC_γ takes as input a message $M \in \mathbb{B}^m$ with m blocks and outputs $\text{CBC}_\gamma(M) := \mathbf{v}_m^\gamma$. This is inductively computed as follows: $\mathbf{v}_0^\gamma(M) = 0^n$, and for all $i \in [m]$, we have

$$\begin{aligned} \mathbf{u}_i^\gamma(M) &:= \mathbf{v}_{i-1}^\gamma(M) \oplus M_i, \\ \mathbf{v}_i^\gamma(M) &:= \gamma(\mathbf{u}_i^\gamma(M)). \end{aligned} \tag{4.1}$$

As before, we call $\mathbf{u}^\gamma(M) := (\mathbf{u}_i^\gamma(M))_{i \in [m]}$ and $\mathbf{v}^\gamma(M) := (\mathbf{v}_i^\gamma(M))_{i \in [m]}$, the *intermediate input* and *intermediate output* vectors, respectively, associated to γ and M . The intermediate input vector $\mathbf{u}^\gamma(M)$ is uniquely determined by $\mathbf{v}^\gamma(M)$ (and does not depend on the function γ). We can write down this association generically as a function $\text{out2in}_M : \mathbb{B}^{m+1} \rightarrow \mathbb{B}^m$ mapping any block vector y^{m+1} to another block vector x^m where $x_i = y_{i-1} \oplus M_i$ for all $i \in [m]$. So for all functions $\gamma \in \text{Func}$, we have $\text{out2in}_M(\mathbf{v}^\gamma(M)) = \mathbf{u}^\gamma(M)$. We drop the superscripts γ and M when they are obvious from the context.

Given the definition of CBC_γ , one can easily define the CBC-MAC family. In the following description, $K^3 \in \mathcal{K}^3$, $F \in \text{BFunc}(\mathcal{K}, \mathbb{B}, \mathbb{B})$, $L, L' \in \mathbb{B}$, and $M \in \{0, 1\}^{\eta^*}$ with $\lceil |M|/n \rceil = m$.

1. CBC-MAC, based on F instantiated with key K_1 , is defined as,

$$\text{CBC-MAC}_{F_{K_1}}(M) := \text{CBC}_{F_{K_1}}(M) \text{ for all } M \in \mathbb{B}^{\eta^+}.$$

2. EMAC, based on F instantiated twice with key tuple $K^{[2]}$, is defined as,

$$\text{EMAC}_{F_{K_1}, F_{K_2}}(M) := F_{K_2}(\text{CBC}_{F_{K_1}}(M)) \text{ for all } M \in \mathbb{B}^{\eta^+}.$$

3. ECBC, based on F instantiated thrice with key tuple $K^{[3]}$, is defined as,

$$\text{ECBC}_{F_{K_1}, F_{K_2}, F_{K_3}}(M) := \begin{cases} F_{K_2}(\text{CBC}_{F_{K_1}}(\overline{M})) & \text{if } |\overline{M}| = |M|, \\ F_{K_3}(\text{CBC}_{F_{K_1}}(\overline{M})) & \text{otherwise.} \end{cases}$$

4. FCBC, based on F instantiated thrice with key tuple $K^{[3]}$, is defined as,

$$\text{FCBC}_{F_{K_1}, F_{K_2}, F_{K_3}}(M) := \begin{cases} F_{K_2}(\text{CBC}_{F_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m) & \text{if } |\overline{M}| = |M|, \\ F_{K_3}(\text{CBC}_{F_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m) & \text{otherwise.} \end{cases}$$

5. XCBC, based on L , L' , and F instantiated with key K_1 , is defined as,

$$\text{XCBC}_{L, L', F_{K_1}}(M) := \begin{cases} F_{K_1}(\text{CBC}_{F_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m \oplus L) & \text{if } |\overline{M}| = |M|, \\ F_{K_1}(\text{CBC}_{F_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m \oplus L') & \text{otherwise.} \end{cases}$$

6. TMAC, based on L , and F instantiated with key K_1 , is defined as,

$$\text{TMAC}_{L, F_{K_1}}(M) := \begin{cases} F_{K_1}(\text{CBC}_{F_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m \oplus L) & \text{if } |\overline{M}| = |M|, \\ F_{K_1}(\text{CBC}_{F_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m \oplus \mu \odot L) & \text{otherwise,} \end{cases}$$

where $\mu \odot L$ denotes the multiplication of $\mu \in \mathbb{F}_{2^n} \setminus \{0, 1\}$ and L over $\mathbb{F}_{2^n}$ (viewing $\mathbb{B}$ as $\mathbb{F}_{2^n}$).

7. OMAC, based on F instantiated with key K_1 , is defined as,

$$\text{OMAC}_{F_{K_1}}(M) := \begin{cases} F_{K_1}(\text{CBC}_{F_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m \oplus F_{K_1}(0)) & \text{if } |\overline{M}| = |M|, \\ F_{K_1}(\text{CBC}_{F_{K_1}}(\overline{M}^{[m-1]}) \oplus \overline{M}_m \oplus \mu \odot F_{K_1}(0)) & \text{otherwise,} \end{cases}$$

where $\mu \odot F_{K_1}(0)$ is defined as in case of TMAC.

We will mostly focus on EMAC, ECBC, FCBC, XCBC, and TMAC. However, our attack, given in Lemma 4.1.3, also works for OMAC and CBC-MAC with fixed length messages.

First, we replace the multiple instantiations of F with independent and uniform random functions $\Gamma_1, \Gamma_2, \Gamma_3 \leftarrow \$ \text{Func}$. This incurs a cost of $\text{Adv}_F^{\text{prf}}(\sigma, t)$, $2\text{Adv}_F^{\text{prf}}(\sigma, t)$, and $3\text{Adv}_F^{\text{prf}}(\sigma, t)$ in case of $\text{XCBC}_{L, L', F_{K_1}}$ and $\text{TMAC}_{L, F_{K_1}}$, $\text{EMAC}_{F_{K_1}, K_2}$, and $\text{ECBC}_{K_1, K_2, K_3}$ and $\text{FCBC}_{K_1, K_2, K_3}$, respectively. Formally, we have

$$\text{Adv}_{\text{EMAC}_{F, F}}^{\text{prf}}(q, \ell, \sigma, t) \leq 2\text{Adv}_F^{\text{prf}}(\sigma, t) + \text{Adv}_{\text{EMAC}_{\Gamma_1, \Gamma_2}}^{\text{prf}}(q, \ell, \sigma) \quad (4.2)$$

$$\text{Adv}_{\text{ECBC}_{F, F, F}}^{\text{prf}}(q, \ell, \sigma, t) \leq 3\text{Adv}_F^{\text{prf}}(\sigma, t) + \text{Adv}_{\text{ECBC}_{\Gamma_1, \Gamma_2, \Gamma_3}}^{\text{prf}}(q, \ell, \sigma) \quad (4.3)$$

$$\text{Adv}_{\text{FCBC}_{F, F, F}}^{\text{prf}}(q, \ell, \sigma, t) \leq 3\text{Adv}_F^{\text{prf}}(\sigma, t) + \text{Adv}_{\text{FCBC}_{\Gamma_1, \Gamma_2, \Gamma_3}}^{\text{prf}}(q, \ell, \sigma) \quad (4.4)$$

$$\text{Adv}_{\text{XCBC}_{L, L', F}}^{\text{prf}}(q, \ell, \sigma, t) \leq \text{Adv}_F^{\text{prf}}(\sigma, t) + \text{Adv}_{\text{XCBC}_{L, L', \Gamma_1}}^{\text{prf}}(q, \ell, \sigma) \quad (4.5)$$

$$\text{Adv}_{\text{TMAC}_{L, F}}^{\text{prf}}(q, \ell, \sigma, t) \leq \text{Adv}_F^{\text{prf}}(\sigma, t) + \text{Adv}_{\text{TMAC}_{L, \Gamma_1}}^{\text{prf}}(q, \ell, \sigma) \quad (4.6)$$

4.1.2 PRF Analysis of CBC-MAC Variants

In general, the PRF analysis of CBC-MAC variants can be reduced to the analysis of some collision events on the underlying CBC function. This has been the common technique for the PRF analysis of PRP instantiated CBC-MAC variants. We will follow the same technique here and establish a relationship between the PRF advantage of CBC-MAC variants and the collision probability of CBC function. Basically, we show that the PRF advantage of $\mathfrak{M}^+ \in \{\text{EMAC}, \text{ECBC}, \text{FCBC}, \text{XCBC}, \text{TMAC}\}$ is asymptotically tight in the collision probability of the underlying CBC function. First, we develop some new terminologies to aid our discussions in rest of the chapter. Following that we build the core results of this chapter in shape of Lemma 4.1.1, 4.1.3, Theorem 4.1.4, 4.1.5 and 4.1.6.

Let M_1 and M_2 be two distinct tuple of blocks with block lengths m_1 and m_2 , respectively. Analogous to $\text{Icoll}^\pi(M_1, M_2)$ and $\text{Ocoll}^\pi(M_1, M_2)$ for some $\pi \in \text{Perm}$, we write $\text{Icoll}^\gamma(M_1, M_2)$ and $\text{Ocoll}^\gamma(M_1, M_2)$ to denote the input and output collision events, i.e., $\mathbf{u}_{m_1}^\gamma(M_1) = \mathbf{u}_{m_2}^\gamma(M_2)$ and $\mathbf{v}_{m_1}^\gamma(M_1) = \mathbf{v}_{m_2}^\gamma(M_2)$, respectively, for a pair of messages M_1 and M_2 and a function $\gamma \in \text{Func}$. The collision events for a tuple of $q \geq 2$ distinct messages M^q is similarly defined as

$$\text{Icoll}^\Gamma(M^q) = \bigcup_{i \neq j} \text{Icoll}^\Gamma(M_i; M_j),$$

and

$$\text{Ocoll}^\Gamma(M^q) = \bigcup_{i \neq j} \text{Ocoll}^\Gamma(M_i; M_j).$$

We define **input-collision probability** as $\text{inCP}^\Gamma(M^q) = \Pr[\text{Icoll}^\Gamma(M^q)]$ and **output-collision probability** as $\text{outCP}^\Gamma(M^q) = \Pr[\text{Ocoll}^\Gamma(M^q)]$. It is easy to observe that $\text{outCP}^\Gamma(M^q)$ can be bounded in terms of $\text{inCP}^\Gamma(M^q)$. When Icoll is true Ocoll is trivially true. Otherwise, Ocoll is true when the underlying random function has a collision at the last intermediate output block. More specifically we have,

$$\text{inCP}^\Gamma(M^q) \leq \text{outCP}^\Gamma(M^q) \leq \text{inCP}^\Gamma(M^q) + \frac{q(q-1)}{2^{n+1}}. \quad (4.7)$$

Compare this with the permutation case, where $\text{inCP}^\Pi(M^q) = \text{outCP}^\Pi(M^q)$. Thus, one can expect additional output collisions in function case. This will be our main attack idea.

Lemma 4.1.1 (PRF–CBC Upper Bound). *For $q, \ell, \sigma \geq 1$ we have,*

$$1. \text{Adv}_{\text{EMAC}_{\Gamma_1, \Gamma_2}}^{\text{prf}}(q, \ell, \sigma) \leq \text{inCP}_{q, \ell, \sigma} + \frac{q(q-1)}{2^{n+1}}.$$

2. $\text{Adv}_{\text{ECBC}_{\Gamma_1, \Gamma_2, \Gamma_3}}^{\text{prf}}(q, \ell, \sigma) \leq \text{inCP}_{q, \ell, \sigma} + \frac{q(q-1)}{2^{n+1}}.$
3. $\text{Adv}_{\text{FCBC}_{\Gamma_1, \Gamma_2, \Gamma_3}}^{\text{prf}}(q, \ell, \sigma) \leq \text{inCP}_{q, \ell, \sigma}.$
4. $\text{Adv}_{\text{XCBC}_{L, L', \Gamma_1}}^{\text{prf}}(q, \ell, \sigma) \leq \text{inCP}_{q, \ell, \sigma} + \frac{\sigma q}{2^n} + \frac{q(q-1)}{2^{n+1}}.$
5. $\text{Adv}_{\text{TMAC}_{L, \Gamma_1}}^{\text{prf}}(q, \ell, \sigma) \leq \text{inCP}_{q, \ell, \sigma} + \frac{\sigma q}{2^n} + \frac{q(q-1)}{2^{n+1}}.$

The proof of this lemma is given in section 4.1.2.1 and 4.1.2.2.

4.1.2.1 Proof of 1, 2 and 3 of Lemma 4.1.1

Following [19, 34, 163], we view EMAC, ECBC, and FCBC as instances of the Hash-then-PRF paradigm [183] (see chapter 2 section 2.4.2.2). We discuss the implication of this for each of the schemes below:

1. EMAC and ECBC: In case of EMAC and ECBC, we consider the output of the underlying CBC function as the output of an almost-universal hash function. For EMAC the final random function Γ_2 acts on the output of this hash function.

For ECBC the final random function is either Γ_2 or Γ_3 depending upon the padding result. Also observe that in this case the output collision on the CBC outputs for two messages, one each from $\mathbb{B}^{\eta^+}$ and $\{0, 1\}^{\eta^*}$ is of no use for the adversary as the final outputs are independent due to the independence of Γ_2 and Γ_3 . So we assume that the messages are from $\mathbb{B}^{\eta^+}$. Now using the Hash-then-PRF paradigm we have,

$$\text{Adv}_{\mathfrak{M}^+}^{\text{prf}}(q, \ell, \sigma) \leq \text{outCP}_{q, \ell, \sigma}. \quad (4.8)$$

for $\mathfrak{M}^+ \in \{\text{EMAC}, \text{ECBC}\}$. The result follows by simple application of Eq. (4.7).

2. FCBC: In case of FCBC, we consider the input to the final random function as the output of an almost-universal hash function. Again using similar arguments as in the case of ECBC we assume that the messages are from $\mathbb{B}^{\eta^+}$. A small difference from EMAC and ECBC lies in the fact that we need to bound the probability of getting a collision on the final input, which is bounded by at most $\text{inCP}_{q, \ell, \sigma}$, whence we have

$$\text{Adv}_{\text{FCBC}}^{\text{prf}}(q, \ell, \sigma) \leq \text{inCP}_{q, \ell, \sigma}. \quad (4.9)$$

4.1.2.2 Proof of 4 and 5 of Lemma 4.1.1

We prove the result for XCBC here. The proof for TMAC can be obtained by slight modification of this proof. We use the coefficient-H technique to prove this proposition. Let Ω denote the set of all transcripts realizable via a uniform random function $\Gamma \leftarrow_{\$} \text{Func}[\{0, 1\}^{\eta^*}, \mathbb{B}]$. A typical transcript $\omega \in \Omega$ is of the form (M^q, T^q) , where $M^q \in (\{0, 1\}^{\eta^*})^q$ and $T^q \in \mathbb{B}^q$. Let $\overline{M}^q := (\overline{M}_i)_{i \in [q]} \in (\mathbb{B}^{\eta^+})^q$ be the padded version of M^q . Also, for all $i \in [q]$, $|\overline{M}_i| = m_i \leq \ell$, and $\sum_{i=1}^q m_i \leq \sigma$.

We take all transcripts $\omega \in \Omega$ to be good, i.e., $\Pr[\Theta_0 \in \Omega_{\text{bad}}] = 0$. We fix a transcript $\omega = (M^q, T^q) \in \Omega$. In the ideal world, we have

$$\Pr[\Theta_0 = \omega] = \Pr[\Gamma(M^q) = T^q] = \frac{1}{2^{nq}}. \quad (4.10)$$

Let $\bar{u}$ denote the random tuple $(u_{m_1}^{\Gamma_1}(\overline{M}_1) \oplus K_1, \dots, u_{m_q}^{\Gamma_1}(\overline{M}_q) \oplus K_q)$, where $K_i = L$ if $M_i \in \mathbb{B}^{\eta^+}$, and $K_i = L'$, otherwise. Let Fresh denote the event,

$$\forall r, r' \in [q] \text{ and } i \in [m_{r'}], \left(\bar{u}_r \neq u_i^{\Gamma_1}(\overline{M}_{r'}) \right) \wedge (r \neq r' \implies \bar{u}_r \neq \bar{u}_{r'}).$$

The output of XCBC is completely random when Fresh holds, as Γ_1 has not been sampled over $\bar{u}$. Thus, we have,

$$\Pr[\text{XCBC}_{L, L', \Gamma_1}(M^q) = T^q | \text{Fresh}] = \Pr[\Gamma_1(\bar{u}) = T^q | \text{Fresh}] = \frac{1}{2^{nq}}.$$

So the interpolation probability in the real world can be written as,

$$\begin{aligned} \Pr[\Theta_1 = \omega] &\geq \Pr[\Theta_1 = \omega | \text{Fresh}] \times \Pr[\text{Fresh}] \\ &\geq \frac{1}{2^{nq}} \left(1 - \Pr[\neg \text{Fresh}] \right). \end{aligned}$$

Once we upper bound $\Pr[\neg \text{Fresh}]$, we are done by the application of coefficient-H technique. The event $\neg \text{Fresh}$ holds if one of the following three events occur:

1. B_1 : $\exists^* r, r' \in [q]$, such that $M_r, M_{r'} \in \mathbb{B}^{\eta^+}$ (or $M_r, M_{r'} \in \{0, 1\}^{\eta^+}$), and $\bar{u}_r = \bar{u}_{r'}$.
2. B_2 : $\exists^* r, r' \in [q]$, such that $M_r \in \mathbb{B}^{\eta^+}$ and $M_{r'} \in \{0, 1\}^{\eta^+}$, and $\bar{u}_r = \bar{u}_{r'}$.
3. B_3 : $\exists r, r' \in [q]$ and $i \in [m_{r'} - 1]$, such that $\bar{u}_r = u_i^{\Gamma_1}(\overline{M}_{r'})$.

BOUNDING $\Pr[B_1]$: Note that, for B_1 the masking keys do not play any role, and there is actually a collision of the form $u_{m_r}^{\Gamma_1}(\overline{M}_r) = u_{m_{r'}}^{\Gamma_1}(\overline{M}_{r'})$. So we can bound $\Pr[B_1]$ in

terms of inCP , i.e.,

$$\Pr [\mathcal{B}_1] \leq \text{inCP}_{q,\ell,\sigma}.$$

BOUNDING $\Pr [\mathcal{B}_2]$: For a fix $\gamma \in \text{Func}$, the conditional probability of $\mathcal{B}_2$ is dependent only on the randomness of the masking keys. Further the two masking keys L and L' are uniformly and independently distributed over $\mathbb{B}$. Thus, we have

$$\begin{aligned} \Pr [\mathcal{B}_2 | \Gamma = \gamma] &\leq \sum_{r < r' \in [q]} \Pr [L \oplus L' = u_{m_r}^\gamma(\overline{M}_r) \oplus u_{m_{r'}}^\gamma(\overline{M}_{r'}) | \Gamma = \gamma] \\ &\leq \frac{\binom{q}{2}}{2^n}. \end{aligned}$$

Since the conditional probability of $\mathcal{B}_2$ given γ is independent of the choice of γ , we can conclude that

$$\Pr [\mathcal{B}_2] \leq \frac{q(q-1)}{2^{n+1}}.$$

BOUNDING $\Pr [\mathcal{B}_3]$: For a fix $\gamma \in \text{Func}$, the conditional probability of $\mathcal{B}_3$ is dependent only on the randomness of the masking keys (either one of them). Thus, we have

$$\begin{aligned} \Pr [\mathcal{B}_3 | \Gamma = \gamma] &\leq \sum_{r \in [q]} \sum_{(r', i) \in [q] \times [m_{r'} - 1]} \Pr [K_r = u_{m_r}^\gamma(\overline{M}_r) \oplus u_i^\gamma(\overline{M}_{r'}) | \Gamma = \gamma] \\ &\leq \frac{\sigma q}{2^n}. \end{aligned}$$

Again, since the conditional probability of $\mathcal{B}_3$ is independent of the choice of γ , so we have

$$\Pr [\mathcal{B}_3] \leq \frac{\sigma q}{2^n}.$$

Thus,

$$\Pr [\neg \text{Fresh}] \leq \text{inCP}_{q,\ell,\sigma} + \frac{\sigma q}{2^n} + \frac{q(q-1)}{2^{n+1}}.$$

The result follows from coefficient-H technique. $\square$

Remark 4.1.2. Note that, the PRF analysis of OMAC is missing in lemma 4.1.1. Our proof technique cannot be applied directly in case of OMAC. We bound the probability of getting a collision at the input block of the final function. For CBC-MAC variants (other than OMAC) this can be argued using the randomness of the independent random functions or the auxiliary keys. In case of OMAC, $\Gamma_1(0)$ is used to mask the final internal input block. Whenever the first message block is 0, $\Gamma_1(0)$ is already defined, hence the current proof technique will not work. Having said that, we believe that identical upper bound should hold for OMAC also.

Lemma 4.1.3 (PRF-CBC Lower Bound). Let $q, \ell \geq 1$ and $M^q := (M_1, \dots, M_q)$ be a q -tuple of distinct messages such that for $i \in [q]$, $M_i \in \mathbb{B} \times (0^n)^{\ell-1}$. Then $\forall \mathfrak{M}^+ \in \{\text{CBC-MAC}, \text{EMAC}, \text{ECBC}, \text{FCBC},$

we have

$$\mathbf{Adv}_{\mathfrak{M}^+}^{\text{prf}}(q, \ell) \geq \text{inCP}(M^q) \left(1 - \frac{q(q-1)}{2^{n+1}}\right).$$

Proof. To show the lower bound we present an adversary $\mathcal{A}$ that achieves the claimed PRF advantage using the given message tuple. Consider the following attack algorithm for $\mathfrak{M}^+$:

1. $\mathcal{A}$ queries $M_i \in M^q$ and observes the corresponding output T_i .
2. If $T_i = T_j$ for some $j < i$ then $\mathcal{A}$ returns 1.

Let $\mathbf{p}_{\mathfrak{M}^+}$ and $\mathbf{p}_{\Gamma}$ denote $\Pr[\mathcal{A}^{\mathfrak{M}^+} = 1]$ and $\Pr[\mathcal{A}^{\Gamma} = 1]$, respectively. Let $\mathbf{cp}_q$ denote the probability of having a collision on q elements chosen uniformly and independently from $\mathbb{B}$. It is well-known [78] that

$$1 - e^{-\frac{k(k-1)}{2^{n+1}}} \leq \mathbf{cp}_q \leq \frac{k(k-1)}{2^{n+1}}. \quad (4.11)$$

Now, we know that,

$$\mathbf{Adv}_{\mathfrak{M}^+}^{\text{prf}}(q, \ell, \sigma) \geq |\mathbf{p}_{\mathfrak{M}^+} - \mathbf{p}_{\Gamma}|.$$

For $\mathfrak{M}^+ \in \{\text{CBC-MAC}, \text{EMAC}, \text{ECBC}, \text{FCBC}\}$ we have,

$$\begin{aligned} |\mathbf{p}_{\text{MAC}} - \mathbf{p}_{\Gamma}| &\stackrel{1}{=} |\text{outCP}(M^q) + (1 - \text{outCP}(M^q)) \cdot \mathbf{cp}_q - \mathbf{cp}_q| \\ &\stackrel{2}{=} |\text{outCP}(M^q) \cdot (1 - \mathbf{cp}_q)| \\ &\stackrel{3}{\geq} |\text{inCP}(M^q) \cdot (1 - \mathbf{cp}_q)| \\ &\stackrel{4}{\geq} \text{inCP}(M^q) \cdot \left(1 - \frac{q(q-1)}{2^{n+1}}\right) \end{aligned}$$

We use Eq. (4.7) from 2 to 3 and Eq. (4.11) from 3 to 4. We can have similar analysis for $\mathfrak{M}^+ \in \{\text{XCBC}, \text{TMAC}, \text{OMAC}\}$ by replacing outCP with inCP in the first equality above. $\square$

Note that, due to the choice of messages (a non-zero block followed by $\ell - 1$ zero blocks) in the attack, the CBC function can also be viewed as an iterated random function $\Gamma^{(\ell)}$ (denotes ℓ times composition of Γ). In other words, our attack also applies on the general iterated random function. Lemma 4.1.1 and 4.1.3 show that the PRF advantages of EMAC, ECBC, FCBC, XCBC, and TMAC are tight in inCP of CBC function.

4.1.3 Summary of Main Results

The following theorems are the main technical results of this chapter which quantify $\text{inCP}_{q,\ell,\sigma}$. The proof of these theorems are postponed to section 4.2 and 4.3.

Theorem 4.1.4 (Upper Bound Theorem). *Let $q, \ell, \sigma \geq 1$. Let $M^q = (M_1, \dots, M_q)$ be a q -tuple of distinct messages such that $M_i \in \mathbb{B}^{m_i}$, $m_i \leq \ell$ for all $i \in [q]$, and $\sum_{i=1}^q m_i \leq \sigma$. Then we have,*

$$\text{inCP}_{q,\ell,\sigma}(M^q) \leq \frac{\sigma q}{2^n} + \frac{\sigma \ell}{2^n} + \frac{8\sigma q \ell^3}{2^{2n}}.$$

PROOF OVERVIEW — The proof strategy is identical to the proof of Lemma 3.5.1 in chapter 3. Note that, the bound in this case is worse than the bound in Lemma 3.5.1. As we will see in the proof (see section 4.2), this is due to the cumulative effect of internal input collisions.

Theorem 4.1.5 (Lower Bound Theorem). *Let $q, \ell, \sigma \geq 1$. Let $M^q = (M_1, \dots, M_q)$ be a q -tuple of distinct messages such that $M_i \in \mathbb{B} \times (0^n)^{\ell-1}$. Then we have,*

$$\begin{aligned} \text{inCP}(M^q) \geq & \binom{q}{2} \frac{\ell-1}{2^n} e^{-\frac{4\ell^2}{2^n}} - 3 \binom{q}{3} \left(\frac{2\ell^2}{2^{2n}} + \frac{6\ell^6}{2^{3n}} \right) \\ & - \frac{1}{2} \binom{q}{2} \binom{q-2}{2} \left(\frac{\ell^2}{2^{2n}} + \frac{6\ell^3 + 2\ell^5}{2^{3n}} + \frac{28\ell^8}{2^{4n}} \right) \end{aligned}$$

For $\ell, q \geq 3$, $\frac{q^2 \ell}{2^n} < 1$ and $\ell < \min\{\frac{2^n}{5184}, \frac{2^{n/2}}{4\sqrt{3}}, \frac{2^{n/3}}{\sqrt[3]{36}}\}$, the above expression is at least $\frac{q^2 \ell}{12 \cdot 2^n}$. Further if we take $\ell = \frac{\sigma}{q}$, the expression is at least $\frac{\sigma q}{12 \cdot 2^n}$.

Note that, in Lemma 4.1.3, the advantage can be lower bounded to $\frac{1}{2} \text{inCP}(M^q)$ for $q < 2^{n/2}$. Using Lemma 4.1.1, 4.1.3, and Theorem 4.1.4, 4.1.5 we have the exact PRF security bounds.

Theorem 4.1.6 (PRF Bound). *Let $q, \ell, \sigma \geq 3$, such that $\frac{q^2 \ell}{2^n} < 1$, $q < 2^{n/2}$, and $\ell < \min\{\frac{2^n}{5184}, \frac{2^{n/2}}{4\sqrt{3}}, \frac{2^{n/3}}{\sqrt[3]{36}}, q\}$. Then, $\forall \mathfrak{M}^+ \in \{\text{EMAC}, \text{ECBC}, \text{FCBC}, \text{XCBC}, \text{TMAC}\}$, the PRF advantage of $\mathfrak{M}^+$ is asymptotically tight in terms of q, ℓ and σ , i.e.,*

$$\text{Adv}_{\mathfrak{M}^+}^{\text{prf}}(q, \ell, \sigma) = \Theta\left(\frac{\sigma q}{2^n}\right).$$

From the above discussion it is clear that, we are only left with the analysis of the collision probability of CBC. More specifically we have to prove Theorem 4.1.4 and 4.1.5. Following chapter 3, we use structure graphs to bound the collision probability, albeit with a slightly different definition.

In the rest of the chapter, we take M^q as the q -tuple of distinct messages such that $M_i \in \mathcal{B}^{m_i}$, $m_i \leq \ell$ for all $i \in [q]$, and $\sum_{i=1}^q m_i \leq \sigma$.

4.1.4 Input-Structure Graph

We mostly use the same set of notations and terminologies as used in section 3.2 of chapter 3. However, we define the block-vertex structure graphs over intermediate input vectors instead of output vectors. This small change is required as we will be mostly interested in Icoll , and in case of functions Ocoll does not imply Icoll . In the following subsections we briefly revisit the notations and definitions on structure graphs. Note that, we prepend the adjective “input” just to emphasize that the vertices in the graph correspond to the intermediate inputs.

4.1.4.1 Intermediate Inputs and Outputs

We define set $\text{Func}_\perp$ as

$$\text{Func}_\perp := \{\gamma \mid \gamma : \mathbb{B} \cup \{\perp\} \rightarrow \mathbb{B} \wedge \gamma(\perp) = 0^n\}.$$

Note that, the uniform distribution over $\text{Func}_\perp$ has the same probability mass function (pmf) as Func , i.e., a constant function taking up the value 2^{-n2^n} .

We extend the definition of CBC function for any $\gamma \in \text{Func}_\perp$ by setting $u_0^\gamma(M_i) = \perp$ for all $i \in [q]$, and then following the normal CBC computation.

Recall that, this extension does not hamper our analysis, as the uniform distribution over $\text{Func}_\perp$ follows the same pmf as $\text{Func}_\perp$. This extension may (or may not) look artificial at the moment, but it will greatly simplify some of the definitions and proofs that we discuss later. From now onward, γ and Γ will have their usual meaning, but over the set $\text{Func}_\perp$.

INDEX SET: We define

$$\mathcal{I} = \{(r, i) : r \in [q], i \in (m_r)\}.$$

The dictionary order $\prec$ over $\mathcal{I}$ is defined analogous to $\mathcal{I}_0$ of section 3.2 of chapter 3. Note that, $\prec$ is a partial order. For $\omega, \omega' \in \mathcal{I}$, we say that ω' is a *successor* of ω if $\omega \prec \omega'$. In such cases, we say that ω is a *predecessor* of ω' . For all $\omega \in \mathcal{I}$ and $i \in \mathbb{N}$, we sometimes write $\omega + i$ to denote the i -th successor of ω in order.

SEQUENCES FOR INTERMEDIATE INPUTS AND OUTPUTS: For any $r \in [q]$, the subsequence $(x_{r,i})_{i \in (m_r)}$ is denoted x_{r*} . The sequences of intermediate inputs and outputs over $\mathcal{I}$ is denoted as $u^\gamma(M^q)$ and $v^\gamma(M^q)$, respectively, where $\forall r \in [q]$, $u_{r*}^\gamma(M^q) := u^\gamma(M_r)$ and $v_{r*}^\gamma(M^q) := v^\gamma(M_r)$.

The *block-vertex input-structure (BINS)* graph $\text{BStruct}^\gamma(M^q) = (\mathcal{V}^\gamma, \mathcal{E}^\gamma)$ for a function γ is defined by the labeled vertex set $\mathcal{V}^\gamma = \hat{\mathbf{u}}^\gamma(M^q)$, and the set of labeled edges

$$\mathcal{E} := \cup_{r=1}^q \{(\mathbf{u}_{(r,i-1)}^\gamma, \mathbf{u}_{(r,i)}^\gamma, M_{(r,i)}) : i \in [m_r]\}.$$

Note that, we skipped M^q from the parametrization as it is quite clear from the context. Clearly, $\perp \in \mathcal{V}^\gamma$ has zero in-degree and positive out-degree, and BStruct^γ is a union of M_i -walks, denoted W_i , for $i \in [q]$. Note that,

$$u \xrightarrow{A} v \Rightarrow \gamma(u) \oplus A = v. \quad (4.12)$$

So, for every $u \in V$, all outward edges (similarly for inward edges) have distinct edge labels. Using this property, it is easy to see that for all $i \in [q]$, the W_i walk is unique in a BINS graph. We denote the set of all BINS graphs by $\text{BStruct}(M^q)$. Recall the minimum index mapping α of section 3.2.2, that maps a vertex $v \in \mathcal{V}^\gamma$ to the minimum index $(r, i) \in \mathcal{I}$ such that $W_{(r,i)} = v$.

Definition 4.1.7 (Input-Structure Graph). The input-structure (INS) graph $\text{Struct}^\gamma = (\tilde{\mathcal{V}}^\gamma, \tilde{\mathcal{E}}^\gamma)$ associated to γ is the α -transformed BStruct^γ , i.e. $\text{Struct}^\gamma = \alpha(\text{BStruct}^\gamma)$.

The following example is a slight modification of example 3.1 given in chapter 3.

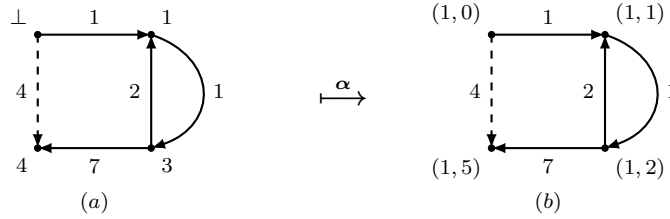


Figure 4.1.1: INS graph corresponding to a BINS graph.

Example 4.1. Let $M_1 = (1, 1, 2, 1, 7)$ and $M_2 = (4)$ be two messages and $\gamma(1) = 2$; $\gamma(3) = 3$ for some $\gamma \in \text{Func}_\perp$. Then, we have $\mathbf{u}^\gamma(M_1) = (\perp, 1, 3, 1, 3, 4)$ and $\mathbf{u}^\gamma(M_2) = (\perp, 4)$. The corresponding BINS graph BStruct^γ , shown in Figure 4.1.1(a), has vertex set $\mathcal{V}^\gamma = \{\perp, 1, 3, 4\}$ and edge set

$$\mathcal{E}^\gamma = \{(\perp, 1, 1), (1, 3, 1), (3, 1, 2), (3, 4, 7), (\perp, 4, 4)\}.$$

The corresponding INS graph Struct^γ , illustrated in Figure 3.2.2(b), has vertex set $\tilde{\mathcal{V}}^\gamma = \{(1, 0), (1, 1), (1, 2), (1, 5)\}$ and edges set

$$\tilde{\mathcal{E}}^\gamma = \{((1, 0), (1, 1), 1), ((1, 1), (1, 2), 1), ((1, 2), (1, 1), 2), ((1, 2), (1, 5), 7), ((1, 0), (1, 5), 4)\}.$$

It is easy to see that an INS graph is again a union of $\widetilde{W}_r$ walks corresponding to message M_r . Further, $(1, 0)$ (i.e. $\alpha(\perp)$) has zero in-degree, and positive out-degree. We denote the set of all INS graphs for M^q by $\text{Struct}(M^q)$.

Definition 4.1.8 (valid block labeling). An injective function $\beta : \widetilde{\mathcal{V}} \rightarrow \mathbb{B} \cup \{\perp\}$ is called a *valid block labeling* for an INS graph $\widetilde{\mathcal{G}} = (\widetilde{\mathcal{V}}, \widetilde{\mathcal{E}})$ if the graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a BINS graph where

1. $\mathcal{V} := \beta(\mathcal{V}) = \{\beta_v := \beta(v) : v \in \widetilde{\mathcal{V}}\}$, and
2. $\mathcal{E}$ is the edge set after relabeling v by β_v , i.e. $\mathcal{E} = \{(\beta_u, \beta_v, x) : (u, v, x) \in \widetilde{\mathcal{E}}\}$.

NECESSARY CONDITIONS FOR A VALID BLOCK LABELING: Analogous to the valid block label in case of permutation-based structure graphs (see Definition 3.2.2) one can identify certain restrictions on valid block labeling for an INS graph. A valid block labeling must map $(1, 0)$ to $\perp$, and $(1, 0)$ should be the only preimage of $\perp$. Second, β must be injective as distinct vertices in a BINS graph have distinct block labels. Lastly, whenever we have edges $e_1 := (u, v, a), e_2 := (u, w, b) \in \widetilde{\mathcal{E}}$, we must have $\beta_v \oplus a = \beta_w \oplus b$, as these are equal to $\gamma(u)$ for some $\gamma \in \text{Func}_{\perp}$.

Note that, if there exists an edge $e_1 := (u, w, a) \in \widetilde{\mathcal{E}}$, then $\gamma(\beta_u) \oplus a = \beta_w$ for some $\gamma \in \text{Func}_{\perp}$. Now, if there exists another edge $e_2 := (v, w, b) \in \widetilde{\mathcal{E}}$, then we must have $\gamma(\beta_u) \oplus \gamma(\beta_v) = a \oplus b$. A collision in $\widetilde{\mathcal{G}}$ is defined by such a triple $\delta = (u, v; w)$. The set $\{u, v\}$ is called the source of the collision whereas w is called the head of the collision. We also say the edges e_1 and e_2 are colliding edges. Thus, a collision $\delta = (u, v; z)$ induces a linear restriction $L_{\delta} : \gamma(\beta_u) \oplus \gamma(\beta_v) = c_{\delta}$, where $c_{\delta} = a \oplus b \in \mathbb{B}$. We denote the set of all collisions in $\widetilde{\mathcal{G}}$ by $\Delta(\widetilde{\mathcal{G}})$. Let $\text{rank}(\widetilde{\mathcal{G}})$ denote the rank of the system of linear equations $L(\widetilde{\mathcal{G}}) := \{L_{\delta} : \delta \in \Delta(\widetilde{\mathcal{G}})\}$.

Definition 4.1.9 (Accident of an INS graph). The accident of an INS graph $\widetilde{\mathcal{G}}$ is defined as $\text{Acc}(\widetilde{\mathcal{G}}) := \text{rank}(\widetilde{\mathcal{G}})$.

Now, we mention some important results on INS graphs which will be useful in our analysis ahead. These results have already been proved in [19, 163] and chapter 3 for the random permutation case. We prove these results for the random function case in section 4.4.

Lemma 4.1.10. For any INS graph $\widetilde{\mathcal{G}}$ with a accidents,

$$\Pr \left[\text{Struct}^{\Gamma} = \widetilde{\mathcal{G}} \right] \leq \frac{1}{2^{na}}.$$

Lemma 4.1.11. The number of INS graphs with a accidents associated to $M^q = (M_1, \dots, M_q)$ is at most $\binom{m}{2}^a$, where $\sum_{i=1}^q m_i = m$.

Corollary 4.1.12. *Let $a \geq 1$ be an integer. Then,*

$$\Pr \left[\text{Acc}(\text{Struct}^\Gamma) \geq a \right] \leq \frac{m^{2a}}{2^{na}}.$$

4.2 Proof of Theorem 4.1.4 [Upper Bound Theorem]

In the previous chapter, we saw that Ocoll is a structure graph event. Similarly, one can show that Icoll is a structure graph event as well. For a fixed tuple of message M^q , Icoll is said to be true if there exists some pair of walks W_i and W_j (corresponding to some $M_i, M_j \in M^q$) in $\text{Struct}^\Gamma(M^q)$ such that $W_{(i,m_i)} = W_{(j,m_j)}$.

Let $\text{Struct}_a(M^q) \subseteq \text{Struct}(M^q)$ denote the set of structure graphs with a accidents for M^q . By extending the notation we write $\text{Struct}_a(M^q)[\text{Icoll}]$ to denote the subset of graphs which additionally satisfy Icoll . We follow the approach of section 3.5.2 to bound $\text{Struct}_a(M^q)[\text{Icoll}]$.

Let us define the event Bad as,

1. for a pair of messages M_i, M_j , $\text{Acc}(\text{Struct}^\Gamma(M_i, M_j)) \geq 2$, or
2. for any message M_i , $\text{Acc}(\text{Struct}^\Gamma(M_i)) \geq 1$.

We aim to bound the inCP in terms of $\Pr[\text{Bad}]$ and $\Pr[\neg \text{Bad}]$. Specifically, we have

$$\text{inCP}_{q,\ell,\sigma} \leq \Pr \left[\text{Icoll}^\Gamma(M^q) \cap \neg \text{Bad} \right] + \Pr[\text{Bad}].$$

So we just need to upper bound the following:

1. Bounding $\Pr[\text{Bad}]$: We use Corollary 4.1.12 to bound the probability of Bad . The probability of first condition is bounded by $\sum_{i < j \in [q]} \frac{(m_i + m_j)^4}{2^{2n}}$ (for all $\binom{q}{2}$ pairs of messages), and the probability of second conditions is bounded by $\sum_{i \in [q]} \frac{m_i^2}{2^n}$ (for all q messages). Finally we have,

$$\begin{aligned} \Pr[\text{Bad}] &\leq \sum_{i < j \in [q]} \frac{(m_i + m_j)^4}{2^{2n}} + \sum_{i \in [q]} \frac{m_i^2}{2^n} \\ &\leq \sum_{i < j \in [q]} \frac{(m_i + m_j) \cdot 8\ell^3}{2^{2n}} + \sum_{i \in [q]} \frac{m_i \cdot \ell}{2^n} \\ &\leq \frac{8\sigma q \ell^3}{2^{2n}} + \frac{\sigma \ell}{2^n}. \end{aligned}$$

2. Bounding $\Pr \left[\text{Icoll}^\Gamma(M^q) \cap \neg \text{Bad} \right]$: $\neg \text{Bad}$ implies that $\text{Acc}(W_i) = 0$ for all $i \in [q]$, or in other words W_i is acyclic. For any pair of messages M_i and M_j , we bound the set $|\text{Struct}_1[\text{Icoll} \wedge \neg \text{Bad}]|$ to at most $\min\{m_i, m_j\}$ (see the following claim), whence we bound the probability to at most $\frac{\min\{m_i, m_j\}}{2^n}$. Hence for q messages we have

$$\Pr \left[\text{Icoll}^\Gamma(M^q) \cap \neg \text{Bad} \right] \leq \sum_{i < j \in [q]} \frac{\min\{m_i, m_j\}}{2^n} \leq \frac{\sigma q}{2^n}.$$

Combining 1 and 2, we have the desired result.

Claim 4.2.1. For any pair of messages $M_1, M_2 \in M^q$, we have

$$|\text{Struct}_1[\text{Icoll} \wedge \neg \text{Bad}]| \leq \min\{m_1, m_2\}.$$

Proof. We prove the claim in two cases:

Case A: M_1 is a prefix of M_2 : In this case M_1 must be a strict prefix of M_2 as M_1 and M_2 are distinct. Further W_1 is a subwalk of W_2 and we have $W_{1,i} = W_{2,i}$ for $i \in (m_1]$. The Icoll event is equivalent to $W_{1,m_1} = W_{2,m_2}$, or $W_{2,m_2} = W_{2,m_1}$. Thus, W_2 must contain a cycle which is not possible. So, $|\text{Struct}_1[\text{Icoll} \wedge \neg \text{Bad}]| = 0$.

Case B: M_1 is not a prefix of M_2 : WLOG assume that $m_1 < m_2$. In this case we must have $p = \text{lcp}(M_1, M_2) < m_1$. $\neg \text{Bad}$ implies that W_1 and W_2 are paths and Icoll implies that $W_{1,m_1} = W_{2,m_2}$. To get $W_{1,m_1} = W_{2,m_2}$ we must have an accident $(W_{1,i}, W_{2,j}; W_{1,i+1})$ for some $p+1 \leq i \leq m_1$ and $j = m_2 - m_1 + i$. Therefore, summing over all values of i we have, $|\text{Struct}_1(M_1, M_2)[\text{Icoll} \wedge \neg \text{Bad}]| \leq m_1 \leq \min\{m_1, m_2\}$.

The result follows from case 1 and 2. □

4.3 Proof of Theorem 4.1.5 [Lower Bound Theorem]

Let $M^q := (M_1, \dots, M_q)$ be a q -tuple of messages such that for $i, j \in [q]$ $M_i = (X_i, 0, \dots, 0) \in \mathcal{B}^\ell$ and $X_i \neq X_j \in \mathcal{B}$. We want to find a lower bound of collision probability that is,

$$\begin{aligned} \text{inCP}(M^q) &= \Pr \left[\text{Icoll}^\Gamma(M^q) \right] = \Pr \left[\bigcup_{1 \leq i < j \leq q} \text{Icoll}^\Gamma(M_i; M_j) \right] \\ &\geq \sum_{i < j} \Pr \left[\text{Icoll}^\Gamma(M_i, M_j) \right] \end{aligned}$$

$$\begin{aligned}
& - 3 \sum_{i < j < k} \Pr \left[\text{Icoll}^\Gamma(M_i, M_j) \wedge \text{Icoll}^\Gamma(M_j, M_k) \right] \\
& - \frac{1}{2} \sum_{\substack{i < j, k < m \\ \{i, j\} \cap \{k, m\} = \emptyset}} \Pr \left[\text{Icoll}^\Gamma(M_i, M_j) \wedge \text{Icoll}^\Gamma(M_k, M_m) \right] \quad (4.13)
\end{aligned}$$

where the inequality follows from Principle of Inclusion-Exclusion and Bonferroni inequality (for details see section A.2.1 of appendix A). We have to compute the following bounds,

- Upper bound for $\Pr [\text{Icoll}^\Gamma(M_i; M_j) \wedge \text{Icoll}^\Gamma(M_j; M_k)]$, where i, j and k are distinct.
- Upper bound for $\Pr [\text{Icoll}^\Gamma(M_i; M_j) \wedge \text{Icoll}^\Gamma(M_k; M_m)]$, where i, j, k and m are distinct.
- Lower bound for $\Pr [\text{Icoll}^\Gamma(M_i; M_j)]$, where i and j are distinct.

We use structure graphs to bound the above mentioned probabilities. Before moving forward, we define a special graph which will be encountered frequently in this section.

Definition 4.3.1 (*t*-unicycle). A *t*-unicycle is a connected directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where $\forall v \in \mathcal{V}, \text{deg}_{\text{out}}(v) \leq 1 \vee \text{deg}_{\text{in}}(v) = 0$, and $\mathcal{E}$ is a union of 1 cycle $\mathcal{C}$ and t distinct paths $\mathcal{P}_1, \dots, \mathcal{P}_t$ where exactly one endpoint of $\mathcal{P}_i$ is a vertex of $\mathcal{C}$ and the other endpoint must have zero in-degree.

The paths $\mathcal{P}_i$ may not be disjoint. Example 4.2 describes a 3-unicycle.

Observe that due to our choice of messages, we have the following property on $\text{Struct}^\Gamma(M^q)$:

$$\forall v \in \mathcal{V}(\text{Struct}^\Gamma) \setminus \{(1, 0)\}, \text{deg}_{\text{out}}(v) \leq 1.$$

This is obvious as all the edge labels (except those involving $(1, 0)$) are identical (0^n). Therefore, Struct^Γ is either a union of paths or a union of unicycles or both. Note that, in either case the graph has no dependent collisions with distinct heads (as that requires $\text{deg}_{\text{out}}(v) \geq 2$ for some v). Further, for a vertex with in-degree $a \geq 1$, we have $a - 1$ independent collisions or accidents.

Remark 4.3.2. We summarize some useful properties derived from the above discussion:

1. A union of k paths has at most $k - 1$ collisions. So the number of accidents is $k - 1$.
2. A k -unicycle has exactly k collisions. So the number of accidents is k .

3. A union of a k_1 -unicycle and k_2 paths has at most $k_1 + k_2 - 1$ accidents.
4. A union of a k_1 -unicycle and a k_2 -unicycle has exactly $k_1 + k_2$ accidents.
5. In general k distinct walks (where each vertex v has $\deg_{\text{out}}(v) \leq 1$), can have at most k accidents.

Example 4.2. In figure 4.3.1,

1. $\mathcal{G}$ is a 3-unicycle with $\text{Acc}(\mathcal{G}) = 3$.
2. $\mathcal{G}'$ is a union of 3 paths with $\text{Acc}(\mathcal{G}') = 1$.

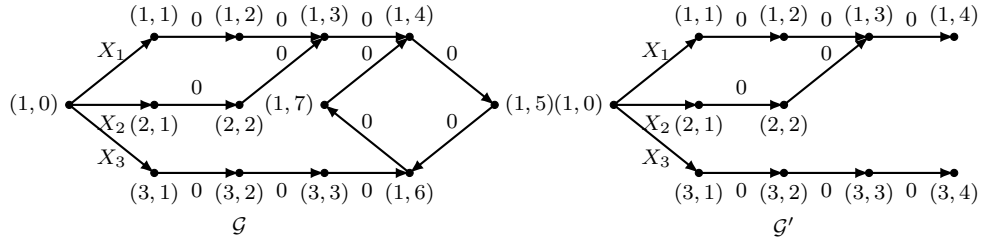


Figure 4.3.1: Unicycles and union of paths.

We bound the above mentioned probabilities in Lemma 4.3.3, 4.3.4, and 4.3.5. The proofs are postponed to section 4.3.1.

Lemma 4.3.3.

$$\Pr \left[\text{Icoll}^\Gamma(M_i; M_j) \wedge \text{Icoll}^\Gamma(M_j; M_k) \right] \leq \frac{2\ell^2}{2^{2n}} + \frac{6\ell^6}{2^{3n}}.$$

Lemma 4.3.4.

$$\Pr \left[\text{Icoll}^\Gamma(M_i; M_j) \wedge \text{Icoll}^\Gamma(M_k; M_m) \right] \leq \frac{\ell^2}{2^{2n}} + \frac{6\ell^3 + 2\ell^5}{2^{3n}} + \frac{28\ell^8}{2^{4n}}.$$

Lemma 4.3.5.

$$\Pr \left[\text{Icoll}^\Gamma(M_i; M_j) \right] \geq \frac{\ell}{2^n} e^{-\frac{4\ell^2}{2^n}}.$$

Combining Equation 4.13 with Lemma 4.3.3, 4.3.4 and 4.3.5 we have,

$$\begin{aligned} \text{inCP}(M^q) &\geq \binom{q}{2} \frac{\ell - 1}{2^n} e^{-\frac{4\ell^2}{2^n}} - 3 \binom{q}{3} \left(\frac{2\ell^2}{2^{2n}} + \frac{6\ell^6}{2^{3n}} \right) \\ &\quad - \frac{1}{2} \binom{q}{2} \binom{q-2}{2} \left(\frac{\ell^2}{2^{2n}} + \frac{6\ell^3 + 2\ell^5}{2^{3n}} + \frac{28\ell^8}{2^{4n}} \right). \end{aligned}$$

If $\ell, q \geq 3$, $\frac{q^2\ell}{2^n} < 1$ and $\ell < \min\{\frac{2^n}{5184}, \frac{2^{n/2}}{4\sqrt{3}}, \frac{2^{n/3}}{\sqrt[3]{36}}\}$, then using the inequality $e^{-x} \geq 1 - x$ and some algebraic manipulations one can show

$$\text{inCP}(M^q) \geq \frac{q^2\ell}{122^n}.$$

Further for $\ell = \sigma/q$ this bound becomes $\frac{q\sigma}{122^n}$. $\square$

4.3.1 Proofs Related to the Lower Bound Theorem

4.3.1.1 Proof of Lemma 4.3.3

Let $\text{Icoll}^{i,j,k}$ denote the event $\text{Icoll}^\Gamma(M_i, M_j) \wedge \text{Icoll}^\Gamma(M_j, M_k)$. From remark 4.3.2 we know that the number of accidents must be ≤ 3 .

1. Observe that $\text{Icoll}^{i,j,k}$ requires at least 2 independent collisions (accidents) (in W_i and W_j paths and W_k and $W_i \cup W_j$), so

$$|\text{Struct}_1[\text{Icoll}^{i,j,k}]| = 0.$$

2. Accident 2 graphs are possible only for union of paths, as the number of accidents correspond to the collision between the three paths. There are at most ℓ many choices for collision between W_i and W_j paths, and at most 2ℓ many choices for collision between W_k and $W_i \cup W_j$. This bounds

$$|\text{Struct}_2[\text{Icoll}^{i,j,k}]| \leq 2\ell^2.$$

3. The graph can have 3 accidents iff it is a 3-unicycle. Suppose the cycle is in W_i . Then W_i is determined by the length of cycle and the distance of $W_{i,0}$ from the cycle which gives ℓ^2 choices for W_i . For each such choice we have at most $2\ell^2$ many choices for $W_i \cup W_j$, and at most $3\ell^2$ many choices for $W_i \cup W_j \cup W_k$. This bounds

$$|\text{Struct}_3[\text{Icoll}^{i,j,k}]| \leq 6\ell^6.$$

The result follows by direct application of Lemma 4.1.10. $\square$

4.3.1.2 Proof of Lemma 4.3.4

Let $\text{Icoll}^{i,j,k,m}$ denote the event $\text{Icoll}^\Gamma(M_i, M_j) \wedge \text{Icoll}^\Gamma(M_k, M_m)$. From remark 4.3.2 we know that the number of accidents must be ≤ 4 . We bound the four sets corresponding to the number of accidents as below:

1. Accident 1 graphs are not possible. Hence,

$$|\text{Struct}_1[\text{Icoll}^{i,j,k,m}]| = 0.$$

2. The accident 2 graphs are possible if and only if $(W_i \cup W_j) \cap (W_k \cup W_m) = \{(1, 0)\}$, where the two accidents correspond to the collision between W_i and W_j , and collision between W_k and W_m . Now there are at most ℓ many choices for collision between W_i and W_j paths, and similarly at most ℓ many choices for collision between W_k and W_m paths. This gives

$$|\text{Struct}_2[\text{Icoll}^{i,j,k,m}]| \leq \ell^2.$$

3. Accident 3 graphs are possible if and only if,

- (a) W_i, W_j, W_k, W_m paths collide and the graph is a union of paths. In this case we have at most ℓ many choices for collision between W_i and W_j paths. Similarly we have at most 2ℓ and 3ℓ many choices for collision between W_k and $W_i \cup W_j$, and W_m and $W_i \cup W_j \cup W_m$ respectively. This gives

$$|\text{Struct}_3[\text{Icoll}^{i,j,k,m}]| \leq 6\ell^3.$$

- (b) $(W_i \cup W_j) \cap (W_k \cup W_m) = \{(1, 0)\}$ and $W_i \cup W_j$ is a 2-unicycle and $W_k \cup W_m$ is a union of paths or vice versa. Without loss of generality assume that $W_i \cup W_j$ is a 2-unicycle. Then there exist a cycle in $W_i \cup W_j$. Suppose the cycle is in W_i . Then W_i is determined by the length of cycle and the distance of $W_{i,0}$ from the cycle which gives ℓ^2 choices for W_i . For each such choice we have at most $2\ell^2$ many choices for $W_i \cup W_j$. And there are at most ℓ many choices for collision between W_k and W_m . This gives,

$$|\text{Struct}_3[\text{Icoll}^{i,j,k,m}]| \leq 2\ell^5.$$

Combining the two subcases we have,

$$|\text{Struct}_3[\text{Icoll}^{i,j,k,m}]| \leq 6\ell^3 + 2\ell^5.$$

4. Accident 4 graphs are possible if and only if,

- (a) $W_i \cup W_j$ and $W_k \cup W_m$ are distinct 2-unicycles. Using similar arguments as used in the previous cases we get a bound of $4\ell^8$.
- (b) W_i, W_j, W_k, W_m form a 4-unicycle. This case can be bounded to $24\ell^8$, using similar approach as used in the previous cases.

Combining the two subcases we have,

$$|\text{Struct}_4[\text{Icoll}^{i,j,k,m}]| \leq 28\ell^8.$$

The result follows by direct application of Lemma 4.1.10. $\square$

4.3.1.3 Proof of Lemma 4.3.5

Let $\text{Icoll}^{i,j}$ denote the event $\text{Icoll}^\Gamma(M_i; M_j)$. We are basically interested in the probability that $\text{Struct}^\Gamma \in \text{Struct}(M_i, M_j)[\text{Icoll}]$. Let Acyclic denote the property that some graph $\mathcal{G} \in \text{Struct}(M_i, M_j)[\text{Icoll}]$ is acyclic graph, and $\text{Struct}(M_i, M_j)[\text{Icoll} \wedge \text{Acyclic}]$ denote the subset of graphs which satisfy both Icoll and Acyclic . Thus, we have

$$\Pr \left[\text{Struct}^\Gamma \in \text{Struct}(M_i, M_j)[\text{Icoll}] \right] \geq \Pr \left[\text{Struct}^\Gamma \in \text{Struct}(M_i, M_j)[\text{Icoll} \wedge \text{Acyclic}] \right] \quad (4.14)$$

We will lower bound $\Pr \left[\text{Struct}^\Gamma \in \text{Struct}(M_i, M_j)[\text{Icoll} \wedge \text{Acyclic}] \right]$. First, convince yourself that for all $\mathcal{G} \in \text{Struct}(M_i, M_j)[\text{Icoll} \wedge \text{Acyclic}]$, we must have $\text{Acc}(\mathcal{G}) = 1$ (as M_i and M_j share a common suffix of length $\ell - 1$). Now, this accident can happen at any one of the index $i \in [2 \dots \ell]$, each contributing exactly one structure graph.

Now fix an index $i \in [2 \dots \ell]$ where the accident occurs and let the corresponding INS graph be $\mathcal{G}_i$. Then, a simple counting shows that the number of valid block labeling for $\mathcal{G}_i$ is exactly $(2^n - 2) \dots (2^n - 2i + 2)$. Each such labeling gives a BINS graph $\mathcal{S}$ with exactly $2i - 2$ positive out-degree vertices (excluding $\perp$ which is trivial) such that $\alpha(\mathcal{S}) = \mathcal{G}$. The probability of getting a BINS graph with $2i - 2$ many vertices having positive out-degree is equal to 2^{2n-2in} (as exactly these many outputs of Γ are fixed). Thus, we get

$$\begin{aligned} \Pr \left[\text{Struct}^\Gamma \in \text{Struct}(M_i, M_j)[\text{Icoll} \wedge \text{Acyclic}] \right] &= \frac{1}{2^n} \sum_{i=2}^{\ell} \left(1 - \frac{2}{2^n} \right) \cdots \left(1 - \frac{2i-2}{2^n} \right) \\ &\geq \frac{\ell-1}{2^n} \prod_{i=1}^{2\ell-2} \left(1 - \frac{i}{2^n} \right) \\ &\geq \frac{\ell-1}{2^n} \left(1 - \frac{2\ell^2}{2^n} \right) \\ &\geq e^{-\frac{4\ell^2}{2^n}} \frac{\ell-1}{2^n}, \end{aligned} \quad (4.15)$$

where the last inequality follows from $(1 - x) \geq e^{-2x}$ for $0 < x < 0.5$, and the assumption that $\ell < 2^{\frac{n}{2}-1}$. The result follows from Eq. (4.14) and (4.15). $\square$

4.4 Proofs of Results on Structure Graph

Proof of Lemma 4.1.10: The proof is similar to the proof of Lemma 3.2.6. $\tilde{\mathcal{G}}$ is an INS graph with a accidents, i.e., $\text{rank}(\tilde{\mathcal{G}}) = a$. Let s denote the number of vertices, excluding $(1, 0)$, in $\tilde{\mathcal{G}}$ with positive out-degree. Using linear algebra, we know that some $s - a$ choices of $\gamma(\beta_i)$ values will uniquely determine the rest of the block labels and so the number of valid block labeling is at most 2^{ns-na} . In fact, the number of block labeling should be much less due to the added restriction of distinctness. Any valid choice of β induces a block-vertex structure graph $\mathcal{S}(\mathcal{V}, \mathcal{E})$ such that $\alpha(\mathcal{S}) = \tilde{\mathcal{G}}$. Note that, s is the number of vertices $v \in \mathcal{V}$ with positive out-degree. So exactly $(2^n)^{2^n-s}$ number of functions can result in BINS graph $\mathcal{S}$. Therefore,

$$\Pr [\text{BStruct}^\Gamma = \mathcal{S}] = \frac{1}{2^{ns}}. \quad (4.16)$$

The result follows by summing over all possible BINS graphs $\mathcal{S}$ such that the INS graph $\alpha(\mathcal{S}) = \tilde{\mathcal{G}}$. $\square$

For an INS graph $\tilde{\mathcal{G}}$ we define *traversal* $\mathcal{T}(\tilde{\mathcal{G}})$ as the sequence of vertices $\mathcal{T}(\tilde{\mathcal{G}}) := (\tilde{W}_{r,i})_{(r,i) \in \mathcal{I}}$. Note that, $\mathcal{T}(\tilde{\mathcal{G}})$ implicitly stores the edges: for every $\omega \in \mathcal{I}$ such that $\omega \neq (r, m_r)$ we have $(\tilde{W}_\omega, \tilde{W}_{\omega+1}) \in \mathcal{E}$ with label $M_{\omega+1}$. We denote the set of edges in $\mathcal{T}(\tilde{\mathcal{G}})$ by $\mathcal{E}(\mathcal{T}(\tilde{\mathcal{G}}))$. The partial traversal $\mathcal{T}_\omega(\tilde{\mathcal{G}})$ of $\mathcal{T}(\tilde{\mathcal{G}})$ is defined as the subsequence till $\omega \in \mathcal{I}$ (using $\prec$ as the ordering). In $\mathcal{T}(\tilde{\mathcal{G}})$, a collision $\delta := (u, v; w)$ can be equivalently written as,

$$\delta := (\tilde{W}_i = u, \tilde{W}_j = v; \tilde{W}_{i+1} = \tilde{W}_{j+1} = w), \quad i \prec j \in \mathcal{I},$$

where i and j are the smallest such indices, and recall that $i+1$ and $j+1$ denote the immediate successor of i and j , respectively. Under this equivalent representation we can define a partial order $\prec_\Delta$ on $\Delta(\tilde{\mathcal{G}})$ as follows:

For $i, j, i', j' \in \mathcal{I}$, $i \prec j$, and $i' \prec j'$, let $\delta = (\tilde{W}_i, \tilde{W}_j; \tilde{W}_{i+1})$ and $\delta' = (\tilde{W}_{i'}, \tilde{W}_{j'}; \tilde{W}_{i'+1})$. $\delta \prec_\Delta \delta'$ if either,

1. $j \prec j'$, or
2. $j = j'$ and $i \prec i'$.

Proposition 4.4.1. Let $\tilde{\mathcal{G}}_1, \tilde{\mathcal{G}}_2 \in \text{Struct}(M^q)$ be two INS graphs and $\mathcal{T}(\tilde{\mathcal{G}}_1)$ and $\mathcal{T}(\tilde{\mathcal{G}}_2)$ be their associated traversals. Then,

$$\forall \omega \in \mathcal{I}, \mathcal{T}_\omega(\tilde{\mathcal{G}}_1) = \mathcal{T}_\omega(\tilde{\mathcal{G}}_2) \iff \tilde{\mathcal{G}}_1^{\mathcal{E}_\omega} = \tilde{\mathcal{G}}_2^{\mathcal{E}_\omega},$$

where $\tilde{\mathcal{G}}_i^{\mathcal{E}_\omega}$ is the edge induced subgraph of $\tilde{\mathcal{G}}_i$ with edge set $\mathcal{E}(\mathcal{T}_\omega(\tilde{\mathcal{G}}_i))$. Particularly for $\omega = (q, m_q)$ we have,

$$\mathcal{T}(\tilde{\mathcal{G}}_1) = \mathcal{T}(\tilde{\mathcal{G}}_2) \iff \tilde{\mathcal{G}}_1 = \tilde{\mathcal{G}}_2.$$

Proof. The necessary condition, i.e.,

$$\tilde{\mathcal{G}}_1^{\mathcal{E}_\omega} = \tilde{\mathcal{G}}_2^{\mathcal{E}_\omega} \implies \mathcal{T}_\omega(\tilde{\mathcal{G}}_1) = \mathcal{T}_\omega(\tilde{\mathcal{G}}_2)$$

is trivially true (by definition of traversals). So we focus on the sufficient condition, i.e.,

$$\mathcal{T}_\omega(\tilde{\mathcal{G}}_1) = \mathcal{T}_\omega(\tilde{\mathcal{G}}_2) \implies \tilde{\mathcal{G}}_1^{\mathcal{E}_\omega} = \tilde{\mathcal{G}}_2^{\mathcal{E}_\omega}.$$

Note that, $\mathcal{T}_\omega(\tilde{\mathcal{G}}_1) = \mathcal{T}_\omega(\tilde{\mathcal{G}}_2) \implies \mathcal{E}(\mathcal{T}_\omega(\tilde{\mathcal{G}}_1)) = \mathcal{E}(\mathcal{T}_\omega(\tilde{\mathcal{G}}_2))$. As the two edge sets are equal, the edge induced subgraphs must also be equal. Thus $\forall \omega \in \mathcal{I}$, $\tilde{\mathcal{G}}_1^{\mathcal{E}_\omega} = \tilde{\mathcal{G}}_2^{\mathcal{E}_\omega}$. $\square$

Definition 4.4.2 (Accident Basis and Dependent Collisions). We define the accident basis $\Delta^{\text{Acc}}(\tilde{\mathcal{G}})$ of the INS graph $\tilde{\mathcal{G}}$ as $\mathcal{C} \subseteq \Delta(\tilde{\mathcal{G}})$ such that $\{L_\delta : \delta \in \mathcal{C}\}$ is the minimal spanning set of $L(\tilde{\mathcal{G}})$ and the elements of $\mathcal{C}$ are smallest with respect to $\prec_\Delta$. Set $\mathcal{D} \subset \Delta(\tilde{\mathcal{G}})$ is called a set of dependent collisions if $L(\mathcal{D})$ is linearly dependent. Note that, $\text{Acc}(\tilde{\mathcal{G}}) = |\Delta^{\text{Acc}}(\tilde{\mathcal{G}})|$ as $\Delta^{\text{Acc}}(\tilde{\mathcal{G}})$ is a basis of $L(\tilde{\mathcal{G}})$.

It is obvious that $\Delta^{\text{Acc}}(\tilde{\mathcal{G}}^{\mathcal{E}_\omega})$ the accident basis corresponding to the edge induced subgraph $\tilde{\mathcal{G}}^{\mathcal{E}_\omega}$ (equivalently to the partial traversal $\mathcal{T}_\omega(\tilde{\mathcal{G}})$) is a subset of $\Delta^{\text{Acc}}(\tilde{\mathcal{G}})$.

Example 4.3. Recall example 4.1 and the corresponding INS graph, say $\tilde{\mathcal{G}}$, in Figure 4.1.1(b). For $\tilde{\mathcal{G}}$, we have

1. $\mathcal{T}(\tilde{\mathcal{G}}) = (\tilde{W}_{(1,0)}, \tilde{W}_{(1,1)}, \tilde{W}_{(1,2)}, \tilde{W}_{(1,3)}, \tilde{W}_{(1,4)}, \tilde{W}_{(1,5)}, \tilde{W}_{(2,0)}, \tilde{W}_{(2,1)})$, where $\tilde{W}_{(1,0)} = (1, 0)$, $\tilde{W}_{(1,1)} = (1, 1)$, $\tilde{W}_{(1,2)} = (1, 2)$, $\tilde{W}_{(1,3)} = \tilde{W}_{(1,1)}$, $\tilde{W}_{(1,4)} = \tilde{W}_{(1,2)}$, $\tilde{W}_{(1,5)} = (1, 5)$, $\tilde{W}_{(2,0)} = \tilde{W}_{(1,0)}$, $\tilde{W}_{(2,1)} = \tilde{W}_{(1,5)}$.
2. $\Delta(\tilde{\mathcal{G}}) = \{((1, 0), (1, 2); (1, 1)), ((1, 2), (1, 0); (1, 5))\}$. The equivalent representation in $\mathcal{T}(\tilde{\mathcal{G}})$ is $\{(\tilde{W}_{(1,0)}, \tilde{W}_{(1,2)}; \tilde{W}_{(1,1)} = \tilde{W}_{(1,3)}), (\tilde{W}_{(1,4)}, \tilde{W}_{(2,0)}; \tilde{W}_{(1,5)} = \tilde{W}_{(2,1)})\}$.
3. $L(\tilde{\mathcal{G}}) = \{\gamma(\beta_{(1,0)}) \oplus \gamma(\beta_{(1,2)}) = 1 \oplus 2; \gamma(\beta_{(1,0)}) \oplus \gamma(\beta_{(1,2)}) = 7 \oplus 4\}$.
4. Clearly, $\text{Acc}(\tilde{\mathcal{G}}) = \text{rank}(\tilde{\mathcal{G}}) = 1$.
5. $\Delta^{\text{Acc}}(\tilde{\mathcal{G}}) = \{((1, 0), (1, 2); (1, 1))\}$.

Proof of Lemma 4.1.11: The proof becomes trivial once we show that each structure graph has a unique accident basis and distinct graphs have distinct accident basis. In other words, we need to show that the mapping from the set of structure graphs to the set of accident basis is injective. It is easy to see that each structure graph has a unique accident basis (by the definition of accident basis). We now show that distinct structure graphs have distinct accident basis.

Claim 4.4.3. Let $\tilde{\mathcal{G}}_1$ and $\tilde{\mathcal{G}}_2$ be two structure graphs. Then,

$$\Delta^{\text{Acc}}(\tilde{\mathcal{G}}_1) = \Delta^{\text{Acc}}(\tilde{\mathcal{G}}_2) \implies \tilde{\mathcal{G}}_1 = \tilde{\mathcal{G}}_2.$$

Using Proposition 4.4.1 it is sufficient to show that

$$\Delta^{\text{Acc}}(\tilde{\mathcal{G}}_1) = \Delta^{\text{Acc}}(\tilde{\mathcal{G}}_2) \implies \mathcal{T}(\tilde{\mathcal{G}}_1) = \mathcal{T}(\tilde{\mathcal{G}}_2).$$

We prove the claim by induction on the dictionary order over the index set $\mathcal{I}$. Let $\omega \in \mathcal{I}$. Suppose $\mathcal{T}_\beta(\tilde{\mathcal{G}}_1) = \mathcal{T}_\beta(\tilde{\mathcal{G}}_2)$ for all $\beta \prec \omega$. If $\omega = (r, m_r)$ for some $r \in [q]$, then the next vertex on $\mathcal{T}(\tilde{\mathcal{G}}_1)$, i.e., $\tilde{W}_{\omega+1}^1 = (1, 0) = \tilde{W}_{\omega+1}^2$, the next vertex on $\mathcal{T}(\tilde{\mathcal{G}}_2)$. Note that, the superscript $b \in \{1, 2\}$ is used to distinguish the vertices of $\tilde{\mathcal{G}}_1$ and $\tilde{\mathcal{G}}_2$. Thus, $\mathcal{T}_\omega(\tilde{\mathcal{G}}_1) = \mathcal{T}_\omega(\tilde{\mathcal{G}}_2)$. Suppose $\omega = (r, i)$ for some $r \in [q]$ and $i \in [m_r - 1]$. We show that the next edge in $\mathcal{T}(\tilde{\mathcal{G}}_2)$, i.e., $e_2 := (\tilde{W}_\omega^2, \tilde{W}_{\omega+1}^2)$ is same as $e_1 := (\tilde{W}_\omega^1, \tilde{W}_{\omega+1}^1)$, the next edge in $\mathcal{T}(\tilde{\mathcal{G}}_1)$. The next edge can lead to one of the following cases:

1. Suppose e_1 leads to a dependent collision δ in $\tilde{\mathcal{G}}_1$. Then, the corresponding linear restriction L_δ must be spanned by $\Delta^{\text{Acc}}(\tilde{\mathcal{G}}_1^{\mathcal{E}_\omega})$. Now $\mathcal{T}_\omega(\tilde{\mathcal{G}}_1) = \mathcal{T}_\omega(\tilde{\mathcal{G}}_2) \implies \Delta^{\text{Acc}}(\tilde{\mathcal{G}}_1^{\mathcal{E}_\omega}) = \Delta^{\text{Acc}}(\tilde{\mathcal{G}}_2^{\mathcal{E}_\omega})$. So L_δ is also spanned by $\Delta^{\text{Acc}}(\tilde{\mathcal{G}}_2^{\mathcal{E}_\omega})$. As the message label is same, we must have $e_2 = e_1$.
2. Suppose e_1 leads to a new accident δ in $\tilde{\mathcal{G}}_1$. As $\Delta^{\text{Acc}}(\tilde{\mathcal{G}}_1) = \Delta^{\text{Acc}}(\tilde{\mathcal{G}}_2)$, $\delta \in \Delta^{\text{Acc}}(\tilde{\mathcal{G}}_2)$. Thus e_2 also leads to same accident δ in $\tilde{\mathcal{G}}_2$. Thus $e_1 = e_2$.
3. Suppose e_1 leads to a new vertex, i.e., $\tilde{W}_{\omega+1}^1 \notin \mathcal{T}_\omega(\tilde{\mathcal{G}}_1)$. As the labels of both e_1 and e_2 are same, e_2 must also lead to a new vertex. Then using the definition of INS graph we have $e_1 = e_2$.

In all three cases we have $e_1 = e_2$, i.e., $\mathcal{T}_\omega(\tilde{\mathcal{G}}_1) = \mathcal{T}_\omega(\tilde{\mathcal{G}}_2)$. This proves the claim. So $|\text{Struct}(M^q)|$ is at most equal to the number of distinct accident basis of size a . Note that, we can have at most m vertices in the graph. So the number of distinct accident basis is at most $\binom{m}{2}^a$. The result follows. $\square$

Corollary 4.1.12 can be easily obtained by combining Lemma 4.1.10 and 4.1.11 followed by some algebraic simplifications.

Chapter 5

Counter-based Input Encoding in MACs

In CRYPTO '95, Bellare et al. [16] proposed a method to construct non-deterministic MACs. The so-called XOR-MAC consisted of a stateful (nonce-based) MAC, XMACC, and a probabilistic MAC, XMACR. At high level, XOR-MAC consists of three sequential steps:

1. Counter-based encoding of input message and parsing as a sequence of encoded blocks;
2. Application of a pseudorandom function (or PRF) F to each of the encoded blocks followed by the accumulation of outputs into a hash value, say h , using simple XOR; and
3. XORing h with $F'(N)$, where F' is a PRF independent of F , and N is either a nonce (non-repeating internal state) or a random salt.

The original security proof of XOR-MAC is based on the assumption that F and F' are (pseudo)random functions. Later, Bernstein [24] provided an improved analysis when F and F' are (pseudo)random permutations. It can be easily observed that XOR-MAC can be viewed as an instance of Wegman-Carter authenticators or Hash-then-Mask (see section 2.4.2.3), where steps 1 and 2 constitute the universal hashing (see Figure 5.0.1), and step 3 is the finalization. Later, Bernstein proposed a Hash-then-PRF variant for XOR-MAC, called *protected counter sum* (or PCS) [23], which applies a PRF F' on the hash value h in step 3 and returns the output as tag. LightMAC by Luykx et al. [126] is the Hash-then-PRP variant of XOR-MAC, where both F and F' are PRPs. We group all these schemes under a common family, called the XMAC family. All the MAC

schemes in the XMAC family are instances of Hash-then- $\{\text{Mask/PRP/PRF}\}$. Naturally, the security of XMAC family mainly relies on the universal property of h which in turn depends on the counter-based encoding of inputs. In this chapter, we formalize the notion of counters in symmetric-key settings, and characterize the properties required from them to guarantee security (under reasonable assumptions) for a large class of symmetric-key schemes, including the XMAC family.

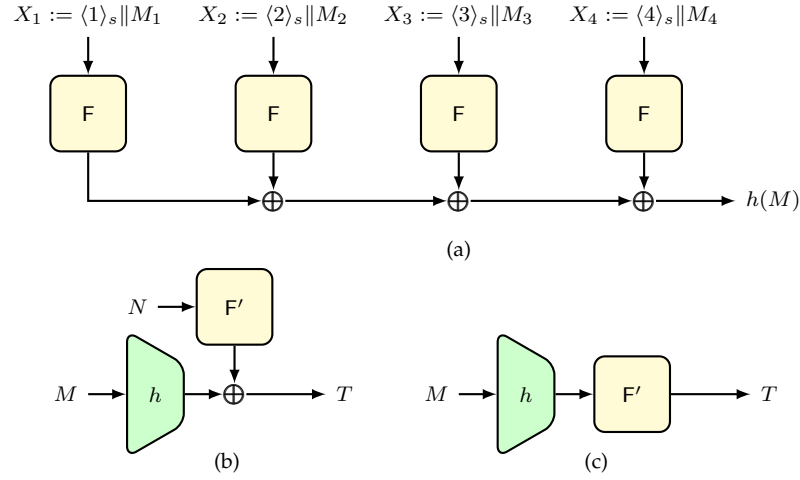


Figure 5.0.1: The XMAC family: (a) hash value computation; (b) finalization used in non-deterministic MACs, i.e., XMCCR and XMACR; (c) finalization used in deterministic MACs, i.e., PCS and LightMAC. Here F denotes a PRF (or PRP).

5.1 Inefficiency of Classical Counters

Counter-based schemes can be classified into two categories: (1) *Counter-as-Input* or CaI, where the counter values are used as a standalone input to the underlying primitive; and (2) *Counter-as-Encoding* or CaE, where the counter values are encoded along with data bits within the input to the underlying primitive. Schemes based on CTR mode [153], such as GCM [130] and SIV [172], fall under the former category, whereas the XMAC family falls under the later category. Here we will concentrate only on CaE schemes.

Classical counter-based encodings are based on a fixed length counter. Let us fix an integer $s \in \mathbb{N}$ as the *counter size* and $L := 2^s$. Now, given a message $M \in \{0, 1\}^\ell$ for some $\ell \leq L$, the encoding works as follows: Let $M' = M \| 1 \| 0^r$ where r is the smallest integer such that $n - s \mid |M'|$. Suppose $M' = (M_1, M_2, \dots, M_b)$, such that for all $i \in [b]$, M_i is of bit-length $n - s$. Then $X := (X_1 := \langle 1 \rangle_s \| M_1, \dots, X_b := \langle b \rangle_s \| M_b)$ is the counter-based encoded input corresponding to M .

The choice of s may depend on the nature of the application. More precisely, s should be chosen in such a way that the number of encoded blocks generated for the longest permissible message should not be more than 2^s , to avoid the possible resetting of counter values.

A typical choice for the maximum permissible message length would allow 2^{64} encoded blocks, i.e., s is exactly 64 bits. Suppose AES-128 cipher is used to instantiate LightMAC [126] scheme. Then, each call of AES-128 will process 64 bits of message, as each encoded block contains 64 bits of counter value and 64 bits of message. So, it would take 2 AES calls per 128 bits of message, independent of whether the message size is 1 kB (2^{13} bits), or 1 GB (2^{33} bits). In short, for a 1 kB message, LightMAC makes about 128 AES-128 calls. On the other hand when $s = 8$, the number of AES-128 calls reduces to 69 (almost twice faster). But this limits the maximum number of encoded blocks to 2^8 . So, a classical counter cannot be efficient over a wide range of message lengths.

Contrary to fixed length counters, one can simply choose the best choice of s for the given message length, instead of fixing it throughout all the invocations. This will definitely speed up the performance for shorter messages and provide very similar performance when the message size approaches the maximum size. A disadvantage of the aforementioned message length dependent counter scheme is the requirement of prior knowledge of the message length. In many scenarios this may not be possible. What can be a good approach when the message length is not known beforehand?

Let us consider an analogous problem from athletics. Consider a race over an unknown distance, where the athletes would only know the finishing point once they actually reach there. What should be a good strategy for running this race? One may consider to run like a marathoner hoping that it is a marathon. In this case, clearly the athlete loses if the race is a hundred meter sprint. The better solution would be to start like a sprinter and then gradually reduce the speed as the race progresses. This could be optimal for short runs.

Our problem is similar to the one just described. We want to find a variable length counter that offers near optimal counter sizes for each message length. Although similar problems are well-studied in the field of prefix codes [92, 175] and data compression [179], to the best of our knowledge it has not seen any interest in cryptography. It would be interesting to investigate the techniques used in prefix coding and construct a near optimal counter scheme when the message length is not known.

Although these ideas are very natural, so far they have not been applied in any of the algorithms in the XMAC family. One possible reason is that the security guarantee is

not obvious for variable length messages. In [16], the authors have given a general framework for constructing XOR-MAC. But even this general framework is somewhat lacking and does not give a unified yet simple treatment for all possible counter-based encodings. In the main theorem of [23], it is assumed that given a message, the encoding produces distinct encoded blocks for distinct primitive calls. Although this points towards a general requirement from any counter-based encoding, the indication is rather implicit. Thus, a formal treatment of counters is required. Furthermore, this generalization should allow a generic proof technique that applies to all counter-based encodings.

5.2 A New Look at Counters

In computer science, counters are used to count the number of times a particular event or process has occurred. In addition to counting, counters have a very special role in cryptography. In XMAC family, the counter values are used to encode the message into distinct blocks. This freshness of inputs actually helps in (possibly improved) the security proof. The classical encoding of i is $\langle i \rangle_s$ which is the s -bit unsigned binary representation of i for a fixed parameter s . Whenever $i < 2^s$, we have distinct binary string corresponding to each counter value. For counter-based algorithms, the parameter s can be chosen as per the needs of the application domain. For example, if we know beforehand that for an application, the message size can not be more than 2^{32} , then one can choose $s = 32$. However, it must remain constant throughout all the executions of the algorithm. This might affect the performance for smaller length messages. In this section we introduce a new and general way of looking at counters which will provide some tools to improve the performance of these schemes without compromising with the security.

Throughout this chapter, we use a fixed $L \in \mathbb{N}$ to denote the maximum permissible bit-length. We will use $\widehat{\ell}$ to denote the block length corresponding to bit-length ℓ , i.e. $\widehat{\ell} := \lceil \frac{\ell}{n} \rceil$.

Definition 5.2.1 (Counter function family). A counter function family (we also use CFF) CNTR is a family of counter functions $\{\text{cntr}_\ell : \ell \leq L\}$, where for all $\ell \leq L$, $\text{cntr}_\ell : \mathbb{N} \rightarrow \cup_{b \in (n-1]} \{0, 1\}^b$.

We say that CNTR is *prefix-free* if for all $\ell \leq L$, cntr_ℓ is prefix-free (i.e., for all $i \neq j$, $\text{cntr}_\ell(i)$ is not a prefix of $\text{cntr}_\ell(j)$).

We use capital letters to denote a function family and small letters for individual functions. The classical (or standard) encoding, as mentioned above, can be viewed as a

CFF STD^s , where $\text{std}_\ell^s(i) = \langle i \rangle_s$, for a fixed positive integer $s < n$. Note that, the standard counter function std_ℓ is actually independent of ℓ and hence for all ℓ , the counter functions are same. We call such CFFs *message length independent*. For a message length independent CFF CNTR, we simply write cntr to denote the counter function cntr_ℓ . Note that, the standard counter is a prefix-free counter. Prefix-free CFF is necessary to avoid repetitions among the inputs to the primitive (see Lemma 5.2.3 below). We also note that the size of the output of std_ℓ is fixed for all counter values. In general for a CFF CNTR, if $\forall \ell, \forall i, j \quad |\text{cntr}_\ell(i)| = |\text{cntr}_\ell(j)|$, then we say that the CFF has *fixed size*. Otherwise, we call it a *variable size CFF*. We will see some examples of message length dependent and variable size CFFs later in the section.

5.2.1 Counter Function Family Based Message Encoding

Recall the encoding scheme discussed in section 5.1. When we use the standard counter STD^s with $(n - s)2^s \leq L$, we first parse a message $M \in \{0, 1\}^\ell$ as $(M_1, \dots, M_{b-1}, M'_b)$ where $b = \lceil \frac{\ell+1}{n-s} \rceil$, $|M_1| = \dots = |M_{b-1}| = n - s$ and $0 \leq |M'_b| < (n - s)$ such that $M = M_1 \parallel \dots \parallel M_{b-1} \parallel M'_b$. Let $M_b = M'_b \parallel 10^d$ where $d = n - s - 1 - |M'_b|$. Thus, $|M_b| = n - s$. Now, we define the b blocks encoding as $X_i = \text{std}^s(i) \parallel M_i$ for all $i \in [b]$. These blocks are used as inputs to the underlying primitive, such as block cipher or compression function.

We extend the same methodology to define the encoding for any other CFF CNTR. For this, we first define the block function $b^{\text{CNTR}}(\ell)$, which associates each message size to a unique number of blocks. We have seen that for standard counter, $b^{\text{STD}}(\ell) = \lceil (\ell + 1)/(n - s) \rceil$.

Definition 5.2.2 (Block function and Average counter length). For any CFF CNTR, we define the *block function* $b^{\text{CNTR}}(\ell)$ as the least integer b such that

$$\ell + 1 \leq \sum_{i=1}^b (n - |\text{cntr}_\ell(i)|) \leq \ell + n.$$

We define the *average counter length* $\mu^{\text{CNTR}}(\ell)$ as

$$\mu^{\text{CNTR}}(\ell) := \frac{1}{b^{\text{CNTR}}(\ell)} \sum_{i=1}^{b^{\text{CNTR}}(\ell)} |\text{cntr}_\ell(i)|.$$

It is easy to see that $\mu^{\text{CNTR}}(\ell) \in (n - 1]$ and $b^{\text{CNTR}}(\ell) = \lceil (\ell + 1)/(n - \mu^{\text{CNTR}}(\ell)) \rceil$. Now given a message $M \in \{0, 1\}^\ell$, $\ell \leq L$, we parse it as $M = M_1 \parallel \dots \parallel M_{b-1} \parallel M'_b$ where $|M_i| = n - |\text{cntr}_\ell(i)|$ for all $i < b$ and $0 \leq |M'_b| < n - |\text{cntr}_\ell(b)|$. We similarly pad the last

block to make it compatible with the corresponding counter. More precisely, we define $M_b = M'_b || 10^d$ where $d = n - |\text{cntr}_\ell(b)| - 1 - |M'_b|$. Thus, $|M_b| = n - |\text{cntr}_\ell(b)|$. We denote the parsing procedure as $(M_1, \dots, M_b) \xleftarrow{\text{CNTR}} M$. Finally, we define the encoding

$$\text{CNTR}(M) := (X_1, \dots, X_b),$$

where $X_i = \text{cntr}_\ell(i) || M_i$ for all $i \in [b]$. Note that, we abuse the notations slightly to view the CFF as an encoding function $\text{CNTR} : \cup_{\ell \in [L]} \{0, 1\}^\ell \rightarrow \mathbb{B}^+$.

Recall that one of the main purpose of counters in cryptography is to generate distinct blocks. Mathematically, a CFF CNTR is called *block-wise collision-free* if for all $M \in \{0, 1\}^\ell$, $\ell \leq L$, $\text{CNTR}(M) = X^b \in (\mathbb{B})_b$, i.e., X_i 's are distinct for all $i \in [b]$. Now, we provide a characterization of block-wise collision-free counters.

Lemma 5.2.3. *CNTR is block-wise collision-free if and only if it is prefix-free.*

Proof. We first prove the “only if” direction. Suppose there exists $i < j \leq b$ such that $\text{cntr}_\ell(i)$ is a prefix of $\text{cntr}_\ell(j)$. Thus, we can find x and y such that $\text{cntr}_\ell(i) || x = \text{cntr}_\ell(j) || y \in \mathbb{B}$. Let $(M_1, \dots, M_b) \xleftarrow{\text{CNTR}} M \in \{0, 1\}^\ell$, for some $\ell \in [L]$. Then, we define M' by replacing M_i and M_j in M by x and y , respectively. It is easy to see that $X'_i = X'_j$, where $\text{CNTR}(M') = (X'_1, \dots, X'_b)$.

To prove the other direction, let us assume $X_i = X_j$ for some $i \neq j$. Therefore, $\text{cntr}_\ell(i) || M_i = \text{cntr}_\ell(j) || M_j$. Clearly, either $\text{cntr}_\ell(i)$ is a prefix of $\text{cntr}_\ell(j)$, or $\text{cntr}_\ell(j)$ is a prefix of $\text{cntr}_\ell(i)$. $\square$

5.2.2 Some (Efficient) Alternatives to the STD CFF

We have already demonstrated the classical or standard CFF STD^s . It is a message-length independent and fixed size CFF. In this section, we study two new examples of CFFs and see their advantages over the standard one.

5.2.2.1 OPT: A Message Length Dependent CFF

Our first CFF is just an optimization of the classical counters. In this case, the counter function maps any integer i to the s -bit unsigned bit representation of its argument, where s depends on the message length instead of a pre-determined fixed parameter. In fact, s is defined as the smallest possible value such that we can represent all the

counter values distinctly for the given message length. Formally, the counter function is defined as

$$\text{opt}_\ell(i) = \langle i \rangle_s, \text{ where } s = \min\{g : 2^g(n - g) > \ell\}.$$

It is an example of message-length dependent and fixed size CFF. Clearly, it is a prefix-free counter.

5.2.2.2 VAR^r: A Variable Size CFF

Till now we have seen counter functions which map integers to their s -bit unsigned binary representation. In case of STD^s, s is fixed (approx. $\log_2 L$), whereas in case of OPT, s depends on the message length ℓ (approx. $\log_2 \ell$). Both STD^s and OPT are fixed size CFFs. Now we will construct a CFF which maps integers to binary strings of monotonically increasing lengths. A similar problem is well-known in the field of source coding. We briefly discuss these techniques assuming that the set of symbols is $\mathbb{N}$. Readers may refer to the references cited for a more detailed exposition.

PREFIX CODES: A mapping $\alpha : \mathbb{N} \rightarrow \{0, 1\}^*$, is called a binary prefix code [175] over integers, if for all $x \neq y \in \mathbb{N}$, $\alpha(x)$ is not a prefix of $\alpha(y)$ and vice-versa. Here $\alpha(x)$ is called the codeword corresponding to x . Huffman codes [92, 175], Shannon-Fano codes [68, 175, 179] and Universal codes [66, 175] are some popular techniques for getting prefix codes.

Huffman codes and Shanon-Fano codes are in general better than universal codes, when the probability (or frequency) distribution over the integers is known. This is generally the case in static data compression settings, such as JPEG File Interchange Format [96] which employs a modified form of Huffman coding, and the ZIP file format [164] which uses a modified version of Shannon-Fano coding. Although Huffman codes and Shannon-Fano codes are better than universal codes, but they require exact probability (or frequency) distribution over the integers.

UNIVERSAL CODES: A universal code [66, 67, 175] is a binary prefix code, with the additional property that if the probability (or frequency) distribution over integers is monotonic, then the expected lengths of the codewords are within a constant factor of the optimal expected lengths for that distribution. Note that, a universal code works for any countably infinite set, and it only requires a monotonic distribution. This is one of the main motivations behind the study of universal codes. Our problem falls precisely in this case, as we can always assume a natural monotonic distribution over the number of blocks. For instance all messages must have at least one block, so 1 has the maximum frequency, followed by 2, and so on.

In 1974, Elias proposed the first prefix code with universal property [66]. In 1975, he followed it up by proposing several new examples of universal codes [67]. Some of the popular examples of universal codes are Elias gamma coding [67, 175], Elias delta coding [67, 175], Elias omega coding [67, 175], Fibonacci coding [73, 175], Levenshtein coding [175] and Exp-Golomb coding [175]. We describe two examples of universal codes, namely, Elias gamma and delta codings, related to our work.

Elias Gamma Coding, γ : A number $i \geq 1$ is encoded as follows:

1. Let $j = \lfloor \log_2 i \rfloor$, i.e., $2^j \leq i < 2^{j+1}$.
2. $\gamma(i) := 0^j \| \langle i \rangle_{j+1}$.

Elias Delta Coding, δ : A number $i \geq 1$ is encoded as follows:

1. Let $j = \lfloor \log_2 i \rfloor$, i.e., $2^j \leq i < 2^{j+1}$.
2. $\delta(i) := \gamma(j+1) \| \text{lsb}_j(\langle i \rangle_{j+1})$.

To represent an integer i , Elias gamma uses $2 \log_2 i + 1$ bits, and Elias delta uses $\log_2 i + 2 \log_2(\log_2 i + 1) + 1$ bits. Clearly, Elias delta is more compact as compared to Elias gamma.

CFF FROM UNIVERSAL CODING: Theoretically, any universal code U can be used to construct a length-independent variable size prefix-free CFF U-CTR. Suppose U is a universal code, then we define U-CTR as follows:

$$\text{U-CTR} = \{\text{u-ctr}_\ell : \ell \leq L\} \text{ where } \text{u-ctr}_\ell = U, 1 \leq \ell \leq L.$$

Clearly, U-CTR is prefix-free as U is universal. Although one might be tempted to use U-CTR straightaway, there is still some scope of improvement in this generic scheme. Note that, the motivation for universal coding is quite different than the cryptographic settings.

First, we are restricting the domain to some L , whereas a universal code is defined over $\mathbb{N}$. Intuitively it seems that the universal codes must have some redundant information that we can avoid. Second, all the CFFs discussed so far have efficient counter update (generating $\text{u-ctr}_\ell(i+1)$ from $\text{u-ctr}_\ell(i)$) mechanism, which is not a mandatory requirement for universal codes. So we should restrict our focus to only those universal codes which support efficient update mechanism, such as Elias delta code. Now we present our second CFF candidate, called VAR^r, which is a modified version of δ -CTR.

VAR^r COUNTER FUNCTION FAMILY: We first fix r , an application parameter chosen suitably (we will see very soon how to choose r). As VAR^r would be message length independent, it would be sufficient to define a function over the domain $\{1, 2, \dots, L\}$. To begin with, we define $\text{var}^r(0) = 0^r$. Now, for all $i \leq L$, we will recursively define $\text{var}^r(i)$ given that $\text{var}^r(j)$ has been defined for all $j < i$. Like STD^s and OPT, we increment the counter by one at every step. In other words, we first define $y = \text{var}^r(i-1) + 1$. If the r most significant bits get altered (this is easy to check, as the remaining bits would all be zero), then we define $\text{var}^r(i) = y \parallel 0$, otherwise we define $\text{var}^r(i) = y$. Mathematically, we can write the recursive definition of var^r for $i \geq 1$, as

$$\text{var}^r(i) = \begin{cases} x + 1 & \text{if } \text{msb}_r(x) = \text{msb}_r(x + 1), \\ (x + 1) \parallel 0 & \text{if } \text{msb}_r(x) \neq \text{msb}_r(x + 1). \end{cases}$$

where $x = \text{var}^r(i-1)$.

Clearly, size of the counter function output increases slowly but monotonically with i . So, VAR^r is an example of message length independent and variable size CFF. To understand this counter we demonstrate how the counter values are computed for $r = 3$. A boldface zero at the least significant bit represents the added zero bit on expansion. The underlined bits are the third most significant bits.

$$00\text{0}, 00\text{1}\underline{\text{0}}, 00\text{1}\underline{\text{1}}, 01\text{0}\underline{\text{00}}, 01\text{0}\underline{\text{01}}, 01\text{0}\underline{\text{10}}, 01\text{0}\underline{\text{11}}, 01\text{1}\underline{\text{000}}, \dots$$

Now we will describe how one should choose the parameter r given that the limit on the message length is L . Note that, when the size of the counter reaches $r+i$ for the first time, the i least significant bits of the counter value are all zero. So the counter will be incremented 2^i many times before the next expansion in counter size. Since we need to keep the r most significant bits non-zero (to avoid repetitions), the following equation must be satisfied:

$$\sum_{i=0}^{2^r-1} (n - r - i)2^i > L.$$

After doing some algebraic simplifications one can show that the smallest r that satisfies the above equation is approx. $\log_2 \log_2 L$ for $L < 2^{c(n) \cdot n}$, where $\frac{1}{2} \leq c(n) < 1$. Note that, for any integer i ,

$$\text{var}_\ell^r(i) = \langle \lfloor \log_2 i \rfloor \rangle_r \parallel \text{lsb}_{\log_2 i}(\langle i \rangle_{\log_2 i+1}),$$

whereas

$$\delta(i) = \gamma(\lfloor \log_2 i + 1 \rfloor) \parallel \text{lsb}_{\log_2 i}(\langle i \rangle_{\log_2 i+1}).$$

Clearly $|\text{var}_\ell^r(i)|$ is less than $|\delta(i)|$ when

$$i \geq 2^{2^{\frac{r-1}{2}}-1}.$$

We generally fix the maximum message length to be 2^{64} bits, which gives $r \approx 6$. So, the average counter size in VAR^r will be less than the average counter size in $\delta\text{-CTR}$, when the number of generated blocks is at least 2^6 . Specifically, for around 2^{16} blocks, the average counter size in VAR^r is shorter than the average counter size in $\delta\text{-CTR}$ by more than 3 bits. Now we show that VAR^r is a prefix-free CFF.

Theorem 5.2.4. *Let r be defined as above then VAR^r is prefix-free.*

Proof. Let $i \neq j$ be non-zero and $x = \text{msb}_r(\text{var}^r(i))$ and $y = \text{msb}_r(\text{var}^r(j))$. If $x \neq y$ then x can not be prefix of y . So assume $x = y$. Because of our choice of r , the r most significant bits does not become 0^r (except for the input 0). So, $\text{var}^r(i)$ and $\text{var}^r(j)$ have same size. As i and j are distinct, the rest of the bits must be different. This proves the prefix-free property. $\square$

5.2.2.3 Word Oriented Adaptation of Our Counters

The counter functions described in the preceding section are aimed to minimize the counter size as much as possible, keeping all the counter values distinct (i.e., prefix-free). However, there are different practical issues while implementing these counter functions. The most important issue is to parse the message into small chunks which are compatible with the counter-based encoding. In practice, we mostly receive messages as a sequence of words (e.g., 8-bit (byte), or 32-bit words). It would be easier for implementation if we can parse the messages in multiples of word size. More formally, let us fix a parameter w (elements of $\{0, 1\}^w$ are called words). We define OPT^w (the word oriented adaption of OPT) as follows:

$$\text{opt}_\ell^w(i) = \langle i \rangle_s, \text{ where } s = \min\{g : w|g, (n - g)2^g > \ell\}.$$

Note that, $\text{OPT} = \text{OPT}^1$. Now we describe how we can generalize our second proposal to $\text{VAR}^{r,w}$ which fits into word oriented implementation, for $r \leq w$. We define the counter function recursively as before except that we add 0^w instead of single 0. More formally, $\text{var}^{r,w}(0) = 0^w$. For $i \geq 1$, let $x = \text{var}^{r,w}(i - 1)$. Then,

$$\text{var}^{r,w}(i) = \begin{cases} x + 1 & \text{if } \text{msb}_r(x) = \text{msb}_r(x + 1), \\ (x + 1) \| 0^w & \text{if } \text{msb}_r(x) \neq \text{msb}_r(x + 1). \end{cases}$$

For this choice of r , the counter function is prefix-free. In the following example we describe how the counter expands for $r = 4$ and $w = 8$.

$$00000000, \dots, 000100000^8, \dots, 001000000000000000^8, \dots$$

To the best of our knowledge such word-oriented definitions are not available for delta codes (for obvious reasons). Even if we use our word-oriented definition, the resulting code will not give better counter function. This can be argued by the simple fact that in case of $\text{VAR}^{r,w}$, r is fixed, so we can simply start with some fixed w and then move as it is. But in case of delta code we have to consider the underlying gamma code also which is variable in nature. Handling two variable components may result in overheads in the counter size as well as the update mechanism.

5.2.3 Comparison of Rates of Counter Function Families

Recall that we have defined the number of blocks in a message of length ℓ as $\widehat{\ell} = \lceil \ell/n \rceil$. As we execute some costly primitive function on these blocks, it would be good to minimize the number of blocks as much as possible.

Definition 5.2.5. We define the rate function $\text{rate}^{\text{CNTR}}(\ell)$ associated with a counter-based encoding CNTR, as the ratio of the number of blocks in the message to the total number of blocks in the encoded message produced by CNTR, i.e., $\text{rate}^{\text{CNTR}}(\ell) = \frac{\widehat{\ell}}{b^{\text{CNTR}}(\ell)}$.

Now, $\widehat{\ell} = \lceil \ell/n \rceil \leq \lceil (\ell+1)/n \rceil \leq \lceil (\ell+1)/(n-k) \rceil$ for any $k \in (n-1]$. In particular, we have $\widehat{\ell} \leq \lceil (\ell+1)/(n - \mu^{\text{CNTR}}(\ell)) \rceil = b^{\text{CNTR}}(\ell)$. Thus, we have $\text{rate}^{\text{CNTR}}(\ell) \leq 1$, where equality holds when n divides $(\ell+1)$ and the counter size is zero. Now we provide a comparison between the rate functions for the three CFFs defined above.

1. For the standard counter STD^s , the rate function is

$$\frac{\lceil \ell/n \rceil}{\lceil (\ell+1)/(n-s) \rceil} \approx \frac{n-s}{n},$$

for $\ell \gg n > s$. Typically, $s \approx \log_2 L$, which reduces the rate when L is large.

2. For OPT, the rate function is

$$\frac{\lceil \ell/n \rceil}{\lceil (\ell+1)/(n - \log_2 \ell) \rceil} \approx \frac{n - \log_2 \ell}{n}.$$

Clearly, the rate of this counter is better than STD^s for all choices of $\ell < L$.

3. For VAR^r , the rate function is a complex function in n and r . We skip the exact algebraic expression here and give an approximation,

$$\frac{\lceil \ell/n \rceil}{\lceil (\ell + n - r + 2)/(n - r + 2 - \log_2 \ell) \rceil} \approx \frac{n - r + 2 - \log_2 \ell}{n} \quad (\text{for } \ell \gg n).$$

Clearly, the rate of this counter is better than STD for small messages and large s . Further the rate is comparable with OPT for $n \gg r$.

5.2.4 Word Oriented Rates of Counter Function Families

Let $w \in \mathbb{N}$ denote the word size. We will assume that $n, \ell, L \in \mathbb{M}_w$, where $\mathbb{M}_w$ denotes the set of all multiples of w . We assume that $L = o(2^n)$. The word oriented rate functions for STD^s and OPT are pretty similar to their bit oriented counterparts. For instance, in STD^s , we can simply choose s to be some multiple of w . So we only derive the word oriented rate function for VAR^r .

5.2.4.1 Estimating Parameter r in $\text{VAR}^{r,w}$

Let $c = \lceil r/w \rceil$. We know that the following inequality holds for the correct value of r ,

$$\sum_{i=0}^{2^r-1} (n - cw - iw) 2^{iw+cw-r} \geq L$$

Let $n' = n - cw$, $r' = cw - r$, $\dot{r} = 2^r$, and $\dot{w} = 2^w$. Now we will get a lower bound on r ,

$$\begin{aligned} \sum_{i=0}^{\dot{r}-1} (n' - iw) 2^{iw+r'} &\geq L \\ 2^{r'} \left(n' \sum_{i=0}^{\dot{r}-1} \dot{w}^i - w \sum_{i=0}^{\dot{r}-1} i \cdot \dot{w}^i \right) &\geq L \\ 2^{r'} \left[\left(n' + \frac{w\dot{w}}{\dot{w}-1} \right) \left(\frac{\dot{w}^{\dot{r}}-1}{\dot{w}-1} \right) - \frac{w\dot{r}\dot{w}^{\dot{r}}}{\dot{w}-1} \right] &\stackrel{1}{\geq} L \\ 2^{r'} \left[(n' + 2w) (2\dot{w}^{\dot{r}-1}) - w\dot{w}^{\dot{r}-1} \right] &\stackrel{2}{\geq} L \\ \dot{w}^{\dot{r}} (2n - w) &\stackrel{3}{\geq} 2L \end{aligned}$$

where 1 can be obtained by simple algebraic simplifications. As $\dot{w} \geq 2$, we have $\frac{1}{\dot{w}-1} \leq \frac{2}{\dot{w}}$. Using this we get 2. Also $c \geq 1$ and $r' \leq w - 1$, which gives us 3. Now simple

algebraic simplifications give us the upper bound on r ,

$$r \geq \log_2 \left\lceil \frac{\log_2 \left(\frac{2L}{2n-w} \right)}{w} \right\rceil \quad (5.1)$$

For $w < n \ll L$, we get $r \approx \log_2 \log_2 L - \log_2 w$.

5.2.4.2 Estimating the Block Function of $\text{VAR}^{r,w}$

For a given message length ℓ , we are interested in a lower bound on $b(\ell)$.¹ For simplicity we also assume $c = r/w$, i.e., $r' = 0$. We restrict our analysis to only those ℓ , for which $\exists k \leq 2^r - 1$ such that the number of blocks,

$$b(\ell) = 2^{r'} \sum_{i=0}^{k-1} \dot{w}^i = \left(\frac{\dot{w}^k - 1}{\dot{w} - 1} \right). \quad (5.2)$$

We find an upper bound for $b(\ell)$ in the following derivation,

$$\begin{aligned} \left(n' \sum_{i=0}^{k-1} \dot{w}^i - w \sum_{i=0}^{k-1} i \cdot \dot{w}^i \right) &\leq \ell + n \\ \left(n' + \frac{w\dot{w}}{\dot{w}-1} - \frac{wk\dot{w}^k}{\dot{w}^k-1} \right) \left(\frac{\dot{w}^k - 1}{\dot{w} - 1} \right) &\stackrel{1}{\leq} \ell + n \\ (n - r + w - wk) \left(\frac{\dot{w}^k - 1}{\dot{w} - 1} \right) &\stackrel{2}{\lesssim} \ell + n \\ (n - r + w - \log_2(\ell + n)) \left(\frac{\dot{w}^k - 1}{\dot{w} - 1} \right) &\stackrel{3}{\lesssim} \ell + n. \end{aligned}$$

$\frac{1}{\dot{w}-1} \geq \frac{1}{\dot{w}}$, and for moderately large k , we get $\frac{\dot{w}^k}{\dot{w}^k-1} \approx 1$. Using these facts we get 2 from 1. Similarly $wk \lesssim \log_2(\ell + n)$ (experimental results show that $wk \approx \log_2 \ell$), which gives 3 from 2. Using (5.2) and 3 we get,

$$b(\ell) \lesssim \frac{\ell + n}{n - r + w - \log_2(\ell + n)} \quad (5.3)$$

RATE FUNCTION FOR WORD ORIENTED $\text{VAR}^{r,w}$: Finally we get the following approximation for lower bound on the rate function of word oriented $\text{VAR}^{r,w}$,

$$\text{rate}^{\text{VAR}^{r,w}}(\ell) \gtrsim \frac{n + w - r - \log_2(\ell + n)}{n}$$

for $w < n \ll \ell$. Note that, we derived this function for some restricted ℓ values. But experimental results, given in Table 5.2.1, shows that the estimation is close to the actual

¹Note that, we have dropped $\text{VAR}^{r,w}$ from the superscript as it is obvious.

function. Figure 5.2.1 gives a graphical comparison between the rates of the three CFFs.

Table 5.2.1: Comparison between the rates offered by the three candidate CFFs for block size, $n = 128$ and $w = 8$.

Length	STD ^s				STD ^{opt,8}	VAR ^{4,8}
	s = 8 bits	s = 16 bits	s = 32 bits	s = 64 bits		
128B	0.89	0.80	0.73	0.47	0.89	0.89
256B	0.89	0.84	0.73	0.48	0.89	0.89
512B	0.91	0.86	0.74	0.49	0.91	0.89
1kB	0.93	0.86	0.74	0.50	0.93	0.88
2kB	0.93	0.87	0.75	0.50	0.93	0.88
4kB	-	0.87	0.75	0.50	0.87	0.88
8kB	-	0.87	0.75	0.50	0.87	0.88
16kB	-	0.87	0.75	0.50	0.87	0.88
32kB	-	0.87	0.75	0.50	0.87	0.88
64kB	-	0.87	0.75	0.50	0.87	0.86
128kB	-	0.87	0.75	0.50	0.87	0.80
256kB	-	0.87	0.75	0.50	0.87	0.77
512kB	-	0.87	0.75	0.50	0.87	0.76
1MB	-	-	0.75	0.50	0.75	0.76

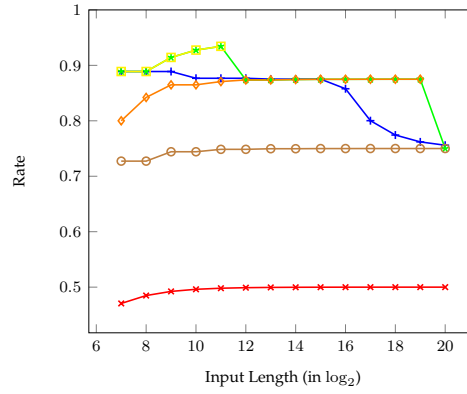


Figure 5.2.1: Rate plot for the three CFFs. Plots marked with +, *, ×, ○, ◇, and □ marked plots correspond to VAR^{4,8}, OPT⁸, STD⁶⁴, STD³², STD¹⁶, and STD⁸, respectively.

5.3 Counter-as-Encoding Constructions

We propose some generic CaE schemes, such as AXU hash and MACs based on CFFs. We also propose a HAIFA [31] based cryptographic hash function. In the last section, we defined prefix-free counter function family. For any such prefix-free counter CNTR and for all M , we know that $X_1, \dots, X_b$ are distinct where $\text{CNTR}(M) = (X_1, \dots, X_b)$. With a slight abuse of notation, we let $\text{CNTR}(M)$ to denote the set $\{X_1, \dots, X_b\}$ as well. Observe that prefix-free property does not say anything about the collisions between the encoded blocks of two distinct messages. Clearly, we will have some intersection between $\text{CNTR}(M)$ and $\text{CNTR}(M')$ (viewed as a set) for any counter function family.

Definition 5.3.1. A prefix-free CFF CNTR is called *injective* if for all $M \neq M'$, $\text{CNTR}(M) \neq \text{CNTR}(M')$ (as a set).

We provide a sufficient condition for injective counter function families.

Lemma 5.3.2. Let CNTR be a prefix-free CFF. It is injective if it satisfies the following condition

$$\forall \ell, \ell' \leq L, \quad b(\ell) = b(\ell') \Rightarrow \text{cntr}_\ell = \text{cntr}_{\ell'}.$$

Proof. Suppose for some $M \in \{0, 1\}^\ell, M' \in \{0, 1\}^{\ell'}, \text{CNTR}(M) = \text{CNTR}(M')$. We require to show that $M = M'$. Note that, $\text{CNTR}(M) = \text{CNTR}(M') \Rightarrow \{X_1, \dots, X_{b(\ell)}\} = \{X'_1, \dots, X'_{b(\ell')}\}$. So $b(\ell) = b(\ell') = b$. By given condition, we have $\text{cntr}_\ell = \text{cntr}_{\ell'}$ and we denote it simply by cntr . Now, we first show that $X_i = X'_i$ for all i . As two sets are equal for any $i \leq b$, there must exist j so that $X_i = X'_j$. This implies that one of $\text{cntr}(i)$ and $\text{cntr}(j)$ is prefix of the other, and hence $i = j$ (as the counter function is block-wise collision-free). Since counter functions for ℓ and ℓ' are same we have $M_i = M'_i$ for all i . This proves that $M = M'$. $\square$

Recall that all our candidate CFFs are prefix-free. Further it is obvious to see that they also satisfy the condition for injectivity. So we get the following corollary.

Corollary 5.3.3. $\text{CNTR} \in \{\text{STD}, \text{OPT}, \text{VAR}^r\}$ is a prefix-free and injective CFF.

In the following subsections we present various schemes based on CFFs. In particular, the MAC schemes with STD are precisely the MACs from the XMAC family. The security of all these schemes follow directly from the prefix-free and injective property of the underlying CFF. Specifically, all the security results in the following subsections hold, when we instantiate these schemes with $\text{CNTR} \in \{\text{STD}, \text{OPT}, \text{VAR}^r\}$.

5.3.1 Almost XOR Universal Hash Function

Let CNTR be a CFF. We define a counter based AXU-hash function, denoted CtH , based on a length-preserving² function Λ over $\mathbb{B}$, and a CFF CNTR. Let $M \in \{0, 1\}^\ell$. We define

$$\text{CtH}_{\Lambda, \text{CNTR}}(M) = \Lambda(X_1) \oplus \dots \oplus \Lambda(X_b),$$

where $\text{CNTR}(M) = (X_1, \dots, X_b)$. Notice that this hash function is a generalization of the hash functions used in the XMAC family [16, 23, 126]. It has a simple design that requires only Λ computation and XOR operations. Furthermore, the hash can be

²A function f is called length-preserving if $|f(x)| = |x|$, for all x .

computed completely in parallel. Now we show that CtH is an AXU hash function whenever Λ is a uniform random permutation or function over $\mathbb{B}$.

Theorem 5.3.4. *Let CNTR be a prefix-free and injective CFF and $b^{\text{CNTR}}(L) \leq 2^{n-2}$. Then, we have*

1. $\text{CtH}_{\Lambda, \text{CNTR}}$ is 2^{-n} -AXU hash if $\Lambda \leftarrow \$ \text{Func}(\mathbb{B}, \mathbb{B})$; and
2. $\text{CtH}_{\Lambda, \text{CNTR}}$ is 2^{1-n} -AXU hash if $\Lambda \leftarrow \$ \text{Perm}(\mathbb{B})$.

Proof. We prove the second part, while the first can be shown analogously. Let $M \neq M'$ be two messages of lengths ℓ and ℓ' , $\text{CTR}(M) = S = \{X_1, \dots, X_b\}$, and $\text{CTR}(M') = S' = \{X_1, \dots, X_{b'}\}$. As CNTR is prefix-free and injective, there exists at least one block Y which appears exactly once in $S \cup S'$. Thus, for any $\delta \in \mathbb{B}$,

$$\begin{aligned} \Pr [\text{CtH}_{\Lambda, \text{CNTR}}(M) \oplus \text{CtH}_{\Lambda, \text{CNTR}}(M') = \delta] &= \Pr [\Lambda(Y) = \oplus_{y \in S_1 \cup S_2 \setminus \{Y\}} \Lambda(y)] \\ &\stackrel{1}{\leq} \frac{1}{2^n - b^{\text{CNTR}}(\ell) - b^{\text{CNTR}}(\ell')} \stackrel{2}{\leq} \frac{2}{2^n}. \end{aligned}$$

where inequality 1 follows by conditioning on the outputs of Λ on all the blocks except Y , and inequality 2 follows from $b^{\text{CNTR}}(L) \leq 2^{n-2}$. $\square$

Remark 5.3.5. The above result has been proved implicitly in [16, 23, 126] for the standard counter. Our result generalizes their results for any prefix-free and injective counter function family.

5.3.2 Message Authentication Codes

Now that we have an AXU hash function, we can apply standard methods discussed in section 2.4.2 to obtain (non-)deterministic MAC schemes from CtH .

5.3.2.1 Deterministic MACs

Let CNTR be a CFF. Given any $M \in \{0, 1\}^\ell$ with $\ell > n$, we write $M = M' \| m$ where m is a block. We define

$$\text{CtMAC1}_{E_{K_1}, E_{K_2}}(M) := E_{K_2}(\text{CtH}_{E_{K_1}}(M') \oplus m).$$

In the following theorem we show that CtMAC1 is a secure PRF.

Theorem 5.3.6. *Let $\text{CtMAC1}_{E_{K_1}, E_{K_2}, \text{CNTR}}$ be defined based on two independent instantiations of a keyed function E , and a prefix-free and injective CFF CNTR . We have*

1. If E is a PRP, then we have

$$\mathbf{Adv}_{\text{CtMAC1}_{E,E,\text{CNTR}}}^{\text{prf}}(q, \hat{\ell}, t) \leq 2\mathbf{Adv}_E^{\text{prp}}(q\hat{\ell}, t) + \frac{1.5q^2}{2^n}.$$

2. If E is a PRF, then we have

$$\mathbf{Adv}_{\text{CtMAC1}_{E,E,\text{CNTR}}}^{\text{prf}}(q, \hat{\ell}, t) \leq 2\mathbf{Adv}_E^{\text{prf}}(q\hat{\ell}, t) + \frac{q^2}{2^n}.$$

Proof. We prove the first part, the second part can be shown analogously. First, we replace the two instantiations of E with independent random permutations Π and Π' , at the cost of $2\mathbf{Adv}_E^{\text{prp}}(q\hat{\ell}, t)$. Now, Theorem 5.3.4 shows that CtH_{Π} is a $2/2^n$ -AXU hash function when CNTR is prefix-free and injective. So, the modified scheme $\text{CtH}_{\Pi}(M') \oplus m$ is a $2/2^n$ -AU hash when CNTR is prefix-free and injective (using Proposition 2.3.5). The result follows by direct application of Proposition 2.4.3. $\square$

Remark 5.3.7. The deterministic MAC schemes from XMAC family, namely PCS [23] and LightMAC [126], are instantiations of CtMAC1 with STD as the underlying CFF. Since, STD is prefix-free and injective, the security of PCS and LightMAC follows directly from Theorem 5.3.6.

5.3.2.2 Non-deterministic MACs

Let N be a seed (either random or nonce). We define

$$\text{CtMAC2}_{E_{K_1}, E_{K_2}}(N, M) = E_{K_2}(N) \oplus \text{CtH}_{E_{K_1}}(M).$$

Depending upon the nature of N , we get either a nonce-based MAC, denoted $\text{CtMAC2}^{\text{st}}$, or a probabilistic MAC CtMAC2^{s} . Using Theorem 5.3.4 and Propositions 2.4.4 and 2.4.5 we get the following result on the security of $\text{CtMAC2}^{\text{st}}$ and CtMAC2^{s} .

Theorem 5.3.8. *Let $\text{CtMAC2}_{E_{K_1}, E_{K_2}, \text{CNTR}}$ be defined based on two independent instantiations of a keyed function E , and a prefix-free and injective CFF CNTR. Then,*

1. If E is a PRP, then we have

- $\mathbf{Adv}_{\text{CtMAC2}^{\text{st}}_{E,E,\text{CNTR}}}^{\text{mac}}(q, \hat{\ell}, t) \leq 2\mathbf{Adv}_E^{\text{prp}}(q\hat{\ell}, t) + \frac{2q_v}{2^n} \left(1 - \frac{q_m}{2^n}\right)^{-\frac{q_m+1}{2}}, \text{ and}$
- $\mathbf{Adv}_{\text{CtMAC2}^{\text{s}}_{E,E,\text{CNTR}}}^{\text{mac}}(q, \hat{\ell}, t) \leq 2\mathbf{Adv}_E^{\text{prp}}(q\hat{\ell}, t) + \frac{2q_v}{2^n} \left(1 - \frac{q_m}{2^n}\right)^{-\frac{q_m+1}{2}} + \frac{0.5q_m^2}{2^n}.$

2. If E is a PRF, then we have

- $\mathbf{Adv}_{\text{CtMAC2}^{\text{st}}_{E,E,\text{CNTR}}}^{\text{mac}}(q, \hat{\ell}, t) \leq 2\mathbf{Adv}_E^{\text{prf}}(q\hat{\ell}, t) + \frac{2q_v}{2^n}, \text{ and}$

$$\bullet \text{Adv}_{\text{CtMAC2}_{\text{E,E,CNTR}}}^{\text{mac}}(q, \hat{\ell}, t) \leq 2\text{Adv}_{\text{E}}^{\text{prf}}(q\hat{\ell}, t) + \frac{0.5q_m^2}{2^n} + \frac{2q_v}{2^n}.$$

Remark 5.3.9. The non-deterministic MAC schemes from XMAC family, namely XMACC and XMACR [16], are PRF-based instantiations of CtMAC2st and CtMAC2^s, respectively, with STD as the underlying CFF. Since, STD is prefix-free and injective, the security of XMACC and XMACR follows directly from Theorem 5.3.8.

5.3.2.3 Necessity of Prefix-free and Injectivity

In all the security results discussed till now, the security proof is implied by the prefix-free and injective property of the CFF CNTR. In other words, prefix-free and injective properties are sufficient for security. Now, we show that these properties are also necessary for CtMAC1 and CtMAC2, and hence for the XMAC family as well.

Case 1: Suppose CNTR doesn't have injective property. As CNTR is a deterministic function (encodings are deterministic), one can easily find two distinct messages M and M' such that $\text{CNTR}(M) = \text{CNTR}(M')$.

Case 2: Suppose CNTR is not prefix-free, i.e., for some $\ell \leq L$, we have $i < j \in \mathbb{N}$ such that $\text{cntr}_{\ell}(i)$ is a prefix of $\text{cntr}_{\ell}(j)$. Thus, we can find two pairs (c, c') and (d, d') such that $c \neq d, c' \neq d', \text{cntr}_{\ell}(i) \| c = \text{cntr}_{\ell}(j) \| c'$ and $\text{cntr}_{\ell}(i) \| d = \text{cntr}_{\ell}(j) \| d'$. Using these two pairs we can construct two distinct messages M and M' which only differ at the i -th and j -th blocks. Further, suppose the i -th block of M and M' is c and d , respectively, and the j -th block of M and M' is c' and d' , respectively. Clearly, the i -th encoded block of M collides with the j -th encoded block of M' and vice-versa, i.e., they cancel out each other in $\text{CtH}(M)$ and $\text{CtH}(M')$.

In both the cases the adversary can construct $M \neq M'$, such that

$$\Pr [\text{CtH}(M) \oplus \text{CtH}(M') = 0] = 1.$$

This leads to trivial forgery attacks on CtMAC1 and CtMAC2, as the adversary can make one MAC query with message M , and forge with M' and the tag T so obtained for M . Thus, prefix-free and injective properties are both necessary and sufficient for CtMAC1 and CtMAC2.

5.3.3 Counter-based Cryptographic Hash

As a side-result, we also propose a cryptographic hash function based on CFFs and block cipher. A hash function H is a function from $\cup_{i=1}^L \{0, 1\}^i$ to $\mathbb{B}$, which aspires to achieve three chief security requirements:

1. *Preimage resistance*: Given a random challenge y , it is hard to find M such that $H(M) = y$. Here M is called *preimage* of y .
2. *Collision resistance*: It is hard to find a pair (M, M') of distinct elements, called *collision pair* for H , such that $H(M) = H(M')$.
3. *Second preimage (2PI) resistance*: Given a random challenge input M , it is hard to find a collision pair of the form (M, M') . In this case M' is also called a *second preimage* of M .

We employ a variant of *HASh Iterative FrAmework* (HAIFA) [31] by Biham and Dunkelman to construct the hash function, which resists several second preimage attacks [6, 59, 116] applicable to a popular class of hash functions, the so-called Merkle-Damgård hash [55, 134]. The main idea of HAIFA is to process counter-encoded inputs in iteration. This roughly means that all primitive inputs are distinct (as counter differ) in a hash computation, which allows for higher security. In fact, Bouillaguet and Fouque [41] have shown that HAIFA has full (secure up to $o(2^n)$ queries) second preimage security in the random oracle model (the underlying primitive is assumed to be a public random function).

5.3.3.1 CtHAIFA Hash Function

Let CNTR be a CFF. We present a counter based HAIFA hash function, called CtHAIFA, based on a block cipher $E_{-n/n}$ (both block and key lengths are n bits). For a message $M \in \{0, 1\}^{\eta^*}$, let $h_0 = IV$, an n -bit constant, and

$$\forall i \in [b], \quad h_i := \text{DM}_E(h_{i-1}, X_i),$$

where X_i is the i -th element of $\text{CNTR}(M)$, and $\text{DM}_E : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ is the Davies-Meyer compression function [166] which is defined by the mapping $(x, y) \mapsto E_y(x) \oplus x$. We define CtHAIFA (illustrated in Figure 5.3.1) based on CNTR as,

$$\text{CtHAIFA}_{E, \text{CNTR}}(M) := h_\star = \text{DM}_E(h_b, \langle |M| \rangle_n).$$

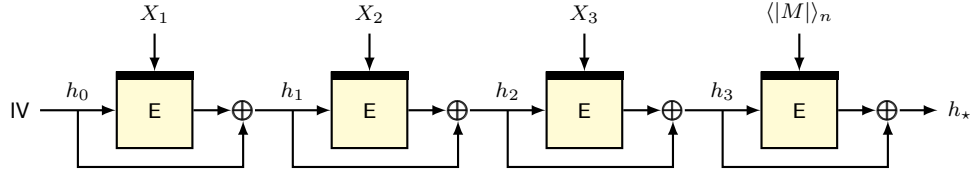


Figure 5.3.1: CtHAIFA hash computation over a 3-block message.

Since, we employ a block cipher instead of a public random function, the random oracle model as used in [41] is not appropriate to prove the security of CtHAIFA. The ideal cipher model (ICM) [50, 51] is widely used as the de facto model for a block cipher based hash function. In this model, the block cipher E is modeled as an *ideal cipher*, where for each $K \in \mathbb{B}$, $E_K \leftarrow \$ \text{Perm}(\mathbb{B})$, and E_K is statistically independent of $E_{K'}$ for all $K' \neq K \in \mathbb{B}$. Further, the adversary has access to E and its inverse E^{-1} .

The preimage and collision security up to $o(2^n)$ and $o(2^{n/2})$ queries, respectively, can be easily shown using simple reductions to the preimage and collision resistance of DM in the ICM. We will concentrate on the second preimage security of CtHAIFA. Formally, the advantage of any adversary $\mathcal{A}$ in finding a second preimage of a hash function H_E under ICM is defined as

$$\text{Adv}_{H_E}^{2\text{PI}}(\mathcal{A}) := \Pr_{\mathcal{E}} \left[\mathcal{A}^{E^{\pm}}(M) = M' : M \leftarrow \$ \cup_{i=1}^L \{0, 1\}^i \wedge H_E(M') = H_E(M) \right].$$

Theorem 5.3.10 gives an upper bound on the second preimage advantage.

Theorem 5.3.10. *Let CNTR be a prefix-free and injective CFF. Then, CtHAIFA has full second preimage security. More specifically, for any second preimage adversary $\mathcal{A}$ that makes $q \leq 2^n/3$ many queries, we have*

$$\text{Adv}_{\text{CtHAIFA}_{E, \text{CNTR}}}^{2\text{PI}}(\mathcal{A}) \leq \frac{3q}{2^n}.$$

Proof. We use a similar approach as used in [41] for HAIFA based on random oracle. In the ideal cipher model, $\mathcal{A}$ can make both encryption and decryption queries to the underlying block cipher. We also assume that the adversary computes $\text{CtHAIFA}(M)$. The adversary can do this by making encryption query (M_i, h_{i-1}) and storing $(h_{i-1}, M_i, h_i := E_{M_i}(h_{i-1}) \oplus h_{i-1})$ for all $i \in [b]$. Actually $\mathcal{A}$ can compute any $\text{DM}_E(h, m)$ by making encryption query (m, h) to E . Let the sequence of intermediate chaining values for the challenge message M be $(h_0, h_1, \dots, h_b, h_{b+1})$.

We denote the i -th query of $\mathcal{A}$ by the tuple $(x_i, m_i, c_i, y_i, o_i)$ such that c_i is the counter value, m_i is the message value, x_i is some chaining value, and $y_i = \text{DM}(x_i, c_i \| m_i) \oplus x_i$. Further,

1. if the i -th query is an encryption query then $o_i = 0$ and $\mathcal{A}$ queried $c_i \| m_i, x_i$ and got $y_i := E_{c_i \| m_i}(x_i)$.
2. if the i -th query is a decryption query then $o_i = 1$ and $\mathcal{A}$ queried $c_i \| m_i, y_i$ and got $x_i := E_{c_i \| m_i}^{-1}(y_i)$.

Now suppose that $\mathcal{A}$ produces a valid second preimage M' . We denote this event by Win . Let $\text{Win}^\neq$ and $\text{Win}^=$ denote the events $\text{Win} \wedge |M| \neq |M'|$ and $\text{Win} \wedge |M| = |M'|$, respectively. Clearly $\text{Win} = \text{Win}^\neq \sqcup \text{Win}^=$. Therefore,

$$\Pr[\text{Win}] = \Pr[\text{Win}^\neq] + \Pr[\text{Win}^=].$$

We bound the probabilities of the two events on the R.H.S. as follows:

1. If $\text{Win}^\neq$ is true then $|M| \neq |M'|$. In this case we must have $\langle |M| \rangle_n \neq \langle |M'| \rangle_n$. Therefore, the adversary found a second preimage for the last block, i.e., there exist a query $i \in [q]$ such that $c_i \| m_i = \langle |M'| \rangle_n \neq \langle |M| \rangle_n$ and $y_i \oplus x_i = h_{b+1}$. This is possible with probability $\frac{1}{2^{n-i+1}}$. Bounding over q queries we have $\Pr[\text{Win}^\neq] \leq \frac{q}{2^n - q}$.
2. If $\text{Win}^=$ is true then $|M| = |M'|$. In this case, if $\mathcal{A}$ succeeds, then $\mathcal{A}$ was successful in creating a connection to some intermediate chaining value h_j . Suppose this connection is established at the i -th query, then c_i must be equal to $\text{ctr}_\ell(j)$ and $y_i \oplus x_i = h_j$. This probability is again $\frac{1}{2^{n-i+1}}$. Bounding over q queries we have $\Pr[\text{Win}^=] \leq \frac{q}{2^n - q}$.

The result follows by combining 1 and 2. □

Remark 5.3.11. It is well-known that Davies-Meyer compression functions have efficiently computable fixed points, $\text{DM}_E(E_y^{-1}(0), y) = E_y^{-1}(0)$ for all $y \in \mathbb{B}$. However, this does not help in inverting any arbitrary hash value. This can be argued as follows: Let h_i be some hash value and the adversary aims to compute a pair (h_{i-1}, m) such that $\text{DM}_E(h_{i-1}, m) = h_i$. Suppose the adversary queries (m, y) to the decryption oracle, then the probability that $E_m^{-1}(y) = x$ such that $E_m(x) \oplus x = h_i$ is at most $1/(2^n - i + 1)$.

Remark 5.3.12. Dean [59] showed an attack on Merkle-Damgård hash functions, when the underlying compression function has efficiently computable fixed points for random message blocks. That attack cannot be extended to CtHAIFA as the underlying CFF has block-wise collision free property.

Remark 5.3.13. In Crypto 2005, Coron et al. [50] proved that Merkle-Damgård hash with prefix-free encoding is indistinguishable from a random oracle. Here prefix-free refers to

the property that the encoding of M is not a prefix of the encoding of M' , if $M \neq M'$. Recall that a prefix-free CFF means $\text{cntr}_\ell(i)$ is not a prefix of $\text{cntr}_\ell(j)$ if $i \neq j$. Not all encoding schemes based on prefix-free CFFs are prefix-free encodings. In fact length padding is necessary to get prefix-free encodings.

5.4 Comparison and Empirical Result

In this section, we explore the feasibility of replacing the classical counter STD with the newly introduced OPT and VAR. We compare the performance of the three candidate counter function families via their application in different CaE schemes. In particular, we present software implementation of CtMAC1, CtMAC2st, and CtHAIFA. We instantiate the underlying primitive with AES-128 [152] block cipher.

5.4.1 Platform Setup

As mentioned earlier we use AES-128 (key size 128 bits) as the underlying block cipher. We use performance data for all inputs of length $\ell = 2^k$ bits where $7 \leq k \leq 20$. We implemented all the schemes on Intel's Sandy Bridge microarchitecture using AES-NI and SSE4 instructions. All measurements were taken on a single core of Intel Xeon E5-2640 processor at 2.5Ghz with Turbo Boost disabled. The warmup parameter is 250000 and the data is averaged over 1000000 repetitions. All results will be either in number of cycles per byte (or CPB) or number of cycles. The baseline performance for AES-128 using AES-NI instruction set is presented in Table 5.4.1.

Table 5.4.1: Baseline CPB value of AES-128 using AES-NI in Sandy Bridge architecture.

	encryption (cycles/byte)	key scheduling (cycles)
(AES, serial)	6.7	117
(AES, parallel)	0.69	117

5.4.2 Implementing Counter Function Families

We choose $w = 8$ as the word size in our implementations. The three counter function families basically differ in their selection of counter sizes. STD^s is a standard counter with a fixed size s ; OPT computes the (optimal) counter size s based on the length of the input; and VAR^r dynamically changes the counter size s based on the number of input blocks processed so far. We choose $r = 4$ in our implementations.

In practice, STD^s requires a beforehand heuristic on the typical input lengths to be encountered. In situations where the input lengths are inconstant, this scheme may not be that efficient. If the input length is known beforehand, then OPT is the best option as it fixes the optimal (machine-dependent) counter size. VAR^r has an edge over the other two in the worst case scenario, i.e., when neither the input length is known nor the inputs have predictable lengths. This scenario is quite frequent in cloud-server applications.

Although OPT and VAR^r are much more flexible in terms of the changes in the input length, they do require some book-keeping operations. OPT requires the computation of a suitable counter size for the given input length. Similarly, VAR^r requires some kind of mechanism to update the counter size when the current counter reaches its limit.

STD^s does not incur such book-keeping overheads, as it is fixed for the construction. Thus, STD^s with $s = 32$ uses 32-bit counter irrespective of whether the input size is 2^{10} or 2^{20} bits. We implement STD^s for $s = w2^{i-1}$ and $i \in [4]$. Similar values are used in OPT and VAR^r while choosing (or expanding) the counter size. For OPT, we store the set of counter sizes ($\{8, 16, 32, 64\}$) along with their respective maximum input lengths. Although this incurs a small increment in code size, it reduces the number of micro-operations. For VAR^r , the overhead can be reduced significantly by reducing the number of times the counter size is increased. For example, we increase the size in steps of multiple of $8 \cdot 2^{i-1}$ (i.e. 8, 16, 32, 64) (instead of $8 \cdot i$). All three counters were implemented in 64-bit registers.

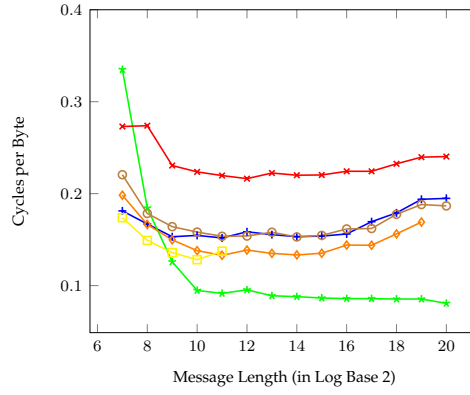
Table 5.4.2 gives the book-keeping cost incurred by the three counters. Figure 5.4.1 presents these characteristics graphically. The cost for OPT is significantly lower than the other two candidates as in this case the major book-keeping operation is executed only once per message. For VAR^r and STD^s the check for last message block accounts for most of the book-keeping cost. VAR^r has a slightly more involved counter update mechanism as compared to STD^s . This leads to a further increase in its book-keeping cost. However, we remark that we have not tried to optimize the counter updation in VAR^r beyond the obvious. So, it is quite possible that the book-keeping cost can be further reduced.

5.4.3 Performance of MACs

We summarize the performance results of the deterministic MACs in Table 5.4.3 and the stateful MACs in Table 5.4.4. Figure 5.4.2 and 5.4.3 give graphical characteristics of the performance data.

Table 5.4.2: Cycles per byte comparison between the book-keeping operations performed by the three CFF candidates.

Length	STD ^s				OPT ⁸	VAR ^{4,8}
	s = 8 bits	s = 16 bits	s = 32 bits	s = 64 bits		
128B	0.17	0.20	0.22	0.27	0.33	0.18
256B	0.15	0.17	0.18	0.27	0.18	0.17
512B	0.14	0.15	0.16	0.23	0.13	0.15
1kB	0.13	0.14	0.16	0.22	0.09	0.15
2kB	0.14	0.13	0.15	0.22	0.09	0.15
4kB	-	0.14	0.15	0.22	0.10	0.16
8kB	-	0.14	0.16	0.22	0.09	0.16
16kB	-	0.13	0.15	0.22	0.09	0.15
32kB	-	0.14	0.15	0.22	0.09	0.15
64kB	-	0.14	0.16	0.22	0.09	0.16
128kB	-	0.14	0.16	0.22	0.09	0.17
256kB	-	0.16	0.18	0.23	0.09	0.18
512kB	-	0.17	0.19	0.24	0.09	0.19
1MB	-	-	0.19	0.24	0.08	0.19

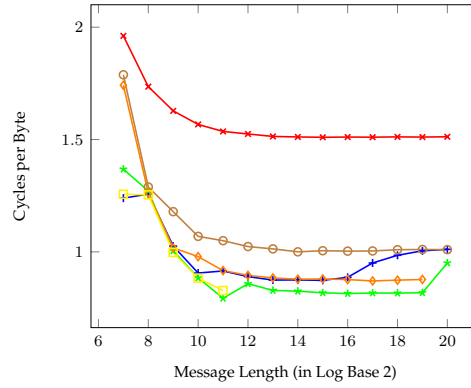
**Figure 5.4.1:** CPB plot for book-keeping operations of the three CFFs. Plots marked with +, *, ×, o, ◇, and □ correspond to VAR^{4,8}, OPT⁸, STD⁶⁴, STD³², STD¹⁶, and STD⁸, respectively.

Some notes on the characteristics: It is evident from Figure 5.4.2 and 5.4.3 that OPT gives the fastest MAC among all the three candidates. Also observe that when the input length is comparable to the counter size both STD^s and VAR^r closely resemble the performance curve for OPT. For instance, in Table 5.4.3, look at the entries corresponding to the range 8 kB (2¹⁶) to 64 kB (2¹⁹ bits). In this range all three counters offer similar cpb of around 0.8-0.9.

For a fixed value of s in STD^s, VAR^r outperforms STD^s, until the input length is significantly smaller than 2^s, or the counter size in VAR^r is smaller than s . For example, for $s = 32$ VAR^r is faster than STD^s, when the input length $\ell \leq 128\text{kB}$ (2²⁰ bits). At 256 kB, the counter size in VAR^r expands to 32 bits which causes a change in the slope of the curve. Even when the input length lies in the optimal range the gain in using STD^s is marginal when compared to the flexibility offered by VAR^r over a diverse range of input lengths.

Table 5.4.3: Comparison between the CPB values of CtMAC1 based on the three CFF candidates.

Length	STD ^s				OPT ⁸	VAR ^{4,8}
	s = 8 bits	s = 16 bits	s = 32 bits	s = 64 bits		
128B	1.26	1.74	1.79	1.96	1.37	1.24
256B	1.25	1.26	1.29	1.74	1.27	1.26
512B	1.00	1.02	1.18	1.63	1.00	1.03
1kB	0.88	0.98	1.07	1.57	0.88	0.91
2kB	0.83	0.92	1.05	1.54	0.79	0.91
4kB	-	0.90	1.02	1.52	0.86	0.89
8kB	-	0.88	1.01	1.51	0.83	0.87
16kB	-	0.88	1.00	1.51	0.83	0.88
32kB	-	0.88	1.01	1.51	0.82	0.87
64kB	-	0.88	1.00	1.51	0.81	0.89
128kB	-	0.87	1.00	1.51	0.82	0.95
256 kB	-	0.87	1.01	1.51	0.82	0.98
512kB	-	0.88	1.01	1.51	0.82	1.00
1MB	-	-	1.01	1.51	0.95	1.01

**Figure 5.4.2:** CPB plots for CtMAC1 based on the three CFF candidates. Plots marked with +, *, ×, o, ◇, and □ correspond to CtMAC1 with VAR^{4,8}, OPT⁸, STD⁶⁴, STD³², STD¹⁶, and STD⁸, respectively.

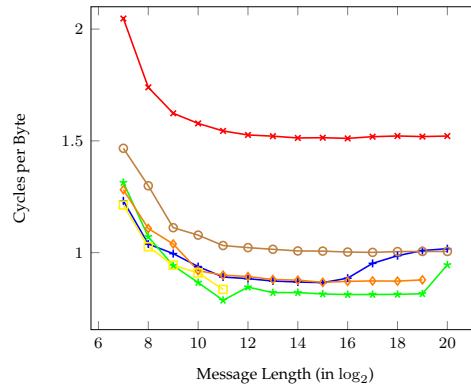
Note that, the effective counter size in VAR^r is always r bits less than the actual counter size. This is because the first r bits are reserved. For example when the counter size is 16 bits, only 12 bits are used. This reduces the maximum message length for 16-bit counter from 2^{23} bits to 2^{19} bits.

5.4.4 Performance of HAIFA

We summarize the performance results of CtHAIFA in Table 5.4.5. The graphical representation is illustrated in Figure 5.4.4. The comparison results are similar to those obtained earlier, in case of CtMAC1 and CtMAC2st.

Table 5.4.4: Comparison between the CPB values of CtMAC2st based on the three CFF candidates.

Length	STD ^s				OPT ⁸	VAR ^{4,8}
	s = 8 bits	s = 16 bits	s = 32 bits	s = 64 bits		
128B	1.21	1.28	1.47	2.05	1.31	1.23
256B	1.03	1.11	1.30	1.74	1.07	1.04
512B	0.94	1.04	1.11	1.62	0.94	1.00
1kB	0.91	0.92	1.08	1.58	0.87	0.94
2kB	0.84	0.90	1.03	1.54	0.79	0.89
4kB	-	0.89	1.02	1.53	0.85	0.88
8kB	-	0.88	1.01	1.52	0.82	0.87
16kB	-	0.88	1.01	1.51	0.82	0.87
32kB	-	0.87	1.01	1.51	0.82	0.87
64kB	-	0.87	1.00	1.51	0.81	0.89
128kB	-	0.87	1.00	1.52	0.81	0.95
256kB	-	0.87	1.00	1.52	0.81	0.99
512kB	-	0.88	1.01	1.52	0.82	1.01
1MB	-	-	1.01	1.52	0.95	1.02

**Figure 5.4.3:** CPB plots for CtMAC2st based on the three CFF candidates. Plots marked with +, *, ×, o, ◇, and □ correspond to CtMAC2st with VAR^{4,8}, OPT⁸, STD⁶⁴, STD³², STD¹⁶, and STD⁸, respectively.**Table 5.4.5:** Comparison between the CPB values of CtHAIFA based on the three CFF candidates.

Length	STD ^s				OPT ⁸	VAR ^{4,8}
	s = 8 bits	s = 16 bits	s = 32 bits	s = 64 bits		
128B	7.42	8.16	8.96	13.83	7.64	7.58
256B	7.37	7.76	8.95	13.33	7.31	7.47
512B	7.17	7.55	8.69	13.15	7.00	7.53
1kB	7.02	7.55	8.71	13.07	6.92	7.65
2kB	6.95	7.48	8.63	13.03	6.88	7.62
4kB	-	7.43	8.65	13.00	7.43	7.59
8kB	-	7.40	8.61	12.92	7.42	7.57
16kB	-	7.42	8.61	12.95	7.42	7.60
32kB	-	7.43	8.63	12.99	7.41	7.62
64kB	-	7.41	8.64	12.97	7.41	7.78
128kB	-	7.41	8.65	12.98	7.41	8.37
256kB	-	7.42	8.66	12.99	7.42	8.66
512kB	-	7.44	8.67	12.99	7.43	8.82
1MB	-	-	8.66	12.99	8.65	8.90

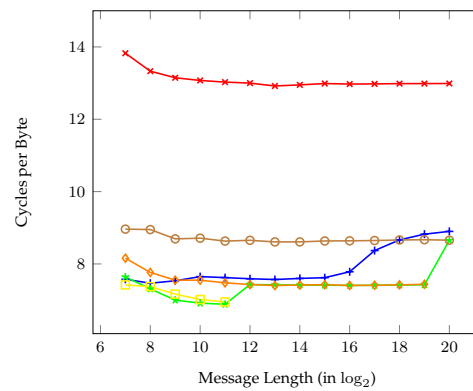


Figure 5.4.4: CPB plots for CtHAIFA based on the three CFF candidates. Plots marked with +, *, ×, ○, ◇, and □ marked plots correspond to CtHAIFA with VAR^{4,8}, OPT⁸, STD⁶⁴, STD³², STD¹⁶, and STD⁸, respectively.

Part II

Analysis of Online and Tweakable Ciphers

Chapter 6

Tweakable Pseudorandom Permutation based Online Ciphers

Till now, we have seen schemes concerned with the authenticity and integrity of data, i.e., message authentication codes. In this chapter, we shift our focus to confidentiality. We study a special class of encryption schemes, called online ciphers [18, 39]. In particular, we study highly secure¹ and efficient² online cipher schemes based on tweakable block ciphers.

Andreeva et al. [7] showed that online ciphers are equivalent to arbitrary tweak length (ATL) TBCs. Although this result is more of theoretical interests, the TBC to online cipher converter can be coupled with XTX [139] to get an online cipher. In this chapter, we study the (un)suitability of straightforward application of this approach. Further, we study variants of this approach to construct efficient and highly secure online ciphers.

6.1 A Design Strategy for TBC-based BBB Online Cipher

The rate of any cryptographic scheme can be defined as the ratio of the number of blocks in any input to the number of primitive (TBC in this case) calls for that input. In most of the cases, rate is independent of the input. A rate ≥ 1 scheme will, in general, be more efficient than a rate $\frac{1}{2}$ scheme when identical primitives are employed. In this chapter, we use rate and the number of field multiplications as our parameters for efficiency.

¹Beyond-the-birthday-bound security.

²Number of primitive calls is at most $\lceil \ell/n \rceil$ where ℓ denotes the message length and n denotes the primitive block size.

6.1.1 The Iterated TBC View of Online Ciphers

In a recent work [7] on a generic construction of offline ciphers (deterministic encryption scheme) using online ciphers as primitives, Andreeva et al. made the following useful observation:

An ideal online cipher is equivalent to an ideal arbitrary tweak length (ATL) TBC.

We reproduce their result in Theorem 6.1.1.

Theorem 6.1.1 ([7, Theorem 1]). *There is a security preserving one-to-one correspondence between online ciphers on $\mathbb{B}^+$ and tweakable block ciphers on $\mathbb{B}$ with tweak space $\mathbb{B}^*$.*

While a more detailed proof is available in [7], we describe their generic construction of online cipher based on a tweakable block cipher with arbitrary tweak space. Let $\widehat{E}_K$ be a TBC with tweak space $\mathbb{B}^*$ and block space $\mathbb{B}$. We define an online cipher $\mathfrak{O}_{\widehat{E}}$ over $\mathbb{B}^+$ as $\forall \ell \geq 1, \forall x := (x_1, \dots, x_\ell) \in \mathbb{B}^\ell$,

$$\mathfrak{O}_{\widehat{E}}^+(x) := \widehat{E}_K^{x_0}(x_1) \parallel \widehat{E}_K^{x^{(1)}}(x_2) \parallel \dots \parallel \widehat{E}_K^{x^{(i-1)}}(x_i) \parallel \dots \parallel \widehat{E}_K^{x^{(\ell-1)}}(x_\ell),$$

where x_0 is some constant. One can easily verify that,

$$\mathbf{Adv}_{\mathfrak{O}_{\widehat{E}}}^{\text{osprp}}(q, \sigma, t) \leq \mathbf{Adv}_{\widehat{E}}^{\text{tsprp}}(\sigma, t'), \quad (6.1)$$

where $t' = t + O(\sigma)$. Specifically, when we replace $\widehat{E}$ with an ideal ATL tweakable random permutation $\widehat{\Pi}$ we get $\mathbf{Adv}_{\mathfrak{O}_{[\widehat{\Pi}]}}^{\text{osprp}}(q, \sigma, t) = 0$. A minor variant of the above mentioned construction may also consider the previous ciphertext blocks along with the plaintext blocks, i.e., tweak for the i -th block is $(x^{(i-1]}, \mathfrak{O}_{\widehat{E}}^+(x^{(i-1]}))$, and the modified definition is,

$$\mathfrak{O}_{\widehat{E}}^+(x) := \widehat{E}_K^{x_0}(x_1) \parallel \widehat{E}_K^{(x^{(1]}, \mathfrak{O}_{\widehat{E}}^+(x^{(1]}))}(x_2) \parallel \dots \parallel \widehat{E}_K^{(x^{(\ell-1]}, \mathfrak{O}_{\widehat{E}}^+(x^{(\ell-1]}))}(x_\ell).$$

It is not hard to see that this does not give any added security advantage. But the utility of this modification will become more apparent as we proceed. Thus any online cipher can be viewed as a chain of iterated TBC. We call this equivalent view of online ciphers, *the iterated TBC view of online cipher*.

6.1.2 $\mathfrak{O}_{\text{XTX}}$: Moving from Theory to Practice

Given the strong security guarantee of Eq. (6.1) and efficiency (rate 1 construction) of $\mathfrak{O}_{\tilde{\mathbf{E}}}$, it is only natural to look for practical instantiations. The next immediate question is: how can we instantiate an ATL TBC based on a fixed tweak length (FTL) TBC?

XTX [139] by Minematsu and Iwata is an elegant way of extending the tweak length of an FTL TBC. At the highest level, XTX employs a pAXU hash which takes a tweak value of arbitrary length as input and computes a fixed length tweak and input/output masking for the underlying FTL TBC. Formally, let $H : \mathbb{B}^* \rightarrow \mathbb{T} \times \mathbb{B}$ be an ϵ -pAXU. For convenience, we have dropped the key of the hash function from the notation, but keep in mind that the key is present. The hash output $H(T)$ is parsed into $H_{\text{twk}}(T) || H_{\text{msk}}(T)$. Using this pAXU and a TBC $\tilde{\mathbf{E}}$ over the tweak space $\mathbb{T}$, XTX is defined as $\text{XTX}_K^T(x) = \tilde{\mathbf{E}}_K^{H_{\text{twk}}(T)}(x \oplus H_{\text{msk}}(T)) \oplus H_{\text{msk}}(T)$. In [139], Minematsu and Iwata have shown the following upper bound on the TSPRP advantage of XTX.

Theorem 6.1.2. $\text{Adv}_{\text{XTX}}^{\text{tsprp}}(\sigma, t') \leq \text{Adv}_{\tilde{\mathbf{E}}}^{\text{tsprp}}(\sigma, t'') + \epsilon\sigma^2$, where $t'' = t' + \text{Time}_H \times O(\sigma)$.

By using Eq. (6.1) and Theorem 6.1.2, we obtain a construction $\mathfrak{O}_{\text{XTX}}$ with OSRP advantage at most $\text{Adv}_{\tilde{\mathbf{E}}}^{\text{tsprp}}(\sigma, t'') + \sigma^2\epsilon$. A recent proposal by Forler et al., called POEx [71, 72], is an implicit example of $\mathfrak{O}_{\text{XTX}}$. Although $\mathfrak{O}_{\text{XTX}}$ is simple (both in description and security proof), it requires very strong bound on ϵ , close to $2^{-(n+\tau)}$. Now we describe how this becomes an issue when coupled with the need for efficiency. In this chapter we focus on field multiplication based hash functions as the use of TBC in hash computation will make the overall construction inefficient (decreases the rate).

SECURITY VS HASH KEY SIZE: As mentioned before, we only consider algebraic hash functions. Note that, $H : \mathbb{B}^* \rightarrow \mathbb{T} \times \mathbb{B}$ is required to be ϵ -pAXU for some ϵ . If we keep hash key size to be $n + \tau$ then $\epsilon' \geq \ell'/2^{n+\tau}$ where $\ell' = \ell n / (n + \tau)$ (the number of $(n + \tau)$ -bit strings present in ℓ blocks input). We can achieve this bound by considering polynomial hash defined over $(n + \tau)$ -bit binary field. If we plug this ϵ we obtain a bound of the form $\sigma^2\ell/2^{n+\tau}$. To get rid of the ℓ factor we need to apply hash functions with larger hash key size, such as Pseudo dot product [36, 87, 188], multi-linear hash [43, 75, 177], and EHC [150], which is practically infeasible.

6.1.2.1 Towards A Possible Remedy

One possible way of solving or avoiding this degradation/inefficiency could be to use a fixed number of previous blocks information instead of the entire history of previous blocks. This will certainly replace the ℓ factor in case of poly-hash. But now we have

to use both plaintext and ciphertext pairs, otherwise the overall design will no longer be secure (the adversary can always keep the required plaintext blocks fixed). Further a direct security reduction like $\mathfrak{D}_{\text{XTX}}$ will not be possible any more and an independent security analysis is required. In section 6.3 we use this design strategy to give an (almost) affirmative solution for (a) in shape of a new construction that achieves a security in the order of $\min(2^n/n, 2^{(n+\tau)/2})$ block-queries.

POEx, though an implicit instance of $\mathfrak{D}_{\text{XTX}}$, tries to use this strategy. It fixes the input of the hash function to just the immediate previous input-output of the underlying TBC to avoid the ℓ factor while maintaining a key size of $n + \tau$ bits. Unfortunately, there is a flaw in their analysis (discussed in next section). It seems that the actual bound for POEx must have an ℓ factor.

6.2 Revisiting POEx

In this section, we review the security of POEx, a rate-1 online cipher based on tweakable block ciphers, which claims Beyond Birthday Security [71].

6.2.1 Description of POEx

POEx is an extended version of POE, the online cipher used under POET [1]. It constructs an online cipher by iterated usage of a fixed tweak length TBC and a pAXU hash function on two block input/output. The algorithmic description of POEx is given in Algorithm 6.2.1 and a schematic illustration of the encryption/decryption process is shown in Figure 6.2.1. On a macro level the construction looks neat and the security claim looks correct. But in the following subsections we describe a birthday bound attack on POEx that invalidates the security claim.

The security of POEx is claimed to be $2^{\frac{n+\tau}{2}}$ block-queries. More specifically we have the exact security claim in Theorem 6.2.1.

Theorem 6.2.1 (Theorem 3 in [71]).

$$\text{Adv}_{\text{POEx}[\tilde{E}, H]}^{\text{osprp}} \leq 2\text{Adv}_{\tilde{E}^\pm}^{\text{tsprp}}(\sigma, O(t)) + 2(\sigma + 1)^2 \epsilon \cdot \left(2 + \frac{2^\tau}{2^n - (\sigma + 1)}\right)$$

6.2.1.1 Flaw in POEx Security Analysis

The security of POEx mainly rely on the pAXU bound (say ϵ) of the underlying hash function. For example one of the bad events in the security proof of POEx is $(T_{(i,j)}, X_{(i,j)})$

Algorithm 6.2.1 Definition of $\text{POEx}_{\tilde{E}, H}^{\pm}$. $\mu, \nu \in \mathbb{F}_{2^n}$, are two application specific constants.

1: function $\text{POEx}_{\tilde{E}, H_L}^+(P)$	1: function $\text{POEx}_{\tilde{E}, H_L}^-(C)$
2: $(X_0, Y_0) \leftarrow (\mu, \nu)$	2: $(X_0, Y_0) \leftarrow (\mu, \nu)$
3: $(P_1, \dots, P_\ell) \xleftarrow{n} P$	3: $(C_1, \dots, C_\ell) \xleftarrow{n} C$
4: for $i \leftarrow 1$ to ℓ do	4: for $i \leftarrow 1$ to ℓ do
5: $(T_i, U_i) \leftarrow H_L(X_{i-1}, Y_{i-1})$	5: $(T_i, U_i) \leftarrow H_L(X_{i-1}, Y_{i-1})$
6: $X_i \leftarrow P_i \oplus U_i$	6: $Y_i \leftarrow C_i \oplus U_i$
7: $Y_i \leftarrow \tilde{E}_K^{T_i}(X_i)$	7: $X_i \leftarrow \tilde{E}_K^{-T_i}(Y_i)$
8: $C_i \leftarrow Y_i \oplus U_i$	8: $P_i \leftarrow X_i \oplus U_i$
9: end for	9: end for
10: return $C := (C_1 \parallel \dots \parallel C_\ell)$	10: return $P := (P_1 \parallel \dots \parallel P_\ell)$
11: end function	11: end function

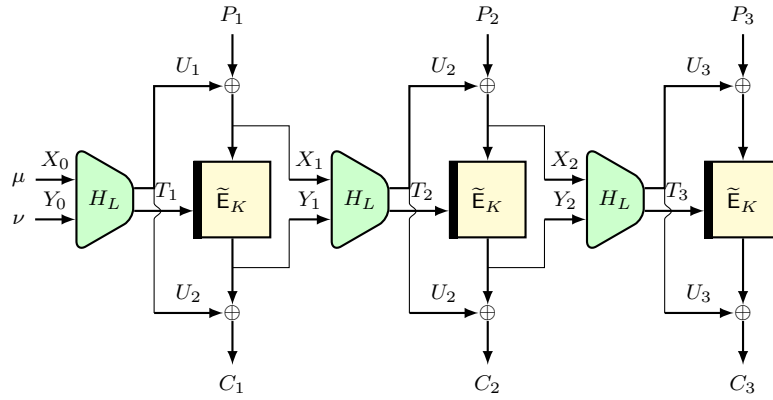


Figure 6.2.1: Schematic of the encryption/decryption process for a 3-block plaintext/-ciphertext using POEx construction.

(tweak-input) collision. This case has been bounded to $\sigma^2\epsilon/2$. The argument being: there can be at most $\binom{\sigma}{2}$ pairs and for each pair the pAXU bound is ϵ . But this argument only holds when all inputs to the hash function are independent of the hash key. In case of POEx this is not true as the input to the hash function is dependent on the previous hash values (linear combination of previous hash value and current message block). For example if we consider the candidate hash function given in [71], the hash-function inputs are dependent and the polynomial degree will accumulate through subsequent inputs to poly-hash. So the security claim is invalid. It seems that POEx requires a stronger assumption, i.e. it needs pAXU assumption on the iterated hash function. This may not be easy to achieve with algebraic hash functions.

6.2.2 Birthday Bound Attack on POEx

We substantiate the flaw discovered in the previous subsection by constructing an artificial example of a pAXU hash function that is not secure when used in iterated fashion.

For the sake of simplicity we take $\tau = n$. Similar attacks can be constructed for any arbitrary τ . The main idea is to construct an AXU hash, say SciFi, that on any given input converges to a fixed value when iterated a moderate number of times. By looking at the masking and tweak generator it can be observed that the adversary can use SciFi to fix the masking to an unknown but constant value. But the same is not possible for the tweak value as the Y_i value is not in the control of the adversary. Once the masking is fixed, the adversary can expect tweak collisions in birthday bound.

6.2.2.1 AXU Hash using Random Mapping over Ranked Nodes

Let $\text{rank} : \mathbb{B} \rightarrow [0..n]$ be a surjective function which is defined as follows:

$$\forall x \in \mathbb{B}, \text{rank}(x) := \begin{cases} 0 & x = 0, \\ i & 2^{i-1} \leq x \leq 2^i - 1. \end{cases}$$

Let $\mathbb{B}_i = \{x \in \mathbb{B} : \text{rank}(x) = i\}$ and $\mathbb{B}_{<i} = \cup_{j<i} \mathbb{B}_j$. Clearly $\mathbb{B} = \sqcup_{i \in [n]} \mathbb{B}_i$. Let

$$\begin{aligned} \Psi &\leftarrow \$ \text{Perm}(\mathbb{B}) \\ \Phi_0 &\leftarrow \$ \text{Func}(\mathbb{B}_0 \times \mathbb{B}, \mathbb{B}_0) \\ \forall i \in [1..n], \Phi_i &\leftarrow \$ \text{Func}(\mathbb{B}_i \times \mathbb{B}, \mathbb{B}_{<i}) \end{aligned}$$

We write Φ to denote the random vector $(\Phi_0, \dots, \Phi_n)$. We define a keyed hash function $\text{SciFi}_{\Psi, \Phi} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ as follows:

$$\forall (x, y) \in \mathbb{B} \times \mathbb{B}, \text{SciFi}_{\Psi, \Phi}(x, y) := \Psi^{-1} \circ \Phi_{\text{rank}(\Psi(x))}(\Psi(x), y).$$

For the sake of simplicity, we drop the subscript Ψ, Φ from $\text{SciFi}_{\Psi, \Phi}$. It is easy to verify that after at most n iterations SciFi returns a fixed value, i.e. for any x and $y^n := (y_1, \dots, y_n) \in \mathbb{B}^n$ we have

$$\text{SciFi}^n(x, y^n) := \text{SciFi}(\dots (\text{SciFi}(\text{SciFi}(x, y_1), y_2) \dots, y_n) = c,$$

where c is some unknown but constant value. This can be argued as follows: after each iteration the rank of the output reduces by at least 1. Since there are finite number of ranks, $n + 1$ to be precise, we must be at rank 0 in n iterations. Once we reach rank 0 the output becomes fixed to $\Psi^{-1}(0)$. We summarize this property in Lemma 6.2.2.

Lemma 6.2.2. *For all $m \geq n$, and $(x, y^m) \in \mathbb{B} \times \mathbb{B}^m$, we have $\text{SciFi}^m(x, y^m) = c$, where $c = \Psi^{-1}(0)$.*

In Lemma 6.2.3 below, we show that SciFi is an $O\left(\frac{n}{2^n}\right)$ -AXU hash function. Thus, security wise it can be a good candidate for AXU hash.

Lemma 6.2.3. *For distinct $(x_1, y_1) \neq (x_2, y_2) \in \mathbb{B} \times \mathbb{B}$ and $\delta \in \mathbb{B}$, we have*

$$\Pr [\text{SciFi}(x_1, y_1) \oplus \text{SciFi}(x_2, y_2) = \delta] \leq \frac{n+2}{2^n}.$$

In other words, SciFi is an $(n+2)2^{-n}$ -AXU hash function.

Proof. Let R_1 and R_2 denote $\text{rank}(\Psi(x_1))$ and $\text{rank}(\Psi(x_2))$, respectively. Let

$$\mathcal{NZ} := [n] \times [n] \text{ and } \mathcal{Z} := (n) \times (n) \setminus \mathcal{NZ}.$$

Now we have,

$$\begin{aligned} & \Pr [\text{SciFi}(x_1, y_1) \oplus \text{SciFi}(x_2, y_2) = \delta] \\ &= \overbrace{\Pr [\text{SciFi}(x_1, y_1) \oplus \text{SciFi}(x_2, y_2) = \delta, (R_1, R_2) \in \mathcal{NZ}]}^{\epsilon_1} \\ &+ \overbrace{\Pr [\text{SciFi}(x_1, y_1) \oplus \text{SciFi}(x_2, y_2) = \delta, (R_1, R_2) \in \mathcal{Z}]}^{\epsilon_2} \end{aligned} \quad (6.2)$$

We bound ϵ_1 and ϵ_2 below:

BOUND ON ϵ_1 : For a fixed $\psi \in \text{Perm}(\mathbb{B})$ if we consider inputs (x_1, y_1) and (x_2, y_2) such that $\psi(x_1) \neq 0$ and $\psi(x_2) \neq 0$, then the bound effectively reduces to bounding the probability that the sum of outputs of two (possibly) independent random functions on distinct inputs equals to δ . This can be bounded by $2^{-\max\{r_1, r_2\}}$ (by conditioning on the output of the function with smaller range). Using the preceding argument and conditioning on Ψ , we have

$$\begin{aligned} \epsilon_1 &\leq \sum_{(r_1, r_2) \in \mathcal{NZ}} \Pr [\Psi^{-1}(\Phi_{r_1}(\Psi(x_1), y_1)) \oplus \Psi^{-1}(\Phi_{r_2}(\Psi(x_2), y_1)) = \delta] \times \frac{2^{r_1+r_2-2}}{2^{2n}} \\ &\leq \sum_{(r_1, r_2) \in \mathcal{NZ}} \frac{1}{2^{\max\{r_1, r_2\}-1}} \times \frac{2^{r_1+r_2-2}}{2^{2n}} \\ &\leq \sum_{(r_1, r_2) \in \mathcal{NZ}} \frac{2^{r_1-1}}{2^{2n}} = \frac{n}{2^n} \end{aligned} \quad (6.3)$$

BOUND ON ϵ_2 : In this case at least one of x_1 or x_2 is mapped to 0 by Ψ . Further $R_1 = R_2 = 0$ is possible if and only if $x_1 = x_2$, in which case we can simply bound the probability to at most 2^{-n} . So without loss of generality we assume that at least one of them is non-zero, say R_1 . Now we can proceed as earlier and

we have

$$\begin{aligned}
\epsilon_2 &\leq \frac{1}{2^n} + \sum_{r_1 \in [n]} \Pr [\Psi^{-1}(\Phi_{r_1}(\Psi(x_1), y_1)) = x_2 \oplus \delta] \times \frac{2^{r_1-1}}{2^{2n}} \\
&\leq \frac{1}{2^n} + \sum_{r_1 \in [n]} \frac{1}{2^{r_1-1}} \times \frac{2^{r_1-1}}{2^{2n}} \\
&\leq \frac{2}{2^n}
\end{aligned} \tag{6.4}$$

The result follows from Eq. (6.2)-(6.4). $\square$

6.2.2.2 Attack Description

Lemma 6.2.3 shows that SciFi is a good AXU hash function, whereas Lemma 6.2.2 shows that the iterated version $\text{SciFi}^{\geq n}$ is a pathetic AXU hash. We use this later fact to construct a birthday bound attack on POEx. Suppose we instantiate the POEx construction using a tuple of AXU hash functions $H := (\Gamma, \text{SciFi})$ where $\Gamma \leftarrow \$ \text{Func}(\mathbb{B} \times \mathbb{B}, \mathbb{B})$ is chosen independently of SciFi. Further the mapping is defined as follows:

$$H(X_i, Y_i) = (\Gamma(X_i, Y_i), \text{SciFi}(X_i, Y_i))$$

Since Γ is a uniform random function over $\text{Func}(\mathbb{B} \times \mathbb{B}, \mathbb{B})$ and hence a 2^{-n} -AXU hash function, using Proposition 2.3.5 we can conclude that H is an $\frac{n+2}{2^{2n}}$ -pAXU hash function. Note that, the choice of universal hash is not rigid. We can take any good candidate of universal hash provided it is keyed independently of SciFi. Now consider an adversary $\mathcal{A}$ that works as follows:

1. $\mathcal{A}$ makes q queries of the form $(P_i \| 0^{\ell-1})$, such that $P^q \in (\mathbb{B})_q$ and $\ell > n$, and observes the outputs $(C_{(i,1)} \| \dots \| C_{(i,\ell)})$.
2. If $(C_{(i,j)}, C_{(i,j+1)}) = (C_{(i',j')}, C_{(i',j'+1)})$ for two distinct block indices (i, j) and (i', j') , then $\mathcal{A}$ returns 1.

For an ideal online cipher this should require roughly 2^n many blocks. But for $\text{POEx}_{\tilde{\Pi}, H}$ this would require roughly $2^{n/2}$ many blocks. This can be argued using Lemma 6.2.2 which, in this case, implies that for each query beyond block index $n - 1$, the input becomes a constant value. So all that is required is a tweak collision which can be achieved if we have roughly $2^{n/2}$ blocks. Hence POEx is at most birthday bound secure online cipher.

Remark 6.2.4. We remark that the hash function notion (pAXU) used in the original POEx design [71] has been modified in a later journal version [72] with due acknowledgments to our observations. The authors have defined a new and stronger notion of hash function, called Chained-Partial-AXU Hash Function (an extension of the pAXU notion for iterated use), and based their security analysis on this notion. However, the authors have not given any practical instantiations for such hash function and we speculate that given the stringent conditions it will be difficult to construct an efficient candidate with close to $2^{-(n+\tau)}$ bound. We refer the readers to [72] for a more detailed exposition. In this chapter, we have considered the original POEx design from [71].

6.3 XTC: Rate-1 (Almost) Optimally Secure Online Cipher

In the preceding section we showed a subtle flaw in the security claim (and proof) of POEx. Here we explore the possibility of another approach towards a practical instantiation of Andreeva et al. [7] TBC to online cipher converter. As discussed in section 6.1, $\mathcal{O}_{\text{XTX}}$ cannot achieve both rate-1 and BBB security simultaneously. A possible remedy is the idea of using just a fixed number of previous input-output block-pairs information. Based on this idea, we propose XTC, that achieves $\min(2^n/n, 2^{\frac{n+\tau}{2}})$ block-queries security. The XTC construction is similar to POEx in the sense that it follows POEx's idea of generating the mask and tweak using a fixed number of previous blocks information. However, POEx uses pAXU property on hash function with at most ℓ -block input, whereas XTC uses pAXU property on hash function with at most three blocks input.

6.3.1 An Impossibility Result

In POEx [71] as well as XTC (see subsection 6.3.2), only a small number of previous blocks are used for tweak and mask computations. POEx uses the immediate previous TBC input and output blocks, where as XTC employs the previous two plaintext and ciphertext blocks. This is done to avoid the loss of ℓ factor in security. In Theorem 6.3.2, we show an impossibility result that would imply that this approach will be futile when the security goal is more than 2^n block-queries, which is a natural possibility for $\tau > n$.

Definition 6.3.1. A pair of distinct tuples $(a_1, \dots, a_m)$ and $(b_1, \dots, b_m)$ is called an m -block *all-but-first collision pair* if $a_i = b_i$ for all $i \in [2 \dots m]$.

Theorem 6.3.2. Given $m \geq 3$, for an ideal online cipher an m -block *all-but-first collision pair* can be constructed in $O(m^2 2^n)$ block-queries.

Proof. Consider the following algorithm $\mathcal{A}$:

1. Make roughly $2^{n/2}$ distinct encryption queries (P_i, x) and stores the result $(C_{(i,1)}, C_{(i,2)})$ in $\mathcal{L}$. At least one collision on the second ciphertext block is expected, say (P_i, x) and (P_j, x) with the corresponding ciphertext $(C_{(i,1)}, y)$ and $(C_{(j,1)}, y)$.
2. Rename $P_i, P_j, C_{(i,1)}$, and $C_{(j,1)}$ as p_1, p'_1, c_1 and c'_1 . Let $p'_2 = p_2 := x$ and $c'_2 = c_2 = y$.
3. For $i \in [3 \dots m]$:
 - (a) For all $a \in \mathbb{B}$: Make 2 queries $(p_1, p_2, \dots, p_{i-1}, a)$ and $(p'_1, p'_2, \dots, p'_{i-1}, a)$, and check if $(c_2, \dots, c_i) = (c'_2, \dots, c'_i)$. If the equality holds for some a , then fix $p'_i = p_i := a$ and move to next i .
4. Set $p^m := (p_1, \dots, p_m)$, $c^m := (c_1, \dots, c_m)$, $p'^m := (p'_1, \dots, p'_m)$, and $c'^m = (c'_1, \dots, c'_m)$.
5. Returns (p^m, c^m) and (p'^m, c'^m) as the m -block all-but-first collision pair.

As the first block is distinct for all queries in step 1 above, there is very high chance of getting a collision pair. In step 3(a) at the i -th iteration $(p_1, p_2, \dots, p_{i-1})$ is distinct from $(p'_1, p'_2, \dots, p'_{i-1})$ so the outputs c_i and c'_i are drawn independently from $\mathbb{B}$, whence we expect one collision in 2^n tries. Hence the total block-query complexity is bounded by $O(m^2 \cdot 2^n)$. $\square$

Step 1 of $\mathcal{A}$ in the proof of Theorem 6.3.2 gives a simple corollary that lower bounds the number of previous blocks information required for n -bit security.

Corollary 6.3.3. *To achieve n -bit security at least 2 previous plaintext-ciphertext blocks information is required.*

Remark 6.3.4. Although we constructed an all-but-first collision pair with start index as the first index of the queries, similar strategy can be applied if the collision pair has to be shifted to a later start index.

As long as m is a polynomial in n and n is sufficiently large, Theorem 6.3.2 and Remark 6.3.4 imply that an adversary can always create a collision on m previous input and output blocks in $O(2^n)$ (ignoring the polynomial factors), and thus the tweak value for the current block will collide leading to an easy distinguisher. Thus, to achieve significantly more than 2^n block-queries security one has to use more than m (polynomial in n) many previous blocks.

6.3.2 Description of XTC

For the sake of simplicity we assume tweak size $\tau = n$ while describing the scheme and its security. Later we give ways to extend the result for all τ . Using Corollary 6.3.3 we know that 2 previous plaintext-ciphertext block-pairs can be sufficient for the desired security goal. The algorithmic description of XTC is given in Algorithm 6.3.1, while Figure 6.3.1 gives a pictorial illustration of the encryption/decryption process. XTC can be viewed as an iteration of XTX, much like $\mathfrak{D}_{\text{XTX}}$, albeit with tweak length fixed to three blocks. We call this equivalent view $\mathfrak{D}'_{\text{XTX}}$. In Figure 6.3.1, this equivalent view is represented by dashed rectangles which denote the underlying XTX component.

Although $\mathfrak{D}_{\text{XTX}}$ and XTC are similar in their use of XTX for tweak length extension, yet XTC is much more efficient while maintaining a satisfactory level of security. For a plaintext-ciphertext pair (P, C) we only use $(S_{i-2} = P_{i-2} \oplus C_{i-2}, P_{i-1}, C_{i-1})$ as the tweak input to the i -th block XTX. While this enables the application of efficient algebraic hash functions within XTX, the security analysis of the overall scheme becomes a bit more involved.

6.3.2.1 Design Choices and Rationale

We choose the pair $(S_{i-2}, P_{i-1}, C_{i-1})$ as it reduces the state size by one block, and is the simplest such pair. Further it can be easily verified that we cannot reduce this to 2 blocks without compromising on security. As far as the choice of hash function is concerned, we need universal property for the tweak part and XOR universal for the masking part. In other words we need H to be a pAXU hash over 3 blocks input. Since we are considering only rate-1 constructions we recommend algebraic hash functions for H .

BRW-BASED PAXU CANDIDATE: BRW hash function is an efficient candidate that requires just one multiplication when the input is restricted to three blocks. It was proposed by Bernstein [28] based on previous works by Rabin and Winograd [169]. For a 3-blocks input (a_1, a_2, a_3) , $\text{BRW}_x(a_1, a_2, a_3)$ is defined as:

$$\text{BRW}_x(a_1, a_2, a_3) := (a_1 \oplus x) \odot (a_2 \oplus x^2) \oplus a_3,$$

where $\odot$ and $\oplus$ denote field multiplication and addition, respectively, over $\mathbb{F}_{2^n}$ generated by some predefined primitive element. It is well-known that BRW hash with three blocks input is $3 \cdot 2^{-n}$ -universal [28, 46].

For $L \in \mathbb{B}^2$, let L_1 and L_2 be the most and least, respectively, significant n bits of L . We define the keyed hash function, $H_L : \mathbb{B} \times \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B} \times \mathbb{B}$ as follows

$$\forall S, P, C \in \mathbb{B}, \quad (T, U) := H_L(S, P, C) = (\text{BRW}_{L_1}(C, S, P), \text{BRW}_{L_2}(C, S, 0)).$$

Note that, the hash function definition is not arbitrary over the three block inputs. For example the security reduces to birthday bound if we just swap the values of T and U as after the swapping, U does not follow AXU. In Lemma 6.3.5 we bound the pAXU probability of H .

Lemma 6.3.5. *For distinct $(S, P, C), (S', P', C') \in \mathbb{B}^3$, and $\delta \in \mathbb{B}$ we have,*

$$\Pr_{\mathbf{L}} [H_{\mathbf{L}}(S, P, C) \oplus H_{\mathbf{L}}(S', P', C') = (0, \delta)] \leq \frac{9}{2^{2n}}.$$

Proof. To compute the output (T, U) , $H_{\mathbf{L}}$ employs two calls to **BRW** hash with independent keys L_1 and L_2 , which enables a universal bound of $9 \cdot 2^{-2n}$. But we need pAXU property, which in this case means that the second output should have AXU property. Note that, the last input block for the second call of **BRW** is always zero. This enables us to consider the difference block as part of the input and reduce the AXU bound to universal bound.

$$\begin{aligned} & \Pr_{\mathbf{L}} [H_{\mathbf{L}}(S, P, C) \oplus H_{\mathbf{L}}(S', P', C') = (0, \delta)] \\ & \leq \Pr_{L_1} [\text{BRW}_{L_1}(C, S, P) = \text{BRW}_{L_1}(C', S', P')] \\ & \quad \times \Pr_{L_2} [\text{BRW}_{L_2}(C, S, 0) \oplus \text{BRW}_{L_2}(C', S', 0) = \delta] \\ & = \Pr_{L_1} [\text{BRW}_{L_1}(C, S, P) = \text{BRW}_{L_1}(C', S', P')] \\ & \quad \times \Pr_{L_2} [\text{BRW}_{L_2}(C, S, \delta) = \text{BRW}_{L_2}(C', S', 0)] \\ & \leq \frac{9}{2^{2n}}. \end{aligned}$$

□

We emphasize here that the above definition is not the only possibility. Indeed, one can use polynomial hash to further reduce the state size at the cost of two more multiplication. In general any good pAXU hash function can be employed. In this work we mainly focus on saving on the number of multiplications.

DERIVING HASH KEYS VIA TBC: XTC requires two hash keys. However the hash keys can be easily derived via TBC by reserving one tweak bit for key generation. For example one can use $(L_1, L_2) = (\tilde{\mathbf{E}}_K^{1\|0}(0), \tilde{\mathbf{E}}_K^{1\|1}(1))$ as the hash key and fix the most

Algorithm 6.3.1 Definition of $\text{XTC}_{\tilde{E}, H}^{\pm}$. We take $\mu = v = 0$ and $\nu = 1$.

1: function $\text{XTC}_{\tilde{E}, H}^{+}(P)$	1: function $\text{XTC}_{\tilde{E}, H}^{-}(C)$
2: $S_{-1} \leftarrow v$	2: $S_{-1} \leftarrow v$
3: $P_0 \leftarrow \mu$	3: $P_0 \leftarrow \mu$
4: $C_0 \leftarrow \nu$	4: $C_0 \leftarrow \nu$
5: $(P_1, \dots, P_\ell) \xleftarrow{n} P$	5: $(C_1, \dots, C_\ell) \xleftarrow{n} C$
6: for $i \leftarrow 1$ to ℓ do	6: for $i \leftarrow 1$ to ℓ do
7: $(T_i, U_i) \leftarrow H_L(S_{i-2}, P_{i-1}, C_{i-1})$	7: $(T_i, U_i) \leftarrow H_L(S_{i-2}, P_{i-1}, C_{i-1})$
8: $X_i \leftarrow P_i \oplus U_i$	8: $Y_i \leftarrow C_i \oplus U_i$
9: $Y_i \leftarrow \tilde{E}_K^{T_i}(X_i)$	9: $X_i \leftarrow \tilde{E}_K^{-T_i}(Y_i)$
10: $C_i \leftarrow Y_i \oplus U_i$	10: $P_i \leftarrow X_i \oplus U_i$
11: $S_i \leftarrow P_i \oplus C_i$	11: $S_i \leftarrow P_i \oplus C_i$
12: end for	12: end for
13: return $C := (C_1 \parallel \dots \parallel C_\ell)$	13: return $P := (P_1 \parallel \dots \parallel P_\ell)$
14: end function	14: end function

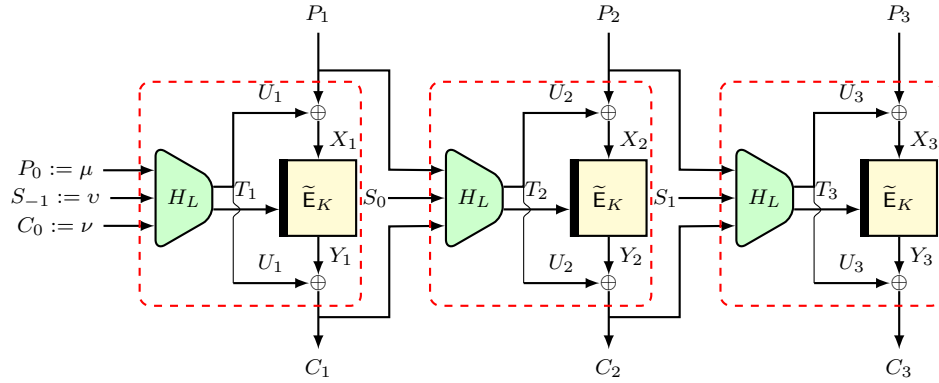


Figure 6.3.1: Schematic of the encryption/decryption process for a 3-block plaintext/-ciphertext using XTC construction. For each $i \geq 0$, $S_i := P_i \oplus C_i$. Dashed rectangles denote the XTX components of XTC and can be used to view XTC as $\mathcal{D}'_{\text{XTX}}$.

significant bit (MSB) of the tweaks for each TBC in block processing to 0. This will lead to a nominal loss of 1 bit security.

6.3.3 Security of XTC

We show that $\text{XTC}_{\tilde{\Pi}, H}$ construction is secure up to $2^n/n$ block-queries given the hash function H is $O(2^{-2n})$ -pAXU hash and $\tilde{\Pi}$ is a uniform tweakable random permutation. More generally, we prove the upper bound result on the OSRP advantage of $\text{XTC}_{\tilde{E}, H}$ in Theorem 6.3.6.

Theorem 6.3.6. *If XTC is defined as above, H is an ϵ -pAXU, and $\sigma \leq 2^{n-3}$, then we have*

$$\text{Adv}_{\text{XTC}_{\tilde{E}, H}}^{\text{osprp}}(q, \ell, \sigma, t) \leq \text{Adv}_{\tilde{E}}^{\text{tsprp}}(\sigma, t'') + \sigma^2 \epsilon + \frac{2(n+2)\sigma}{2^n}.$$

PROOF OVERVIEW — Before proceeding with the proof it would be better to discuss the main crux of the proof briefly. We employ the iterated TBC view (see Section 6.1) of an online cipher. The main steps of the proof are shown in Figure 6.3.2. Basically,

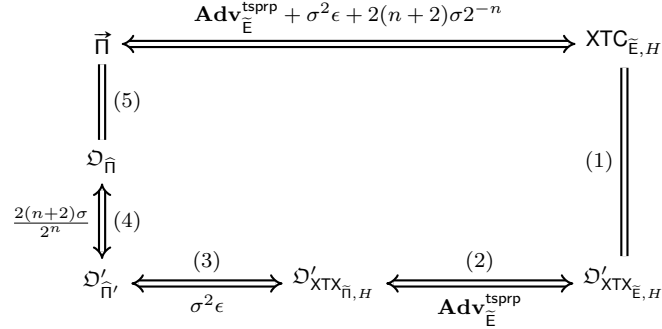


Figure 6.3.2: Main steps in the proof are given in (1)–(5). $\longleftrightarrow$ with overlying text denotes the distance between the corresponding schemes and $=$ denotes the equivalence between the corresponding schemes. Sequence of steps: (1) XTX based view of XTC; (2) and (3) TSPRP security of XTX; (4) Distance between $\mathfrak{D}'_{\hat{\Pi}}$ and $\mathfrak{D}_{\hat{\Pi}}$.

we reduce the original XTC construction based on $\tilde{E}$ and H to a variant of $\mathfrak{D}_{\tilde{E}}$, that we call $\mathfrak{D}'_{\hat{\Pi}'}$ based on an ATL tweakable uniform random permutation $\hat{\Pi}'$. $\mathfrak{D}'_{\hat{\Pi}'}$ behaves exactly as $\mathfrak{D}_{\hat{\Pi}}$, except that it restricts the i -th block tweak to $(S_{i-2}, P_{i-1}, C_{i-1})$. Finally, we bound the distance between $\mathfrak{D}_{\hat{\Pi}}$ and $\mathfrak{D}'_{\hat{\Pi}'}$. Intuitively, the two oracles will behave identically until there is a tweak collision in one of them which is not reciprocated by the other one. Now a tweak collision in $\mathfrak{D}_{\hat{\Pi}}$ always implies a tweak collision in $\mathfrak{D}'_{\hat{\Pi}'}$. But the converse is not true, and we bound the probability of this event to complete the proof.

Proof. We follow the series of steps shown in Figure 6.3.2. Step (1) simply transforms XTC to $\mathfrak{D}'_{\text{XTX}}$ and step (5) transforms $\vec{\Pi}$ to its iterated TBC view $\mathfrak{D}_{\hat{\Pi}}$. Steps (2) and (3) are used to replace the underlying $\text{XTX}_{\tilde{E}, H}$ with an ATL TRP, $\hat{\Pi}$. Using Theorem 6.1.2, steps (2) and (3) are bounded by $\text{Adv}_{\tilde{E}}^{\text{tsprp}}(\sigma, t'') + \epsilon\sigma^2$. In step (4) we upper bound the distance between $\mathfrak{D}_{\hat{\Pi}}$ and $\mathfrak{D}'_{\hat{\Pi}'}$. We employ coefficient-H technique (see Corollary 2.2.2).

Let Ω denote the set of all attainable transcripts for the ideal oracle $\mathcal{O}_0 := \mathfrak{D}_{\hat{\Pi}}$. A typical transcript $\omega \in \Omega$ is of the form (P^q, C^q) , where $P^q, C^q \in (\mathbb{B}^{\eta^+})_q$, $|P_i|_n = p_i \leq \ell$ and $\sum_{i \in [q]} p_i \leq \sigma$. In addition, (P^q, C^q) satisfies the online property.

We say that a transcript $\omega \in \Omega$ is bad, denoted $\omega \in \Omega_{\text{bad}}$, if the following condition holds:

$\text{TColl} : \exists (i, j) \in [q] \times [p_i], (i', j') \in [q] \times [p_{i'}]$, such that

$$P_i^{(j-1)} \neq P_{i'}^{(j'-1)} \wedge (S_{(i,j-2)}, P_{(i,j-1)}, C_{(i,j-1)}) = (S_{(i',j'-2)}, P_{(i',j'-1)}, C_{(i',j'-1)}).$$

It is quite straightforward to see that for all $\omega \in \Omega \setminus \Omega_{\text{bad}}$, we have

$$\Pr[\Theta_1 = \omega] = \Pr[\Theta_0 = \omega],$$

since $\neg \text{TCo11}$ implies that a tweak collision in the real world means a tweak collision in the ideal world and vice-versa. So, all that we are left with is bounding the probability of TCo11 (viewed as the event associated with the predicate TCo11).

BOUND ON $\Pr[\text{TCo11}]$: In the following discussion we assume that all the queries are of encryption type. This will not hamper the correctness of our analysis due to the online property, rather it will greatly simplify the analysis. The basic idea is to first bound the number of multicollisions on $(S_{(i,j-2)}, P_{(i,j-1)})$, and then for each (i, j) bound the probability of $C_{(i,j-1)}$ collisions over the multicollision set that contains $(S_{(i,j-2)}, P_{(i,j-1)})$. Note that, $(P_i^{(j-1)} \neq P_{i'}^{(j'-1)}) \wedge (P_{(i,j-1)} = P_{(i',j'-1)}) \implies (P_i^{(j-2)} \neq P_{i'}^{(j'-2)})$, otherwise the probability of TCo11 is zero. Let

$$\mathcal{I} := \left\{ (u, v) \in [q] \times [p_u] : \forall (u', v') < (u, v) \in [q] \times [p_{u'}], P_u^{(v)} \neq P_{u'}^{(v')} \right\}.$$

We define the multicollision relation $\sim$ on $\mathcal{I}$ as follows:

$$\forall (u, v), (u', v') \in \mathcal{I}, (u, v) \sim (u', v') \iff (S_{(u,v-1)}, P_{(u,v)}) = (S_{(u',v'-1)}, P_{(u',v')}).$$

Clearly $\sim$ is an equivalence relation. Let $\mathcal{P}_\alpha$ denote the equivalence class containing $\alpha \in \mathcal{I}$ and $\#_{\text{mc}} := \max_{\alpha \in \mathcal{I}} |\mathcal{P}_\alpha|$. Let MC_n denote the event $\#_{\text{mc}} > n$. We make the following claim on $\#_{\text{mc}}$.

Claim 6.3.7. For $\sigma \leq 2^{n-3}$, we have

$$\Pr[\text{MC}_n] \leq \frac{4\sigma}{2^n}.$$

We are interested in the conditional probability of TCo11 given $\neg \text{MC}_n$. It is clear that for each $(i, j-1) \in \mathcal{I}$ we have $|\mathcal{P}_{(i,j-1)}| \leq n$, and we have to bound the probability of $C_{(i,j-1)}$ collisions for at most these many pairs.

$$\begin{aligned} \Pr[\text{TCo11}] &\leq \Pr[\text{MC}_n] + \Pr[\text{TCo11} \mid \neg \text{MC}_n] \\ &\stackrel{1}{\leq} \Pr[\text{MC}_n] + \sum_{(i,j-1) \in \mathcal{I}} \sum_{(i',j') \in \mathcal{P}_{(i,j-1)}} \Pr[C_{(i,j-1)} = C_{(i',j')}] \\ &\stackrel{2}{\leq} \Pr[\text{MC}_n] + \sum_{(i,j-1) \in \mathcal{I}} \sum_{(i',j') \in \mathcal{P}_{(i,j-1)}} \frac{1}{2^n - s_{(i,j-2)}} \\ &\stackrel{3}{\leq} \Pr[\text{MC}_n] + \sum_{(i,j-1) \in \mathcal{I}} \sum_{(i',j') \in \mathcal{P}_{(i,j-1)}} \frac{1}{2^n - \sigma} \end{aligned}$$

$$\begin{aligned}
&\stackrel{4}{\leq} \Pr[\text{MC}_n] + \sum_{(i,j-1) \in \mathcal{I}} \frac{n}{2^n - \sigma} \\
&\stackrel{5}{\leq} \frac{4\sigma}{2^n} + \frac{2n\sigma}{2^n}.
\end{aligned}$$

Note that, $P_i^{(j-2]} \neq P_{i'}^{(j'-1]}$, so the tweaks are distinct, whence the transition from 1 to 2, where $s_{(i,j-2)}$ denotes the number of (i_1, j_1) such that $P_{i_1}^{(j_1]} = P_i^{(j-2]}$. Using $s_{(i,j-2)} \leq \sigma$ we have 2 to 3. Using $\#\text{mc} \leq n$, $|\mathcal{I}| \leq \sigma$ and Claim 5.1 we have 3 to 5. $\square$

Proof of Claim 6.3.7: Let $\#\text{mc} = m$ and for some $\alpha \in \mathcal{I}$, let $|\mathcal{P}_\alpha| = m$. For each $(u, v) \in \mathcal{P}_\alpha$, let $s_{(u,v)}$ denote the number of $(u', v') \in \mathcal{I}$ such that $P_{u'}^{(v')]} = P_u^{(v)}]$. Let us fix one index, say lexicographically first one, (u_1, v_1) as our reference index. Then we get a system of $(m-1)$ equations of the form $\{S_{(u_1, v_1-1)} = S_{(u_a, v_a-1)}\}_{a \in [2 \dots m]}$. We can argue that all these equations are independent as $P_{u_a}^{(v_a-1]} \neq P_{u_b}^{(v_b-1]}$ where $a \neq b \in [m]$. So, we have

$$\Pr[\#\text{mc} = m] \leq \frac{\binom{\sigma}{m}}{(2^n - s_{(i_2, j_2)}) \cdots (2^n - s_{(i_m, j_m)})} \leq \frac{\binom{\sigma}{m}}{(2^n - \sigma)^{m-1}}.$$

Summing over all $m \geq n+1$, we have

$$\begin{aligned}
\Pr[\text{MC}_n] &= \sum_{m=n+1}^{\infty} \Pr[\#\text{mc} = m] \\
&\stackrel{1}{\leq} \sum_{m=n+1}^{\infty} \frac{\binom{\sigma}{m}}{(2^n - \sigma)^{m-1}} \\
&\stackrel{2}{\leq} \frac{1}{2} \sum_{m=n+1}^{\infty} \left(\frac{4\sigma}{2^n}\right)^m \\
&\stackrel{3}{\leq} \frac{4\sigma}{2^n}.
\end{aligned}$$

From 1 to 2 we use the fact that $\sigma < 2^{n-1}$, and $n(m-1) > (n-1)m$ whenever $m > n$. Using the convergence result on infinite geometric series and assuming $\sigma \leq 2^{n-3}$, we get the result from 2 to 3. $\square$

As a direct consequence of Theorem 6.3.6 and Lemma 6.3.5 we get the following corollary for the BRW-based instantiation of H .

Corollary 6.3.8. *If H is defined as in Section 6.3.2.1, and $\sigma \leq 2^{n-3}$, then we have*

$$\text{Adv}_{\text{XTC}_{\tilde{E}, H}}^{\text{osprp}}(q, \ell, \sigma, \tau) \leq \text{Adv}_{\tilde{E}}^{\text{tsprp}}(\sigma, \tau'') + \frac{9\sigma^2}{2^{2n}} + \frac{2(n+2)\sigma}{2^n}.$$

6.3.4 Extending XTC to Handle Arbitrary Tweak Lengths

In the following discussion, we extend the initial XTC scheme with BRW based hash function to handle arbitrary tweak lengths.

- For $\tau < n$, we replace the tweak generating part of H by $\text{msb}_\tau(\text{BRW}_{L_1}(C, S, 0) \odot (P \oplus L_1^3))$. It can be easily shown that $\text{BRW}_{L_1}(C, S, 0) \odot (P \oplus L_1^3)$ is a $6/2^n$ -AXU hash. Using Proposition 2.3.5, we can establish that $\text{msb}_t(\text{BRW}_{L_1}(C, S, 0) \odot (P \oplus L_1^3))$ is a $6/2^\tau$ -AU hash. Finally, this gives a bound of $\min(2^n/n, 2^{\frac{n+\tau}{2}})$ block-queries.
- For $\tau > n$, we already know a 2^n block-queries attack (using Theorem 6.3.2) on XTC. While we cannot improve over it without incorporating significantly more previous blocks, we can still get a sub-optimal upper bound of $2^n/n$ block-queries. This is achieved by padding the n -bit tweak generated by the hash function with zeros to make it τ -bit. Clearly, the previous bounds are directly applicable.

Combining the two cases we conclude that XTC is secure while $\sigma \ll \min(2^n/n, 2^{\frac{n+\tau}{2}})$.

Chapter 7

Security of Cascaded LRW2

In the previous chapter, we discussed online ciphers based on tweakable block ciphers (TBCs). This chapter studies the security of a popular way of constructing TBCs using standard block ciphers, called the *Cascaded LRW2* or CLRW2. Landecker et al. [123] first suggested the cascading of two independent LRW2 [125] instances to get a beyond the birthday bound (BBB) secure TBC, called CLRW2. They proved that CLRW2 is a secure TBC up to approx. $2^{2n/3}$ queries. Later, Procter [168] pointed out a flaw in the security proof, which was subsequently fixed by both Landecker et al. and Procter to recover the claimed security bounds. Lampe and Seurin [122] studied the $\ell \geq 2$ independent cascades of LRW2, and proved that it is secure up to approx. $2^{\frac{\ell n}{\ell+2}}$ queries.

In [132], Mennink presented an improved security analysis of CLRW2. The major contribution was an attack in approx. $n^{1/2}2^{3n/4}$ queries (see section 7.2.2). Following on the insights from the attack, he also gave a security proof of the same order under three assumptions:

1. The hash functions are 4-wise independent AXU.
2. The maximum number of tweak repetitions is restricted to $2^{n/4}$.
3. A limited variant of Patarin's mirror theory [160, 161] is true for $q < 2^{3n/4}$.

Among the three assumptions, the first two are at least plausible. But the last assumption is questionable as barring certain restricted cases, the proof of mirror theory has several critical gaps which are still open or unproven [54, 58]. In this chapter we explore the possibility of relaxing the above mentioned assumptions while maintaining a similar security bound.

7.1 A Note on Patarin's Mirror Theory

In [160] Patarin defines Mirror theory as a technique to estimate the number of solutions of linear systems of equalities and linear non equalities in finite groups. In its most general case, the mirror theory proof is tractable up to the order of $2^{2n/3}$ security bound, but it readily becomes complex and extremely difficult to verify, as one aims for the optimal bound [54, 58]. We remark here that this in no way suggests that the result is incorrect, and in future, we might even get some independent verifications of the result.

We restrict ourselves to the binary field $\mathbb{B}$ with $\oplus$ as the group operation. We will use Mennink and Neves interpretation [133] of mirror theory. For ease of understanding and notational coherency, we sometime use different parametrization and naming conventions. Let $q \geq 1$ and let $\mathcal{L}$ be the system of linear equations

$$\{e_1 : Y_1 \oplus V_1 = \lambda_1, e_2 : Y_2 \oplus V_2 = \lambda_2, \dots, e_q : Y_q \oplus V_q = \lambda_q\}$$

where Y^q and V^q are unknowns, and $\lambda^q \in (\mathbb{B})^q$ are knowns. In addition there are (in)equality restrictions on Y^q and V^q , which uniquely determine $\widehat{Y}^q$ and $\widehat{V}^q$. We assume that $\widehat{Y}^q$ and $\widehat{V}^q$, are indexed in an arbitrary order by the index sets $[q_Y]$ and $[q_V]$, where $q_Y = |\widehat{Y}^q|$ and $q_V = |\widehat{V}^q|$. This assumption is without any loss of generality as this does not affect the system $\mathcal{L}$. Given such an indexing, we can define two surjective index mappings:

$$\varphi_Y : \begin{cases} [q] \rightarrow [q_Y] \\ i \mapsto j \text{ if and only if } Y_i = \widehat{Y}_j. \end{cases} \quad \varphi_V : \begin{cases} [q] \rightarrow [q_V] \\ i \mapsto j \text{ if and only if } V_i = \widehat{V}_j. \end{cases}$$

It is easy to verify that $\mathcal{L}$ is uniquely determined by $(\varphi_Y, \varphi_V, \lambda^q)$, and vice-versa. Consider a labeled bipartite graph $\mathcal{G}(\mathcal{L}) = ([q_Y], [q_V], \mathcal{E})$ associated with $\mathcal{L}$, where $\mathcal{E} = \{(\varphi_Y(i), \varphi_V(i), \lambda_i) : i \in [q]\}$, λ_i being the label of edge. Clearly, each equation in $\mathcal{L}$ corresponds to a unique labeled edge (assuming no duplicate equations). We give three definitions with respect to the system $\mathcal{L}$ using $\mathcal{G}(\mathcal{L})$.

Definition 7.1.1 (cycle-freeness). $\mathcal{L}$ is said to be cycle-free if and only if $\mathcal{G}(\mathcal{L})$ is acyclic.

Definition 7.1.2 ($\xi_{\max}$ -component). Two distinct equations (or unknowns) in $\mathcal{L}$ are said to be in the same component if and only if the corresponding edges (res. vertices) in $\mathcal{G}(\mathcal{L})$ are in the same component. The size of any component $\mathcal{C}$ in $\mathcal{L}$, denoted $\xi(\mathcal{C})$, is the number of vertices in the corresponding component of $\mathcal{G}(\mathcal{L})$, and the maximum component size is denoted by $\xi_{\max}(\mathcal{L})$ (or simply $\xi_{\max}$).

Definition 7.1.3 (non-degeneracy). $\mathcal{L}$ is said to be non-degenerate if and only if there does not exist a path of length at least 2 in $\mathcal{G}(\mathcal{L})$ such that the labels along the edges on this path sum up to zero.

Theorem 7.1.4 (Fundamental Theorem of Mirror Theory [160]). *Let $\mathcal{L}$ be a system of equations over the unknowns $(\widehat{Y}^q, \widehat{V}^q)$, that is (i) cycle-free, (ii) non-degenerate, and (iii) $\xi_{\max}^2 \cdot \max\{q_Y, q_V\} \leq 2^n$. Then, the number of solutions $(y_1, \dots, y_{q_Y}, v_1, \dots, v_{q_V})$ of $\mathcal{L}$, denoted h_q , such that $y_i \neq y_j$ and $v_i \neq v_j$ for all $i \neq j$, satisfies*

$$h_q \geq \frac{(2^n)_{q_Y} \cdot (2^n)_{q_V}}{2^{nq}}. \quad (7.1)$$

A sketchy proof for this theorem is given in [160]. As mentioned before, the proof is quite involved with some claims remaining open or unproved. On the other hand, the same paper contains results for various other cases. For instance, for $\xi = 2$, several sub-optimal bounds have been shown. By sub-optimal, we mean that a factor of $(1 - \epsilon)$, for some $\epsilon > 0$, is multiplied to the right hand side of Eq. (7.1). Inspired by this, we give the following terminology which will be useful in later references to mirror theory.

For $\xi \geq 2$, $\epsilon > 0$, we write (ξ, ϵ) -restricted mirror theory theorem to denote the mirror theory result in which the number of solutions, h_q , of a system of equations with $\xi_{\max} = \xi$, satisfies $h_q \geq (1 - \epsilon) \frac{(2^n)_{q_Y} \cdot (2^n)_{q_V}}{2^{nq}}$.

Here ϵ can be viewed as the degree of deviation from random function behavior. Mirror theory has been primarily used for bounding the pseudorandomness of some random system with respect to a random function. Accordingly, one finds a term of the form 2^{nq} in mirror theory bounds. When combined with the coefficient-H technique, we get an ϵ term in the distinguishing advantage bound.

In [132], $(4, q^4/2^{3n})$ -restricted mirror theory theorem is used. In section 7.4, we study the $(\xi, q^4/2^{3n})$ case, for $\xi \in \{2, 3\}$ and present a variant of mirror theory suitable for tweakable permutation scenario.

7.2 Revisiting Mennink's Improved Bound on CLRW2

Recall the notion of ℓ -wise independent XOR universal hash functions defined in Definition 2.3.3 of chapter 2. This notion will be used for the description of CLRW2 (for $\ell = 2$), as well as Mennink's improved bound on CLRW2 (for $\ell = 4$).

7.2.1 Description of the Cascaded LRW2 Construction

Let $E \in \text{BPerm}(\{0, 1\}^\kappa, \mathbb{B})$ be a block cipher. Let $\mathcal{H}$ be a hash function family from $\mathbb{T}$ to $\mathbb{B}$. We define the tweakable block cipher $\text{LRW2}_{E, \mathcal{H}}$, based on the block cipher E and the hash function family $\mathcal{H}$, by the following mapping: $\forall (k, h, t, m) \in \{0, 1\}^\kappa \times \mathcal{H} \times \mathbb{T} \times \mathbb{B}$,

$$\text{LRW2}_{E, \mathcal{H}}(k, h, t, m) := E_k(m \oplus h(t)) \oplus h(t). \quad (7.2)$$

For $r \in \mathbb{N}$, the r -round cascaded LRW2 construction, denoted $\text{CLR2}_{E, \mathcal{H}, r}$, is a cascade of r independent LRW2 instances, i.e. $\text{CLR2}_{E, \mathcal{H}, r}$ is a tweakable block cipher, based on the block cipher E and the hash function family $\mathcal{H}$, defined as follows: $\forall (k^r, h^r, t, m) \in \{0, 1\}^{r\kappa} \times \mathcal{H}^r \times \mathbb{T} \times \mathbb{B}$,

$$y_i := \begin{cases} \text{LRW2}_{E, \mathcal{H}}(t, m) & \text{for } i = 1, \\ \text{LRW2}_{E, \mathcal{H}}(t, y_{i-1}) & \text{otherwise.} \end{cases}$$

$$\text{CLR2}_{E, \mathcal{H}, r}(k^r, h^r, t, m) := y_r. \quad (7.3)$$

The 2-round CLRW2, was first analyzed by Landecker et al. [123], whereas the $r > 2$ case was studied by Lampe and Seurin [122]. Since we mainly focus on the $r = 2$ case, we use the nomenclatures, CLRW2 and cascaded LRW2, interchangeably with 2-round CLRW2. Figure 7.2.1 gives a pictorial description of the cascaded LRW2 construction. In [123] the CLRW2 construction was shown to be a BBB secure (up to $2^{2n/3}$ queries)

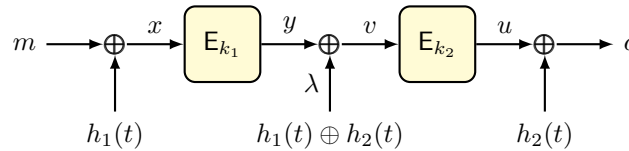


Figure 7.2.1: The cascaded LRW2 construction.

TSPRP, provided the underlying block cipher is an SPRP, and the hash function families are AXU. However, an attack with matching bounds eluded up until quite recently.

7.2.2 Mennink's Attack on CLRW2

In [132] Mennink gave an $O(n^{1/2}2^{3n/4})$ query attack on CLRW2. The attack is generic in nature as it does not exploit the weaknesses in the underlying block cipher. Rather it assumes that the block cipher instances are independent random permutations. Also the attack works for any hash function. We briefly describe the attack and refer the readers to [132] for a more concrete and formal description, analysis and experimental verification of the attack.

ATTACK DESCRIPTION: Suppose in the transcript generated by a distinguisher, there exist four queries (t, m_1, c_1) , (t', m_2, c_2) , (t, m_3, c_3) , and (t', m_4, c_4) , such that the following equations hold:

$$\begin{aligned} m_1 \oplus h_1(t) &= m_2 \oplus h_1(t') \\ c_2 \oplus h_2(t') &= c_3 \oplus h_2(t) \\ m_3 \oplus h_1(t) &= m_4 \oplus h_1(t') \end{aligned} \tag{7.4}$$

Using notations analogous to Figure 7.2.1, we equivalently have, $x_1 = x_2$; $u_2 = u_3$; and $x_3 = x_4$. Since $x^4 \rightsquigarrow y^4$ and $v^4 \rightsquigarrow u^4$, looking at the equations generated by the corresponding y and v values, we have $v_1 = y_1 \oplus \lambda(t) = y_2 \oplus \lambda(t) = v_2 \oplus \lambda(t') \oplus \lambda(t) = v_3 \oplus \lambda(t) \oplus \lambda(t') = y_3 \oplus \lambda(t') = v_4$. This immediately gives $u_1 = u_4$, i.e.

$$c_4 \oplus h_2(t') = c_1 \oplus h_2(t). \tag{7.5}$$

In other words, Eq. (7.5) is implied by the existence of Eq. (7.4), and by combining all four equations, we have

$$\begin{aligned} m_1 \oplus m_2 &= m_3 \oplus m_4 = \alpha, \\ c_1 \oplus c_4 &= c_2 \oplus c_3 = \beta, \end{aligned}$$

where $\alpha = h_1(t) \oplus h_1(t')$ and $\beta = h_2(t) \oplus h_2(t')$. While the distinguisher does not know α and β , it can exploit the relations:

$$m_1 \oplus m_2 = m_3 \oplus m_4, \tag{7.6}$$

$$c_1 \oplus c_4 = c_2 \oplus c_3, . \tag{7.7}$$

If for some value a we have about 2^n many quadruples satisfying

$$m_1 \oplus m_2 = m_3 \oplus m_4 = a, \tag{7.8}$$

then, for CLRW2, the expected number of solutions for Eq. (7.6)-(7.7) is approximately 2 for $a = \alpha$. On the other hand, for $\tilde{\Pi}$, the expected number of solutions is always close to 1 for any $a \in \mathbb{B}$. In [132], it has been shown that approximately $2n^{1/2}2^{3n/4}$ queries are sufficient for the distinguisher to ensure that Eq. (7.8) has about 2^n solutions. Given these many queries the distinguisher can attack by observing the number of solutions for Eq. (7.6)-(7.7) for each value of a .

7.2.3 Mennink's Proof Approach

The proof in [132] applies coefficient-H technique coupled with mirror theory. The main focus is to identify a suitable class of bad events on (x^q, u^q) , where q is the number of queries, which makes mirror theory inapplicable. Crudely, the bad events correspond to cases where for some query there is no randomness left (in the sampling of y^q and v^q) in the ideal world. Given a good transcript, mirror theory is applied to bound the number of solutions of the system of equation $\{Y_i \oplus V_i = \lambda_i : i \in [q]\}$, where Y_i and V_i are unknowns satisfying $x^q \rightsquigarrow Y^q$ and $V^q \rightsquigarrow u^q$, and λ^q is fixed. The proof relies on three major assumptions:

Assumption 1. $\mathcal{H}$ is AXU₄ hash function family.

Assumption 2. For any $t' \in \mathbb{T}$, $\mu_{t'} = \mu(t^q, t') \leq \gamma = 2^{n/4}$.

Assumption 3. $(4, \frac{q^4}{2^{3n}})$ -restricted mirror theory theorem is correct.

TRANSCRIPT GRAPH: A graphical view on x^q and u^q was used to characterize all bad events. Basically, each transcript is mapped to a unique bipartite graph on x^q, u^q , as defined in Definition 7.2.1.

Definition 7.2.1 (Transcript Graph). A transcript graph $\mathcal{G} = (\mathcal{X}^q, \mathcal{U}^q, \mathcal{E}^q)$ associated with (x^q, u^q) , denoted $\mathcal{G}(x^q, u^q)$, is defined as a bipartite graph with vertex sets $\mathcal{X} := \{(x_i, 0) : i \in [q]\}$; $\mathcal{U} := \{(u_i, 1) : i \in [q]\}$; and edge set $\mathcal{E} := \{((x_i, 0), (u_i, 1)) : i \in [q]\}$. We also associate the value $\lambda_i = h_1(t_i) \oplus h_2(t_i)$ with edge $((x_i, 0), (u_i, 1)) \in \mathcal{E}$.

Note that, the graph may not be simple, i.e., it can contain parallel edges. Here $\mathcal{X}$ and $\mathcal{U}$ are just the disjointified representations of x^q and u^q , respectively. For all practical purposes we may drop the 0 and 1 for $(x, 0) \in \mathcal{X}$ and $(u, 1) \in \mathcal{U}$, as they can be easily distinguished. Further, for some $i, j \in [q]$, if $x_i = x_j$ (or $u_i = u_j$), then they share the same vertex $x_i = x_j = x_{i,j}$ (or $u_i = u_j = u_{i,j}$). The event $x_i = x_j$ and $u_i = u_j$, although extremely unlikely, will lead to a parallel edge in $\mathcal{G}$. Finally each edge $(x_i, u_i) \in \mathcal{E}$ corresponds to a query index $i \in [q]$, so we can equivalently view (and call) the edge (x_i, u_i) as index i . Figure 7.2.2 gives an example graph for $\mathcal{G}$.

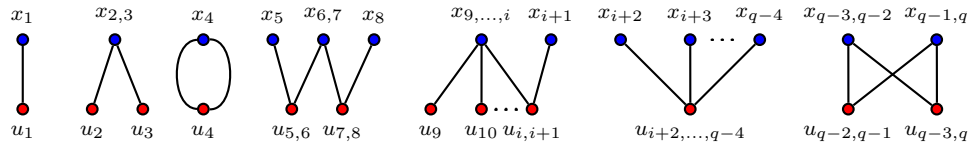


Figure 7.2.2: A possible transcript graph $\mathcal{G}(x^q, u^q)$ associated with (x^q, u^q) . Vertices in x^q are colored blue and vertices in u^q are colored red, for illustration only.

BAD TRANSCRIPTS: A transcript graph $\mathcal{G}(x^q, u^q)$ is called bad if:

1. it has a cycle of size = 2.
2. it has two adjacent edges i and j such that $\lambda_i \oplus \lambda_j = 0$.
3. it has a component of length ≥ 4 edges.

All subgraphs in Figure 7.2.2, except the first two from left, are considered bad in [132]. Conditions 1 and 2 correspond to the cases which might lead to degeneracy in the real world. Condition 3 may lead to a cycle of length ≥ 4 edges. The non-fulfillment of condition 1,2 and 3 satisfies the cycle-free and non-degeneracy properties required in mirror theory. It also bounds $\xi_{\max} \leq 4$. Condition 1 and 2 contribute small and insignificant terms and can be ignored from this discussion. We focus on the major bottleneck, i.e. condition 3. The subgraphs corresponding to condition 3 are given in Figure 7.2.3. Configuration (D), (E), and (F) are symmetric to (A), (B), and (C). So we can study (A), (B), and (C), and the other three can be similarly analyzed.

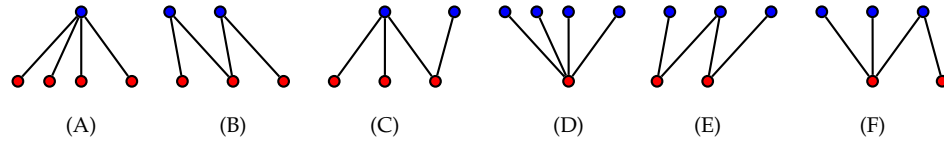


Figure 7.2.3: Possible configuration of size = 4 edge subgraphs. Vertices in x^q are colored blue and vertices in u^q are colored red, and vertex labels are omitted for brevity.

BOTTLENECK 1: BOUND ON THE PROBABILITY OF (A), (C), (D) AND (F) — This can be divided into two parts:

(a) Configuration (A) arises for the event

$$\exists^* i, j, k, l \text{ such that } x_i = x_j = x_k = x_l.$$

This event is upper bounded to $q^4 \epsilon^3$ using assumption 1 on hash functions. Similar argument holds for (D).

(b) Configuration (C) (similarly for F) arises for the event

$$\exists^* i, j, k, \ell \in [q] \text{ such that } x_i = x_j = x_k \wedge u_k = u_\ell.$$

In this case we can apply assumption 1 (even AXU₃ would suffice) to get an upper bound of $q^4 \epsilon^3$.

BOTTLENECK 2: BOUND ON THE PROBABILITY OF (B) — Configuration (B) arises for the event

$$\exists^* i, j, k, l \text{ such that } x_i = x_j \wedge u_j = u_k \wedge x_k = x_l.$$

This is probably the trickiest case, which requires assumption 2, i.e. restriction on tweak repetition. Specifically consider the case $t_i = t_k$ and $t_j = t_\ell$. This is precisely the case exploited in Mennink's attack on CLRW2 [132] (see subsection 7.2.2). In this case for a fixed i, j, k, ℓ the probability is bounded by ϵ^2 . There are at most q^2 many choices for (i, j) , at most $(\mu_{t_i} - 1)$ choices for k and a single choice for ℓ given i, j and k . Thus the probability is bounded by $q^2 \gamma \epsilon^2$ (using assumption 2). Similar argument holds for (E).

BOTTLENECK 3: MIRROR THEORY BOUND — The final hurdle is the use of mirror theory in computation of real world interpolation probability, which requires assumption 3. Yet another issue is the nature of the mirror theory bound. A straightforward application of mirror theory bound leads to a term of the form

$$\frac{\prod_{t' \in \hat{t}^q} (2^n)^{\mu_{t'}}}{(2^n)^q} (1 - O(q/2^n)),$$

in the ratio of interpolation probabilities, where $\sum_{t' \in \hat{t}^q} \mu_{t'} = q$. Here the numerator is due to the tweakable random permutation. In the worst case, $\mu_{t'} = O(q)$, which gives a lower bound of the form $1 - q^2/2^n$. But using assumption 2, we get a bound of $1 - q\gamma/2^n$ as $\mu_{t'} \leq \gamma$.

7.2.3.1 Severity of the Assumptions in [132]

Among the three assumptions, assumption 1 and 2 are quite plausible in the sense that real life use-cases exist for assumption 2 and practical instantiations are possible for assumption 1. Another point of note is the fact that $\gamma < 2^{n/4}$ is imposed only due to bottleneck 3. Otherwise a better bound of $\gamma < 2^{n/2}$ could have been used. While assumption 1 and 2 are plausible to a large extent, assumption 3 is disputable.

Although the proof in [132] requires the above mentioned assumptions, the proof approach seems quite simple and in some cases it highlights the bottlenecks in getting tight security. In the remainder of this chapter, we aim to resolve all the bottlenecks discussed here, while relaxing all the assumptions made in [132]. Specifically, bottleneck 1(a) is incorporated within good transcripts, bottleneck 2 is resolved using the tools from section 7.3, and bottleneck 3 is resolved using the tools from section 7.4 and a careful application of the expectation method in section 7.5. The only unresolved bottleneck is 1(b) for which we will use the AXU₃ assumption on hash functions.

7.3 The Alternating Collisions and Events Lemmata

Let $\mathcal{H} = \{h \mid h : \mathcal{T} \rightarrow \mathcal{B}\}$ be an ϵ -AU hash function family. A pair (t_i, t_j) is called a colliding pair for some $h \in \mathcal{H}$ if $h(t_i) = h(t_j)$. The probability of having at least one colliding pair from q distinct elements $t^q \in (\mathcal{T})_q$ for a randomly chosen hash function $H \leftarrow \mathcal{H}$ is at most $\binom{q}{2} \cdot \epsilon$. This is straightforward from the union bound.

Suppose $H_1, H_2 \leftarrow \mathcal{H}$ are two independently drawn universal hash functions and $t^q \in (\mathcal{T})_q$. Then, by applying independence and union bound, we have

$$\Pr [\exists^* i, j, k \in [q], H_1(t_i) = H_1(t_j) \wedge H_2(t_j) = H_2(t_k)] \leq q(q-1)(q-2) \cdot \epsilon^2.$$

Now we go one step further. We would like to bound the probability of the following event:

$$\exists^* i, j, k, l \in [q], H_1(t_i) = H_1(t_j) \wedge H_2(t_j) = H_2(t_k) \wedge H_1(t_k) = H_1(t_l).$$

For any fixed distinct i, j, k and l , we cannot claim that the probability of the event $H_1(t_i) = H_1(t_j) \wedge H_2(t_j) = H_2(t_k) \wedge H_1(t_k) = H_1(t_l)$ is ϵ^3 as the first and last event are no longer independent. In this section we show how we can get an improved bound even in the dependent situation. In particular, we prove the following lemma.

Lemma 7.3.1 (Alternating Collisions Lemma). *Suppose $H_1, H_2 \leftarrow \mathcal{H}$ are two independently drawn ϵ -universal hash functions and $t^q \in (\mathcal{T})_q$. Then,*

$$\Pr [\exists^* i, j, k, l \in [q], H_1(t_i) = H_1(t_j) \wedge H_1(t_k) = H_1(t_l) \wedge H_2(t_j) = H_2(t_k)] \leq q^2 \epsilon^{1.5}.$$

Proof. For any $h \in \mathcal{H}$, we define the following useful set:

$$\mathcal{I}_h = \{(i, j) : h(t_i) = h(t_j)\}.$$

Let us denote the size of the above set by I_h . So, I_h is the number of colliding pairs for the hash functions h . We also define a set $\mathcal{H}_{\leq} = \{h : I_h \leq \frac{1}{\sqrt{\epsilon}}\}$ which collects all hash functions having a small number of colliding pairs. We denote the complement set by $\mathcal{H}_{>}$. Now, by using double counting of the set $\{(h, i, j) : h(t_i) = h(t_j)\}$ we get

$$\sum_h I_h \leq q(q-1) \cdot \epsilon \times |\mathcal{H}|. \quad (7.9)$$

Basically for every h , we have exactly I_h many choices of (i, j) and so the size of the set $\{(h, i, j) : h(t_i) = h(t_j)\}$ is exactly $\sum_h I_h$. On the other hand, for any $i < j \in [q]$, there are at most $\epsilon \cdot |\mathcal{H}|$ many hash functions h , such that (t_i, t_j) is a colliding pair for h .

This follows from the definition of the universal hash function. From Eq. (7.9) and the definition of $\mathcal{H}_{\leq}$, we have

$$\frac{|\mathcal{H}_{>}|}{\sqrt{\epsilon}} + \sum_{h \in \mathcal{H}_{\leq}} I_h \leq \sum_h I_h \leq q(q-1) \cdot \epsilon \cdot |\mathcal{H}|. \quad (7.10)$$

Let E denote the event that there exists distinct i, j, k, l such that $H_1(t_i) = H_1(t_j) \wedge H_1(t_k) = H_1(t_l) \wedge H_2(t_j) = H_2(t_k)$. Now we proceed to bound the probability of this event.

$$\begin{aligned} \Pr[E] &= \sum_h \Pr[E \wedge H_1 = h] \\ &= \sum_h \Pr[H_1 = h] \times \Pr[E \wedge H_1 = h \mid H_1 = h] \\ &\leq \sum_h \Pr[H_1 = h] \times \min\{1, I_h^2 \cdot \epsilon\} \\ &= \Pr[H_1 \in \mathcal{H}_{>}] + \sum_{h \in \mathcal{H}_{\leq}} \Pr[H_1 = h] \cdot I_h^2 \cdot \epsilon \\ &\leq \frac{|\mathcal{H}_{>}|}{|\mathcal{H}|} + \sum_{h \in \mathcal{H}_{\leq}} \frac{I_h \cdot \sqrt{\epsilon}}{|\mathcal{H}|} \\ &= \frac{\sqrt{\epsilon}}{|\mathcal{H}|} \times \left(\frac{|\mathcal{H}_{>}|}{\sqrt{\epsilon}} + \sum_{h \in \mathcal{H}_{\leq}} I_h \right) \\ &\leq q(q-1)\epsilon^{1.5}. \end{aligned}$$

We justify the first inequality. Let $\mathcal{I}_h$ be the set of pairs (i, j) with $h_1(t_i) = h_1(t_j)$ (i.e. $|\mathcal{I}_h| = I_h$). Given $H_1 = h$, the probability of the event E is same as the probability of the following event:

$$\exists^*(i, j), (k, l) \in \mathcal{I}_h, H_2(t_j) = H_2(t_k).$$

There are at most I_h^2 many pairs of pairs and for each pair of pairs and the collision probability of $H_2(t_j) = H_2(t_k)$ is at most ϵ . So probability of the above event can be at most $\min\{1, I_h^2 \cdot \epsilon\}$. The last inequality follows from Eq. (7.10). $\square$

Now we generalize the above result for a more general setting which will also be used in this chapter. The proof of this result is similar to the previous proof.

Lemma 7.3.2 (Alternating Events Lemma). *Suppose for all $i < j \in [q]$, $E_{i,j}$ are events, possibly dependent. Each event holds with probability at most ϵ . For any distinct i, j, k, l , $F_{i,j,k,l}$ are events which holds with probability at most ϵ' . Moreover, the collection of events $(F_{i,j,k,l})_{i,j,k,l}$ is independent with the collection of event $(E_{i,j})_{i,j}$. Then,*

$$\Pr[\exists^* i, j, k, l \in [q], E_{i,j} \wedge E_{k,l} \wedge F_{i,j,k,l}] \leq q^2 \cdot \epsilon \cdot \sqrt{\epsilon'}.$$

Note that, Lemma 7.3.1 is a direct corollary of the above Lemma (the event $E_{i,j}$ denotes that (t_i, t_j) is a colliding pair of H_1 and $F_{i,j,k,l}$ denotes that (t_j, t_k) is a colliding pair of H_2).

Proof. We follow a similar proof approach as considered in Lemma 7.3.1. We define a indicator random vector $\mathbf{l} = (l_{i,j})_{i,j \in [q]}$ where $l_{i,j}$ takes value 1 if $E_{i,j}$ holds, otherwise zero. The sample space of the random vector is Ω , the set of all binary vectors indexed by all pairs (i, j) . For any vector $w \in \Omega$, we write $\#w$ to represent the number of 1's appeared in w . Let $\Omega_{\leq} = \{w : \#w \leq \frac{1}{\sqrt{\epsilon'}}\}$ and its complement set by $\Omega_{>}$.

We define a random variable $N = \sum_{i \neq j} l_{i,j}$, i.e., the number of E-events hold. As $E_{i,j}$ holds with probability at most ϵ ,

$$\begin{aligned} q(q-1)\epsilon &\geq \mathbb{E}[N] \\ &= \sum_w \#w \cdot \Pr[\mathbf{l} = w] \\ &\geq \sum_{w \in \Omega_{\leq}} \#w \cdot \Pr[\mathbf{l} = w] + \frac{\Pr[\mathbf{l} \in \Omega_{>}]}{\sqrt{\epsilon'}}. \end{aligned} \quad (7.11)$$

Let E denote the event that there exists distinct i, j, k, l such that $E_{i,j} \wedge E_{k,l} \wedge F_{i,j,k,l}$. Now we proceed for bounding the probability of the event.

$$\begin{aligned} \Pr[E] &= \sum_w \Pr[E \wedge \mathbf{l} = w] \\ &= \sum_w \Pr[\mathbf{l} = w] \times \Pr[E \wedge \mathbf{l} = w \mid \mathbf{l} = w] \\ &\leq \sum_w \Pr[\mathbf{l} = w] \times \min\{1, (\#w)^2 \cdot \epsilon'\} \\ &= \Pr[\mathbf{l} \in \Omega_{>}] + \sum_{w \in \Omega_{\leq}} \Pr[\mathbf{l} = w] \cdot (\#w)^2 \cdot \epsilon' \\ &\leq \Pr[\mathbf{l} \in \Omega_{>}] + \sum_{w \in \Omega_{\leq}} \Pr[\mathbf{l} = w] \cdot \#w \cdot \sqrt{\epsilon'} \\ &= \sqrt{\epsilon'} \cdot \left(\sum_{w \in \Omega_{\leq}} \#w \cdot \Pr[\mathbf{l} = w] + \frac{\Pr[\mathbf{l} \in \Omega_{>}]}{\sqrt{\epsilon'}} \right) \\ &\leq q(q-1)\epsilon \cdot \sqrt{\epsilon'}. \end{aligned}$$

The first inequality follows exactly by the same reason argued in the proof of Lemma 7.3.1. The last inequality follows from Eq. (7.11). This completes the proof. $\square$

7.4 Mirror Theory for Tweakable Permutations

As evident from bottleneck 4 of section 7.2.3, a straightforward application of mirror theory bound would lead to a sub-optimal bound. In order to circumvent this sub-optimality, [132] uses a restriction on tweak repetitions (assumption 2 of section 7.2.3). Specifically, a bound of the form $O(q/2^{3n/4})$ requires $\mu(t^q, t') < 2^{n/4}$ for all $t' \in \hat{t}^q$, where t^q is the q -tuple denoting the tweaks used in the q queries. In order to avoid this assumption, we need a different approach.

A closer inspection of the mirror theory proof reveals that we can actually avoid the restrictions on tweak repetitions. In fact, rather surprisingly, we will see that tweak repetitions are actually helpful in the sense that mirror theory bound is good. In the remainder of this section, we develop a modified version of mirror theory, apt for applications in tweakable permutation settings. We will only consider $\xi_{\max}(\mathcal{L}) = 3$ case.

7.4.1 General setup and notations

Following the notations and definitions from section 7.1, consider a system of equation $\mathcal{L}$ with $\xi_{\max}(\mathcal{L}) = 3$. For brevity, we simply write “sub-system of $\mathcal{L}$ with $\xi = x$ ”, for some $x \in \{2, 3\}$, to denote the sub-system of $\mathcal{L}$ consisting of equations belonging to components with $\xi = x$.

Following this nomenclature, let $\mathcal{C}_1$ denote the sub-system of $\mathcal{L}$ with $\xi = 2$. Let $\mathcal{C}_2$ (res. $\mathcal{C}_3$) denote the sub-system of $\mathcal{L}$ with $\xi = 3$ such that for $i \in [q]$, if equation $e_i \in \mathcal{C}_2$ (res. $e_i \in \mathcal{C}_3$) then $\mu(V^q, V_i) = 1$ (res. $\mu(Y^q, Y_i) = 1$).

For $i \in [3]$, let $c_i = |\mathcal{C}_i|$ and q_i be the number of equations in $\mathcal{C}_i$, i.e. $q_1 = c_1$, $q_2 = 2c_2$, $q_3 = 2c_3$, and $q_1 + q_2 + q_3 = q$. We also write $c = c_2 + c_3$. For $i \in [q]$, let $\delta_i := \mu(\lambda^{i-1}, \lambda_i)$, where $\delta_1 = 0$ by convention.

Note that, the equations in $\mathcal{L}$ can be arranged in any arbitrary order without affecting the number of solutions. For the sake of simplicity, we fix the following order on $\mathcal{L}$:

$$\begin{array}{ccc|ccc|ccc}
 Y_1 \oplus V_1 = \lambda_1 & & & Y_{q_1+1} \oplus V_{q_1+1} = \lambda_{q_1+1} & & & Y_{q_1+q_2+1} \oplus V_{q_1+q_2+1} = \lambda_{q_1+q_2+1} & & \\
 \vdots & & & \vdots & & & \vdots & & \\
 Y_{q_1} \oplus V_{q_1} = \lambda_{q_1} & & & Y_{q_1+q_2} \oplus V_{q_1+q_2} = \lambda_{q_1+q_2} & & & Y_{q_1+q_2+q_3} \oplus V_{q_1+q_2+q_3} = \lambda_{q_1+q_2+q_3} & & \\
 \hline
 \underbrace{\hspace{10em}}_{\mathcal{C}_1} & & \underbrace{\hspace{10em}}_{\mathcal{C}_2} & & \underbrace{\hspace{10em}}_{\mathcal{C}_3}
 \end{array}$$

where for all $i \in [c_1], j \in [c_2]$, $Y_{q_1+2i-1} = Y_{q_1+2i}$, and $V_{q_1+q_2+2j-1} = V_{q_1+q_2+2j}$, otherwise all Y 's are pairwise distinct and V 's are pairwise distinct. Therefore, we get the following equalities on φ_Y and φ_V :

1. for all $i \in [c_2]$, $\varphi_Y(q_1 + 2i - 1) = \varphi_Y(q_1 + 2i)$.
2. for all $i \in [c_3]$, $\varphi_V(q_1 + q_2 + 2i - 1) = \varphi_V(q_1 + q_2 + 2i)$.

Also, for all $i \in [c]$, $\lambda_{q_1+2i-1} \neq \lambda_{q_1+2i}$. So all the equations in $\mathcal{C}_1$ come first, followed by all the equations in $\mathcal{C}_2$, and finally all the equations in $\mathcal{C}_3$. Now, our goal is to give a lower bound on the number of solutions of $\mathcal{L}$, such that the $\hat{Y}_i$ values are pairwise distinct and $\hat{V}_i$ values are pairwise distinct. More formally we aim to prove the following result.

Theorem 7.4.1. *Let $\mathcal{L}$ be the system of linear equations satisfying the restrictions as above in section 7.4.1 with $q < 2^{n-2}$. Then the number of solutions $(y_1, \dots, y_{q_Y}, v_1, \dots, v_{q_V})$ of $\mathcal{L}$, denoted h_q , such that $y_i \neq y_j$ and $v_i \neq v_j$, for all $i \neq j$, satisfies:*

$$h_q \geq \left(1 - \frac{604q^2c}{2^{2n}} - \frac{128qc}{2^{2n}} - \frac{192q^3c}{2^{3n}} - \frac{36q^4}{2^{3n}} - \frac{204q^2}{2^{2n}} - \frac{64q^3}{2^{3n}} - \frac{56q}{2^{2n}}\right) \frac{(2^n)_{q_1+c_2+q_3}(2^n)_{q_1+q_2+c_3}}{\prod_{\lambda' \in \hat{\lambda}^q} (2^n)_{\mu(\lambda^q, \lambda')}}.$$

The following corollary is just a simplified variant of Theorem 7.4.1 that is more suitable for applications. We will use this variant later on in our main result on CLRW2.

Corollary 7.4.2. *Let $\mathcal{L}$ be the system of linear equations satisfying the restrictions as above in section 7.4.1 with $q < 2^{n-2}$. Then the number of solutions $(y_1, \dots, y_{q_Y}, v_1, \dots, v_{q_V})$ of $\mathcal{L}$, denoted h_q , such that $y_i \neq y_j$ and $v_i \neq v_j$, for all $i \neq j$, satisfies:*

$$h_q \geq \left(1 - \frac{780q^2c}{2^{2n}} - \frac{100q^4}{2^{3n}} - \frac{260q^2}{2^{2n}}\right) \frac{(2^n)_{q_1+c_2+q_3}(2^n)_{q_1+q_2+c_3}}{\prod_{\lambda' \in \hat{\lambda}^q} (2^n)_{\mu(\lambda^q, \lambda')}}.$$

We note here that the bound in Theorem 7.4.1 and Corollary 7.4.2 are parametrized in q and c . This is a bit different from the traditional mirror theory bounds. Further, we note that the bounds in Theorem 7.4.1 and Corollary 7.4.2, become $1 - O(q^4/2^{3n})$ in average case, when the expected value of c is $O(q^2/2^n)$. The proof of Theorem 7.4.1 uses an inductive approach similar to the one in [160]. We postpone the complete proof to section 7.6.

7.5 Improved Security Bound of CLRW2

Based on the tools we developed in section 7.3 and 7.4, we now show that the CLRW2 construction achieves security up to the query complexity approximately $2^{3n/4}$, given

that the underlying hash functions are AXU_3 . Given Mennink's attack [132] (see section 7.2.2) in roughly these many queries we can conclude that the bound is tight.

Theorem 7.5.1. *Let $\kappa, \tau, n \in \mathbb{N}$ and $\epsilon > 0$. Let $E \in \text{BPerm}(\{0, 1\}^\kappa, \mathbb{B})$, and let $\mathcal{H}$ be an ϵ - AXU_3 hash function family from $\mathbb{T}$ to $\mathbb{B}$. Consider*

$$\text{CLRW2}_{E, \mathcal{H}} : \{0, 1\}^{2\kappa} \times \mathcal{H}^2 \times \mathbb{T} \times \mathbb{B} \rightarrow \mathbb{B}.$$

For $q \leq 2^{n-2}$ and $t > 0$, the TSPRP security of $\text{CLRW2}_{E, \mathcal{H}}$ against $\mathbb{A}(q, t)$ is given by

$$\text{Adv}_{\text{CLRW2}_{E, \mathcal{H}}}^{\text{tsprp}}(q, t) \leq 2\text{Adv}_E^{\text{sprp}}(q, t') + \Delta,$$

where $t' = O(t + qt_{\mathcal{H}})$, $t_{\mathcal{H}}$ being the time complexity for computing the hash function $\mathcal{H}$, and

$$\Delta \leq \frac{780q^4\epsilon}{2^{2n}} + \frac{100q^4}{2^{3n}} + \frac{260q^2}{2^{2n}} + 2q^2\epsilon^2 + 2q^2\epsilon^{1.5} + q^4\epsilon^3 + \frac{8q^2\epsilon}{2^{n/2}} + \frac{4q^4\epsilon^2}{2^n}. \quad (7.12)$$

On putting $\epsilon = 1/2^n$, in Eq. (7.12) and further simplifying, we get

Corollary 7.5.2. *For $q \leq 2^{n-2}$ and $\epsilon = \frac{1}{2^n}$, we have*

$$\text{Adv}_{\text{CLRW2}_{E, \mathcal{H}}}^{\text{tsprp}}(q, t) \leq 2\text{Adv}_E^{\text{sprp}}(q, t') + \frac{885q^4}{2^{3n}} + \frac{10q^2}{2^{3n/2}} + \frac{262q^2}{2^{2n}}. \quad (7.13)$$

Specifically, the advantage is negligible up to $q = \min(2^{\frac{3n}{4}-2.5}, 2^{n-4.5})$ queries.

PROOF OVERVIEW — The proof of Theorem 7.5.1 employs the Expectation method (see Lemma 2.2.1) coupled with an adaptation of $(3, q^4/2^{3n})$ -restricted mirror theory theorem [160] in tweakable permutation settings. While our use of mirror theory is somewhat inspired by its recent use in [132], in contrast to [132], we apply the modified version of mirror theory developed in section 7.4, and that too for a restricted subset of queries. The alternating collisions lemma of section 7.3 is used to bound the probability of the most crucial bad event (Bottleneck 2 in section 7.2.3). The complete proof of Theorem 7.5.1 is given in the remainder of this section.

7.5.1 Initial Step

Consider the instantiation $\text{CLRW2}_{E_{K_1}, E_{K_2}, H_1, H_2}$ of $\text{CLRW2}_{E, \mathcal{H}}$, where K_1, K_2, H_1, H_2 are independent and $(K_1, K_2) \leftarrow \$ (\{0, 1\}^\kappa)^2$, $(H_1, H_2) \leftarrow \$ \mathcal{H}^2$. As the first step, we switch to the information-theoretic setting, i.e. we replace (E_{K_1}, E_{K_2}) with $(\Pi_1, \Pi_2) \leftarrow \$ \text{Perm}(\mathbb{B})^2$. For the sake of simplicity, we write the modified instantiation $\text{CLRW2}_{\Pi_1, \Pi_2, H_1, H_2}$ as

CLRW2, i.e. without any parametrization. This switching is done via a standard hybrid argument that incurs a cost of $2\text{Adv}_E^{\text{sprp}}(q, t')$ where $t' = O(t + qt_{\mathcal{H}})$. Thus, we have

$$\text{Adv}_{\text{CLRW2}_{E, \mathcal{H}}}^{\text{tsprp}}(q, t) \leq 2\text{Adv}_E^{\text{sprp}}(q, t') + \text{Adv}_{\text{CLRW2}}^{\text{tsprp}}(q). \quad (7.14)$$

So, in Eq. (7.14), we have to give an upper bound on $\text{Adv}_{\text{CLRW2}}^{\text{tsprp}}(q)$. At this point, we are in the information-theoretic setting. In other words, we consider computationally unbounded distinguisher $\mathcal{A}$. Without loss of generality, we assume that $\mathcal{A}$ is deterministic and non-trivial. Under this setup, we are now ready to apply the expectation method.

7.5.2 Oracle Description

The two oracles of interest are: $\mathcal{O}_1$, the real oracle, that implements CLRW2; and, $\mathcal{O}_0$, the ideal oracle, that implements $\tilde{\Pi} \leftarrow \$ \text{BPerm}(\tau, n)$. We consider an extended version of these oracles, the one in which they release some additional information. We use notations analogously as given in Figure 7.2.1 to describe the transcript generated by $\mathcal{A}$'s interaction with its oracle.

7.5.2.1 Description of the Real Oracle, $\mathcal{O}_1$

The real oracle $\mathcal{O}_1$ faithfully runs CLRW2. We denote the transcript random variable generated by $\mathcal{A}$'s interaction with $\mathcal{O}_1$ by the usual notation Θ_1 , which is a 10-ary q -tuple

$$(T^q, M^q, C^q, X^q, Y^q, V^q, U^q, \lambda^q, H_1, H_2),$$

defined as follows: The initial transcript consists of (T^q, M^q, C^q) , where for all $i \in [q]$:

T_i : i -th tweak value, M_i : i -th plaintext value, C_i : i -th ciphertext value

where $C^q = \text{CLRW2}(T^q, M^q)$. At the end of the query-response phase $\mathcal{O}_1$ releases some additional information $(X^q, Y^q, V^q, U^q, \lambda^q, H_1, H_2)$, where for all $i \in [q]$:

- (X_i, Y_i) : i -th input-output pair for Π_1 ,
- (V_i, U_i) : i -th input-output pair for Π_2 ,
- λ_i : i -th internal masking, H_1, H_2 : the hash keys.

Note that, X^q, U^q , and λ^q are completely determined by the hash keys H_1, H_2 , and the initial transcript (T^q, M^q, C^q) . But, we include them anyhow to ease the analysis.

7.5.2.2 Description of the ideal oracle, $\mathcal{O}_0$

The ideal oracle $\mathcal{O}_0$ has access to $\tilde{\Pi}$. Since $\mathcal{O}_1$ releases some additional information, $\mathcal{O}_0$ must generate these values as well. The ideal transcript random variable Θ_0 is also a 10-ary q -tuple

$$(T^q, M^q, C^q, X^q, Y^q, V^q, U^q, \lambda^q, H_1, H_2),$$

defined below. Note that, we use the same notation to represent the variables of transcripts in the both world. However, the probability distributions of these would be determined from their definitions. The initial transcript consists of (T^q, M^q, C^q) , where for all $i \in [q]$:

T_i : i -th tweak value, M_i : i -th plaintext value, C_i : i -th ciphertext value,

where $C^q = \tilde{\Pi}(T^q, M^q)$. Once the query-response phase is over $\mathcal{O}_0$ samples $(H_1, H_2) \leftarrow \mathcal{H}^2$ and computes X^q, U^q, λ^q , where for all $i \in [q]$:

$$X_i := H_1(T_i) \oplus M_i, \quad U_i := H_2(T_i) \oplus C_i, \quad \lambda_i := H_1(T_i) \oplus H_2(T_i).$$

This means that X^q, U^q , and λ^q are defined honestly. Given the partial transcript $\Theta'_0 := (T^q, M^q, C^q, X^q, U^q, \lambda^q, H_1, H_2)$ we wish to characterize the hash key $H := (H_1, H_2)$ as good or bad. We write $\mathcal{H}_{\text{bad}}$ for the set of bad hash keys, and $\mathcal{H}_{\text{good}} = \mathcal{H}^2 \setminus \mathcal{H}_{\text{bad}}$. We say that a hash key $H \in \mathcal{H}_{\text{bad}}$ (or H is bad) if and only if one of the following predicates is true:

1. H_1 : $\exists^* i, j \in [q]$ such that $X_i = X_j \wedge U_i = U_j$.
2. H_2 : $\exists^* i, j \in [q]$ such that $X_i = X_j \wedge \lambda_i = \lambda_j$.
3. H_3 : $\exists^* i, j \in [q]$ such that $U_i = U_j \wedge \lambda_i = \lambda_j$.
4. H_4 : $\exists^* i, j, k, \ell \in [q]$ such that $X_i = X_j \wedge U_j = U_k \wedge X_k = X_\ell$.
5. H_5 : $\exists^* i, j, k, \ell \in [q]$ such that $U_i = U_j \wedge X_j = X_k \wedge U_k = U_\ell$.
6. H_6 : $\exists^* i, j, k, \ell \in [q]$ such that $X_i = X_j = X_k \wedge U_k = U_\ell$.
7. H_7 : $\exists^* i, j, k, \ell \in [q]$ such that $U_i = U_j = U_k \wedge X_k = X_\ell$.

CASE 1: H IS BAD — If the hash key H is bad, then Y^q and V^q values are sampled arbitrarily.

CASE 2: H IS GOOD — To characterize the transcript corresponding to a good hash key it will be useful to study a graph, similar to the one in section 7.2, associated with

(X^q, U^q) . Specifically, we consider the random transcript graph $\mathcal{G}(X^q, U^q)$ arising due to $H \in \mathcal{H}_{\text{good}}$. Lemma 7.5.3 and Figure 7.5.1 characterizes the different types of possible components in $\mathcal{G}(X^q, U^q)$. Note that, type-4 and type-5 graphs are the same as configuration (A) and (D) of Figure 7.2.3, for size ≥ 4 edges. These graphs are considered as bad in [132], whereas we allow such components.

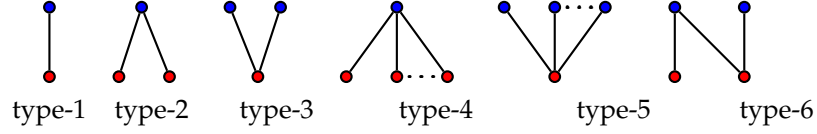


Figure 7.5.1: Enumerating all possible types of components of a transcript graph corresponding to a good hash key: type-1 is the only possible component of size = 1 edge; type-2 and type-3 are the only possible components of size = 2 edges; type-4, type-5 and type-6 are the only possible components of size = 3 edges; type-4 and type-5 can have size > 3 edges, and type-6 can have degree 2 vertices in both $\mathcal{X}$ and $\mathcal{U}$.

Lemma 7.5.3. *The transcript graph $\mathcal{G}$ corresponding to (X^q, U^q) generated by a good hash key H has the following properties:*

1. $\mathcal{G}$ is simple, acyclic and has no isolated vertices.
2. $\mathcal{G}$ has no two adjacent edges i and j such that $\lambda_i \oplus \lambda_j = 0$.
3. $\mathcal{G}$ has no subgraph $\mathcal{G}'$ of size ≥ 4 edges such that $\exists X \in \mathcal{X}$ and $\exists U \in \mathcal{U}$ with $\deg(X), \deg(U) \geq 2$ and $(X, U) \in \mathcal{E}(\mathcal{G}')$.

In fact the all possible types of components of $\mathcal{G}$ are enumerated in Figure 7.5.1.

Proof. Property 1 holds by definition and the non-existence of bad hash key conditions 1, 4, and 5. Property 2 holds due to the non-existence of bad hash key conditions 2, and 3. Property 3 holds due to the non-existence of bad hash key conditions 4, 5, 6, and 7. It is easy to verify that given Property 1, 2, and 3, Figure 7.5.1 enumerates all possible types of components of $\mathcal{G}$. $\square$

In what follows we describe the sampling of Y^q and V^q when $H \in \mathcal{H}_{\text{good}}$. We collect the indices $i \in [q]$ corresponding to the edges in all type-1, type-2, type-3, type-4, type-5 and type-6 components, in the index sets $\mathcal{I}_1, \mathcal{I}_2, \mathcal{I}_3, \mathcal{I}_4, \mathcal{I}_5$, and $\mathcal{I}_6$, respectively. Clearly, the six sets are disjoint, and $[q] = \mathcal{I}_1 \sqcup \mathcal{I}_2 \sqcup \mathcal{I}_3 \sqcup \mathcal{I}_4 \sqcup \mathcal{I}_5 \sqcup \mathcal{I}_6$. Let $\mathcal{I} = \mathcal{I}_1 \sqcup \mathcal{I}_2 \sqcup \mathcal{I}_3$. Consider the system of equation

$$\mathcal{L} = \{Y_i \oplus V_i = \lambda_i : i \in \mathcal{I}\},$$

where $Y_i = Y_j$ (res. $V_i = V_j$) if and only if $X_i = X_j$ (res. $U_i = U_j$) for all $i, j \in [q]$. The solution set of $\mathcal{L}$ is precisely the set

$$\mathcal{S} = \{(y^{\mathcal{I}}, v^{\mathcal{I}}) : y^{\mathcal{I}} \rightsquigarrow X^{\mathcal{I}} \wedge v^{\mathcal{I}} \rightsquigarrow U^{\mathcal{I}} \wedge y^{\mathcal{I}} \oplus v^{\mathcal{I}} = \lambda^{\mathcal{I}}\}.$$

Given these definitions, the ideal oracle $\mathcal{O}_0$ samples (Y^q, V^q) as follows:

- $(Y^{\mathcal{I}}, V^{\mathcal{I}}) \leftarrow \$ \mathcal{S}$, i.e. $\mathcal{O}_0$ uniformly samples one valid assignment from the set of all valid assignments.
- Let $\mathcal{G} \setminus \mathcal{I}$ denote the subgraph of $\mathcal{G}$ after the removal of edges and vertices corresponding to $i \in \mathcal{I}$. For each component $\mathcal{C}$ of $\mathcal{G} \setminus \mathcal{I}$:
 - Let (X_i, U_i) be the edge in $\mathcal{C}$ with the smallest index $i \in [q] \setminus \mathcal{I}$. Then $Y_i \leftarrow \$ \mathbb{B}$ and $V_i = Y_i \oplus \lambda_i$.
 - For all other indices $i' \in [q] \setminus \mathcal{I}$, corresponding to edges $(X_{i'}, U_{i'}) \in \mathcal{C}$, $(Y_{i'}, V_{i'})$ is defined in the following way:
 - * Suppose $i, i' \in \mathcal{I}_4$ (res. $\mathcal{I}_5$), then $Y_{i'} = Y_i$ (res. $V_{i'} = V_i$), and $V_{i'} = Y_i \oplus \lambda_{i'}$ (res. $Y_{i'} = V_i \oplus \lambda_{i'}$).
 - * Suppose $i, i' \in \mathcal{I}_6$, and assume that i corresponds to the edge where $X_i = X_{i'}$ and $U_i = U_{i''}$, where i, i', i'' are the three edges in the component. Then $Y_{i'} = Y_i$ and $V_{i'} = Y_i \oplus \lambda_{i'}$. The other cases can be handled in a similar fashion.

At this point, $\Theta_0 = (\mathbb{T}^q, \mathbb{M}^q, \mathbb{C}^q, \mathbb{X}^q, \mathbb{Y}^q, \mathbb{V}^q, \mathbb{U}^q, \lambda^q, H_1, H_2)$ is completely defined. In this way we maintain both the consistency of equations of the form $Y_i \oplus V_i = \lambda_i$ (as in the case of real world), and the permutation consistency within each component, when $H \in \mathcal{H}_{\text{good}}$. However, there might be collisions among Y or V values from different components.

7.5.3 Definition and Analysis of Bad Transcripts

Given the description of the transcript random variable corresponding to the ideal oracle we can define the set of attainable transcripts Ω as the set of all tuples $\omega = (t^q, m^q, c^q, x^q, y^q, v^q, u^q, \lambda^q, h_1, h_2)$, where $t^q \in (\mathbb{T})^q$; $m^q, c^q, y^q, v^q \in (\mathbb{B})^q$; $(h_1, h_2) \in \mathcal{H}^2$; $x^q = h_1(t^q) \oplus m^q$; $y^q = h_2(t^q) \oplus c^q$; $\lambda^q = h_1(t^q) \oplus h_2(t^q)$; and $(t^q, m^q) \rightsquigarrow (t^q, c^q)$.

Our bad transcript definition is inspired by two requirements:

1. Eliminate all x^q, u^q , and λ^q tuples such that y^q and v^q are trivially restricted by way of linear dependence. For example, a cycle in the transcript graph would lead to such a restriction.
2. Eliminate all x^q, u^q, y^q, v^q tuples such that $x^q \not\leftrightarrow y^q$ or $u^q \not\leftrightarrow v^q$.

Among the two, requirement 2 is trivial as $x^q \leftrightarrow y^q$ and $u^q \leftrightarrow v^q$ is always true for real world transcript. Requirement 1 is more of a technical one that helps in the ideal world sampling of y^q and v^q .

BAD TRANSCRIPT DEFINITION: We first define certain attainable transcripts as bad depending upon the characterization of hash keys. Inspired by the ideal world description, we say that a hash key $(h_1, h_2) \in \mathcal{H}_{\text{bad}}$ (or (h_1, h_2) is bad) if and only if the following predicate is true:

$$H_1 \vee H_2 \vee H_3 \vee H_4 \vee H_5 \vee H_6 \vee H_7.$$

We say that ω is *hash induced bad* transcript, if $(h_1, h_2) \in \mathcal{H}_{\text{bad}}$. We write this event as BAD-HASH , and by a slight abuse of notations,¹ we have

$$\text{BAD-HASH} = \bigcup_{i=1}^7 H_i. \quad (7.15)$$

This takes care of the first requirement. For the second one we have to enumerate all the conditions which might lead to $x^q \not\leftrightarrow y^q$ or $u^q \not\leftrightarrow v^q$. Since we sample arbitrarily when the hash key is bad, we assume that the transcript is *trivially inconsistent* in this case. For good hash keys, if $x_i = x_j$ (or $u_i = u_j$) then we always have $y_i = y_j$ (res. $v_i = v_j$); hence the inconsistency will not arise. So, given that the hash key is good, we say that ω is *sampling induced bad* transcript, if one of the following conditions is true: for some $\alpha \in [6]$ and $\beta \in [\alpha \dots 6]$, we have

- $\text{Ycoll}_{\alpha\beta} : \exists i \in \mathcal{I}_\alpha, j \in \mathcal{I}_\beta$, such that $x_i \neq x_j \wedge y_i = y_j$, and
- $\text{Vcoll}_{\alpha\beta} : \exists i \in \mathcal{I}_\alpha, j \in \mathcal{I}_\beta$, such that $u_i \neq u_j \wedge v_i = v_j$,

where $\mathcal{I}_i$ is defined as before in section 7.5.2. By varying α and β over all possible values, we get all 42 conditions which might lead to $x^q \not\leftrightarrow y^q$ or $u^q \not\leftrightarrow v^q$. Here we remark that some of these 42 conditions are never satisfied due to the sampling mechanism prescribed in section 7.5.2. These are $\text{Ycoll}_{11}, \text{Ycoll}_{12}, \text{Ycoll}_{13}, \text{Ycoll}_{22}, \text{Ycoll}_{23}, \text{Ycoll}_{33}, \text{Vcoll}_{11}, \text{Vcoll}_{12}, \text{Vcoll}_{13}, \text{Vcoll}_{22}, \text{Vcoll}_{23}$, and Vcoll_{33} . We listed them here only for the sake of completeness. We write the combined event that one of

¹We use the notation H_i to denote the event that the predicate H_i is true.

the 42 conditions hold as BAD-SAMP. Again by an abuse of notations, we have

$$\text{BAD-SAMP} = \bigcup_{\alpha \in [6], \beta \in [\alpha \dots 6]} (\text{Ycoll}_{\alpha\beta} \cup \text{Vcoll}_{\alpha\beta}). \quad (7.16)$$

Finally, a transcript ω is called bad, i.e. $\omega \in \Omega_{\text{bad}}$, if it is either a hash or a sampling induced bad transcript. All other transcripts are called good.

BAD TRANSCRIPT ANALYSIS: We analyze the probability of realizing a bad transcript in the ideal world. By definition, this is possible if and only if one of BAD-HASH or BAD-SAMP occurs. So, we have

$$\begin{aligned} \epsilon_{\text{bad}} &= \Pr[\Theta_0 \in \Omega_{\text{bad}}] = \Pr_{\Theta_0}[\text{BAD-HASH} \cup \text{BAD-SAMP}] \\ &= \underbrace{\Pr_{\Theta_0}[\text{BAD-HASH}]}_{\epsilon_{\text{hash}}} + \underbrace{\Pr_{\Theta_0}[\text{BAD-SAMP}]}_{\epsilon_{\text{samp}}}. \end{aligned} \quad (7.17)$$

Lemma 7.5.4 upper bounds ϵ_{hash} to $2q^2\epsilon^2 + 2q^2\epsilon^{1.5} + q^4\epsilon^3$ and Lemma 7.5.5 upper bounds ϵ_{samp} to $8q^2\epsilon^{2-n/2} + 4q^4\epsilon^2 2^{-n}$. Substituting these values in Eq. (7.17), we get

$$\epsilon_{\text{bad}} \leq 2q^2\epsilon^2 + 2q^2\epsilon^{1.5} + q^4\epsilon^3 + \frac{8q^2\epsilon}{2^{n/2}} + \frac{4q^4\epsilon^2}{2^n}. \quad (7.18)$$

Lemma 7.5.4. $\epsilon_{\text{hash}} \leq 2q^2\epsilon^2 + 2q^2\epsilon^{1.5} + q^4\epsilon^3$.

Proof. Using Eq. (7.15) and (7.17), we have

$$\epsilon_{\text{hash}} = \Pr[\text{H}_1 \cup \text{H}_2 \cup \text{H}_3 \cup \text{H}_4 \cup \text{H}_5 \cup \text{H}_6 \cup \text{H}_7] \leq \sum_{i=1}^7 \Pr[\text{H}_i].$$

H_1 is true if for some distinct i, j both $X_i = X_j$, and $U_i = U_j$. Now $T_i = T_j \implies M_i \neq M_j$. Hence $X_i \neq X_j$ and H_1 is not true. So suppose $T_i \neq T_j$. Then for a fixed i, j we get an upper bound of ϵ^2 as $\mathcal{H}$ is ϵ -AXU, and we have at most $\binom{q}{2}$ pairs of i, j . Thus, $\Pr[\text{H}_1] \leq \binom{q}{2}\epsilon^2$. Following a similar line of argument one can bound $\Pr[\text{H}_2] \leq \binom{q}{2}\epsilon^2$ and $\Pr[\text{H}_3] \leq \binom{q}{2}\epsilon^2$.

In the remaining, we bound the probability of H_4 and H_6 , while the probability of H_5 and H_7 can be bounded analogously. For any function $f : \mathbb{T} \in \mathbb{B}$, let $f' : \mathbb{T} \times \mathbb{B} \rightarrow \mathbb{B}$ be defined as $f'(t, m) = f(t) \oplus m$. So $X_i = \text{H}'_1(T_i, M_i)$, and $U_i = \text{H}'_2(T_i, C_i)$, for all $i \in [q]$. It is easy to see that H'_b is ϵ -universal if H_b is ϵ -AXU for $b \in \{0, 1\}$. Using the renewed description, H_4 is true if for some distinct i, j, k, ℓ ,

$$\text{H}'_1(T_i, M_i) = \text{H}'_1(T_j, M_j) \wedge \text{H}'_2(T_j, C_j) = \text{H}'_2(T_k, C_k) \wedge \text{H}'_1(T_k, M_k) = \text{H}'_1(T_\ell, M_\ell).$$

Since $(t_i, m_i) \neq (t_j, m_j)$ and $(t_i, c_i) \neq (t_j, c_j)$ for distinct i and j , we can apply the alternating collisions lemma of Lemma 7.3.1 to get $\Pr [\mathbb{H}_4] \leq q^2 \epsilon^{1.5}$.

For $\mathbb{H}_6$ we use the AXU₃ property of $\mathbb{H}_1$. Basically for some distinct i, j, k the probability that $\mathbb{H}'_1(\mathbb{T}_i, \mathbb{M}_i) = \mathbb{H}'_1(\mathbb{T}_j, \mathbb{M}_j) = \mathbb{H}'_1(\mathbb{T}_i, \mathbb{M}_i)$ is upper bounded by ϵ^2 (as $\mathbb{H}_1$ is ϵ -AXU₃ hash). Now for each such triple, the probability that $\mathbb{H}'_2(\mathbb{T}_k, \mathbb{C}_k) = \mathbb{H}'_2(\mathbb{T}_\ell, \mathbb{C}_\ell)$ is upper bounded by ϵ for a fixed ℓ . We have at most $\binom{q}{4}$ such i, j, k, ℓ and hence $\Pr [\mathbb{H}_6] \leq \binom{q}{4} \epsilon^3$. The result follows by summing up all the individual probabilities followed by some algebraic simplifications. $\square$

Lemma 7.5.5. $\epsilon_{\text{samp}} \leq \frac{8q^2\epsilon}{2^{n/2}} + \frac{4q^4\epsilon^2}{2^n}$.

Proof. Using Eq. (7.16) and (7.17), we have

$$\begin{aligned} \epsilon_{\text{samp}} &= \Pr \left[\bigcup_{\alpha \in [6], \beta \in [\alpha \dots 6]} (\text{Ycoll}_{\alpha\beta} \cup \text{Vcoll}_{\alpha\beta}) \right] \\ &\leq \sum_{\alpha \in [6]} \sum_{\beta \in [\alpha \dots 6]} \left(\Pr [\text{Ycoll}_{\alpha\beta}] + \Pr [\text{Vcoll}_{\alpha\beta}] \right). \end{aligned}$$

We bound the probabilities of the events on the right hand side in groups as given below:

Bounding $\sum_{\alpha \in [3], \beta \in [\alpha \dots 3]} \Pr [\text{Ycoll}_{\alpha\beta}] + \Pr [\text{Vcoll}_{\alpha\beta}]$: Recall that the sampling of $\mathbb{Y}$ and $\mathbb{V}$ values is always done consistently for indices belonging to $\mathcal{I} = \mathcal{I}_1 \sqcup \mathcal{I}_2 \sqcup \mathcal{I}_3$. Hence,

$$\sum_{\alpha \in [3], \beta \in [\alpha \dots 3]} \Pr [\text{Ycoll}_{\alpha\beta}] + \Pr [\text{Vcoll}_{\alpha\beta}] = 0. \quad (7.19)$$

Bounding $\sum_{\beta \in \{4,5\}} \Pr [\text{Ycoll}_{1\beta}] + \Pr [\text{Vcoll}_{1\beta}]$: Let's consider the event Ycoll_{14} , which translates to there exist indices $i \in \mathcal{I}_1$ and $j \in \mathcal{I}_4$ such that $\mathbb{X}_i \neq \mathbb{X}_j \wedge \mathbb{Y}_i = \mathbb{Y}_j$. Since $j \in \mathcal{I}_4$, there must exist $k, \ell \in \mathcal{I}_4 \setminus \{j\}$, such that $\mathbb{X}_j = \mathbb{X}_k = \mathbb{X}_\ell$. To bound the probability of Ycoll_{14} , we can thus look at the joint event

$$\exists i \in \mathcal{I}_1, \exists^* j, k, \ell \in \mathcal{I}_4, \text{ such that } \mathbb{Y}_i = \mathbb{Y}_j \wedge \mathbb{X}_j = \mathbb{X}_k \wedge \mathbb{X}_k = \mathbb{X}_\ell.$$

Note that, the event $\mathbb{Y}_i = \mathbb{Y}_j$ is independent of $\mathbb{X}_j = \mathbb{X}_k \wedge \mathbb{X}_k = \mathbb{X}_\ell$, as both $\mathbb{Y}_i$ and $\mathbb{Y}_j$ are sampled independent of the hash key. Thus, we get

$$\begin{aligned} \Pr [\text{Ycoll}_{14}] &= \Pr [\exists i \in \mathcal{I}_1, \exists^* j, k, \ell \in \mathcal{I}_4, \text{ such that } \mathbb{Y}_i = \mathbb{Y}_j \wedge \mathbb{X}_j = \mathbb{X}_k \wedge \mathbb{X}_k = \mathbb{X}_\ell] \\ &= \sum_{i \in \mathcal{I}_1} \sum_{j < k < \ell \in \mathcal{I}_4} \Pr [\mathbb{Y}_i = \mathbb{Y}_j] \cdot \Pr [\mathbb{X}_j = \mathbb{X}_k \wedge \mathbb{X}_k = \mathbb{X}_\ell] \end{aligned}$$

$$\leq q \binom{q}{3} \frac{\epsilon^2}{2^n},$$

where the last inequality follows from the uniform randomness of Y_j and the ϵ -AXU₃ property of H_1 (by viewing $X_m = H_m(T_m) \oplus M_m$ for $m \in \{j, k, \ell\}$). We can bound the probabilities of $Y_{\text{coll}_{15}}$, $V_{\text{coll}_{14}}$, $V_{\text{coll}_{15}}$ in a similar manner as in the case of $Y_{\text{coll}_{14}}$. So we skip the argumentation for these cases, and summarize the probability for this group as

$$\sum_{\beta \in \{4,5\}} \Pr[Y_{\text{coll}_{1\beta}}] + \Pr[V_{\text{coll}_{1\beta}}] \leq 4q \binom{q}{3} \frac{\epsilon^2}{2^n}. \quad (7.20)$$

Bounding $\sum_{\alpha \in [6]} \Pr[Y_{\text{coll}_{\alpha 6}}] + \Pr[V_{\text{coll}_{\alpha 6}}]$: Let's consider the event $Y_{\text{coll}_{16}}$ which translates to there exist indices $i \in \mathcal{I}_1$ and $j \in \mathcal{I}_6$ such that $X_i \neq X_j \wedge Y_i = Y_j$. Here as $j \in \mathcal{I}_6$, there must exist $k, \ell \in \mathcal{I}_6 \setminus \{j\}$ such that one of the following happens

$$X_j = X_k \wedge U_k = U_\ell \quad U_j = U_k \wedge X_k = X_\ell \quad X_j = X_k \wedge U_j = U_\ell$$

Without loss of generality we assume that the first configuration happens. So to bound the probability of $Y_{\text{coll}_{16}}$, we can look at the joint event

$$\exists i \in \mathcal{I}_1, \exists^* j, k, \ell \in \mathcal{I}_6, \text{ such that } Y_i = Y_j \wedge X_j = X_k \wedge U_k = U_\ell.$$

In this case the three events $Y_i = Y_j$, $X_j = X_k$, and $U_k = U_\ell$ are independent as Y_j , H_1 , and H_2 are all independent. Thus, we get

$$\begin{aligned} \Pr[Y_{\text{coll}_{16}}] &= \Pr[\exists i \in \mathcal{I}_1, \exists^* j, k, \ell \in \mathcal{I}_6, \text{ such that } Y_i = Y_j \wedge X_j = X_k \wedge U_k = U_\ell] \\ &= \sum_{i \in \mathcal{I}_1} \sum_{j < k < \ell \in \mathcal{I}_6} \Pr[Y_i = Y_j] \cdot \Pr[X_j = X_k] \cdot \Pr[U_k = U_\ell] \\ &\leq q \binom{q}{3} \frac{\epsilon^2}{2^n}, \end{aligned}$$

where the last inequality follows from the uniform randomness of Y_j and the ϵ -AXU₂ property of H_1 and H_2 . The probabilities of all the remaining events in this group can be bounded in a similar fashion. We emphasize here that, in case of $Y_{\text{coll}_{66}}$ and $V_{\text{coll}_{66}}$, it is of note that both i and j cannot lie in the same component due to the sampling mechanism. In summary, we have

$$\sum_{\alpha \in [6]} \Pr[Y_{\text{coll}_{\alpha 6}}] + \Pr[V_{\text{coll}_{\alpha 6}}] \leq 12q \binom{q}{3} \frac{\epsilon^2}{2^n}. \quad (7.21)$$

Bounding $\sum_{\alpha \in \{2,4\}} \Pr [\text{Ycoll}_{\alpha 4}] + \Pr [\text{Vcoll}_{\alpha 4}]$: In this case we consider the probability of event Ycoll_{24} , while the probabilities of all other events can be similarly upper bounded. Now Ycoll_{24} translates to there exist $i \in \mathcal{I}_2$ and $j \in \mathcal{I}_4$ such that $X_i \neq X_j \wedge Y_i = Y_j$. Since $i \in \mathcal{I}_2$ and $j \in \mathcal{I}_4$, there must exist $k \in \mathcal{I}_2 \setminus \{i\}$ and $\ell \in \mathcal{I}_4 \setminus \{j\}$ such that $X_i = X_k \wedge X_j = X_\ell$. To bound Ycoll_{24} , we look at the joint event

$$\exists^* i, k \in \mathcal{I}_2, \exists^* j, \ell \in \mathcal{I}_4, \text{ such that } X_k = X_i \wedge Y_i = Y_j \wedge X_j = X_\ell.$$

Here, the event $Y_i = Y_j$ is independent of $X_k = X_i$ and $X_j = X_\ell$ (Y_i and Y_j are sampled independently of H_1). So we can apply the alternating events lemma, which gives

$$\begin{aligned} \Pr [\text{Ycoll}_{24}] &= \Pr [\exists^* i, k \in \mathcal{I}_2, \exists^* j, \ell \in \mathcal{I}_4, \text{ such that } X_k = X_i \wedge Y_i = Y_j \wedge X_j = X_\ell] \\ &\leq q^2 \frac{\epsilon}{2^{n/2}}, \end{aligned}$$

where the inequality follows from Lemma 7.3.2, and the fact $|\mathcal{I}_2|, |\mathcal{I}_4| \leq q$. The probability of this group is summarized below

$$\sum_{\alpha \in \{2,4\}} \Pr [\text{Ycoll}_{\alpha 4}] + \Pr [\text{Vcoll}_{\alpha 4}] \leq 4q^2 \frac{\epsilon}{2^{n/2}}. \quad (7.22)$$

Bounding $\sum_{\alpha \in \{3,5\}} \Pr [\text{Ycoll}_{\alpha 5}] + \Pr [\text{Vcoll}_{\alpha 5}]$: This is analogous to the previous case, and can be bounded as below

$$\sum_{\alpha \in \{3,5\}} \Pr [\text{Ycoll}_{\alpha 5}] + \Pr [\text{Vcoll}_{\alpha 5}] \leq 4q^2 \frac{\epsilon}{2^{n/2}}. \quad (7.23)$$

Bounding $\sum_{(\alpha,\beta) \in \{(2,5),(3,4),(4,5)\}} \Pr [\text{Ycoll}_{\alpha\beta}] + \Pr [\text{Vcoll}_{\alpha\beta}]$: We consider the probability of the event Ycoll_{25} which translates to there exist $i \in \mathcal{I}_2$ and $j \in \mathcal{I}_5$, such that $X_i \neq X_j \wedge Y_i = Y_j$. As $i \in \mathcal{I}_2$ and $j \in \mathcal{I}_5$, there must exist $k \in \mathcal{I}_2 \setminus \{i\}$ and $\ell \in \mathcal{I}_5 \setminus \{j\}$, such that $X_i = X_k \wedge Y_j = U_\ell$. We consider the joint event

$$\exists^* i, k \in \mathcal{I}_2, j, \ell \in \mathcal{I}_5, \text{ such that } X_k = X_i \wedge Y_i = Y_j \wedge U_j = U_\ell.$$

Using a similar line of argument as in the case of group 3, we get

$$\Pr [\text{Ycoll}_{25}] \leq q \binom{q}{3} \frac{\epsilon^2}{2^n}.$$

And finally, we have

$$\sum_{(\alpha,\beta) \in \{(2,5),(3,4),(4,5)\}} \Pr [\text{Ycoll}_{\alpha\beta}] + \Pr [\text{Vcoll}_{\alpha\beta}] \leq 6q \binom{q}{3} \frac{\epsilon^2}{2^n}. \quad (7.24)$$

The result follows by combining Eq. (7.19-7.24), followed by some algebraic simplifications. $\square$

7.5.4 Good Transcript Analysis

From section 7.5.2, we know the types of components present in the transcript graph corresponding to a good transcript ω are exactly as in Figure 7.5.1. Let

$$\omega = (t^q, m^q, c^q, x^q, y^q, v^q, u^q, \lambda^q, h_1, h_2)$$

be the good transcript at hand. From the bad transcript description of section 7.5.3, we know that for a good transcript $(t^q, m^q) \rightsquigarrow (t^q, c^q)$, $x^q \rightsquigarrow y^q$, $v^q \rightsquigarrow u^q$, and $y^q \oplus v^q = \lambda^q$.

We add some new parameters with respect to ω to aid our analysis of good transcripts. For $i \in [6]$, let $c_i(\omega)$ and $q_i(\omega)$ denote the number of components and number of indices (corresponding to the edges), respectively of type- i in ω . Note that, $q_1(\omega) = c_1(\omega)$, $q_i(\omega) = 2c_i(\omega)$, for $i \in \{2, 3\}$, and $q_6(\omega) = 3c_6(\omega)$. Obviously, for a good transcript $q = \sum_{i=1}^6 q_i(\omega)$. Let $c(\omega) = c_2(\omega) + c_3(\omega)$. For all these parameters, we will drop the ω parametrization whenever it is understood from the context.

INTERPOLATION PROBABILITY FOR THE REAL ORACLE: In the real oracle, $H^2 \leftarrow \$ \mathcal{H}^2$, Π_1 is called exactly $q_1 + c_2 + q_3 + c_4 + q_5 + 2c_6$ times and Π_2 is called exactly $q_1 + q_2 + c_3 + q_4 + c_5 + 2c_6$ times. Thus, we have

$$\Pr[\Theta_1 = \omega] = \frac{1}{|\mathcal{H}|^2} \cdot \frac{1}{(2^n)_{q_1+c_2+q_3+c_4+q_5+2c_6}} \cdot \frac{1}{(2^n)_{q_1+q_2+c_3+q_4+c_5+2c_6}}. \quad (7.25)$$

INTERPOLATION PROBABILITY FOR THE IDEAL ORACLE: In the ideal oracle, the sampling is done in parts:

- I. $\tilde{\Pi}$ sampling: Let $(t'_1, t'_2, \dots, t'_r)$ denote the tuple of distinct tweaks in t^q , and for all $i \in [r]$, let $a_i = \mu(t^q, t'_i)$, i.e. $r \leq q$ and $\sum_{i=1}^r a_i = q$. Then, we have

$$\Pr[\tilde{\Pi}(t^q, m^q) = c^q] \leq \frac{1}{\prod_{i=1}^r (2^n)_{a_i}}.$$

- II. Hash key sampling: The hash keys are sampled uniformly from $\mathcal{H}^2$, whence

$$\Pr[(H_1, H_2) = (h_1, h_2)] = \frac{1}{|\mathcal{H}|^2}.$$

- III. Internal variables sampling: The internal variables Y^q and V^q are sampled in two stages.

- (A). *type-1, type-2 and type-3 sampling*: Recall the sets $\mathcal{I}_1$, $\mathcal{I}_2$, and $\mathcal{I}_3$, from section 7.5.3. Consider the system of equation

$$\mathcal{L} = \{Y_i \oplus V_i = \lambda_i : i \in \mathcal{I}\}.$$

Let $(\lambda'_1, \lambda'_2, \dots, \lambda'_s)$ denote the tuple of distinct elements in $\lambda^\mathcal{I}$, and for all $i \in [s]$, let $b_i = \mu(\lambda^\mathcal{I}, \lambda'_i)$. From Figure 7.5.1 we know that $\mathcal{L}$ is cycle-free, non-degenerate and has $\xi_{\max} \leq 3$. So, we can apply Corollary 7.4.2 to get a lower bound on the the number of valid solutions, $|S(\mathcal{L})|$ for $\mathcal{L}$. Using the fact that $(Y^\mathcal{I}, V^\mathcal{I}) \leftarrow \$ S(\mathcal{L})$, and Corollary 7.4.2, we have

$$\Pr[(Y^\mathcal{I}, V^\mathcal{I}) = (y^\mathcal{I}, v^\mathcal{I})] \leq \frac{\prod_{i=1}^s (2^n)^{b_i}}{\zeta(\omega) (2^n)^{q_1+c_2+q_3} (2^n)^{q_1+q_2+c_3}},$$

where

$$\zeta(\omega) = \left(1 - \frac{780q^2c(\omega)}{2^{2n}} - \frac{100q^4}{2^{3n}} - \frac{260q^2}{2^{2n}}\right),$$

- (B). *type-4, type-5 and type-6 sampling*: For the remaining indices, one value is sampled uniformly for each of the components, i.e. we have

$$\Pr\left[\left(Y^{[q]\setminus\mathcal{I}}, V^{[q]\setminus\mathcal{I}}\right) = \left(y^{[q]\setminus\mathcal{I}}, v^{[q]\setminus\mathcal{I}}\right)\right] = \frac{1}{(2^n)^{c_4+c_5+c_6}}.$$

By combining I, II, III, and rearranging the terms, we have

$$\Pr[\Theta_0 = \omega] \leq \frac{1}{|\mathcal{H}|^2} \cdot \frac{1}{\zeta(\omega)} \cdot \frac{\prod_{i=1}^s (2^n)^{b_i}}{\prod_{i=1}^r (2^n)^{a_i} (2^n)^{p_1} (2^n)^{p_2} (2^n)^{c_4+c_5+c_6}}, \quad (7.26)$$

where $p_1 = q_1 + c_2 + q_3$, and $p_2 = q_1 + q_2 + c_3$.

7.5.5 Ratio of Interpolation Probabilities

First, we give two inequality results in Propositions 7.5.7 and 7.5.8, which will be used to compute the ratio of interpolation probabilities. The proofs of these results are postponed to the end of this section.

Definition 7.5.6. For $r \geq s$, let $a = (a_i)_{i \in [r]}$ and $b = (b_j)_{j \in [s]}$ be two sequences over $\mathbb{N}$. We say that a *compresses to* b , if there exists a partition $\mathcal{P}$ of $[r]$ such that $\mathcal{P}$ contains exactly s many cells, say $\mathcal{P}_1, \dots, \mathcal{P}_s$, and $\forall i \in [s]$, $b_i = \sum_{j \in \mathcal{P}_i} a_j$.

Proposition 7.5.7 (Fact 1 in [108]). For $r \geq s$, let $a = (a_i)_{i \in [r]}$ and $b = (b_j)_{j \in [s]}$ be sequences over $\mathbb{N}$, such that a compresses to b . Then for any $n \in \mathbb{N}$, such that $2^n \geq \sum_{i=1}^r a_i$, we have $\prod_{i=1}^r (2^n)^{a_i} \geq \prod_{j=1}^s (2^n)^{b_j}$.

We remark here that Fact 1 in [108] is a variant of Proposition 7.5.7, which is in fact false. However, the proof of Lemma 3 in [108] implicitly used Proposition 7.5.7, and hence stands correct.

Proposition 7.5.8. *For $r \geq 2$, let $c = (c_i)_{i \in [r]}$ and $d = (d_i)_{i \in [r]}$ be two sequences over $\mathbb{N}$. Let $a_1, a_2, b_1, b_2 \in \mathbb{N}$, such that $c_i \leq a_j$, $c_i + d_i \leq a_j + b_j$ for all $i \in [r]$ and $j \in [2]$, and $\sum_{i=1}^r d_i = b_1 + b_2$. Then, for any $n \in \mathbb{N}$, such that $a_j + b_j \leq 2^n$ for $j \in [2]$, we have $\prod_{i=1}^r (2^n - c_i)_{d_i} \geq (2^n - a_1)_{b_1} (2^n - a_2)_{b_2}$.*

Proposition 7.5.8 is quite intuitive, in the sense, that the starting value in each of the falling factorial term on the left is at least as much as the starting values on the right, and the total number of terms are same on both the sides.

Now, on dividing Eq. (7.25) by Eq. (7.26), and simplifying the expression, we get

$$\begin{aligned} \frac{\Pr [\Theta_1 = \omega]}{\Pr [\Theta_0 = \omega]} &\stackrel{1}{\geq} \zeta(\omega) \cdot \frac{\prod_{i=1}^r (2^n)_{a_i}}{\prod_{i=1}^s (2^n)_{b_i} (2^n - p_1 - c_4 - c_6)_{q_5 + c_6} (2^n - p_2 - c_5)_{q_4 + 2c_6}} \\ &\stackrel{2}{\geq} \zeta(\omega) \cdot \frac{\prod_{i=1}^r (2^n)_{d_i} \prod_{i=1}^r (2^n - d_i)_{a_i - d_i}}{\prod_{i=1}^s (2^n)_{b_i} (2^n - p_1 - c_4 - c_6)_{q_5 + c_6} (2^n - p_2 - c_5)_{q_4 + 2c_6}} \\ &\stackrel{3}{\geq} \zeta(\omega) \cdot \frac{\prod_{i=1}^r (2^n - d_i)_{a_i - d_i}}{(2^n - p_1 - c_4 - c_6)_{q_5 + c_6} (2^n - p_2 - c_5)_{q_4 + 2c_6}} \Bigg\} A \\ &\stackrel{4}{\geq} \zeta(\omega). \end{aligned} \tag{7.27}$$

From inequality 1 to 2, we rewrite the numerator such that $d_i = \mu(t^{\mathcal{I}}, t'_i)$ for $i \in [r]$. At inequality 2, $r \geq s$, as number of distinct internal masking values is at most the number of distinct tweaks, and $\widehat{t^{\mathcal{I}}}$ compresses to $\widehat{\lambda^{\mathcal{I}}}$. So using Proposition 7.5.7, we can justify the transition from 2 to 3. At inequality 3, for $i \in \{2, 3, 4, 5, 6\}$, $c_i(\omega) > 0$ if and only if $r \geq 2$. Also, $d_i \leq c_1 + c_2 + c_3 \leq p_1 + c_4 + c_6$ and $d_i \leq p_2 + c_5$ for $i \in [r]$. Similarly, $a_i \leq c_1 + c_2 + c_3 + c_4 + c_5 + 2c_6 \leq p_1 + c_4 + q_5 + 2c_6$, and $a_i \leq p_2 + q_4 + c_5 + 2c_6$. Thus, A satisfies the conditions given in Proposition 7.5.8, and hence $A \geq 1$. This justifies the transition from 2 to 3.

We define $\epsilon_{\text{ratio}} : \Omega \rightarrow [0, \infty)$ by the mapping

$$\epsilon_{\text{ratio}}(\omega) = 1 - \zeta(\omega).$$

Clearly ϵ_{ratio} is non-negative and the ratio of real to ideal interpolation probabilities is at least $1 - \epsilon_{\text{ratio}}(\omega)$ (using Eq. (7.27)). Let C , C_2 , and C_3 denote the random variables $c(\Theta_0)$, $c_2(\Theta_0)$, and $c_3(\Theta_0)$. Thus, we can use Lemma 2.2.1 to get

$$\text{Adv}_{\text{CLRW2}}^{\text{tsprp}}(q) \leq \frac{780q^2}{2^{2n}} \text{Ex}[C] + \frac{100q^4}{2^{3n}} + \frac{260q^2}{2^{2n}} + \epsilon_{\text{bad}}$$

$$\leq \frac{780q^4\epsilon}{2^{2n}} + \frac{100q^4}{2^{3n}} + \frac{260q^2}{2^{2n}} + \epsilon_{\text{bad}}, \quad (7.28)$$

where the second inequality follows from the fact that $\text{Ex}[C] \leq q^2\epsilon$. This is due to the fact that $C = C_2 + C_3$, and for $i \in \{2, 3\}$, $\text{Ex}[C_i] \leq q^2\epsilon/2$, which itself follows from the ϵ -AXU₂ property of $\mathcal{H}$. Theorem 7.5.1 follows from Eq. (7.14), (7.18), and (7.28). $\square$

Proof of Proposition 7.5.7: Suppose a compresses to b due to a partition $\mathcal{P}$. Then, we call $\mathcal{P}$ the compressing partition of a and b . For $s \geq 1$, let $p(s)$ denote the claimed statement. We prove the result by induction on s . We first handle the base case, $s = 1$. In this case, we have $b_1 = \sum_{i=1}^r a_i$. Thus, $a_i \leq b_1$ for all $i \in [r]$. Now, a term by term comparison gives

$$\prod_{i=1}^r (2^n)_{a_i} \geq (2^n)_{b_1},$$

which shows that the base case $p(1)$ is true. Suppose $p(s)$ is true for all $s = n$, for some $n > 1$. We now show that $p(n+1)$ is true.

Let $a = (a_i)_{i \in [r]}$ and $b = (b_j)_{j \in [s+1]}$ be two sequences over $\mathbb{N}$, such that $r \geq s+1$ and a compresses to b . Suppose $\mathcal{P}$ is a compressing partition of a and b . Consider the sequences $a' = (a_i)_{i \in \mathcal{P}_{s+1}}$ and $b' = (b_{s+1})$. We have $|\mathcal{P}_{s+1}| \geq 1$, and $b_{s+1} = \sum_{i \in \mathcal{P}_{s+1}} a_i$, which means a' compresses to b' . Further, $2^n \geq \sum_{i \in \mathcal{P}_{s+1}} a_i$. Thus, we can apply $p(1)$ result on a' and b' to get

$$\prod_{i \in \mathcal{P}_{s+1}} (2^n)_{a_i} \geq (2^n)_{b_{s+1}}. \quad (7.29)$$

For the remaining, let $a'' = (a_i)_{i \in [r] \setminus \mathcal{P}_{s+1}}$ and $b'' = (b_j)_{j \in [s]}$. Again, we have $r - |\mathcal{P}_{s+1}| \geq s$, and $b_i = \sum_{j \in \mathcal{P}_i} a_j$ for all $i \in [s]$. Thus, we can apply the induction hypothesis for $p(s)$ on a'' and b'' to get

$$\prod_{i \in [r] \setminus \mathcal{P}_{s+1}} (2^n)_{a_i} \geq \prod_{j \in [s]} (2^n)_{b_j}. \quad (7.30)$$

The combination of Eq. (7.29) and (7.30) shows that $p(s+1)$ is true. The result follows by induction. $\square$

Proof of Proposition 7.5.8: For $r \geq 2$, let $p(r)$ denote the claimed statement. We prove the result by induction on r . For now, assume $p(2)$ to be true, as we handle this case later. Suppose the proposition statement, denoted $p(r)$, is true for all $r \geq 2$. We show that the statement $p(r+1)$ is true. Fix some arbitrary $n \in \mathbb{N}$.

Let $a_1, a_2, b_1, b_2, c_1, \dots, c_{r+1}, d_1, \dots, d_{r+1} \in \mathbb{N}$, such that $c_i \leq a_i$ and $c_i + d_i \leq a_i + b_j \leq 2^n$, for all $i \in [r+1]$ and $j \in [2]$. Let i' be the smallest index in $[r+1]$, such that $d_{i'} = \min\{d_1, \dots, d_{r+1}\}$ (such an element exist by well ordering principle). Without loss of generality, we assume that $b_1 \geq b_2$. We compare the terms, $(2^n - c_{i'} - j + 1)$ and $(2^n - a_1 - j + 1)$, for all $j \in [d_{i'}]$. Since $c_{i'} \leq a_1$, we must have $(2^n - c_{i'} - j + 1) \geq (2^n - a_1 -$

$j+1$), for all $j \in [d_{i'}]$. Now, we must have $d_{i'} \leq b_1$, otherwise $d_{i'} > b_1 \geq b_2$ which leads to $\sum_{i \in [r]} d_i > b_1 + b_2$. Suppose $d_{i'} < b_1$, then using $(2^n - c_{i'} - j + 1)/(2^n - a_1 - j + 1) \geq 1$, we remove all the $(2^n - c_{i'} - j + 1)$, $(2^n - a_1 - j + 1)$ terms for all $j \in [d_{i'}]$. This reduces the claimed statement to $p(r)$, which is true by hypothesis. If $d_{i'} = b_1$, then we are left with $\prod_{i \in [r+1] \setminus \{i'\}} (2^n - c_i) \cdots (2^n - c_i - d_i + 1)$ on the left, where $r \geq 2$, and $(2^n - a_2) \cdots (2^n - a_2 - b_2 + 1)$ on the right. Using a similar line of argument as above we can again reduce the claimed statement to $p(r)$, which is true by hypothesis. So $p(r+1)$ is true.

Now the base case $p(2)$ can be handled in a similar manner. In this case we assume without loss of generality that $d_1 \leq d_2$ and $b_1 \geq b_2$, where $d_1 + d_2 = b_1 + b_2$. Since $c_1 \leq a_1$, we must have $(2^n - c_1 - j + 1) \geq (2^n - a_1 - j + 1)$, for all $j \in [d_1]$. Now, we must have $d_1 \leq b_1$, otherwise $d_1 > b_1 \geq b_2$ which leads to $d_1 + d_2 > b_1 + b_2$. If $d_1 = b_1$, then after removing all the terms corresponding to (c_1, d_1) and (a_1, b_1) , we have $(2^n - c_2) \cdots (2^n - c_2 - d_2 + 1)$ on the left and $(2^n - a_2) \cdots (2^n - a_2 - b_2 + 1)$, where $c_2 \leq a_2$ and $c_2 + b_2 \leq a_2 + b_2$, whence $(2^n - c_2) \cdots (2^n - c_2 - d_2 + 1) \geq (2^n - a_2) \cdots (2^n - a_2 - b_2 + 1)$. If $d_1 < b_1$, then we compare terms from $(2^n - c_2) \cdots (2^n - c_2 - d_2 + 1)$ with $(2^n - a_1 - d_1) \cdots (2^n - a_1 - b_1 + 1)(2^n - a_2) \cdots (2^n - a_2 - b_2 + 1)$. First $(2^n - c_2 - d_2 + j) \geq (2^n - a_2 - b_2 + j)$ for $j \in [b_2]$, as $c_2 + d_2 \leq a_2 + b_2$. We remove all these terms to get $(2^n - c_2) \cdots (2^n - c_2 - d_2 + b_2 + 1)$ on the left and $(2^n - a_1 - d_1) \cdots (2^n - a_1 - b_1 + 1)$ on the right, where the number of terms $d_2 - b_2 = b_1 - d_1$. Since $c_2 \leq a_1$, $(2^n - c_2 - j + 1) \geq (2^n - a_1 - d_1 - j + 1)$ for all $j \in [b_1 - d_1]$. This shows that $p(2)$ is true. $\square$

7.6 Proof of Mirror Theory for Tweakable Permutations

The induction is defined on the number of components. Apropos to this, we consider the parameter $\mathfrak{h}_i$ for $i \in [q_1 + c]$, which denotes the number of solutions for the sub-system consisting of the first i components of $\mathcal{L}$, denoted $\mathcal{L}_{|i}$. Note that, $h_i = \mathfrak{h}_i$ for $i \in [q_1]$, and $h_q = \mathfrak{h}_{q_1+c}$. We describe the proof in two steps. First, in section 7.6.1, we bound the number of solutions for sub-system $\mathcal{C}_1$. Given the number of solutions for $\mathcal{C}_1$, we then bound the number of solutions for sub-systems $\mathcal{C}_2$ and $\mathcal{C}_3$, in section 7.6.2, which essentially gives a bound for the complete system $\mathcal{L}$.

7.6.1 Bound for Sub-System with $\xi = 2$

For all $i \in [q_1]$, denote

$$H_i = \prod_{\lambda' \in \widehat{\lambda}^i} (2^n)_{\mu(\lambda^i, \lambda')} \cdot \mathfrak{h}_i,$$

and $J_i = (2^n)_i^2$. As noted before, we want to bound $\mathfrak{h}_i$ by induction on i , i.e. we want to evaluate $\mathfrak{h}_{i+1}$ from $\mathfrak{h}_i$. Inspired by Patarin's mirror theory argument [142, 160], we will study the relation between H_i and J_i for all $i \in [q_1]$. First, we give two supplementary results in Lemma 7.6.1 and 7.6.3, which will be used later on to prove the main result.

Lemma 7.6.1. *For $i \in [q_1]$,*

$$\mathfrak{h}_{i+1} = \mathfrak{h}_i (2^n - 2i + \delta_{i+1}) + \sum_{(j,k) \in \mathcal{M}} \mathfrak{h}'_i(j, k, \lambda_{i+1}),$$

where

$$\mathcal{M} = \{(j, k) : j, k \in [i], j \neq k, \lambda_{i+1} \neq \lambda_j, \lambda_{i+1} \neq \lambda_k\},$$

and $\mathfrak{h}'_i(j, k, \lambda_{i+1})$ denotes the number of solutions of $\mathcal{L}'_i(j, k, \lambda_{i+1}) := \mathcal{L}_i \cup \{Y_j \oplus V_k = \lambda_{i+1}\}$, for some $j, k \in [i]$.

Proof. Let $\mathcal{S}_i$ denote the solution space of $\mathcal{L}_i$, i.e. $\mathfrak{h}_i = |\mathcal{S}_i|$. For a fix $(y^i, v^i) \in \mathcal{S}_i$, we want to compute the number of (y_{i+1}, v_{i+1}) pairs such that $(y^{i+1}, v^{i+1}) \in \mathcal{S}_{i+1}$. Now, some pair $(x, x \oplus \lambda_{i+1})$ is valid if $x \neq y_j$ and $x \oplus \lambda_{i+1} \neq v_k$, for $j, k \in [i]$. This means that $x \notin \mathcal{Y} \cup \mathcal{V}$, where $\mathcal{Y} = \{y_j : j \in [i]\}$ and $\mathcal{V} = \{v_j \oplus \lambda_{i+1} : j \in [i]\}$. As all y_j values are pairwise distinct and v_j values are pairwise distinct, we must have $|\mathcal{Y}| = |\mathcal{V}| = i$. Thus, we have

$$\begin{aligned} \mathfrak{h}_{i+1} &= \sum_{(y^i, v^i) \in \mathcal{S}_i} (2^n - |\mathcal{Y} \cup \mathcal{V}|) \\ &= \sum_{(y^i, v^i) \in \mathcal{S}_i} (2^n - |\mathcal{Y}| - |\mathcal{V}| + |\mathcal{Y} \cap \mathcal{V}|) \\ &= \mathfrak{h}_i \cdot (2^n - 2i) + \sum_{(y^i, v^i) \in \mathcal{S}_i} |\mathcal{Y} \cap \mathcal{V}| \\ &\stackrel{1}{=} \mathfrak{h}_i \cdot (2^n - 2i) + \sum_{(y^i, v^i) \in \mathcal{S}_i} \sum_{j, k \in [i]} \phi(j, k) \\ &\stackrel{2}{=} \mathfrak{h}_i \cdot (2^n - 2i) + \sum_{j, k \in [i]} \mathfrak{h}'_i(j, k, \lambda_{i+1}) \\ &\stackrel{3}{=} \mathfrak{h}_i \cdot (2^n - 2i) + \mathfrak{h}_i \cdot \delta_{i+1} + \sum_{(j,k) \in \mathcal{M}} \mathfrak{h}'_i(j, k, \lambda_{i+1}) \\ &= \mathfrak{h}_i \cdot (2^n - 2i + \delta_{i+1}) + \sum_{(j,k) \in \mathcal{M}} \mathfrak{h}'_i(j, k, \lambda_{i+1}), \end{aligned} \tag{7.31}$$

where $\phi(j, k)$ is the indicator variable that takes the value of 1 when $y_j \oplus v_k = \lambda_{i+1}$, and 0 otherwise. The equality from 1 to 2 follows from the definition of $\mathfrak{h}'_i(j, k, \lambda_{i+1})$, and the equality from 2 to 3 follows from the fact that exactly δ_{i+1} many (j, k) pairs exist such that $k = j$, $\lambda_{i+1} = \lambda_j$, and $y_j \oplus v_j = \lambda_{i+1}$. For these the number of solutions is

exactly the same as $\mathfrak{h}_i$ (since $Y_j \oplus V_k = \lambda_{i+1}$ is already in $\mathcal{L}_{|i}$). The remaining valid (j, k) pairs, must have $\lambda_j, \lambda_k \neq \lambda_{i+1}$, else they contradict $\mathcal{L}$. The set of these remaining (j, k) pairs is the set $\mathcal{M}$. $\square$

The following corollary of Lemma 7.6.1 will be quite useful. The proof is immediate from the proof of Lemma 7.6.1.

Corollary 7.6.2. For $i \geq 1$, $(2^n - 2i)\mathfrak{h}_i \leq \mathfrak{h}_{i+1} \leq (2^n - i)\mathfrak{h}_i$.

Lemma 7.6.3. For all $(j, k) \in \mathcal{M}$, and for all $\beta \in \{0, 1\}^n$,

$$\mathfrak{h}'_i(j, k, \beta) \geq \frac{\mathfrak{h}_i}{2^n - i + 1} \cdot \left(1 - \frac{2(i-2)}{2^n - 2(i-2)}\right).$$

Proof. We are interested in $\mathfrak{h}'_i(j, k, \beta)$, which is the number of solutions of $\mathcal{L}'_{|i}(j, k, \beta)$, $j, k \in \mathcal{M}$. The sub-system containing j and k equations is of the form

$$Y_j \oplus V_j = \lambda_j, \quad Y_j \oplus V_k = \beta, \quad Y_k \oplus V_k = \lambda_k,$$

where once we fix $Y_j = y_j$, all other unknowns are completely determined by linearity. Thus, $\mathfrak{h}'_i(j, k, \beta)$ is at most $\mathfrak{h}_{i-1}$, where $\mathfrak{h}_{i-1}$ is the number of solutions of $\mathcal{L}_{|i-1}$, the system obtained by removing the equations $Y_j \oplus V_k = \beta$ and $Y_k \oplus V_k = \lambda_k$ from $\mathcal{L}'_{|i}(j, k, \beta)$. Now a solution among the $\mathfrak{h}_{i-1}$ solutions of $\mathcal{L}_{i-1}$ is not valid to be counted in $\mathfrak{h}'_i(j, k, \beta)$, if there exists $\ell \in [i] \setminus \{k\}$, such that $y_j \oplus v_\ell = \beta$ or $y_j \oplus v_\ell = \beta \oplus \lambda_k \oplus \lambda_\ell$. The first case leads to $V_k = V_\ell$, and the second case leads to $Y_k = Y_\ell$, where $k \neq \ell$ is obvious. Therefore, the two cases correspond to the terms $\mathfrak{h}'_{i-1}(j, \ell, \beta)$ and $\mathfrak{h}'_{i-1}(j, \ell, \beta \oplus \lambda_k \oplus \lambda_\ell)$, whence we have

$$\mathfrak{h}'_i(j, k, \beta) \geq \mathfrak{h}_{i-1} - \sum_{\ell \in [i] \setminus \{j, k\}} \mathfrak{h}'_{i-1}(j, \ell, \beta) - \sum_{\ell \in [i] \setminus \{j, k\}} \mathfrak{h}'_{i-1}(j, \ell, \beta \oplus \lambda_k \oplus \lambda_\ell)$$

Using similar line of argument as above we bound $\mathfrak{h}'_{i-1}(j, \ell, \beta) \leq \mathfrak{h}_{i-2}$ and $\mathfrak{h}'_{i-1}(j, \ell, \beta \oplus \lambda_k \oplus \lambda_\ell) \leq \mathfrak{h}_{i-2}$, where the two $\mathfrak{h}_{i-2}$ terms correspond to the number of solutions of $\mathcal{L}'_{|i-2}(j, \ell, \beta)$ and $\mathcal{L}'_{|i-2}(j, \ell, \beta \oplus \lambda_k \oplus \lambda_\ell)$. Finally, we have

$$\begin{aligned} \mathfrak{h}'_i(j, k, \beta) &\geq \mathfrak{h}_{i-1} - \sum_{\ell \in [i] \setminus \{j, k\}} \mathfrak{h}_{i-2} - \sum_{\ell \in [i] \setminus \{j, k\}} \mathfrak{h}_{i-2} \\ &\stackrel{1}{\geq} \mathfrak{h}_{i-1} - 2(i-2)\mathfrak{h}_{i-2} \\ &\stackrel{2}{\geq} \mathfrak{h}_{i-1} \left(1 - \frac{2(i-2)}{2^n - 2(i-2)}\right) \\ &\stackrel{3}{\geq} \frac{\mathfrak{h}_i}{2^n - i + 1} \left(1 - \frac{2(i-2)}{2^n - 2(i-2)}\right), \end{aligned}$$

where 1-3 follows from Corollary 7.6.2. Note that, we switch from $\mathfrak{h}_{i-2}$ to $\mathfrak{h}_{i-1}$ by reintroducing the equation $Y_\ell \oplus V_\ell = \lambda_\ell$, and from $\mathfrak{h}_{i-1}$ to $\mathfrak{h}_i$ by reintroducing the equation $Y_k \oplus V_k = \lambda_k$. The readers may use Figure 7.6.1 to get a pictorial view of the switchings between different systems of equations. $\square$

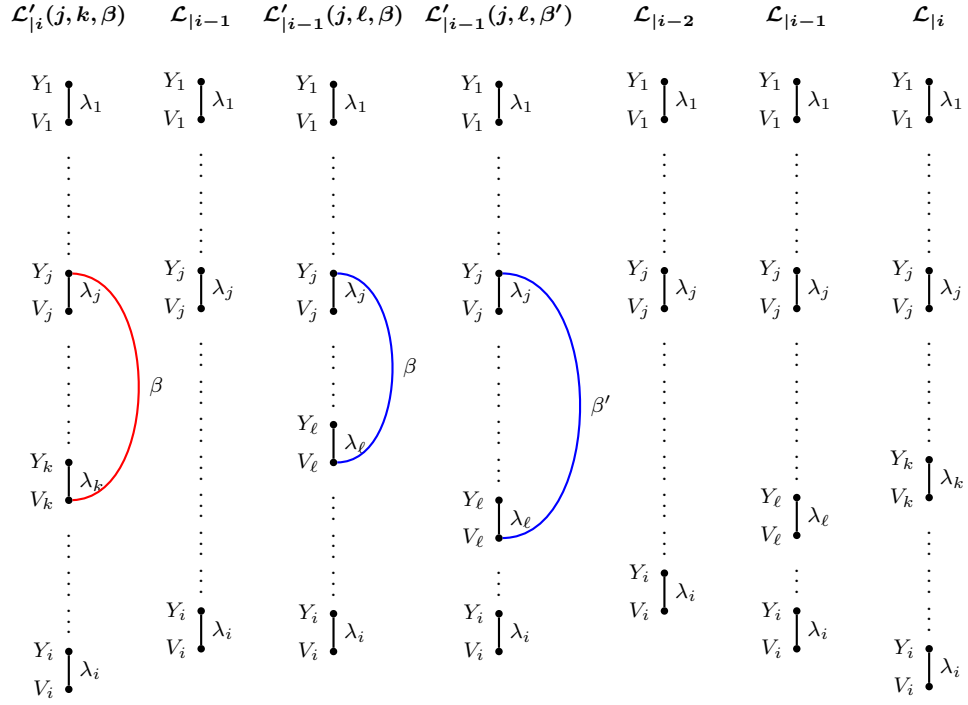


Figure 7.6.1: The switchings used in the proof of Lemma 7.6.3. From left to right: $\mathcal{L}'_{|i}(j, k, \beta)$ is the system $\mathcal{L}_{|i} \cup \{Y_j \oplus V_k = \beta\}$; $\mathcal{L}_{|i-1}$ is obtained by removing the equations involving V_k from $\mathcal{L}'_{|i}(j, k, \beta)$; $\mathcal{L}'_{|i-1}(j, l, \beta)$ is the system $\mathcal{L}_{|i-1} \cup \{Y_j \oplus V_\ell = \beta\}$; $\mathcal{L}'_{|i-1}(j, l, \beta')$ is the system $\mathcal{L}_{|i-1} \cup \{Y_j \oplus V_\ell = \beta'\}$, where $\beta' = \beta \oplus \lambda_k \oplus \lambda_\ell$; $\mathcal{L}_{|i-2}$ is obtained by removing the equations involving V_ℓ from $\mathcal{L}'_{|i-1}(j, l, \beta)$ or $\mathcal{L}'_{|i-1}(j, l, \beta')$. Note that, there should have been two $\mathcal{L}_{|i-2}$ switchings, one each for $\mathcal{L}'_{|i-1}(j, l, \beta)$ and $\mathcal{L}'_{|i-1}(j, l, \beta')$. We have drawn just once for economical reasons. Similar clarification applies to switchings from $\mathcal{L}_{|i-2}$ to $\mathcal{L}_{|i-1}$, and from $\mathcal{L}_{|i-1}$ to $\mathcal{L}_{|i}$.

Remark 7.6.4. In [160, Theorem 11] a result similar to Lemma 7.6.3 has been proved for random function scenario. While the proof of that theorem is correct, there is a notational issue which is worth pointing out. The $\mathfrak{h}'$ notation is used in an unparameterized fashion, with an explicit hint in [160, Theorem 8] that this is done for simplification. But this simplification leads to a rather peculiar technical issue in [160, Theorem 11], where both lower and upper bounds are required on $\mathfrak{h}'$ values, requiring different switchings. Without the parametrization it is difficult to understand (and verify) the switchings.

Remark 7.6.5. The proof of Lemma 7.6.3 should also give an idea of the proof complexity. Since we only want $\epsilon = O(q^4/2^{3n})$, we needed a somewhat crude estimate of $\mathfrak{h}'$ values.

In actual mirror theory as we move towards $\epsilon = O(q/2^n)$, we have to make a good estimate of $\mathfrak{h}'$ values, which does not seem to be easy.

Now, we state the main result of this section.

Lemma 7.6.6. *For $q_1 < 2^{n-2}$, we have*

$$\frac{H_{q_1}}{J_{q_1}} \geq \left(1 - \frac{4q_1^2}{2^{2n}} - \frac{36q_1^4}{2^{3n}}\right).$$

Proof. We prove by induction on $i \in [q_1]$, the number of components. First, $H_1 = 2^2n = J_1$. So the statement is true for $i = 1$. By definition, the ratio $\frac{H_{i+1}}{H_i} = (2^n - \delta_{i+1}) \cdot \frac{\mathfrak{h}_{i+1}}{\mathfrak{h}_i}$, and $J_{i+1} = (2^n - i)^2 J_i$. So we have

$$\frac{H_{i+1}}{J_{i+1}} = \frac{(2^n - \delta_{i+1}) \frac{\mathfrak{h}_{i+1}}{\mathfrak{h}_i} H_i}{(2^n - i)^2 J_i}. \quad (7.32)$$

From Lemma 7.6.1 and 7.6.3, we have

$$\mathfrak{h}_{i+1} \geq \mathfrak{h}_i \left((2^n - 2i + \delta_{i+1}) + \frac{|\mathcal{M}|}{2^n - i + 1} \left(1 - \frac{2(i-2)}{2^n - 2(i-2)}\right) \right). \quad (7.33)$$

Recall that $\mathcal{M} = \{(j, k) : j, k \in [i], j \neq k, \lambda_j, \lambda_k \neq \lambda_{i+1}\}$. As there are δ_{i+1} many $i' \in [i]$ such that $\lambda_{i+1} = \lambda_{i'}$, we must have $|\mathcal{M}| \geq (i - \delta_{i+1})(i - \delta_{i+1} - 1)$. On substituting this value for $|\mathcal{M}|$ in Eq. (7.33), and using the resulting lower bound for $\mathfrak{h}_{i+1}$ in Eq. (7.32), we get

$$\begin{aligned} \frac{H_{i+1}}{J_{i+1}} &\stackrel{1}{\geq} \frac{(2^n - \delta_{i+1}) \left((2^n - 2i + \delta_{i+1}) + \frac{(i - \delta_{i+1})(i - \delta_{i+1} - 1)}{2^n - i + 1} \left(1 - \frac{2(i-2)}{2^n - 2(i-2)}\right) \right)}{(2^n - i)^2} \frac{H_i}{J_i} \\ &\stackrel{2}{\geq} \frac{(2^n - i)^2 - i - \frac{\delta_{i+1}(i - \delta_{i+1})(i - \delta_{i+1} - 1)}{2^n} - \frac{2(i-2)(i - \delta_{i+1})(i - \delta_{i+1} - 1)}{2^n - 2(i-2)}}{(2^n - i)^2} \frac{H_i}{J_i} \\ &\stackrel{3}{\geq} \left(1 - \frac{4i}{2^{2n}} - \frac{36i^3}{2^{3n}}\right) \frac{H_i}{J_i} \\ &\stackrel{4}{\geq} \left(1 - \frac{4i}{2^{2n}} - \frac{36i^3}{2^{3n}}\right)^i \frac{H_1}{J_1} \\ &\stackrel{5}{\geq} \left(1 - \frac{4i^2}{2^{2n}} - \frac{36i^4}{2^{3n}}\right). \end{aligned} \quad (7.34)$$

Here 1 to 2 is just a simplification of the expression; from 2 to 3 we use $\delta_{i+1} \leq i$, and $i \leq 2^{n-2}$; and from 3 to 4 we recursively apply the induction hypothesis. The result follows by induction. $\square$

7.6.2 Bound for Sub-System with $\xi = 3$

At this point, we have the bound for the sub-system consisting the first q_1 many components, and we want to extend it to get the bound on $\mathfrak{h}_{q_1+c}$. We first extend to get $\mathfrak{h}_{q_1+c_2}$, and then further extend to get $\mathfrak{h}_{q_1+c}$. Before moving forward, we add some more notations. For $i \in [c_2]$, let $i_- = q_1 + 2i - 1$, $i_+ = i_- + 1$, $i' = q_1 + i$. Now we give a natural modification in the definition of $H_{i'}$ and $J_{i'}$ to accommodate for two equations per component. For all $i \in [c_2]$, denote

$$H_{i'} = \prod_{\lambda' \in \widehat{\lambda}^{i+}} (2^n)_{\mu(\lambda^{i+}, \lambda')} \cdot \mathfrak{h}_{i'},$$

and $J_{i'} = (2^n)_{q_1+i}(2^n)_{q_1+2i}$. Then, we have the following relations for $i \in \{0, \dots, c_2\}$:

$$H_{q_1+i+1} = (2^n - \delta_{(i+1)_-})(2^n - \delta_{(i+1)_+}) \frac{\mathfrak{h}_{q_1+i+1}}{\mathfrak{h}_{q_1+i}} H_{q_1+i} \quad (7.35)$$

$$J_{q_1+i+1} = (2^n - q_1 - i)(2^n - q_1 - 2i)(2^n - q_1 - 2i - 1) J_{q_1+i}. \quad (7.36)$$

Our approach will be similar to the previous case, and we will first establish a lower bound on $\mathfrak{h}_{i'+1}$ in terms of $\mathfrak{h}_{i'}$ in Lemma 7.6.7.

Lemma 7.6.7. For $i \geq 0$,

$$\mathfrak{h}_{i'+1} \geq (2^n - 3q_1 - 5i + \delta_{(i+1)_-} + \delta_{(i+1)_+}) \mathfrak{h}_{i'},$$

with the convention that $\mathfrak{h}_{0'} = \mathfrak{h}_{q_1}$.

Proof. Let $\mathcal{S}_{i'}$ denote the solution space of $\mathcal{L}_{|i'}$, i.e. $\mathfrak{h}_{i'} = |\mathcal{S}_{i'}|$. For a fix $(y^{i+}, v^{i+}) \in \mathcal{S}_{i'}$, we want to compute the number of $(y_{(i+1)_-}, v_{(i+1)_-})$ and $(y_{(i+1)_+}, v_{(i+1)_+})$ pairs such that $(y^{(i+1)+}, v^{(i+1)+}) \in \mathcal{S}_{i'+1}$. Clearly the two tuples should be of the form $(x, x \oplus \lambda_-)$ and $(x, x \oplus \lambda_+)$ for some $x \in \mathbb{B}$. Now, some x is valid if $x \neq y_j$ and $x \oplus \lambda_{i+1} \neq v_k$, for $j, k \in [i_+]$. This means that $x \notin \mathcal{Y} \cup \mathcal{V}$, where $\mathcal{Y} = \{y_j : j \in [i_+]\}$ and $\mathcal{V} = \{v_j \oplus \lambda_{(i+1)_-}, v_j \oplus \lambda_{(i+1)_+} : j \in [i_+]\}$. Now $|\mathcal{Y}| = q_1 + i$, since exactly q_1 components from sub-system $\mathcal{C}_1$, and i many components from sub-system $\mathcal{C}_2$ have been evaluated so far, each contributing 1 y value in $\mathcal{Y}$. And, $|\mathcal{V}| \leq 2q_1 + 4i$, since each component in $\mathcal{C}_1$ and $\mathcal{C}_2$ contributes at most 2 and 4 v values, respectively. Further, exactly $\delta_{(i+1)_-} + \delta_{(i+1)_+}$ many equations share λ value with one of $\lambda_{(i+1)_-}$, or $\lambda_{(i+1)_+}$, whence $|\mathcal{Y} \cap \mathcal{V}| \geq \delta_{(i+1)_-} + \delta_{(i+1)_+}$. Thus, we have

$$\mathfrak{h}_{i'+1} = \sum_{(y^{i+}, v^{i+}) \in \mathcal{S}_{i'}} (2^n - |\mathcal{Y} \cup \mathcal{V}|)$$

$$\begin{aligned}
&= \sum_{(y^{i+}, v^{i+}) \in \mathcal{S}_{i'}} (2^n - |\mathcal{Y}| - |\mathcal{V}| + |\mathcal{Y} \cap \mathcal{V}|) \\
&\geq \sum_{(y^{i+}, v^{i+}) \in \mathcal{S}_{i'}} (2^n - 3q_1 - 5i + \delta_{(i+1)-} + \delta_{(i+1)+}) \\
&= (2^n - 3q_1 - 5i + \delta_{(i+1)-} + \delta_{(i+1)+}) \cdot \mathfrak{h}_{i'}.
\end{aligned}$$

□

Now, we state the lemma that bounds the number of solutions of $\mathcal{C}_1 \cup \mathcal{C}_2$ given the bound for $\mathcal{C}_1$.

Lemma 7.6.8. For $q' = q_1 + q_2 \leq 2^{n-2}$, we have

$$\frac{H_{c'_2}}{J_{c'_2}} \geq \left(1 - \frac{196q'^2 c_2}{2^{2n}} - \frac{40q' c_2}{2^{2n}} - \frac{24q'^3 c_2}{2^{3n}} - \frac{48q'^2}{2^{2n}} - \frac{16q'}{2^{2n}} - \frac{8q'^3}{2^{3n}}\right) \frac{H_{q_1}}{J_{q_1}}.$$

Proof. We use induction on $i \in [c_2]$. In the base case for $i = 1$, it can be easily verified that we have

$$\begin{aligned}
\frac{H_{1'}}{J_{1'}} &\geq \frac{(2^n - \delta_{1-})(2^n - \delta_{1+})(2^n - 3q_1 + \delta_{1-} + \delta_{1+})}{(2^n - q_1)(2^n - q_1)(2^n - q_1 - 1)} \frac{H_{q_1}}{J_{q_1}} \\
&\geq \left(1 - \frac{48q_1^2}{2^{2n}} - \frac{16q_1}{2^{2n}} - \frac{8q_1^3}{2^{3n}}\right) \frac{H_{q_1}}{J_{q_1}},
\end{aligned} \tag{7.37}$$

where we have used the fact that $\delta_{1-}, \delta_{1+} \leq q_1$, and $q_1 \leq 2^{n-1}$. Note that, when $q_1 = 0$, $H'_1 = 2^{3n} > 2^{2n}(2^n - 1) = J_{1'}$, by definition. For $i > 1$, on dividing Eq. (7.35) by Eq. (7.36), we have

$$\frac{H_{i'+1}}{J_{i'+1}} = \frac{(2^n - \delta_{(i+1)-})(2^n - \delta_{(i+1)+}) \frac{\mathfrak{h}_{i'+1}}{\mathfrak{h}_{i'}}}{(2^n - q_1 - i)(2^n - q_1 - 2i)(2^n - q_1 - 2i - 1)} \frac{H_{i'}}{J_{i'}}. \tag{7.38}$$

On substituting the lower bound on $\mathfrak{h}_{i'+1}$ from Lemma 7.6.7 in Eq. (7.38), we get

$$\begin{aligned}
\frac{H_{i'+1}}{J_{i'+1}} &\stackrel{1}{\geq} \frac{(2^n - \delta_{(i+1)-})(2^n - \delta_{(i+1)+})(2^n - 3q_1 - 5i + \delta_{(i+1)-} + \delta_{(i+1)+})}{(2^n - q_1 - i)(2^n - q_1 - 2i)(2^n - q_1 - 2i - 1)} \frac{H_{i'}}{J_{i'}} \\
&\stackrel{2}{\geq} \left(1 - \frac{(24q'^2 + 5q')2^n + 8q'^3}{(2^n - q_1 - i)(2^n - q_1 - 2i)(2^n - q_1 - 2i - 1)}\right) \frac{H_{i'}}{J_{i'}} \\
&\stackrel{3}{\geq} \left(1 - \frac{196q'^2}{2^{2n}} - \frac{40q'}{2^{2n}} - \frac{24q'^3}{2^{3n}}\right) \frac{H_{i'}}{J_{i'}} \\
&\stackrel{4}{\geq} \left(1 - \frac{196q'^2}{2^{2n}} - \frac{40q'}{2^{2n}} - \frac{24q'^3}{2^{3n}}\right)^i \frac{H_{1'}}{J_{1'}} \\
&\stackrel{5}{\geq} \left(1 - \frac{196q'^2 i}{2^{2n}} - \frac{40q' i}{2^{2n}} - \frac{24q'^3 i}{2^{3n}} - \frac{48q'^2}{2^{2n}} - \frac{16q'}{2^{2n}} - \frac{8q'^3}{2^{3n}}\right) \frac{H_{q_1}}{J_{q_1}},
\end{aligned} \tag{7.39}$$

where, $q' = q_1 + q_2$, 1-2 follows from $\delta_{(i+1)-}, \delta_{(i+1)+}, i, q_1 \leq q$; 2-3 follows from $q_1 + 2i < 2^{n-1}$; 3-4 follows from recursive application of induction hypothesis; 4-5 follows from Eq. (7.37) and $q_1 \leq q$. $\square$

At this point, we have $\mathfrak{h}_{q_1+c_2}$, and we want to extend it to $\mathfrak{h}_{q_1+c}$. The extension follows from exactly the same line of argument as above. We skip the formal analysis and just provide the final result in Lemma 7.6.9. For the sake of verification we provide some modified definitions. For $i \in [c_3]$, let $i_- = q_1 + q_2 + 2i - 1$, $i_+ = i_- + 1$, $i' = q_1 + c_1 + i$. For all $i \in [c_3]$, denote

$$H_{i'} = \prod_{\lambda' \in \widehat{\lambda}^{i_+}} (2^n)_{\mu(\lambda^{i_+}, \lambda')} \cdot \mathfrak{h}_{i'},$$

and $J_{i'} = (2^n)_{q_1+2c_2+i} (2^n)_{q_1+c_2+2i}$.

Lemma 7.6.9. *For $q = q_1 + q_2 + q_3 < 2^{n-2}$, and $c = c_2 + c_3$, we have*

$$\frac{H_{q_1+c}}{J_{q_1+c}} \geq \left(1 - \frac{604q^2c}{2^{2n}} - \frac{128qc}{2^{2n}} - \frac{192q^3c}{2^{3n}} - \frac{200q^2}{2^{2n}} - \frac{56q}{2^{2n}} - \frac{64q^3}{2^{3n}} \right) \frac{H_{q_1}}{J_{q_1}}.$$

Theorem 7.4.1 follows from the definitions of H_{q_1+c} and J_{q_1+c} , and Lemmata 7.6.6 and 7.6.9.

Part III

Analysis of Authenticated Encryptions

Chapter 8

Random Read Access in OCB

Now that we have studied the security of various message authentication and encryption schemes, we shift our focus to authenticated encryption schemes. In this chapter, we study one of the most popular AE family, called *Offset Codebook* or OCB. The first version of OCB, called OCB1, was proposed by Rogaway et al. [174] in 2001. Later, Rogaway [171] proposed a second version of OCB, called OCB2. The latest version of OCB, called OCB3, was proposed by Krovetz and Rogaway [119]. OCB3 is one of the winners in the “high performance applications” category of the recently concluded [42] competition. OCB achieves both authentication and encryption with minimal performance overhead compared to classical single-pass encryption only modes such as CTR mode [153]. In fact OCB3 [119] has, to a larger extent, already achieved the theoretical bounds for efficiency in software implementation.

Recall the random read access and out-of-sequence decryption properties (see section 1.3.3 of chapter 1). The efficiency of OCB family in random read access depends on the efficiency of the underlying mask generating function or MGF in direct computation. OCB masks the input and output of each block cipher call with the output of an MGF. So, if the MGF is efficient in direct computation¹ then OCB can achieve random read access almost as efficiently as CTR [153] based designs, most notably CCM [154] and GCM [130]. Next, we revisit some popular choices of MGFs.

8.1 Revisiting Small-Domain AXU Hash Functions

SOME MORE NOTATIONS: For $m \in \mathbb{N}$, I_m denotes the identity matrix of size m . We use $\oplus$ and $\odot$ to denote the field addition (XOR) and field multiplication, respectively,

¹Computation over arbitrary inputs without following any particular order.

over the finite field $\mathbb{F}_{2^n} = \mathbb{B}$. We use α to denote a primitive element of $\mathbb{F}_{2^n}$, and for any $x \in \mathbb{F}_{2^n}$, the operation $x \odot \alpha$ will be referred to as the α -multiplication operation on x . For some $m \in \mathbb{N}$, for $x \in \{0, 1\}^m$, $b \in [m]$, $x \gg b$ denotes x shifted b bits to the right, i.e., $0^b \| x^{[m-b]}$, while $x \ll b$ denotes x shifted b bits to the left. For a binary string x , $\text{ntz}(x)$ denotes the number of trailing zero bits in x .

8.1.1 Examples of Small-Domain AXU Hash Function

Symmetric-key cryptographic literature is rich in AXU hash function candidates. In this work we concentrate only on *small-domain* hash functions that can be used to generate masks in OCB. Further, the range of these hash functions is $\mathbb{B}$. In what follows, we discuss some examples of AXU hash functions, with a summary given in Table 8.1.1.

Table 8.1.1: Feature summary of some small-domain AXU hash function candidates, where $\oplus$, $\odot$, $\ll$, $\|$, $\oplus?$ denote addition, multiplication, shifts, concatenation and conditional addition over appropriate fields; AESRD denotes AES round function, wt and ntz denote the hamming weight and trailing zeros function.

Hash Family	Sequential (operations)	Direct (operations)	Key Size	Domain Size in $\log_2$	AXU Bound
<i>xtimes</i> [174]	$1 \ll, 1 \oplus?$	$O(\log_2 i) \odot$	128	128	2^{-128}
<i>gray</i> [35, 119]	$1 \text{ ntz}(i), 1 \oplus$	$1 \oplus, 1 \ll, 1 \odot$	128	128	2^{-128}
<i>mtrx</i> [45, 81]	$2 \ll, 1 \oplus^\#$	$O(\log_2 i) \odot$	128	128	2^{-128}
<i>m1in</i>	$\text{wt}(i) \oplus$	$\text{wt}(i) \oplus$	$128r$	r	2^{-128}
<i>clh</i> [137]	$\text{wt}(i) \ll, \text{wt}(i) \oplus$	$\text{wt}(i) \ll, \text{wt}(i) \oplus$	128	127	2^{-128}
<i>sts</i> [119]	$1 \ , 1 \oplus, 3 \ll$	$1 \ , 1 \oplus, 3 \ll$	128	6	2^{-128}
<i>lcube</i>	$1 \oplus, 2 \odot$	$1 \oplus, 2 \odot$	128	128	2^{-128}
<i>linv</i> [174]	$1 \oplus, 1 \odot$	$1 \oplus, 1 \odot$	128	128	2^{-128}
<i>aes4</i> [135, 141]	4 AESRD	4 AESRD	512	128	2^{-113}
<i>aes2</i> (Section 8.2)	2 AESRD	2 AESRD	256	32	2^{-113}
<i>aes1</i> (Section 8.2)	1 AESRD	1 AESRD	128	8	2^{-96}

[#] Note that, [81] defines many three-operation maskings for 128 bit binary field.

Example 8.1 (α -multiplication based hash [174]). For some reasonably large $\ell \in [2^n - 1]$, let $\mathcal{X} = [\ell]$ and $\mathcal{K} = \mathbb{B}$. The hash function *xtimes*, defined as $\forall L \in \mathbb{B}, i \in [\ell]$,

$$\text{xtimes}_L(i) = \text{xtimes}(L, i) := \alpha^i \odot L, \quad (8.1)$$

is a 2^{-n} -AXU hash. *xtimes* has been used to generate position dependent input masks in many symmetric-key designs including the OCB family [119, 171, 174], OTR [138], PMAC [35], PMAC+[190], EME [88], COPA [3] and EIMD [40]. The popularity of *xtimes* is mainly due to its efficiency in sequential computation, as we have

$$\forall i \geq 1, L \in \mathbb{B}, \text{xtimes}_L(i+1) = \alpha \odot \text{xtimes}_L(i),$$

and α -multiplication is efficient (one shift and one conditional bit-wise XOR operation). While the sequential computation of $\text{xtimes}_L(i)$ is very efficient, the same is not reflected in direct

computation of $\mathbf{xtimes}_L(i)$ when i is a large integer. In general it may require $O(\log_2 i)$ many field multiplications. Even when the α^i values are precomputed, we need one field multiplication which is not as efficient as one α -multiplication. Indeed we can even avoid the precomputation and simply use $i \odot L$ as the hash definition.

Example 8.2 (Gray-code based hash [35, 119]). For reasonably large $\ell \in [2^n - 1]$, let $\mathcal{X} = [\ell]$ and $\mathcal{K} = \mathbb{B}$. The hash function **gray** is defined as,

$$\forall L \in \mathbb{B}, i \in [\ell], \mathbf{gray}_L(i) = \mathbf{gray}(L, i) := \gamma(i) \odot L \quad (8.2)$$

where $\gamma(i) := i \oplus (i \gg 1)$ is the gray code value for i . The Gray code sequence has a very nice feature that for $i \geq 1$, the i -th gray code can be written in terms of the $(i - 1)$ -th gray code, i.e

$$\gamma(i) = \gamma(i - 1) \oplus 2^{\text{ntz}(i)},$$

When we substitute this in (8.2) we get an alternate definition $\forall L \in \mathbb{B}, i \in [\ell]$,

$$\mathbf{gray}_L(i) = \mathbf{gray}(L, i) := \bigoplus_{j=1}^i L_{\text{ntz}(j)}. \quad (8.3)$$

where $L_j = \alpha^j \odot L$ for all $j < n$. In [119] and RFC7253 [120], **gray** has been shown to be a 2^{-n} AXU hash function. Note that, we have

$$\forall i \geq 1, L \in \mathbb{B}, \mathbf{gray}_L(i + 1) = \mathbf{gray}_L(i) \oplus L_{\text{ntz}(i+1)}.$$

So if the L_j values are precomputed, then in sequential computation **gray** hash only requires, ntz computation along with a single n -bit XOR operation. Hence it is very efficient in sequential computations. This hash function is applied in **OCB3** [119] (RFC7253 [120]) and **PMAC** [35].

Example 8.3 (Matrix-powered hash [45, 81]). Let $n = wn'$ for some positive integer w , called word size, whose values are generally software friendly such as 8, 16, 32, 64 etc. Let $\mathcal{W} = \{0, 1\}^w$ and $\mathcal{K} = \mathcal{W}^{n'}$ and $\mathcal{X} = [\ell]$ for some reasonably large $\ell \in [2^n - 1]$. Let M be an invertible $n' \times n'$ matrix whose entries are from the field² $\mathcal{W}$ such that for all $i \in [2^n - 1]$, $I_{n'} + M^i$ is invertible. Then the hash function **mtrx** (called matrix-powered hash), defined as

$$\forall L \in \mathcal{W}^{n'}, i \in [\ell], \mathbf{mtrx}_L(i) = \mathbf{mtrx}(L, i) := M^i \cdot L, \quad (8.4)$$

is a 2^{-n} -AXU. Note here that L is viewed as a vector over $\mathcal{W}$. Various matrix-powered hash candidates are given in [81] and [45]. But, similar to **xtimes** hash, all those hash functions are efficient in sequential computation and inefficient in direct computation.

² $\{0, 1\}^m$ can be viewed as the binary field by fixing a deg m primitive polynomial.

The matrix-powered hash function is a very general candidate. For example, `xtimes` and `gray` hash functions can be viewed as instances of matrix-powered hash for appropriate choices of the matrix M . The readers are directed to [81] and [45] for a detailed exposition on matrix-powered hash functions. A table of various word-oriented matrices has been listed in [81].

Example 8.4. *Some other finite field based constructions, with $\mathcal{K} = \mathcal{X} = \mathbb{B}$, are the following:*
 $\forall L \in \mathbb{B}, x \in \mathbb{B}$

- $lcube_L(x) = (L \oplus x)^3$;
- $linv_L(x) = (L \oplus x)^{-1}$ (if $x \neq L$), 0 otherwise.

It is easy to verify that `lcube` is 2^{1-n} -AXU and `linv` is 2^{2-n} -AXU hash function.

Example 8.5 (Multi-linear hash). *For $\ell \in [2^n - 1]$, let $\mathcal{X} = (\ell]$ and $\mathcal{K} = \mathbb{B}^{\log_2 \ell}$. The hash function `mlin` (called multi-linear hash), is defined as*

$$\forall L \in \mathcal{K}, b \in \mathcal{X}, \text{mlin}_L(b) = \text{mlin}(L, b) := \bigoplus_{i=1}^r b_i L_i, \quad (8.5)$$

is a 2^{-n} -AXU hash function, where $L_1, \dots, L_r \in \mathbb{B}$ and $b_1, \dots, b_r \in \{0, 1\}$. In `Grain-128a` [2], the tag is computed using `mlin`, where the keys are derived using Toeplitz matrix.

Example 8.6 (Circulant hash [137]). *In [137] Minematsu presented a small-domain hash function based on data-dependent rotations. For $\ell \in [2^{n-1} - 1]$, let $\mathcal{X} = (\ell]$ and $\mathcal{K} = \mathbb{B}$. The hash function `clh` (called circulant hash) is defined as $\forall K \in \mathcal{K}, x \in \mathcal{X}$,*

$$\text{clh}_L(x) = \text{clh}(L, x) := \bigoplus_{i \in [\log_2 \ell]: x_i=1} (L \ll (i-1)). \quad (8.6)$$

For direct computation, the hash function might require $n - 1$ rotations and $n - 2$ XORs in the worst case; for sequential computation some optimization is possible for the i -th input given the $(i - 1)$ -th output. In [137], `clh` is shown to be both 2^{-128} -AXU and 2^{-128} -uniform hash function.

Example 8.7 (Stretch-then-shift hash [119]). *In [119] Krovetz and Rogaway presented a highly efficient AXU hash function over a small domain $\mathcal{X} = [64]$. It follows the general data-dependent rotation technique by Minematsu [137]. Fix $c \in \mathbb{N}$. For $\mathcal{X} = (\psi(c) - 1]$ and $\mathcal{K} = \mathbb{B}$, where $\psi(c)$ is a positive integer denoting the maximum domain size for a fixed c . The hash function `sts` (called stretch-then-shift hash) is defined as $\forall L \in \mathbb{B}, i \in [0.. \psi(c) - 1]$,*

$$\text{sts}_L(i) = \text{sts}(L, i) := \text{msb}_n(\text{stretch}_c(L) \ll i), \quad (8.7)$$

where $\forall L \in \mathbb{B}$, $\text{stretch}_c(L) := L \parallel (L \oplus (L \ll c))$. The hash function first stretches the key based on a parameter c , and then makes data-dependent shift before outputting the most significant n bits; hence the name. For $n = 128$, $c = 8$ and $\psi(c) = 64$, Krovetz and Rogaway showed that **sts** achieves 2^{-128} -AXU and 2^{-128} -uniform bound [119].

Table 8.1.1 clearly shows that most of the above given small-domain hash functions are indeed quite efficient when computed sequentially over the indices; but all of them are inefficient for direct computation at any arbitrary index. A possible way out is to use iterations of some cryptographic round function with good differential probability. In the next section we take this approach.

8.2 AES-based Small-Domain AXU Hash Functions

In the previous section we listed some hash functions based on finite field operations and data dependent rotations. Now we will propose some hash functions based on the differential properties of the AES rounds. Before moving forward, we give the standard and well-known [141, 157] definition for differential probability of a (keyed) permutation.

Definition 8.2.1. Let π be a permutation over $\mathbb{F}_{2^n}$ and Π_K be a keyed permutation over $\mathbb{F}_{2^n}$ with key $K \leftarrow \mathcal{K}$. For any given non-zero $a, b \in \mathbb{F}_{2^n}$ the differential probability of π , is defined as

$$\text{dp}_\pi(a, b) := \Pr_X [\pi(X) \oplus \pi(X \oplus a) = b].$$

The maximum differential probability (MDP) of π is defined as

$$\text{MDP}(\pi) := \max_{a \neq 0, b} \text{dp}_\pi(a, b).$$

Similarly, the expected differential probability (EDP) of Π_K , is defined as

$$\begin{aligned} \text{EDP}_{\Pi_K}(a, b) &:= \Pr_{K, X} [\Pi_K(X) \oplus \Pi_K(X \oplus a) = b] \\ &= \sum_{k \in \mathcal{K}} \Pr_X [\Pi_k(X) \oplus \Pi_k(X \oplus a) = b \mid K = k] \times \Pr[K = k] \\ &= \sum_{K \in \mathcal{K}} \text{dp}_{\Pi_K}(a, b) \times \Pr[K = k] = \text{Ex}_K [\text{dp}_{\Pi_K}(a, b)], \end{aligned}$$

The maximum expected differential probability (MEDP) of Π_K is defined as

$$\text{MEDP}(\Pi_K) := \max_{a \neq 0, b} \text{EDP}_{\Pi_K}(a, b).$$

We have also used the term differential probability in the context of AXU hash functions in Definition 2.3.1. Now that we have formally defined the differential probability of a (keyed) permutation, it is imperative that we discuss the similarity and differences between the two probabilities. The major difference is the source of randomness. In the AXU view of differential probability, the randomness is due to the hash key, whereas here one of the input is chosen randomly. Even though this seems to be a major difference, but for certain class of functions the two views have a very nice relationship. Suppose π is a (possibly keyed) permutation over $\mathbb{F}_{2^n}$ with keyspace $\mathcal{K}$ (empty set when π is keyless). We define a function $\pi' : \mathbb{F}_{2^n} \times \mathcal{K} \times \mathbb{F}_{2^n} \rightarrow \mathbb{F}_{2^n}$ as a keyed family of function indexed by $K, K' \in \mathbb{F}_{2^n} \times \mathcal{K}$, such that

$$\forall K, K', x \in \mathbb{F}_{2^n} \times \mathcal{K} \times \mathbb{F}_{2^n}, \pi'_{K,K'}(x) := \pi_{K'}(K \oplus x).$$

We will drop K' from the subscript, whenever π is unkeyed. Lemma 8.2.2 is a well-known result that we prove here just for the sake of completeness.

Lemma 8.2.2. *For some $\epsilon \geq 0$,*

1. *If π is unkeyed and $\text{MDP}(\pi) = \epsilon$ then π' is an ϵ -AXU hash for $K \leftarrow \$ \mathbb{F}_{2^n}$.*
2. *If π is keyed and $\text{MEDP}(\pi) = \epsilon$ then π' is an ϵ -AXU hash for $(K, K') \leftarrow \$ \mathbb{F}_{2^n} \times \mathcal{K}$.*

Proof. We prove the unkeyed version of the lemma. The keyed version can be proved in similar fashion. Let $x, x' \in \mathbb{F}_{2^n}$ such that $x \neq x'$ and $\delta \in \mathbb{F}_{2^n}$. If $\delta = 0$ then the result is vacuously true as π' is a permutation. Suppose $\delta \neq 0$. Then, we have

$$\begin{aligned} \Pr_K [\pi'_K(x) \oplus \pi'_K(x') = \delta] &= \Pr_K [\pi(K \oplus x) \oplus \pi(K \oplus x') = \delta] \\ &= \Pr_X [\pi(X) \oplus \pi(X \oplus x' \oplus x) = \delta] \\ &\leq \text{dp}(\pi) = \epsilon. \end{aligned}$$

□

8.2.1 Revisiting MEDP bounds for the AES

AES is a Substitution Permutation Network (SPN) with block size $n = 128$, internal s-box input size $b = 8$, where all s-boxes are identical, denoted by S . The permutation layer, denoted by ϕ , consists of a bitwise permutation followed by four identical 32-bit linear transformations applied in parallel. One round of AES, denoted by 1-AESRD (without the subkey mixing) is nothing but $\phi \circ S$. Using the 1-AESRD round function, we

can define the keyed function i -AESRD for all integers $i > 1$ as follows: $\forall x \in \mathbb{F}_{2^{128}}, K \in \mathbb{F}_{2^{128}}^{i-1}$,

For $i = 2$:

$$2\text{-AESRD}_K(x) = 1\text{-AESRD}(K_1 \oplus 1\text{-AESRD}(x))$$

For $i > 2$:

$$i\text{-AESRD}_K(x) = 1\text{-AESRD}(K_{i-1} \oplus (i-1)\text{-AESRD}_{K_{[i-2]}}(x))$$

The differential properties of AES is a well-studied [52, 53, 114, 115, 157] topic in symmetric-key cryptography. In fact, one of the design criteria for AES [52, 152] was protection against linear [127] and differential cryptanalysis [32].

In [53], Daemen and Rijmen first observed that due to the specific nature of the AES permutation layer ϕ , the differential probabilities over 2 AES rounds are equivalent to those over a reduced SPN structure, which they called a *super box*. Basically the two rounds of AES over 128-bit input can be viewed as four parallel invocations of the super box with independent keys (disjoint substrings of a uniform string are independent) each working with distinct 32 bits of the input. We denote an AES super box instantiated with a subkey K by aesSB_K .

Keliher and Sui [114, 115] later used this idea to derive a tight MEDP bound for 2-AESRD. Specifically, in [114, Theorem 1, Theorem 2] and [115, section 4.1] they proved that the MEDP for AES super box is 1.656×2^{-29} , which is equivalent to the MEDP for 2-AESRD. We refer the readers to [53, 115] for further exposition on AES super box and its relation with 2-AESRD. Using the fact that the upper bound on the MEDP for 4 or more rounds of AES is equal to the 4-th power of the upper bound on the MEDP for 2-AESRD, Keliher and Sui [115] further showed that MEDP for t -AESRD for $t \geq 4$ is upper bounded by 1.881×2^{-114} . We summarize these results in Proposition 8.2.3.

Proposition 8.2.3. [115, section 4.1] Let $X^3 \leftarrow \$ (\mathbb{F}_{2^{128}})^3$ and $Y \leftarrow \$ \mathbb{F}_{2^{32}}$. Then,

1. $\text{MEDP}(\text{aesSB}_Y) \approx 1.656 \times 2^{-29}$.
2. $\text{MEDP}(2\text{-AESRD}_{X_1}) \approx 1.656 \times 2^{-29}$.
3. $\text{MEDP}(4\text{-AESRD}_{X^3}) \leq 1.881 \times 2^{-114}$.

8.2.2 Our AES-based proposals

In the following discussion we fix $n = 128$, i.e. $\mathbb{B} = \{0, 1\}^{128}$. Before presenting our proposals we start with a simple 4-AESRD based hash function in the following example.

Example 8.8. Let $\mathcal{X} = \mathbb{B}$ and $\mathcal{K} = (\mathbb{B})^4$. For $L^4 \in (\mathbb{B})^4$ and $x \in \mathbb{B}$, the 4-AESRD based hash function **aes4** is defined as

$$\mathbf{aes4}_{L^4}(x) = \mathbf{aes4}(L^4, x) := \mathbf{4-AESRD}_{L[2\dots 4]}(L_1 \oplus x).$$

From Lemma 8.2.2 and Proposition 8.2.3, it is straightforward to see that **aes4** is a 1.88×2^{-114} -AXU hash function. Minematsu used such a hash function in [135]. In [141] Minematsu and Tsunoo used a variant of **aes4** to construct a universal hash function and subsequently extended it to construct a MAC. Jakimoski and Subbalakshmi [100] further built upon their work to get more efficient hash functions and MAC. Bellare et al. [20] proposed a cryptographic hash based on 4-AESRD.

8.2.2.1 1-Round AES based Hash Function

The MDP for AES s-box S is known to be 2^{-6} [157]. We define a keyed function $S' : \mathbb{F}_{2^8} \times \mathbb{F}_{2^8} \rightarrow \mathbb{F}_{2^8}$ as

$$\forall K, x \in \mathbb{F}_{2^8}, S'_K(x) := S(K \oplus x).$$

Clearly S and S' satisfy the conditions in Lemma 8.2.2, hence S' is 2^{-6} -AXU hash function. Now, let $\mathcal{X} = \{0, 1\}^8$ (sometimes viewed as [256]) and $\mathcal{K} = \mathbb{B}$ (viewed as $\mathcal{X}^{16}$). Given a key L , and an input $i \in \mathcal{X}$, we define the one round AES hash, **aes1** as

$$\begin{aligned} \mathbf{aes1}_L(i) &= \mathbf{aes1}(L, i) := \mathbf{1-AESRD}(L \oplus i^{16}) \\ &= \phi(S'_{L_1}(i) \parallel \cdots \parallel S'_{L_{16}}(i)), \end{aligned} \tag{8.8}$$

where i^{16} denotes i duplicated 16 times. Recall that ϕ is the linear transformation or permutation layer used in AES. It is well-known that the AXU property is preserved under linear composition over an AXU hash function. Thus ϕ being a linear transformation does not affect the AXU bound for **aes1** and can be ignored. When L is uniformly drawn from $\mathcal{K}$ and viewed as an element of $\mathcal{X}^{16}$ (i.e. as a 16-tuple with elements from $\mathcal{X}$), then L_i 's are mutually independent. So by repeatedly using Proposition 2.3.5 and Lemma 8.2.2, we get the AXU bound for **aes1** in Lemma 8.2.4.

Lemma 8.2.4. ***aes1** as defined above is 2^{-96} -AXU hash function.*

8.2.2.2 2-Round AES based Hash Function

From Proposition 8.2.3 we already now that the MDP for aesSB is upper bounded by 1.656×2^{-29} . Again we define a keyed function $\text{aesSB}' : \mathbb{F}_{2^{32}} \times \mathbb{F}_{2^{32}} \times \mathbb{F}_{2^{32}} \rightarrow \mathbb{F}_{2^{32}}$ as

$$\forall K^2, x \in (\mathbb{F}_{2^{32}})^3, \text{aesSB}'_{K^2}(x) := \text{aesSB}_{K_2}(K_1 \oplus x).$$

Clearly aesSB' is 1.656×2^{-29} -AXU hash function (using Lemma 8.2.2). Now let $\mathcal{X} = \{0, 1\}^{32}$ (viewed as $[2^{32}]$) and $\mathcal{K} = \mathbb{B} \times \mathbb{B}$ (viewed as $\mathcal{X}^4 \times \mathcal{X}^4$). Given a key $L = (L, L') \in \mathcal{X}^4 \times \mathcal{X}^4$, and an input $i \in \mathcal{X}$, we define the two round AES hash, aes2 as

$$\begin{aligned} \text{aes2}_{L,L'}(i) &= \text{aes2}(L, L', i) := 2\text{-AESRD}_{L'}(L \oplus i^4) \\ &= \phi(\text{aesSB}'_{L'_1}(L_1 \oplus i) \parallel \dots \parallel \text{aesSB}'_{L'_4}(L_4 \oplus i)), \end{aligned}$$

where i^4 denotes i duplicated 4 times and each L_i/L'_i is a 32-bit strings. Using a similar line of argument as used in case of aes1 we get the AXU bound for aes2 in Lemma 8.2.5.

Lemma 8.2.5. *aes2 as defined above is 1.881×2^{-114} -AXU hash function.*

So we see that aes1 and aes2 are sub-optimum, but decent AXU hashes given the domains they are applied upon. In addition, they would allow efficient random read access. In fact it is quite surprising that this fact was not discovered yet. However, going by the existing security proofs, the straightforward application of these hash functions in OCB is not advisable as they give sub-optimum security bounds. This is due to AXU assumption on the underlying MGF.

8.3 Generic view of OCB

In previous two sections we have seen various examples of AXU hashes over small domain $[\ell]$, where ℓ is typically a large positive integer in $[2^n]$. These hash functions are useful when the goal is to embed position-based dependency (called masks) to the input blocks of the underlying primitive, typically a block cipher. This type of embedding is used in many encryption, MAC and AE algorithms. Some prominent schemes among these belong to the OCB like design paradigms. We call the embedding function — *mask-generating function* (MGF).

MASK GENERATING FUNCTION: Typically, an MGF, $\lambda : \mathcal{L} \times \mathcal{N} \times \mathbb{N} \rightarrow \mathbb{B}$ is a $(\mathcal{L}, \mathcal{N} \times \mathbb{N})$ -hash function family indexed by the key space $\mathcal{L}$. Here $\mathcal{N}$ is typically called the nonce

space. For notational simplicity we will sometime write $\lambda(i) = \lambda_L(N, i)$, i.e. $N \in \mathcal{N}$ and $L \in \mathcal{L}$ will be clear from the context. Next we define generic abstraction for OCB, called GOCB, based on $\Pi \leftarrow \$ \text{Perm}(\mathbb{B})$ and λ . Figure 8.3.1 illustrates the schematic view of the encryption-decryption algorithms for GOCB and the complete algorithmic descriptions is given in Algorithm 8.3.1. For $i \in \mathbb{N}$, we will refer M_i , A_i and C_i as message block, associated data block and ciphertext block. For $i \in \mathbb{N}$, we will refer U_i and X_i as input blocks; V_i and Y_i as output blocks; and S_i as final block. In GOCB $X_\oplus$ and $Y_\oplus$ will be referred as the tag input and output block respectively. We will use the term GPHash³ for the associated data processing phase of GOCB.

Any binary string X can be mapped uniquely to a block sequence $\hat{X} = (\hat{X}_1, \dots, \hat{X}_m, \hat{X}_*) \xleftarrow{n} X$, where $\ell = \lfloor |X|/n \rfloor$, i.e., $\hat{X}_i$ are complete blocks ($|\hat{X}_i| = n$) for all $i \in [\ell]$, and $\hat{X}_*$ is an incomplete (possibly empty). In the rest of this chapter, we use this methodology to parse any $X \in \{0, 1\}^*$ into block sequences.

Note that, the constructions employ different MGFs for special blocks (last block, checksum block etc.) processing. Usually for concrete constructions these MGFs are simple variants of the usual MGF λ . Similarly the key for the MGF is usually derived from the underlying random permutation. For the sake of simplicity we will assume that the MGF key is drawn independently, and the MGFs for special blocks are also defined independently.

In this chapter, we only focus on the mask generation component and its properties. This is mainly because GOCB greatly resemble simple variant of ECB mode when the masking is ignored. Hence both efficiency and security mostly rely on the mask-generation phase, albeit the two issues are mutually orthogonal. We discuss the two issues separately starting with efficiency.

8.3.0.1 Mask-generation and Efficiency

As noted before, if we ignore the input masking, GOCB is simply a variant of ECB, which is arguably the most efficient cryptographic mode of operation, both in hardware and software. So the MGF should preferably have the following properties:

- *Sequential Computation* — MGF should be fast enough to compute the mask outputs for consecutive inputs. More formally, $\lambda(i), \lambda(i+1), \dots, \lambda(i+r-1)$ should be efficiently computable for some r and for any i . Here r can be viewed as a parameter. One may observe the performance for different choices of r and then choose the best possible one.

³PHash is a commonly used terminology for the hash component in OCB.

Algorithm 8.3.1 GOCB encryption/decryption process applied over associated data A and plaintext/ciphertext M/C having a and m complete blocks and an incomplete (possibly empty) block, respectively; λ_1 is a keyed (i.e. the key is implicit) mask-generating function, and λ_i for $1 < i \leq 6$ are minor variants of λ_1 , deployed in various boundary conditions, like incomplete block processing, tag generation etc.

<pre> 1: function GOCB_{Π,λ}⁺(N, A, M) 2: $S \leftarrow 0^n$ 3: $M_{\oplus} \leftarrow 0^n$ 4: $(A_1, \dots, A_a, A_*) \xleftarrow{n} A$ 5: $(M_1, \dots, M_m, M_*) \xleftarrow{n} M$ 6: for $i \leftarrow 1$ to a do 7: $U_i \leftarrow A_i \oplus \lambda_5(N, i)$ 8: $V_i \leftarrow \Pi(U_i)$ 9: $S \leftarrow S \oplus V_i$ 10: end for 11: if $A_* > 0$ then 12: $U_* \leftarrow \text{pad}(A_*) \oplus \lambda_6(N, a)$ 13: $V_* \leftarrow \Pi(U_*)$ 14: $S \leftarrow S \oplus V_*$ 15: end if 16: for $i \leftarrow 1$ to m do 17: $X_i \leftarrow M_i \oplus \lambda_1(N, i)$ 18: $Y_i \leftarrow \Pi(X_i)$ 19: $C_i \leftarrow Y_i \oplus \lambda_1(N, i)$ 20: $M_{\oplus} \leftarrow M_{\oplus} \oplus M_i$ 21: end for 22: if $M_* = 0$ then 23: $X_{\oplus} \leftarrow M_{\oplus} \oplus \lambda_3(N, m)$ 24: else 25: $X_* \leftarrow \lambda_2(N, *)$ 26: $Y_* \leftarrow \Pi(X_*)$ 27: $\overline{C}_* \leftarrow Y_* \oplus \text{pad}(M_*)$ 28: $C_* \leftarrow \text{msb}_{ M_* }(\overline{C}_*)$ 29: $M_{\oplus} \leftarrow M_{\oplus} \oplus \text{pad}(M_*)$ 30: $X_{\oplus} \leftarrow M_{\oplus} \oplus \lambda_4(N, m)$ 31: end if 32: $C \leftarrow (C_1, \dots, C_m, C_*)$ 33: $Y_{\oplus} \leftarrow \Pi(X_{\oplus})$ 34: $T \leftarrow Y_{\oplus} \oplus S$ 35: return (C, T) 36: end function </pre>	<pre> 1: function GOCB_{Π,λ}⁻(N, A, C, T) 2: $S \leftarrow 0^n$ 3: $M_{\oplus} \leftarrow 0^n$ 4: $(A_1, \dots, A_a, A_*) \xleftarrow{n} A$ 5: $(C_1, \dots, C_m, C_*) \xleftarrow{n} C$ 6: for $i \leftarrow 1$ to a do 7: $U_i \leftarrow A_i \oplus \lambda_5(N, i)$ 8: $V_i \leftarrow \Pi(U_i)$ 9: $S \leftarrow S \oplus V_i$ 10: end for 11: if $A_* > 0$ then 12: $U_* \leftarrow \text{pad}(A_*) \oplus \lambda_6(N, a)$ 13: $V_* \leftarrow \Pi(U_*)$ 14: $S \leftarrow S \oplus V_*$ 15: end if 16: for $i \leftarrow 1$ to m do 17: $Y_i \leftarrow C_i \oplus \lambda_1(N, i)$ 18: $X_i \leftarrow \Pi^{-1}(Y_i)$ 19: $M_i \leftarrow X_i \oplus \lambda_1(N, i)$ 20: $M_{\oplus} \leftarrow M_{\oplus} \oplus M_i$ 21: end for 22: if $C_* = 0$ then 23: $X_{\oplus} \leftarrow M_{\oplus} \oplus \lambda_3(N, m)$ 24: else 25: $X_* \leftarrow \lambda_2(N, m)$ 26: $Y_* \leftarrow \Pi(X_*)$ 27: $\overline{M}_* \leftarrow Y_* \oplus \text{pad}(C_*)$ 28: $M_* \leftarrow \text{msb}_{ C_* }(\overline{M}_*)$ 29: $M_{\oplus} \leftarrow M_{\oplus} \oplus \text{pad}(M_*)$ 30: $X_{\oplus} \leftarrow M_{\oplus} \oplus \lambda_4(N, m)$ 31: end if 32: $M \leftarrow (M_1, \dots, M_m, M_*)$ 33: $Y_{\oplus} \leftarrow \Pi(X_{\oplus})$ 34: $T' \leftarrow Y_{\oplus} \oplus S$ 35: if $T' = T$ then 36: return M 37: else 38: return $\perp$ 39: end if 40: end function </pre>
---	--

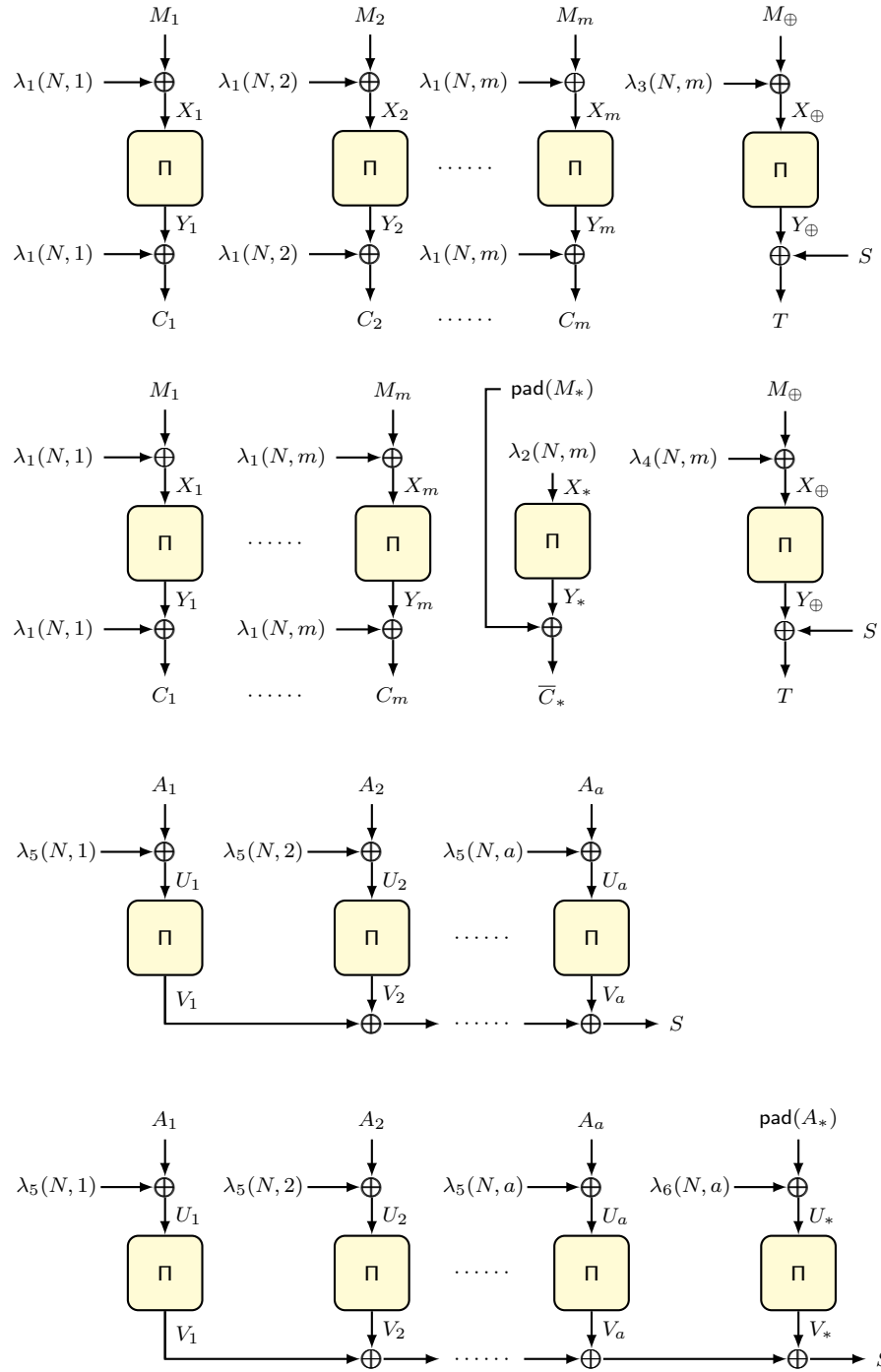


Figure 8.3.1: Schematic of GOCB— **Top to Bottom:** encrypting M when $n \mid |M|$; encrypting M when $n \nmid |M|$; hashing A when $n \mid |A|$; hashing A when $n \nmid |A|$.

- *Direct Computation* — MGF should be easily computable on any arbitrary input. In other words, for all i , $\lambda(i)$ should be efficiently computable without the knowledge of any other mask outputs. This is the usual constraint for hash functions. Direct computation of masks is an important feature as it leads to features like random read access and massive parallelism. Given large number of parallel processing units (multiple CPU cores or GPUs) GOCB is completely parallel when the underlying MGF is efficient in direct computation. This holds even when the message length is not known beforehand, a scenario that is common in data streams.
- *Constant-time Computation* — A secondary requirement for any MGF is constant computation time for all inputs. While this is a secondary requirement as far as efficiency is concerned, it helps in mitigating a form of side channel attacks called timing attack [26, 82].

8.3.0.2 Mask-Generation and Security

From the security point of view, earlier works [30, 119, 174] required strong (almost 2^{-n}) AXU and 1-universal (or regular, i.e $\Pr[H(x) = y] = O(2^{-n})$) property (some time implicitly) from the underlying MGFs. Since almost all of the hash functions discussed in this work are 1-regular, we will implicitly assume that MGF has this property. We will focus on the more dominant, AXU property. In this setting the advantage is generally bounded in terms of ϵ , the AXU bound of MGF. For instance the bound for OCB is roughly $O(\sigma^2\epsilon)$ respectively; hence the need for $O(2^{-n})$ -AXU bound. Naturally, both `aes1` and `aes2` would suffer significant security loss in the existing setting. This is probably one of the reasons, these hash functions have not got much attention. Yet another reason is the fact that `aes1` and `aes2` are defined over restricted domains $\{0, 1\}^8$ and $\{0, 1\}^{32}$ respectively.

8.3.1 Locally-Imperfect XOR Universal Hash Functions

The immediate question in light of the above discussion is as follows:

Can we relax the AXU condition on MGFs to use efficient directly computable functions like `aes1` and `aes2`, while maintaining comparable security?

We answer this in affirmative by proposing a slightly different universal notion — *Locally-Imperfect XOR Universal (or LIXU) Hash Functions*. The idea is to model a hash

function H whose differential probability depends on the grouping of the given inputs. More precisely, we partition the inputs in several disjoint subsets with bounded cardinality, say at most r .⁴ Now the additional property introduced by LIXU is as follows: for two inputs say x and y from distinct partitions the differential probability is at most 2^{-n} , and the probability degrades to $\epsilon \geq 2^{-n}$, when x and y belong to the same partition. Formally we define the notion of Locally-Imperfect XOR Universal in Definition 8.3.1.

Definition 8.3.1 (Locally-Imperfect XOR Universal Hash Function). Let $H : \mathcal{L} \times \mathcal{X} \rightarrow \mathbb{B}$ be a $(\mathcal{L}, \mathcal{X})$ -hash function family indexed by the keyspace $\mathcal{L}$. For a fixed $r \in \mathbb{N}$ and $\epsilon \geq 2^{-n}$, we say that H is an (ϵ, r) -locally-imperfect XOR universal, or LIXU, hash function if and only if there exists a partitioning $\mathcal{X} = \mathcal{P}_1 \sqcup \mathcal{P}_2 \sqcup \dots \sqcup \mathcal{P}_k$ with $k \geq 2$, such that $|\mathcal{P}_i| \leq r$ for $i \in [k]$, and for distinct $x, y \in \mathcal{X}$, and $\delta \in \mathbb{B}$,

$$\Pr_{L \leftarrow \mathcal{L}} [H_L(x) + H_L(y) = \delta] \leq \begin{cases} \epsilon & \text{if } x, y \in \mathcal{P}_i, \\ \frac{1}{2^n} & \text{if } x \in \mathcal{P}_i, y \in \mathcal{P}_j, i \neq j. \end{cases}$$

Here r is called the *width* of H . We say that $x, y \in \mathcal{X}$ are *local pairs* (to each other and as a pair as well) if and only if they belong to the same partition. So a LIXU hash function behaves as a perfect XOR universal hash function when the inputs are not local pairs, and behaves as an imperfect (or almost) XOR universal hash function when the inputs are local pairs. Note that, an (ϵ, r) -LIXU hash implies an ϵ -AXU hash, but the vice-versa may not be true. Further it is rather trivial to establish that any 2^{-n} -AXU hash function over the domain $\mathcal{X}$ is also a $(2^{-n}, r)$ -LIXU hash function for all $r \leq |\mathcal{X}| - 1$. We note this simple fact in Proposition 8.3.2.

Proposition 8.3.2. A 2^{-n} -AXU hash H is a $(2^{-n}, r)$ -LIXU hash for all $r \leq |\mathcal{X}| - 1$, where $\mathcal{X}$ is the domain of H . Notably, H is a $(2^{-n}, 1)$ -LIXU hash function.

In this chapter, we use a specific type of LIXU hash functions defined over the domain $\mathcal{N} \times \mathbb{N}$, where the domain is partitioned as follows: $(N, i) \neq (N', i') \in \mathcal{N} \times \mathbb{N}$ are said to be local pairs if and only if $N = N'$ and $\lceil i/r \rceil = \lceil i'/r \rceil$. Here, $\mathcal{N}$ denotes the nonce space. We sometime use chunk and partition interchangeably, and for any $i \in \mathbb{N}$, we call $\lceil i/r \rceil$ the chunk number of i .

8.3.2 LIXU Hash as MGF in GOCB

Previous works on OCB have assumed that the underlying MGF is an AXU hash. We depart from this setting and employ LIXU hash as MGF. Since LIXU and AXU are not

⁴In fact we try to minimize the value of r .

exactly compatible, we will require a slightly different analysis. Particularly the TBC based abstraction [171] for OCB is not useful here. We now present the main security result of the chapter, along with an overview of our proof approach. The formal proof is postponed to Section 8.4.

Theorem 8.3.3 (GOCB NAEAD Bound). *For a fixed $r \leq \ell \in \mathbb{N}$ and $\epsilon \geq 0$, let λ be an (ϵ, r) -LIXU hash function over $\mathcal{N} \times \mathbb{N}$. Let $\mathbb{A}(q, \tilde{q}, \ell, \sigma, \tilde{\sigma})$ be the class of all adversaries that make q encryption queries consisting of μ message blocks in all with at most ℓ blocks per query, and ν associated data blocks in all, and $\tilde{q}$ decryption queries in a NAEAD security game against $\text{GOCB}_{\Pi, \lambda}$. Then $\forall \mathcal{A} \in \mathbb{A}(q, \tilde{q}, \ell, \sigma)$ we have, $\text{Adv}_{\text{GOCB}_{\Pi, \lambda}}^{\text{naead}}(\mathcal{A}) = \epsilon_{\text{naead}}$, where*

$$\epsilon_{\text{naead}} \leq \frac{0.5\sigma^2 + \tilde{\sigma}^2}{2^n} + \frac{q\sigma + \tilde{q} + 0.5q^2}{2^n - \sigma} + \frac{\tilde{\sigma}(\sigma + 3)}{2^n} + (\sigma + 2\tilde{\sigma})r\epsilon + \tilde{q}\epsilon.$$

Here $\sigma = \mu + \nu$ is the total number of blocks in messages and associated data queried in the encryption queries.

PROOF OVERVIEW — The proof of Theorem 8.3.3 is given in Section 8.4. While we do get a factor of $\sigma r \epsilon$ in the bound for GOCB, the original bound of OCB remains intact asymptotically if $r\epsilon = o(2^{-\frac{n}{2}})$. To understand why we don't get any degradation in security we have to look into the bound more precisely. Generally the OCB bound is dominated by the probability of collision among the input/output of the underlying block cipher. As per the earlier analyses the collisions can be bounded in terms of the AXU bound of λ , which gives $\sigma^2\epsilon/2$ bound. This evaluates to $\sigma^2 2^{1-n}$ when $\epsilon = 2^{-n}$. However when we replace AXU with LIXU the straightforward analysis will result in security degradation. Instead we first bound the probability of collision between input blocks which are non-local pairs. Since LIXU hashes are perfectly XOR universal on non-local pairs we get a bound of $\sigma^2 2^{1-n}$. Now for each input block there are at most $r - 1$ local pairs and there are at most σ input blocks which gives a bound of roughly $r\sigma\epsilon$.

8.3.3 Instantiating LIXU hash in GOCB

8.3.3.1 Deriving OCB3 from GOCB

We define the mask-generating function used in OCB3 in LIXU setting and derive concrete bound for $n = 128$. OCB3 employs gray code based masking defined using the gray hash function discussed in Section 8.1. As noted earlier gray is a 2^{-128} -AXU hash function.

For OCB3, Krovetz and Rogaway defined the **gray** function as: For any $i \in \mathbb{N}$, $\text{gray}_L(i) = 4\gamma(i) \odot L$. For a fixed $j \in [n]$ (fixed as an application parameter) and $N \in \mathbb{B} \setminus \{0\}$, $f(N) = \text{msb}_j(N) \parallel 0^{n-j}$, and $g(N) = \text{lsb}_j(N)$. The OCB3 NAEAD advantage [119] can be obtained from GOCB by instantiating the definition of GOCB as follows:

- $L = (L_1, L_2)$, where L_2 is a keyed function from $\mathbb{B}$ to $\mathbb{B}$, with $L_1 := E_K(0)$ and $\forall N \in \mathbb{B}, L_2(N) := \text{sts}_{E_K(f(N))}(g(N))$.
- For $i \in [2^n]$, and $N \in \mathbb{B} \setminus \{0\}$:
 - $\lambda_1(L, N, i) := L_2(N) \oplus \text{gray}_{L_1}(i)$; $\lambda_5(L, N, i) := \text{gray}_{L_1}(i)$;
 - $\lambda_2(L, N, i) := L_2(N) \oplus \text{gray}_{L_1}(i) \oplus L_1$; $\lambda_6(L, N, i) := \text{gray}_{L_1}(i) \oplus L_1$;
 - $\lambda_3(L, N, i) := L_2(N) \oplus \text{gray}_{L_1}(i) \oplus 2L_1$.
 - $\lambda_4(L, N, i) := L_2(N) \oplus \text{gray}_{L_1}(i) \oplus 3L_1$.

We write λ_i as GR_i to emphasize that λ_i is defined via **gray** hash function.

Here we have derived L through K , which adds a insignificant factor in the overall advantage. In [119], all variants of **GR** were shown to be 2^{-n} -AXU hash over the domain $\{0, 1\}^{128}$. This in combination with Proposition 8.3.2, establishes that **GR** variants are $(2^{-n}, 1)$ -LIXU hash. Using Theorem 8.3.3, $q \leq \sigma$, $\tilde{q} \leq \tilde{\sigma}$, and assuming $\sigma, \tilde{\sigma} < 2^{n-1}$, we get the security bound for the original OCB3 design [119].

Corollary 8.3.4. $\forall \mathcal{A} \in \mathbb{A}(q, \ell, \sigma)$ we have,

$$\text{Adv}_{\text{OCB3}_{E_K^\pm, \text{GR}}}^{\text{naead}}(\mathcal{A}) \leq \text{Adv}_{E_K^\pm}^{\text{sprp}}(\sigma + \tilde{\sigma}) + \frac{6.5\sigma^2 + 4\tilde{\sigma}^2 + 7\sigma\tilde{\sigma} + 3\sigma + 10\tilde{\sigma}}{2^n}.$$

8.3.3.2 Instantiating GOCB with aes1 and aes2

We set $n = 128$ and $\mathcal{N} \subset \mathbb{B}$, and define

- **A1** : $\text{Func}(\mathbb{B}, \mathbb{B}) \times \mathcal{N} \times \mathbb{F}_{2^8} \rightarrow \mathbb{B}$ as, $\forall F \in \text{Func}(\mathbb{B}, \mathbb{B}), N \in \mathcal{N}, i \in \mathbb{F}_{2^8}$,

$$\mathbf{A1}_F(N, i) := \text{aes1}_{F(N)}(i).$$

- **A2** : $\text{Func}(\mathbb{B}, \mathbb{B}) \times \mathbb{B} \times \mathcal{N} \times \mathbb{F}_{2^{32}} \rightarrow \mathbb{B}$ as, $\forall (F, L, N, i) \in \text{Func}(\mathbb{B}, \mathbb{B}) \times \mathbb{B} \times \mathcal{N} \times \mathbb{F}_{2^{32}}$,

$$\mathbf{A2}_{(F, L)}(N, i) := \text{aes2}_{(F(N), L)}(i).$$

Next we show that A1 and A2 are LIXU hash functions for appropriate values of ϵ and r . In Lemma 8.3.5 and Lemma 8.3.6 we show that A1 and A2 are $(2^{-96}, 2^8)$ -LIXU and $(2^{-113}, 2^{32})$ -LIXU hash functions, respectively. The proofs of these lemmata are given in section 8.4.

Lemma 8.3.5. *If $\Gamma \leftarrow \$ \text{Func}(\mathbb{B}, \mathbb{B})$, then $A1_\Gamma$ is a $(2^{-96}, 2^8)$ -LIXU hash function.*

Lemma 8.3.6. *If $(\Gamma, L) \leftarrow \$ \text{Func}(\mathbb{B}, \mathbb{B}) \times \mathbb{B}$, then $A2_{\Gamma, L}$ is a $(2^{-113}, 2^{32})$ -LIXU hash function.*

Description of $\text{GOCB}_{E, A1}$ — We define GOCB based on A1 as follows:

- The maximum message and associated data size per query is limited to 2^6 blocks, i.e. 1 Kilobytes, and the nonce space is $\mathbb{B} \setminus \{0\}$.
- $L = F$, where F is a keyed function from $\mathbb{B}$ to $\mathbb{B}$, where $\forall N \in \mathbb{B}, F(N) := E_K(N)$.
- For $i \in [2^6]$, and $N \in \mathbb{B} \setminus \{0\}$:
 - $\lambda_1(F, N, i) := A1_F(N, 4i + 0)$; and $\lambda_5(F, N, i) := A1_F(N, 4i + 0) \oplus E_K(0)$.
 - $\lambda_2(F, N, i) := A1_F(N, 4i + 1)$; and $\lambda_6(F, N, i) := A1_F(N, 4i + 1) \oplus E_K(0)$.
 - $\lambda_3(F, N, i) := A1_F(N, 4i + 2)$;
 - $\lambda_4(F, N, i) := A1_F(N, 4i + 3)$;

where $4i$ denotes the integer multiplication of i with 4.

Here we need to add a factor of $(q + 1)(q + \sigma)2^{-128}$ to bound the probability that some masked input matches with some nonce value. Further a factor of $q^2 2^{-129}$ is required for PRF-PRP switch. Using Theorem 8.3.3, $q < \sigma$, $\tilde{q} < \tilde{\sigma}$, and $\sigma, \tilde{\sigma} < 2^{127}$, we get the security result for $\text{GOCB}_{E_K, A1}$ as in Corollary 8.3.7.

Corollary 8.3.7. *$\forall \mathcal{A} \in \mathbb{A}(q, \tilde{q}, \sigma)$ we have,*

$$\text{Adv}_{\text{GOCB}_{E_K^\pm, A1}}^{\text{naead}}(\mathcal{A}) \leq \text{Adv}_{E_K^\pm}^{\text{sprp}}(\sigma + \tilde{\sigma}) + \frac{6.5\sigma^2 + 4\tilde{\sigma}^2 + 7\sigma\tilde{\sigma} + 2\sigma + 7\tilde{\sigma}}{2^{128}} + \frac{\sigma + 2\tilde{\sigma}}{2^{88}} + \frac{\tilde{q}}{2^{96}}.$$

Description of $\text{GOCB}_{E, A2}$ — $\text{GOCB}_{E, A2}$ can be defined analogously using mask hash key $L = (L_1, L_2, F)$, where $L_1 = E_K(0)$ and $L_2 = E_K(1)$. We define GOCB based on A2 as follows:

- The maximum message and associated data size per query is limited to 2^{30} blocks, i.e. 16 Gigabytes, and the nonce space is $\mathbb{B} \setminus \{0, 1\}$.

- $L = (L_1, F)$, where $L_1 = E_K(0)$ and F is a keyed function from $\mathbb{B}$ to $\mathbb{B}$. For all $N \in \mathbb{B} \setminus \{0, 1\}$, $F(N) := E_K(N)$.
- For $i \in [2^{30}]$, and $N \in \mathbb{B}$:
 - $\lambda_1(L, N, i) := A2_L(N, 4i + 0)$; and $\lambda_1(L, N, i) := A2_L(N, 4i + 0) \oplus E_K(1)$.
 - $\lambda_2(L, N, i) := A2_L(N, 4i + 1)$; and $\lambda_2(L, N, i) := A2_L(N, 4i + 1) \oplus E_K(1)$.
 - $\lambda_3(L, N, i) := A2_L(N, 4i + 2)$;
 - $\lambda_4(L, N, i) := A2_L(N, 4i + 3)$;

where $4i$ denotes the integer multiplication of i with 4.

Using similar argument as above we can derive the security bound, given in Corollary 8.3.8, for GOCB instantiated with A2.

Corollary 8.3.8. $\forall \mathcal{A} \in \mathbb{A}(q, \tilde{q}, \sigma)$ we have,

$$\text{Adv}_{\text{GOCB}_{E_K^\pm, A2}}^{\text{naead}}(\mathcal{A}) \leq \text{Adv}_{E_K^\pm}^{\text{sprp}}(\sigma + \tilde{\sigma}) + \frac{6.5\sigma^2 + 4\tilde{\sigma}^2 + 7\sigma\tilde{\sigma} + 4\sigma + 9\tilde{\sigma}}{2^{128}} + \frac{\sigma + 2\tilde{\sigma}}{2^{81}} + \frac{\tilde{q}}{2^{113}}.$$

8.3.3.3 Extending the Domain of $\text{GOCB}_{E, A1}$ and $\text{GOCB}_{E, A2}$

The A1 and A2 based instantiations have a restriction on the maximum input length. This is more prominent in case of $\text{GOCB}_{E, A1}$, which allows message lengths up to 1 KB. Here we give some ways to extend the domain of $\text{GOCB}_{E, A1}$ and $\text{GOCB}_{E, A2}$. Our general idea remains the same in all the methods. We improve the maximum input size restriction to roughly 2^{50} bytes or 2^{46} blocks, a usual limit on input size [156]. We need 46 bits to represent each block of input uniquely. We divide the 46-bit length representation into two parts $i||j$, where j is 6-bit long for $\text{GOCB}_{E, A1}$ and j is 30-bit long for $\text{GOCB}_{E, A2}$. We will handle the j -value in the same way as before, and employ different techniques to handle (N, i) input of the mask generating function.

1. DOMAIN EXTENSION VIA THE F FUNCTION: We can consider a keyed function over larger domain, say $\mathcal{N} \times [2^{128}]$. For example, consider the definition $F(N, i) := E_K(i \oplus E_K(N))$. This would add an extra $O(\sigma^2/2^n)$ term, a rough upper bound to avoid block cipher input collisions involving $i \oplus E_K(N)$ for some (N, i) . This method can clearly extend the message size to 2^{46} but at the cost of an extra block cipher call every time the j value resets to zero, plus it needs one more encryption call for the nonce.

2. DOMAIN EXTENSION VIA NONCE SIZE REDUCTION: Another way is to reduce the nonce size to 88 bits and 112 bits for $\text{GOCB}_{E,A1}$ and $\text{GOCB}_{E,A2}$, respectively, and let $F(N, i) := E_K(N \| i)$. This is slightly better than the previous method as we can save one block cipher call. As before, this adds an extra $O(\sigma^2/2^n)$ term, a rough upper bound to avoid block cipher input collisions.

3. DOMAIN EXTENSION VIA SUM OF LIXU HASH: We explain the method for $\text{GOCB}_{E,A2}$. But similar technique is also applicable to $\text{GOCB}_{E,A1}$. We modify the λ function as follows:

- First, we redefine $L = (L_1, L_2, F, F')$, where $L_1 = E_K(0)$ and $L_2 = E_K(1)$. The nonce space is $\mathcal{N} = \{0, 1\}^{n-1} \setminus \{0, 1\}$, and F and F' are keyed functions from $\mathcal{N}$ to $\mathbb{B}$, defined as $F_K(N) = E_K(N \| 0)$ and $F'_K(N) = E_K(N \| 1)$.
- Second, $\lambda_1(L, N, i \| j) := A2_{L_1, F}(N, i) \oplus A2_{L_2, F'}(N, j)$. The other variants are defined analogously as before. Further for AD processing, we XOR $E_K(2)$ (instead of $E_K(1)$) to the modified λ_1 and λ_2 values.

It can be easily shown that this modification offers asymptotically similar security as before.

8.4 Security Proofs

In this section we provide the proof for our main result Theorem 8.3.3, along with the proofs for Lemma 8.3.5 and Lemma 8.3.6.

8.4.1 Inaptness of the Existing TBC-based Abstraction

In [171], Rogaway introduced a very nice abstraction for OCB like constructions based on tweakable block ciphers (TBCs). More formally, the TBC abstraction of [171] views the masked input-output block cipher as a way to construct TBCs from block ciphers and efficient mask generating functions, called XEX. In this view any OCB like construction can be viewed as an instance of the TBC-based authenticated encryption called ΘCB , where each encryption TBC call takes a tuple of tweak values (N, i) , where N denotes the nonce and i denotes the block index.

This abstraction simplifies as well as modularizes the proof, as it can be shown (using results from [171]) that the NAEAD advantage is bounded by the tweakable strong pseudorandom permutation (TSPRP) advantage of XEX, which is bounded by at most $\sigma^2\epsilon$, where σ denotes the total number of TBC calls and ϵ denotes the AXU bound of

the underlying MGF. Consequently, this approach was used in several previous works, most notably [45] and [81].

While the reduction from the NAEAD game for OCB to the TSPRP game for XEX gives tight estimate on the privacy bounds (exactly same as the TSPRP bound for XEX), the reduction is rather loose in case of authenticity bound, as noted in [30]. A straightforward approach gives an authenticity bound of the form $O(\sigma^2\epsilon) + O(1/2^n)$ for single forgery. The first term here is due to the TSPRP advantage of XEX. Here the $O(\sigma^2\epsilon)$ term is due to the XEX to tweakable random permutation reduction. When extended to multiple verification queries, this results in a bound of the form $O(\tilde{q}\sigma^2\epsilon) + O(\tilde{q}/2^n)$, where $\tilde{q}$ denotes the number of verification queries. Clearly the security degrades for multiple verification queries.

Yet another approach gives a security bound of the form $O(\sigma^2\epsilon) + O(\tilde{\sigma}^2\epsilon) + O(\tilde{q}/2^n)$, where $\tilde{\sigma}$ denotes the number of TBC calls in all decryption queries. Here the $O(\sigma^2\epsilon)$ and $O(\tilde{\sigma}^2\epsilon)$ terms are due to the XEX to tweakable random permutation reduction.

A constrained variant of the XEX based abstraction could also be used to get a modularized privacy bound for GOCB. In this abstraction the adversary against tweakable block cipher is restricted to at most r many queries per chunk number. This restriction is perfectly fine, as the NAEAD adversary is nonce-respecting for encryption queries and for each nonce the message length is at most r . This gives a privacy bound of the form $O(\sigma^2/2^n) + O(\sigma r\epsilon)$. But the same is not true for authenticity, as the adversary can make all the decryption queries with tweaks within the same chunk. This results in a term of the form $O(\tilde{\sigma}^2\epsilon)$, which is clearly sub-optimal.

So, we use a more direct approach as also employed in [30]. The proof for Theorem 8.3.3 uses coefficient-H technique.

MORE NOTATIONS: We say a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ is *partial* or *restricted*, if we know the values of f on a strict subset of $\mathcal{X}$. This subset is called $\text{domain}(f)$. A partial function f can be viewed as a restriction of f to $\text{domain}(f)$. The range of this restricted function is called $\text{range}(f)$. We will treat a partial function as *updatable*: for some $x \in \mathbb{B} \setminus \text{domain}(f)$ and some $y \in \mathcal{Y}$, (x, y) may be added to f , so that $\text{domain}(f)$ expands to $\text{domain}(f) \cup \{x\}$, and $\text{range}(f)$ becomes $\text{range}(f) \cup \{y\}$.

8.4.2 Proof of GOCB NAEAD Bound (Theorem 8.3.3)

Let $\mathcal{O}_1^\pm$ denote the real oracle corresponding to GOCB, and $\mathcal{O}_0^\pm$ denote the ideal oracle corresponding to $(\Gamma, \perp)$. We start off by setting up the necessary notations related to the query-response transcript of $\mathcal{A}$. The transcript generated by $\mathcal{A}$ consists of

- *Encryption query-response tuples:* for $i \in [q]$, the i -th encryption query-response tuple is built of
 - N_i : the queried nonce block, such that for any $i' < i$, $N_i \neq N_{i'}$.
 - A_i : the queried associated data, consisting of a_i complete blocks and an incomplete (possibly empty) block $A_{(i,*)}$ at the end;
 - M_i : the queried message, consisting of m_i complete blocks and an incomplete (possibly empty) block $M_{(i,*)}$ at the end;
 - C_i : the response ciphertext, such that $|C_i| = |M_i|$;
 - T_i : the response tag block.
- *Decryption query-response tuples:* for $i \in [\tilde{q}]$ the i -th decryption query-response tuple is built of
 - $\tilde{N}_i$: the queried nonce block. Note that, $\tilde{N}_i$'s can be repeated.
 - $\tilde{A}_i$: the queried associated data, consisting of $\tilde{a}_i$ complete blocks and an incomplete (possibly empty) block $\tilde{A}_{(i,*)}$ at the end;
 - $\tilde{C}_i$: the queried ciphertext, consisting of $\tilde{m}_i$ complete blocks and an incomplete (possibly empty) block $\tilde{C}_{(i,*)}$ at the end;
 - $\tilde{T}_i$: the queried tag block.
 - $\tilde{M}_i$: the response message, such that $|\tilde{M}_i| = |\tilde{C}_i|$. Note that, the decryption oracle may return $\perp$, in which case $\tilde{M}_i$ just denotes the unauthenticated decrypted message to be used internally.

The internal variables arising in one call to the encryption oracle are analogously as given in Algorithm 8.3.1 and Figure 8.3.1, while the internal variables from the decryption oracle are defined identically, but topped with a tilde to differentiate them from the encryption variables.

Let $\mathcal{I}$ and $\mathcal{J}$ denote the encryption query indices with incomplete-block message and associated data respectively. The decryption counterparts are $\tilde{\mathcal{I}}$ and $\tilde{\mathcal{J}}$. Let $I = I_1 \cup I_2$ and $O = O_1 \cup O_2$ be multisets, where for all $i \in [2]$, I_i and O_i are defined as follows

$$\begin{aligned}
 I_1 &:= \{U_{(i,j)} : (i,j) \in [q] \times [a_i]\} \cup \{U_{(i,*)} : i \in \mathcal{J}\}, \\
 I_2 &:= \{X_{(i,j)} : (i,j) \in [q] \times ([m_i] \cup \{\oplus\})\} \cup \{X_{(i,*)} : i \in \mathcal{I}\}, \\
 O_1 &:= \{V_{(i,j)} : (i,j) \in [q] \times [a_i]\} \cup \{V_{(i,*)} : i \in \mathcal{J}\}, \\
 O_2 &:= \{Y_{(i,j)} : (i,j) \in [q] \times ([m_i] \cup \{\oplus\})\} \cup \{Y_{(i,*)} : i \in \mathcal{I}\}.
 \end{aligned}$$

8.4.2.1 Oracle Behavior and Transcript Extension

Consider a modified security game where we let the real oracle $\mathcal{O}_1$ reveal all the internal input and output blocks appearing in the encryption query phase. Consequently we have to make appropriate behavioral changes in the ideal oracle $\mathcal{O}_0$ to release these internal variables. We describe the sampling behavior of $\mathcal{O}_0$ in greater detail in the subsequent paragraphs. During the sampling $\mathcal{O}_0$ might set $\text{bad} = 1$, whereafter its behavior is undefined.

SAMPLING BEHAVIOR OF $\mathcal{O}_0$

Query Phase: For $i \in [q]$, on the i -th encryption query, for each $j \in [m_i]$, $C_{(i,j)} \leftarrow \$ \mathbb{B}$; $T_i \leftarrow \$ \mathbb{B}$; and if $i \in \mathcal{I}$, $\bar{C}_{(i,*)} \leftarrow \$ \mathbb{B}$, set $C_{(i,*)} = \text{msb}_{|M_{(i,*)}|}(\bar{C}_{(i,*)})$; return $C_i = C_{(i,1)} \parallel \dots \parallel C_{(i,*)}$ and T_i to $\mathcal{A}$. For $i \in [\tilde{q}]$, on the i -th decryption query, return $\perp$ to $\mathcal{A}$.

Post-query Phase: Let $L \leftarrow \$ \mathcal{L}$ and Π , a permutation over $\mathbb{B}$, be undefined on all blocks, i.e. $\text{domain}(\Pi) = \text{range}(\Pi) = \emptyset$.

Step 1: Extending the encryption query-response tuple — Set the following values:

- for $i \in [q]$, $j \in [m_i]$ set $X_{(i,j)} = M_{(i,j)} \oplus \lambda_1(L, N_i, j)$ and $Y_{(i,j)} = C_{(i,j)} \oplus \lambda_1(L, N_i, j)$;
- for $i \in \mathcal{I}$, set $X_{(i,*)} = \lambda_2(L, N_i, m_i)$ and $Y_{(i,*)} = \bar{C}_{(i,*)} \oplus \text{pad}(M_{(i,*)})$;
- for $i \in [q] \setminus \mathcal{I}$, set $M_{(i,\oplus)} = \bigoplus_{j=1}^{m_i} M_{(i,j)}$ and $X_{(i,\oplus)} = M_{(i,\oplus)} \oplus \lambda_3(L, N_i, m_i)$;
- for $i \in \mathcal{I}$, set $M_{(i,\oplus)} = \bigoplus_{j=1}^{m_i} M_{(i,j)} \oplus \text{pad}(M_{(i,*)})$ and $X_{(i,\oplus)} = M_{(i,\oplus)} \oplus \lambda_4(L, N_i, m_i)$;
- for $i \in [q]$, $j \in [a_i]$ set $U_{(i,j)} = A_{(i,j)} \oplus \lambda_5(L, N_i, j)$;
- for $i \in \mathcal{J}$, set $U_{(i,*)} = \text{pad}(A_{(i,a_i)}) \oplus \lambda_6(L, N_i, a_i)$;
- set $\text{bad} = 1$, if I or O (the partial multiset containing $Y_{(i,j)}$ values for $i \in [q]$ and $j \in [m_i] \cup \{*\}$) contains a non-trivial colliding pair (pair of duplicate elements). Note that, all colliding pairs are non-trivial at this stage;
- for all $i \in [q]$, $j \in [m_i]$, fix $\Pi(X_{(i,j)}) = Y_{(i,j)}$, and for all $i \in \mathcal{I}$, fix $\Pi(X_{(i,*)}) = Y_{(i,*)}$;
- for all $i \in [q]$, $j \in [a_i]$, fix $\Pi(U_{(i,j)}) = V_{(i,j)} \leftarrow \$ \mathbb{B} \setminus \text{range}(\Pi)$, and for all $i \in \mathcal{J}$, fix $\Pi(U_{(i,*)}) = V_{(i,*)} \leftarrow \$ \mathbb{B} \setminus \text{range}(\Pi)$;
- for $i \in [q] \setminus \mathcal{J}$, set $S_i = \bigoplus_{j=1}^{a_i} V_{(i,j)}$;
- for $i \in \mathcal{J}$, set $S_i = \bigoplus_{j=1}^{a_i} V_{(i,j)} \oplus V_{(i,*)}$;
- for $i \in [q]$ set $Y_{(i,\oplus)} = S_i \oplus T_i$.

- set $\text{bad} = 1$, if O (the complete multiset) contains a non-trivial colliding pair (pair of duplicate elements).
- for $i \in [q]$, fix $\Pi(X_{(i,\oplus)}) = Y_{(i,\oplus)}$.

Note that, the partial sampling of Π remains permutation-compatible in step 1 as long as $\text{bad} = 0$.

Step 2: Extending the decryption query-response tuple — Set the following values:

- at the start all intermediate variables are set as undefined.
- for $i \in [\tilde{q}]$, $j \in [\tilde{m}_i]$, set $\tilde{Y}_{(i,j)} = \tilde{C}_{(i,j)} \oplus \lambda_1(L, \tilde{N}_i, j)$; if $i \in \tilde{\mathcal{I}}$, set $\tilde{X}_{(i,*)} = \lambda_2(L, \tilde{N}_i, \tilde{m}_i)$.
- for $i \in [\tilde{q}]$, $j \in [\tilde{m}_i]$, if $\tilde{Y}_{(i,j)} \in \text{range}(\Pi)$ then set $\tilde{X}_{(i,j)} = \Pi^{-1}(\tilde{Y}_{(i,j)})$ and $\tilde{M}_{(i,j)} = \tilde{X}_{(i,j)} \oplus \lambda_1(L, \tilde{N}_i, j)$; if $i \in \tilde{\mathcal{I}}$ and $\tilde{X}_{(i,*)} \in \text{domain}(\Pi)$, then set $\tilde{Y}_{(i,*)} = \Pi(\tilde{X}_{(i,*)})$ and $\tilde{M}_{(i,*)} = \text{msb}_{|\tilde{C}_{(i,*)}|}(\tilde{Y}_{(i,*)}) \oplus \tilde{C}_{(i,*)}$; if $\tilde{M}_i$ is defined then set $\tilde{X}_{(i,\oplus)} = \tilde{M}_{(i,\oplus)} \oplus \lambda_3(L, \tilde{N}_i, \tilde{m}_i)$.
- for $i \in [\tilde{q}]$, $j \in [\tilde{a}_i]$, set $\tilde{U}_{(i,j)} = \tilde{A}_{(i,j)} \oplus \lambda_5(L, \tilde{N}_i, j)$; if $i \in \tilde{\mathcal{J}}$, set $\tilde{U}_{(i,*)} = \lambda_6(L, \tilde{N}_i, \tilde{a}_i)$.
- for $i \in [\tilde{q}]$, $j \in [\tilde{a}_i]$, if $\tilde{U}_{(i,j)} \in \text{domain}(\Pi)$ then set $\tilde{V}_{(i,j)} = \Pi(\tilde{U}_{(i,j)})$; if $\tilde{V}_i$ is defined then set $\tilde{Y}_{(i,\oplus)} = \tilde{S}_i \oplus \tilde{T}_i$.
- At this stage all those intermediate variables, which can be derived through the MGF key L , and adversary's query, are completely determined. Also, some other variables are trivially derived due to the extended encryption transcript.
- set $\text{bad} = 1$, if there exists $(i, j) \in [\tilde{q}] \times ([\tilde{m}_i] \cup \{*, \oplus\})$, and one of the following is true:
 - $\exists (i', j') \in [q] \times ([m_{i'}] \cup \{*, \oplus\})$ such that $(\tilde{N}_i, \tilde{C}_{(i,j)}) \neq (N_{i'}, C_{(i',j')})$ and $\tilde{Y}_{(i,j)} = Y_{(i',j')}$.
 - $\exists (i', j') \in [q] \times ([a_{i'}] \cup \{*\})$ such that $\tilde{Y}_{(i,j)} = V_{(i',j')}$.
- set $\text{bad} = 1$, if there exists $i \in [\tilde{q}] \times ([\tilde{a}_i] \cup \{*\})$, and one of the following is true:
 - $\exists (i', j') \in [q] \times ([m_{i'}] \cup \{*, \oplus\})$ such that $\tilde{U}_{(i,j)} = X_{(i',j')}$.
 - $\exists (i', j') \in [q] \times ([a_{i'}] \cup \{*\})$ such that $(\tilde{N}_i, \tilde{A}_{(i,j)}) \neq (N_{i'}, \tilde{A}_{(i',j')})$ and $\tilde{U}_{(i,j)} = U_{(i',j')}$.

Extend the transcript by including all the internal variables computed thus far and return the extended transcript to $\mathcal{A}$.

IDENTIFYING THE BAD TRANSCRIPTS: Let Ω denote the set of all realizable transcripts. We say that $\omega \in \Omega$ is a bad transcript if it causes $\text{bad} = 1$. In other words, we say that ω is bad if one of the following events occur:

EEcoll:

- $\exists (i, j) \in [q] \times [m_i] \cup \{*, \oplus\}, (i', j') \in [q] \times [m_{i'}] \cup \{*, \oplus\} : (i, j) \neq (i', j') \wedge X_{(i,j)} = X_{(i',j')}.$
- $\exists (i, j) \in [q] \times [a_i] \cup \{*\}, (i', j') \in [q] \times [a_{i'}] \cup \{*\} : (i, j) \neq (i', j') \wedge U_{(i,j)} = U_{(i',j')}.$
- $\exists (i, j) \in [q] \times [m_i] \cup \{*, \oplus\}, (i', j') \in [q] \times [a_{i'}] \cup \{*\} : X_{(i,j)} = U_{(i',j')}.$
- $\exists (i, j) \in [q] \times [m_i] \cup \{*\}, (i', j') \in [q] \times [m_{i'}] \cup \{*\} : (i, j) \neq (i', j') \wedge Y_{(i,j)} = Y_{(i',j')}.$

ETcoll:

- $\exists i \in [q], (i', j') \in [q] \times [m_{i'}] \cup \{*\} : Y_{(i,\oplus)} = Y_{(i',j')}.$
- $\exists^* i, i' \in [q] : Y_{(i,\oplus)} = Y_{(i',\oplus)}.$
- $\exists i \in [q], (i', j') \in [q] \times [a_{i'}] \cup \{*\} : Y_{(i,\oplus)} = V_{(i',j')}.$

DEcoll:

- $\exists (i, j) \in [\tilde{q}] \times [\tilde{m}_i] \cup \{*\}, (i', j') \in [q] \times [m_{i'}] \cup \{*, \oplus\} : (\tilde{N}_i, \tilde{C}_{(i,j)}) \neq (N_{i'}, C_{(i',j')}) \wedge \tilde{Y}_{(i,j)} = Y_{(i',j')}.$
- $\exists (i, j) \in [\tilde{q}] \times [\tilde{m}_i] \cup \{*\}, (i', j') \in [q] \times [a_{i'}] \cup \{*\} : \tilde{Y}_{(i,j)} = V_{(i',j')}.$
- $\exists (i, j) \in [\tilde{q}] \times [\tilde{a}_i] \cup \{*\}, (i', j') \in [q] \times [a_{i'}] \cup \{*\} : (\tilde{N}_i, \tilde{A}_{(i,j)}) \neq (N_{i'}, A_{(i',j')}) \wedge \tilde{U}_{(i,j)} = U_{(i',j')}.$
- $\exists (i, j) \in [\tilde{q}] \times [\tilde{a}_i] \cup \{*\}, (i', j') \in [q] \times [m_{i'}] \cup \{*, \oplus\} : \tilde{U}_{(i,j)} = X_{(i',j')}.$

DEFcoll:

- $\exists i \in [\tilde{q}], i' \in [q] : (\tilde{N}_i, \tilde{A}_i, \tilde{T}_i) = (N_{i'}, A_{i'}, T_{i'}) \wedge \tilde{C}_i = C_{i'}^{[\tilde{m}_i]} \wedge \tilde{X}_{(i,\oplus)} \in I.$

DDFcoll:

- $\exists (i, j) \in [\tilde{q}] \times [\tilde{m}_i] \cup \{*\}, i' \in [q] : (\tilde{N}_i, \tilde{A}_i, \tilde{C}_i, \tilde{T}_i) = (N_{i'}, A_{i'}, C_{i'}^{[\tilde{m}_i]}, T_{i'}) \wedge \tilde{X}_{(i,\oplus)} = \tilde{X}_{(i,j)}.$
- $\exists (i, j) \in [\tilde{q}] \times [\tilde{a}_i] \cup \{*\}, i' \in [q] : (\tilde{N}_i, \tilde{A}_i, \tilde{C}_i, \tilde{T}_i) = (N_{i'}, A_{i'}, C_{i'}^{[\tilde{m}_i]}, T_{i'}) \wedge \tilde{X}_{(i,\oplus)} = \tilde{U}_{(i,j)}.$
- $\exists i \in [\tilde{q}], \exists^* j, j' \in [\tilde{m}_i] \cup \{*, \oplus\} : \tilde{Y}_{(i,j)} = \tilde{Y}_{(i,j')}.$

Let Ω_{bad} be the set of all transcripts which are bad and $\Omega_{\text{good}} = \Omega \setminus \Omega_{\text{bad}}$. We bound $\text{ip}_{\mathcal{O}_0}[\Omega_{\text{bad}}]$ to $O(\sigma^2 2^{-n} + r\sigma\epsilon + q\sigma 2^{-n} + q^2 2^{-n} + \tilde{\sigma}\sigma 2^{-n} + \tilde{\sigma}r\epsilon + \tilde{\sigma} 2^{-n} + \tilde{q}\epsilon)$ in Lemma 8.4.1.

Lemma 8.4.1.

$$\Pr[\Theta_0 \in \Omega_{\text{bad}}] \leq \frac{0.5\sigma^2}{2^n} + \frac{\tilde{\sigma}^2}{2^n} + \frac{q\sigma + 0.5q^2}{2^n - \sigma} + \frac{\tilde{\sigma}(\sigma + 3)}{2^n} + (\sigma + 2\tilde{\sigma})r\epsilon + \tilde{q}\epsilon.$$

Proof. We bound the probability of getting a bad transcript as follows:

$$\begin{aligned} \Pr[\Theta_0 \in \Omega_{\text{bad}}] &= \Pr[\text{EEcoll} \vee \text{ETcoll} \vee \text{DEcoll} \vee \text{DEFcoll} \vee \text{DDFcoll}] \\ &\leq \Pr[\text{EEcoll}] + \Pr[\text{ETcoll} \mid \neg \text{EEcoll}] \\ &\quad + \Pr[\text{DEcoll} \mid \neg(\text{EEcoll} \vee \text{ETcoll})] \\ &\quad + \Pr[\text{DEFcoll} \mid \neg(\text{EEcoll} \vee \text{ETcoll} \vee \text{DEcoll})] \\ &\quad + \Pr[\text{DDFcoll} \mid \neg(\text{EEcoll} \vee \text{ETcoll} \vee \text{DEcoll} \vee \text{DEFcoll})] \end{aligned}$$

BOUNDING $\Pr[\text{EEcoll}]$: Let P and Q be a colliding pair. We bound the probability in two cases:

Case 1: The colliding pair is a non-local pair — As λ is a LIXU hash function, we can bound the probability of this case by 2^{-n} . Further we have at most $\sigma(\sigma - 1)/2$ many such pairs. Hence

$$\Pr[\text{EEcoll} \wedge \text{Case 1}] \leq \frac{\sigma^2}{2^{n+1}}.$$

Case 2: The colliding pair is a local pair — This means that the colliding pair must belong to a single message or associated data. We can have two cases: (2.1) The colliding pair belongs to the i -th message; (2.2) The colliding pair belongs to the i -th associated data. Without loss of generality we assume case (2.1). Let $n_i = \lfloor m_i/r \rfloor$ and $r_i = m_i - n_i r$. As λ is an (ϵ, r) -LIXU hash function, we can bound the probability of a fixed pair by ϵ . Hence

$$\begin{aligned} \Pr[\text{EEcoll} \wedge \text{Case 2.1}] &\leq \sum_{i=1}^q \sum_{j=1}^{n_i} \binom{r}{2} \epsilon + \sum_{i=1}^q \binom{r_i}{2} \epsilon \\ &\stackrel{1}{\leq} r' \epsilon \sum_{i=1}^q r \cdot n_i + \sum_{i=1}^q \binom{r_i}{2} \epsilon \\ &\stackrel{2}{\leq} r' \mu \epsilon - \left(r' \epsilon \sum_{i=1}^q r_i - \sum_{i=1}^q \binom{r_i}{2} \epsilon \right) \\ &\stackrel{3}{\leq} r' \mu \epsilon. \end{aligned}$$

Here $r' = (r - 1)/2$; 1 to 2 follows from $rn_i = m_i - r_i$ and $\sum m_i = \mu$; 2 to 3 follows from the fact that $(r - 1)r_i \geq r_i(r_i - 1)$ for all i . Similarly the probability in case (2.2) can be bounded to $r'\nu\epsilon$. Hence the total probability in case 2 is bounded by $r'\sigma\epsilon$.

Combining case 1 and 2, we have

$$\Pr[\text{EEcoll}] \leq \frac{\sigma^2}{2^{n+1}} + r\sigma\epsilon.$$

BOUNDING $\Pr[\text{ETcoll} \mid \neg\text{EEcoll}]$: This event bounds the probability that the checksum output block collides with some previous output block given that no other input/output collision occurred. At this instant at most $\sigma = \mu + \nu$ many points are defined for Π , whence we can have at most $q\sigma + q^2/2$ many possible colliding pairs. So we have,

$$\Pr[\text{ETcoll} \mid \neg\text{EEcoll}] \leq \frac{q\sigma}{2^n - \sigma} + \frac{q^2}{2(2^n - \sigma)}.$$

Bounding $\Pr[\text{DEcoll} \mid \neg(\text{EEcoll} \vee \text{ETcoll})]$: This event bounds the probability that some intermediate output ($\tilde{Y}_{(i,j)}$) or input ($\tilde{U}_{(i,j)}$) input block non-trivially belongs in O or I , respectively. First, suppose $\tilde{Y}_{(i,j)}$ belongs to O for some $(i, j) \in [\tilde{q}] \times [\tilde{m}_i]$. Now as in case of EEcoll above: i) the colliding encryption block can either be a non-local pair of (i, j) , in which case we bound the probability to at most $\sigma/2^n$; or ii) the colliding encryption block can be a local pair of (i, j) , in which case we bound the probability to at most $r\epsilon$. Since there are at most $\tilde{\mu}$ many decryption ciphertext blocks, the probability that there exists $(i, j) \in [\tilde{q}] \times [\tilde{m}_i]$, such that $\tilde{Y}_{(i,j)}$ belongs in O non-trivially is bounded by $\tilde{\mu}\sigma/2^n + \tilde{\mu}r\epsilon$. Similarly, one can bound the probability that there exists $(i, j) \in [\tilde{q}] \times [\tilde{a}_i]$, such that $\tilde{U}_{(i,j)}$ belongs in I non-trivially to at most $\tilde{\nu}\sigma/2^n + \tilde{\nu}r\epsilon$. Combining the two cases, we have

$$\Pr[\text{DEcoll} \mid \neg(\text{EEcoll} \vee \text{ETcoll})] \leq \frac{\tilde{\sigma}\sigma}{2^n} + \tilde{\sigma}r\epsilon.$$

Bounding $\Pr[\text{DEFcoll} \mid \neg(\text{EEcoll} \vee \text{ETcoll} \vee \text{DEcoll})]$: This case bounds the probability that some decryption checksum input block collides with some encryption input block with a restriction that the decryption input blocks must all be defined. In this case we have $\tilde{X}_{(i,\oplus)} = \sum_{j=1}^{\tilde{m}_i} \tilde{M}_{(i,j)} \oplus \lambda_3(L, \tilde{N}, \tilde{m}_i)$. Now we can have two cases: i) the checksum block collides with $X_{(i',\oplus)}$, which can be generously bounded by ϵ (assuming $(\tilde{N}_i, \tilde{m}_i)$ and $(N_{i'}, m_{i'})$ are local pair); and ii) the checksum block collides with any other $X_{(i',j')}$, in which case we bound the probability by 2^{-n} using the 1-universal property of λ . In summary, we have

$$\Pr[\text{DEFcoll} \mid \neg(\text{EEcoll} \vee \text{ETcoll} \vee \text{DEcoll})] \leq \frac{\tilde{q}}{2^n} + \tilde{q}\epsilon.$$

Bounding $\Pr [\text{DDFcoll} | \neg(\text{EEcoll} \vee \text{ETcoll} \vee \text{DEcoll} \vee \text{DEFcoll})]$: This case simply bound the probability that there is no collisions within a decryption query. Let us fix a decryption query index $i \in \tilde{q}$. Then the first case is bounded by at most $\tilde{m}_i 2^{-n}$; the second case is bounded by at most $\tilde{a}_i 2^{-n}$; the third case is bounded by at most $\tilde{m}_i(\tilde{m}_i - r)2^{-n} + \tilde{m}_i r \epsilon$. On combining the three cases, we have

$$\Pr [\text{DDFcoll} | \neg(\text{EEcoll} \vee \text{ETcoll} \vee \text{DEcoll} \vee \text{DEFcoll})] \leq \frac{2\tilde{\sigma}}{2^n} + \frac{\tilde{\sigma}^2}{2^n} + \tilde{\sigma} r \epsilon.$$

The result follows by combining all the individual bounds above, and simplifying using $\tilde{q} \leq \tilde{\sigma}$. $\square$

RATIO OF INTERPOLATION PROBABILITIES: Fix an arbitrary transcript $\omega \in \Omega \setminus \Omega_{\text{bad}}$. In Lemma 8.4.2, we show that the ratio of real to ideal interpolation probabilities for ω is at least $(1 - O(\tilde{q}2^{-n}))$.

Lemma 8.4.2. *For any $\omega \in \Omega \setminus \Omega_{\text{bad}}$, we have*

$$\frac{\Pr [\Theta_1 = \omega]}{\Pr [\Theta_0 = \omega]} \geq \left(1 - \frac{\tilde{q}}{2^n - \sigma}\right).$$

Proof. In $\mathcal{O}_0$, for encryption phase: first $\mu + q$ ciphertext and tag outputs are sampled in with replacement fashion, followed by the sampling of ν output blocks in without replacement manner from a subset of $\mathbb{B}$ of size $(2^n - \mu)$ (Π is already defined on μ many input-output blocks); for decryption phase the oracle always returns $\perp$. Hence

$$\text{ip}_{\mathcal{O}_0}[\omega] = \frac{1}{(2^n)^{\mu+q}(2^n - \mu)^\nu}. \quad (8.9)$$

In $\mathcal{O}_1$, for any ω we denote the encryption tuples by ω_e and the decryption tuples by ω_d . Then we have

$$\begin{aligned} \Pr [\Theta_1 = \omega] &= \Pr_{\mathcal{O}_1} [\omega_e, \omega_d] \\ &= \Pr_{\mathcal{O}_1} [\omega_e] \times \Pr_{\mathcal{O}_1} [\omega_d | \omega_e] \\ &= \Pr_{\mathcal{O}_1} [\omega_e] \times \left(1 - \Pr_{\mathcal{O}_1} [\neg \omega_d | \omega_e]\right) \end{aligned} \quad (8.10)$$

where $\omega_d = (\tilde{N}_i, \tilde{A}_i, \tilde{C}_i, \tilde{T}_i, \perp)_{i \in [\tilde{q}]}$, as the ideal oracle always returns $\perp$ on decryption queries. Note that, we have slightly abused the notation to use $\neg \omega_d$ as the event that: for some $i \in [\tilde{q}]$ the i -th decryption query successfully decrypts.

For encryption tuples, exactly $\mu + \nu + q$ many calls are made to Π : one for each of the μ message blocks, one for each of the ν associated data blocks; and one for each of the q

tags. As $\sigma = \mu + \nu$ we have

$$\Pr_{\mathcal{O}_1}[\omega_e] = \frac{1}{(2^n)_{\sigma+q}}. \quad (8.11)$$

Now we upper bound the probability of $\neg\omega_d$. It is enough to bound the probability for a fixed index $i \in [\tilde{q}]$ corresponding to a successful decryption query. By union bound, the probability of $\neg\omega_d$ is at most $\tilde{q}$ times more than the single forgery probability. For simplicity, we bound the probability for $i \in [\tilde{q}] \setminus (\tilde{\mathcal{I}} \cup \tilde{\mathcal{J}})$. For $i \in \tilde{\mathcal{I}} \cup \tilde{\mathcal{J}}$, we get similar bounds. Note that, the adversary succeeds in forgery if the adversary somehow makes the i -th query in such a way that the following equation holds.

$$\Pi(\tilde{X}_{(i,\oplus)}) \oplus \tilde{Y}_{(i,\oplus)} = 0. \quad (8.12)$$

Now based on the i -th decryption query we can have different cases:

Case 1: $\exists j \in [q], \tilde{N}_i = N_j$ — Based on $\tilde{C}_i$ we can have two subcases:

Subcase 1.1: $\tilde{C}_i = C_{i'}^{[\tilde{m}_i]}$ — In this case $\tilde{M}_{(i,\oplus)}$ is pre-determined as $\oplus_{k=1}^{\tilde{m}_i} M_{(i',k)}$. Suppose $\tilde{m}_i = m_{i'}$, i.e. the two ciphertexts are exactly the same. Then we must have some j such that $\tilde{A}_{(i,j)} \neq A_{(i',j)}$, otherwise i -th decryption query is a duplicate of i' -th encryption query. Since the transcript is good, $\tilde{U}_{(i,j)}$ is fresh. So, by exploiting the conditional randomness of Π we can bound the probability to at most $1/(2^n - \sigma)$. Now suppose $\tilde{C}_i$ is a proper prefix of $C_{i'}$, then we must have a fresh $\tilde{X}_{(i,\oplus)}$ (as the transcript is good), whence we can again bound the probability to at most $1/(2^n - \sigma)$.

The two cases discussed above are mutually exclusive, hence the probability that the adversary forges in subcase 2.1 is at most $1/(2^n - \sigma)$.

Subcase 1.2: $\tilde{C}_i \neq C_{i'}^{[\tilde{m}_i]}$ — In this case there exists at least one $k \in [\tilde{m}_i]$ such that $\tilde{C}_{(i,k)} \neq C_{(i',k)}$. We consider first such block $\tilde{C}_{(i,k)}$. Since the transcript is good, we must have a fresh $\tilde{Y}_{(i,k)}$. Observe that we can rewrite Eq. (8.12) as

$$\Pi^{-1}(\tilde{Y}_{(i,k)}) = \Pi^{-1}(\tilde{Y}_{(i,\oplus)}) \oplus \delta \oplus \lambda_3(L, \tilde{N}_i, \tilde{m}_i),$$

where δ denotes the checksum of all the blocks of i -th query, except the k -th block. Again, by conditioning on all points except $\tilde{Y}_{(i,k)}$, we bound the probability of this case by $1/(2^n - \sigma)$.

Case 2: $\forall j \in [q], \tilde{N}_i \neq N_j$: In this case $\tilde{N}_i$ has not been used for mask generation till now. Without loss of generality, we assume that $\tilde{m}_i \geq 1$. Since the transcript is good, we must have a fresh $\tilde{Y}_{(i,1)}$. Now using a similar line of argument as in sub case 1.2 we bound the probability of this event by $1/(2^n - \sigma)$.

Note that, case 1.1, case 1.2 and case 2 are all mutually exclusive so we can take the maximum over all three. As $\sigma \geq a_i + a^j$ we have

$$\Pr_{\mathcal{O}_1}[\neg\omega_d \mid \omega_e] \leq \sum_{i=1}^{\tilde{q}} \frac{1}{2^n - \sigma} \leq \frac{\tilde{q}}{2^n - \sigma} \quad (8.13)$$

On substituting (8.11) and (8.13) in (8.10) and dividing the result by (8.9) we have

$$\begin{aligned} \frac{\text{ip}_{\mathcal{O}_1}[\omega]}{\text{ip}_{\mathcal{O}_0}[\omega]} &\geq \frac{(2^n)^{\mu+q} \cdot (2^n - \mu)^{\underline{\nu}}}{(2^n)^{\sigma+q}} \left(1 - \frac{\tilde{q}}{2^n - \sigma}\right) \\ &\geq \frac{(2^n)^q}{(2^n - \sigma)^{\underline{q}}} \left(1 - \frac{\tilde{q}}{2^n - \sigma}\right) \\ &\geq \left(1 - \frac{\tilde{q}}{2^n - \sigma}\right). \end{aligned}$$

□

We get the desired result using Lemma 8.4.1, Lemma 8.4.2 and Corollary 2.2.2.

8.4.3 Proof of Lemma 8.3.5 and Lemma 8.3.6

The proofs of these lemmata are quite similar. We give the proof for Lemma 8.3.5, while the proof for Lemma 8.3.6 can be obtained analogously.

Fix distinct $(N, i), (N', i') \in \mathcal{N} \times \mathbb{F}_{2^8}$ and $\delta \in \mathbb{B}$. Now we have two cases:

Case 1: (N, i) and (N', i') are local pairs — We know that $N = N'$ and $i \neq i'$. Hence,

$$\Pr[\mathbf{A1}_\Gamma(N, i) \oplus \mathbf{A1}_\Gamma(N', i') = \delta] = \Pr[\mathbf{aes1}_{\Gamma(N)}(i) \oplus \mathbf{aes1}_{\Gamma(N)}(i') = \delta] \leq \frac{1}{2^{96}},$$

where the last inequality follows from Lemma 8.2.4.

Case 2: (N, i) and (N', i') are not local pairs — We know that $N \neq N'$. Hence,

$$\begin{aligned} \Pr[\mathbf{A1}_\Gamma(N, i) \oplus \mathbf{A1}_\Gamma(N', i') = \delta] &= \Pr[\mathbf{aes1}_{\Gamma(N)}(i) \oplus \mathbf{aes1}_{\Gamma(N')}(i') = \delta] \\ &\leq \Pr[\Gamma(N) = 1\text{-AESRD}^{-1}(\mathbf{aes1}_{\Gamma(N')}(i') \oplus \delta) \oplus i] \\ &\leq \frac{1}{2^{128}}. \end{aligned}$$

The result follows from the bounds in case 1 and case 2.

□

8.5 Software Performance

In this section, we present software implementation of GOCB instantiated with AES-128 as the underlying block cipher, and A1 and A2 as the underlying MGFs, and compare the performance of the proposed designs against the standard OCB3-AES-128. We do the benchmarking in two cases: i) sequential processing (the conventional implementation), and ii) random read access (introduced in this chapter).

IMPLEMENTATION — We reuse the optimized C code of OCB3 [119] by Krovetz. Further, we introduce minimal changes, as required, over the OCB3 code to generate the C code for $\text{GOCB}_{\text{AES-128,A1}}$, and $\text{GOCB}_{\text{AES-128,A2}}$. We also reuse the time measurement mechanism as used in [119]. In summary, we utilized the hardware support on our benchmarking platform for Intel’s SSE4 vector instructions [94], Intel’s `pclmulqdq` instruction for multiplication over $\mathbb{F}_{2^{128}}$, and Intel’s AES-NI library [82] for operations involving AES-128 round functions. We made some simplifying assumptions for the implementations:

- message length is treated to be equal to the length of the associated data, and
- all messages and associated data are assumed to have complete blocks.

PLATFORM — We performed the benchmarking on:

- Intel Core i7-6500U “Skylake” (2.5 GHz with 64 KB L1 cache, 256 KB L2 cache, 4 MB L3 cache) with Ubuntu 16.04 LTS (kernel version 4.4),

which supports Intel’s SSE4 vector instructions [94], standard AES-NI instructions [82] and `pclmuldq` [83], the carry-less multiplication instruction.

SETUP — The tests for benchmark were compiled using `gcc` version 5.4, with optimization flag `-O3`, and instruction set architecture flag `-march=native` (as instructed in [119]). All the tests were run on isolated core, after turning off processor frequency scaling and power management options such as Intel’s Turbo Boost and Hyper-Threading technologies. The reference baseline performance for AES-128, using AES-NI instruction set, is presented in Table 8.5.1. All performance figures presented are throughputs, in units of cycles-per-byte (CPB).

TESTING METHODOLOGY — Following [119], we measure the CPB value for every message length from 1 to 1024 bytes, as well as 1500 and 4096 bytes. To avoid any discrepancies arising due to memory subsystem, we execute warm up runs so that the all

Table 8.5.1: Baseline CPB value of AES-128 using AES-NI in Skylake architecture.

Microarchitecture	Encryption (CPB)
Intel Core i7-6500U “Skylake”	0.66

code and data is in the cache before benchmarking begins. To capture the average cost of encryption, we use the `rdtsc` instruction — a time-stamp counter available on Skylake processors — to time encryption of the same message: i) 64 times for sequential processing; ii) 1536 times for random access. Note that, we average over a larger number of repetitions in random access case to get an appreciable measurement for a single block encryption. The median of 15 such averaged values is reported as the number of cycles. The cycles per byte value is obtained by dividing the median value by the length of the corresponding message.

Table 8.5.2: Performance comparison between sequential implementations of OCB3-AES-128, $\text{GOCB}_{\text{AES-128,A1}}$, and $\text{GOCB}_{\text{AES-128,A2}}$. The performance figures presented are throughputs, in units of cycles-per-byte (CPB).

Length	OCB3-AES-128	$\text{GOCB}_{\text{AES-128,A1}}$	$\text{GOCB}_{\text{AES-128,A2}}$
128 bytes	1.48	2.11	2.15
256 bytes	1.15	1.41	1.45
512 bytes	0.85	1.07	1.10
1024 bytes	0.75	0.90	0.93
4096 bytes	0.68	0.77	0.81

Table 8.5.3: Summary of IPI values for OCB3-AES-128, $\text{GOCB}_{\text{AES-128,A1}}$, and $\text{GOCB}_{\text{AES-128,A2}}$.

	OCB3-AES-128	$\text{GOCB}_{\text{AES-128,A1}}$	$\text{GOCB}_{\text{AES-128,A2}}$
IPI:	1.05	1.24	1.28

SEQUENTIAL PROCESSING: RESULTS AND DISCUSSION — Table 8.5.2 summarizes the CPB values for OCB3-AES-128, $\text{GOCB}_{\text{AES-128,A1}}$, and $\text{GOCB}_{\text{AES-128,A2}}$, for moderate to large message lengths, in sequential processing. Following [119], we also present IPI (Internet Performance Index) values — a weighted average CPB value for usual message lengths observed over internet — in Table 8.5.3, for OCB3-AES-128, $\text{GOCB}_{\text{AES-128,A1}}$, and $\text{GOCB}_{\text{AES-128,A2}}$. As per our experiments, OCB3 with 10 rounds of AES-128 is approximately, 9% and 10% faster than OCB3 with 11 and 12 rounds, respectively, of AES-128. One may think that $\text{GOCB}_{\text{AES-128,A1}}$ and $\text{GOCB}_{\text{AES-128,A2}}$ should have similar degradation in performance. But the IPI clearly indicates that the A1 and A2 variants are much more slower, approximately 18% and 22%, respectively, than OCB3 over usual internet data. This is mainly due to the lower masking (just after encryption), which is the contentious instruction that breaks the pipelining benefits.

However, the important point to note is that for large messages (i.e. ≥ 4096 bytes), which is our main area of focus, the CPB values for $\text{GOCB}_{\text{AES-128,A1}}$ and $\text{GOCB}_{\text{AES-128,A2}}$

tend towards the CPB values for OCB3-AES-128 with 11 and 12 rounds, respectively. Specifically, we have observed that OCB3-AES-128 with 11 and 12 rounds have 0.74 and 0.81 CPBs, respectively, for 4096 bytes message. This is quite close to the CPB values achieved by $\text{GOCB}_{\text{AES-128,A1}}$ and $\text{GOCB}_{\text{AES-128,A2}}$, i.e. 0.77 and 0.81, respectively. Beyond this point the CPB values for $\text{GOCB}_{\text{AES-128,A1}}$ and $\text{GOCB}_{\text{AES-128,A2}}$ are expected to saturate and follow the CPB values of OCB3-AES-128 with 11 and 12 rounds, respectively. In other words, for messages with length ≥ 4096 bytes, $\text{GOCB}_{\text{AES-128,A1}}$ and $\text{GOCB}_{\text{AES-128,A2}}$ are only $\approx 10\%$ and $\approx 20\%$ slower than OCB3.

Table 8.5.4: Summary of CPB values for OCB3-AES-128, $\text{GOCB}_{\text{AES-128,A1}}$, and $\text{GOCB}_{\text{AES-128,A2}}$ to process a single block of data in random access.

OCB3-AES-128	$\text{GOCB}_{\text{AES-128,A1}}$	$\text{GOCB}_{\text{AES-128,A2}}$
3.93	3.12	3.29

RANDOM ACCESS: RESULTS AND DISCUSSION — To the best of our knowledge, optimized 128-bit field multiplication algorithms (using `pclmulqdq`) are very close to 5 rounds of AES (using AES-NI) in terms of performance. Thus, theoretically one would expect that $\text{GOCB}_{\text{AES-128,A1}}$, and $\text{GOCB}_{\text{AES-128,A2}}$ will have significantly better random access performance as compared to OCB3, which requires a 128-bit field multiplication.

$\text{GOCB}_{\text{AES-128,A1}}$ and $\text{GOCB}_{\text{AES-128,A2}}$ would require close to 44 and 48 cycles, respectively, for AES round calls, as 1-round AES call requires 4 cycles (latency) in Skylake. Further, roughly 4 cycles are required to create the initial masking state. So, theoretically $\text{GOCB}_{\text{AES-128,A1}}$ and $\text{GOCB}_{\text{AES-128,A2}}$ are expected to have approximately 3-3.1 and 3.2-3.3 CPB. On the other hand, OCB3 requires close to 60 cycles for field multiplication and the AES call. Apart from this approx. 3 cycles are required for gray code generation. So, OCB3 is expected to have around 3.9-4 CPB. In other words, $\text{GOCB}_{\text{AES-128,A1}}$ and $\text{GOCB}_{\text{AES-128,A2}}$ are expected to be close to 25% and 20%, respectively, better than OCB3 in terms of random access. Note that, we have ignored the overhead due to nonce processing. This is reasonable given the assumption that random read/write does not span across different messages.

Table 8.5.4 summarizes the CPB values to process an arbitrary (random access) block using OCB3-AES-128, $\text{GOCB}_{\text{AES-128,A1}}$, and $\text{GOCB}_{\text{AES-128,A2}}$. Clearly, the experimental data justifies the theoretical speedup described above.

Appendix A

Graphs and Probability Theory

A.1 Directed Edge-Labeled Graphs

DIRECTED EDGE-LABELED GRAPH: A directed edge-labeled graph is a pair $\mathcal{G} := (\mathcal{V}, \mathcal{E})$ with $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V} \times \mathcal{L}$ where $\mathcal{V}$ is the set of vertices, $\mathcal{L}$ is the set of edge labels, and $\mathcal{E}$ is the set of edges along with their corresponding labels. We write $u \xrightarrow{a} v$ to mean that $(u, v, a) \in \mathcal{E}$. By a slight abuse of notation, $\mathcal{E}$ also denotes the set of unlabeled edges. For any graph $\mathcal{G}$, we write $\mathcal{V}(\mathcal{G})$ and $\mathcal{E}(\mathcal{G})$ to denote the vertex set and edge set of $\mathcal{G}$.

An *undirected (edge-labeled) graph* is a special case of directed (edge-labeled) graph where $\mathcal{E}$ is a set of two-sets,¹ i.e., the edges (u, v) and (v, u) are denoted by a two-set $\{u, v\}$. For brevity, we extend the notation of ordered edge (u, v) for undirected graphs as well.

When viewed as the set of unlabeled edges, $\mathcal{E}$ is a multiset. Recall that μ denotes the multiplicity function 2.1.

We say, that a graph has *parallel edges* between two vertices u and v , if $(u, v) \in \mathcal{E}$ and $\mu(\mathcal{E}, (u, v)) \geq 2$. We say, that a graph has *self loop* on vertex u , if $(u, u) \in \mathcal{E}$.

SIMPLE GRAPH: A graph $\mathcal{G}$ is called simple if $\mathcal{G}$ has no parallel edges and self loops.

A graph $\mathcal{H}$ is called the *subgraph* of another graph $\mathcal{G}$ if $\mathcal{V}(\mathcal{H}) \subseteq \mathcal{V}(\mathcal{G})$ and $\mathcal{E}(\mathcal{H}) \subseteq \mathcal{E}(\mathcal{G})$.

Definition A.1.1 (Isomorphism). Let $\mathcal{G}_1 = (\mathcal{V}_1, \mathcal{E}_1)$ and $\mathcal{G}_2 = (\mathcal{V}_2, \mathcal{E}_2)$ be two directed edge-labeled graphs. A function $\alpha : \mathcal{V}_1 \rightarrow \mathcal{V}_2$ is an isomorphism from $\mathcal{G}_1$ to $\mathcal{G}_2$ if α is a bijection and $(u, v, a) \in \mathcal{E}_1$ if and only if $(\alpha(u), \alpha(v), a) \in \mathcal{E}_2$. In this case, we write $\mathcal{G}_1 \cong \mathcal{G}_2$.

¹A set with cardinality 2.

A.1.1 Degree of a Vertex

For a directed edge $e := (u, v)$, vertex u is called the *predecessor* of v , and v is called the *successor* of u . For all vertex v , we define two sets:

1. Predecessor set of a vertex v is $\text{nbd}(* \rightarrow v) := \{u : (u, v) \in \mathcal{E}\}$.
2. Successor set of a vertex v is $\text{nbd}(v \rightarrow *) := \{u : (v, u) \in \mathcal{E}\}$.

By extending the definitions to undirected graph, we see that $\text{nbd}(* \rightarrow v) = \text{nbd}(v \rightarrow *)$ (the order does not matter).

Definition A.1.2 (Degree). In-degree $\text{deg}_{\text{in}}(v)$ of a vertex v is defined as $\text{deg}_{\text{in}}(v) = |\text{nbd}(* \rightarrow v)|$. Similarly, out-degree $\text{deg}_{\text{out}}(v)$ is defined as $\text{deg}_{\text{out}}(v) = |\text{nbd}(v \rightarrow *)|$.

In a directed graph $\mathcal{G}$, degree $\text{deg}(v)$ of $v \in \mathcal{V}(\mathcal{G})$ is defined as the sum of $\text{deg}_{\text{in}}(v)$ and $\text{deg}_{\text{out}}(v)$, i.e., $\text{deg}(v) = \text{deg}_{\text{in}}(v) + \text{deg}_{\text{out}}(v)$.

In an undirected graph $\mathcal{G}$, degree $\text{deg}(v)$ of $v \in \mathcal{V}(\mathcal{G})$ is defined as $\text{deg}(v) = \text{deg}_{\text{in}}(v) = \text{deg}_{\text{out}}(v)$.

A.1.2 Walk, Path, Cycle and Component

We define these terms for directed edge-labeled graphs. They can be analogously defined for undirected graphs as well.

Definition A.1.3 (Walk). A *walk of length s* is defined as a vertex sequence $W := (W_0, \dots, W_s)$, such that $W_{i-1} \xrightarrow{a_i} W_i$ for all $i \in [s]$. The sequence $a^s = (a_1, \dots, a_s)$ is called the label of W . We also say that W is an a^s -walk. Any *subwalk* of $W^{(s)}$ is a contiguous subsequence $W^{[a \dots b]}$ where $a \leq b \in [s]$.

A walk $W^{(s)}$ is said to be: a *path* if it has $s + 1$ distinct vertices, i.e., $W_i \neq W_j$ for all $i < j \in [s]$; a *cycle* if it has at most s distinct nodes and $W_s = W_0$. Note that, a self loop (or simply a loop) is also a cycle with $s = 1$.

A graph $\mathcal{G}$ is called *connected* if for any two distinct vertices $u, v \in \mathcal{V}(\mathcal{G})$, there exists a path $W^{(s)}$ such that $W_0 = u$ and $W_s = v$. Any connected subgraph $\mathcal{C}$ of $\mathcal{G}$ is called a *connected component* (or simply component).

A.2 Some Basic Results in Probability Theory

In the following, we assume reader's acquaintance with basic discrete probability theory. See Feller [69] for a primer on basic probability theory.

A.2.1 Bonferroni's Inequalities

For a finite² collection of events $A_1, A_2, \dots, A_n$, we define $S_1 := \sum_{i \in [n]} \Pr[A_i]$, $S_2 := \sum_{i < j \in [n]} \Pr[A_i \cap A_j]$, and $S_k := \sum_{i_1 < \dots < i_k \in [n]} \Pr[A_{i_1} \cap \dots \cap A_{i_k}]$ for all $k \in [3 \dots n]$. Then, for odd $k \in [n]$, we have

$$\Pr \left[\bigcup_{i \in [n]} A_i \right] \leq \sum_{j \in [k]} (-1)^{j-1} S_j,$$

and for even $k \in [2 \dots n]$, we have

$$\Pr \left[\bigcup_{i \in [n]} A_i \right] \geq \sum_{j \in [k]} (-1)^{j-1} S_j.$$

The equality holds for $k = n$, and the resulting identity is the principle of inclusion-exclusion.

²The inequalities hold for countable collections. However, for our uses just the finite case will suffice.

Bibliography

- [1] Farzaneh Abed, Scott R. Fluhrer, Christian Forler, Eik List, Stefan Lucks, David A. McGrew, and Jakob Wenzel. Pipelineable on-line encryption. In *Fast Software Encryption – FSE '14, Revised Selected Papers*, pages 205–223, 2014.
- [2] Martin Ågren, Martin Hell, Thomas Johansson, and Willi Meier. Grain-128a: a new version of Grain-128 with optional authentication. *IJWMC*, 5(1):48–59, 2011.
- [3] Elena Andreeva, Andrey Bogdanov, Atul Luykx, Bart Mennink, Elmar Tischhauser, and Kan Yasuda. Parallelizable and authenticated online ciphers. In *Advances in Cryptology – ASIACRYPT '13, Proceedings, Part I*, pages 424–443, 2013.
- [4] Elena Andreeva, Atul Luykx, Bart Mennink, and Kan Yasuda. COBRA: A parallelizable authenticated online cipher without block cipher inverse. In *Fast Software Encryption – FSE '14, Revised Selected Papers*, pages 187–204, 2014.
- [5] Elena Andreeva, Andrey Bogdanov, Nilanjan Datta, Atul Luykx, Bart Mennink, Mridul Nandi, Elmar Tischhauser, and Kan Yasuda. COLM. Specification document v1, CAESAR Submission, 2016. Online: <https://competitions.cr.yp.to/round3/colmv1.pdf> (Accessed: 02 September, 2019).
- [6] Elena Andreeva, Charles Bouillaguet, Orr Dunkelman, Pierre-Alain Fouque, Jonathan J. Hoch, John Kelsey, Adi Shamir, and Sébastien Zimmer. New second-preimage attacks on hash functions. *J. Cryptol.*, 29:657–696, 2016.
- [7] Elena Andreeva, Guy Barwell, Ritam Bhaumik, Mridul Nandi, Dan Page, and Martijn Stam. Turning online ciphers off. *IACR Trans. Symmetric Cryptol.*, 2017(2), 2017.
- [8] Subhadeep Banik, Sumit Kumar Pandey, Thomas Peyrin, Yu Sasaki, Siang Meng Sim, and Yosuke Todo. GIFT: A small present - towards reaching the limit of lightweight encryption. In *Cryptographic Hardware and Embedded Systems – CHES '17, Proceedings*, pages 321–345, 2017.

- [9] Subhadeep Banik, Andrey Bogdanov, Atul Luykx, and Elmar Tischhauser. SUN-DAE: small universal deterministic authenticated encryption for the internet of things. *IACR Trans. Symmetric Cryptol.*, 2018(3):1–35, 2018.
- [10] M. Baugher, D. McGrew, M. Naslund, E. Carrara, and K. Norrman. The secure real-time transport protocol (SRTP). RFC 3711, Internet Engineering Task Force (IETF), 2004. Online: <https://tools.ietf.org/html/rfc3711> (Accessed: 02 May, 2020).
- [11] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The Simon and Speck Block Ciphers on AVR 8-Bit Microcontrollers. In *Lightweight Cryptography for Security and Privacy – LightSec ’14, Revised Selected Papers*, pages 3–20, 2014.
- [12] Christof Beierle, J  r  my Jean, Stefan K  lbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. In *Advances in Cryptology – CRYPTO ’16, Proceedings, Part II*, pages 123–153, 2016.
- [13] Mihir Bellare. Practice-oriented provable security. In *Lectures on Data Security, Modern Cryptology in Theory and Practice, Summer School*, pages 1–15, 1998.
- [14] Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In *Advances in Cryptology – EURO-CRYPT ’06, Proceedings*, pages 409–426, 2006.
- [15] Mihir Bellare, Joe Kilian, and Phillip Rogaway. The security of cipher block chaining. In *Advances in Cryptology – CRYPTO ’94, Proceedings*, pages 341–358, 1994.
- [16] Mihir Bellare, Roch Gu  rin, and Phillip Rogaway. XOR macs: New methods for message authentication using finite pseudorandom functions. In *Advances in Cryptology – CRYPTO ’95, Proceedings*, pages 15–28, 1995.
- [17] Mihir Bellare, Oded Goldreich, and Hugo Krawczyk. Stateless evaluation of pseudorandom functions: Security beyond the birthday barrier. In *Advances in Cryptology – CRYPTO ’99, Proceedings*, pages 270–287, 1999.
- [18] Mihir Bellare, Alexandra Boldyreva, Lars R. Knudsen, and Chanathip Namprempre. Online ciphers and the hash-CBC construction. In *Advances in Cryptology – CRYPTO 2001, Proceedings*, pages 292–309, 2001.
- [19] Mihir Bellare, Krzysztof Pietrzak, and Phillip Rogaway. Improved Security Analyses for CBC MACs. In *Advances in Cryptology – CRYPTO ’05, Proceedings*, pages 527–545, 2005.

- [20] Mihir Bellare, Viet Tung Hoang, and Sriram Keelveedhi. Cryptography from compression functions: The UCE bridge to the ROM. In *Advances in Cryptology – CRYPTO '14, Proceedings, Part I*, pages 169–187, 2014.
- [21] A. Berendschot, B. den Boer, J. Boly, A. Bosselaers, J. Brandt, D. Chaum, I. Damgård, M. Dichtl, W. Fumy, M. van der Ham, C. Jansen, P. Landrock, B. Preneel, G. Roelofsen, P. de Rooij, and J Vandewalle. Final Report of RACE Integrity Primitives. *Lecture Notes in Computer Science, Springer-Verlag*, 1995, 1007, 1995.
- [22] Robert Berke. On the security of iterated macs. Diploma thesis, ETH Zurich, 2003.
- [23] Daniel J. Bernstein. How to stretch random functions: The security of protected counter sums. *J. Cryptol.*, 12(3):185–192, 1999.
- [24] Daniel J. Bernstein. The poly1305-aes message-authentication code. In *Fast Software Encryption – FSE '05, Revised Selected Papers*, pages 32–49, 2005.
- [25] Daniel J. Bernstein. Stronger Security Bounds for Wegman-Carter-Shoup Authenticators. In *Advances in Cryptology – EUROCRYPT '05, Proceedings*, pages 164–180, 2005.
- [26] Daniel J Bernstein. Cache-timing attacks on AES. Online: <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>, 2005. (Accessed: 02 September, 2019).
- [27] Daniel J. Bernstein. A short proof of the unpredictability of cipher block chaining. Online: <https://cr.yp.to/antiforgery/easycbc-20050109.pdf>, 2005. (Accessed: 02 September, 2019).
- [28] Daniel J. Bernstein. Polynomial evaluation and message authentication. Online: <http://cr.yp.to/antiforgery/pema-20071022.pdf>, 2007. (Accessed: 02 September, '19).
- [29] Ritam Bhaumik and Mridul Nandi. OleF: an inverse-free online cipher. an online SPRP with an optimal inverse-free construction. *IACR Trans. Symmetric Cryptol.*, 2016(2):30–51, 2016.
- [30] Ritam Bhaumik and Mridul Nandi. Improved security for OCB3. In *Advances in Cryptology – ASIACRYPT '17, Proceedings, Part II*, pages 638–666, 2017.
- [31] Eli Biham and Orr Dunkelman. A Framework for Iterative Hash Functions - HAIFA. *IACR Cryptology ePrint Archive*, 2007(278), 2007. Online: <https://eprint.iacr.org/2007/278.pdf> (Accessed: 02 May, 2020).

- [32] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. In *Advances in Cryptology – CRYPTO '90, Proceedings*, pages 2–21, 1990.
- [33] John Black and Martin Cochran. MAC reforgeability. In *Fast Software Encryption – FSE '09, Proceedings*, pages 345–362, 2009.
- [34] John Black and Phillip Rogaway. CBC macs for arbitrary-length messages: The three-key constructions. In *Advances in Cryptology - CRYPTO '00, Proceedings*, pages 197–215, 2000.
- [35] John Black and Phillip Rogaway. A block-cipher mode of operation for parallelizable message authentication. In *Advances in Cryptology – EUROCRYPT '02, Proceedings*, pages 384–397, 2002.
- [36] John Black, Shai Halevi, Hugo Krawczyk, Ted Krovetz, and Phillip Rogaway. UMAC: fast and secure message authentication. In *Advances in Cryptology – CRYPTO '99, Proceedings*, pages 216–233, 1999.
- [37] Manuel Blum and Silvio Micali. How to generate cryptographically strong sequences of pseudo random bits. In *Foundations of Computer Science – FOCS '82, Proceedings*, pages 112–117, 1982.
- [38] Andrey Bogdanov, Lars R. Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, Yannick Seurin, and C. Viskelson. PRESENT: an ultra-lightweight block cipher. In *Cryptographic Hardware and Embedded Systems – CHES '07, Proceedings*, pages 450–466, 2007.
- [39] Alexandra Boldyreva and Nut Taesombut. Online encryption schemes: New security notions and constructions. In *Topics in Cryptology – CT-RSA '04, Proceedings*, pages 1–14, 2004.
- [40] Lilian Bossuet, Nilanjan Datta, Cuauhtemoc Mancillas-López, and Mridul Nandi. ELmD: A pipelineable authenticated encryption and its hardware implementation. *IEEE Trans. Computers*, 65(11):3318–3331, 2016.
- [41] Charles Bouillaguet and Pierre-Alain Fouque. Practical Hash Functions Constructions Resistant to Generic Second Preimage Attacks Beyond the Birthday Bound, Submitted for publication, (2010).
- [42] CAESAR. Competition for Authenticated Encryption: Security, Applicability, and Robustness. Online: <https://competitions.cr.yp.to/caesar.html>, 2014. (Accessed: 02 May, 2020).
- [43] Larry Carter and Mark N. Wegman. Universal classes of hash functions. *J. Comput. Syst. Sci.*, 18(2):143–154, 1979.

- [44] Avik Chakraborti, Nilanjan Datta, Mridul Nandi, and Kan Yasuda. Beetle family of lightweight and secure authenticated encryption ciphers. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(2):218–241, 2018.
- [45] Debrup Chakraborty and Palash Sarkar. A general construction of tweakable block ciphers and different modes of operations. *IEEE Trans. Information Theory*, 54(5):1991–2006, 2008.
- [46] Debrup Chakraborty, Sebati Ghosh, and Palash Sarkar. A fast single-key two-level universal hash function. *IACR Trans. Symmetric Cryptol.*, 2017(1):106–128, 2017.
- [47] Donghoon Chang and Mridul Nandi. A Short Proof of the PRP/PRF Switching Lemma. *IACR Cryptology ePrint Archive*, 2008(78), 2008. Online: <https://eprint.iacr.org/2008/78.pdf> (Accessed: 02 May, 2020).
- [48] Shan Chen and John P. Steinberger. Tight security bounds for key-alternating ciphers. In *Advances in Cryptology – EUROCRYPT ’14, Proceedings*, pages 327–350, 2014.
- [49] Benoît Cogliati and Yannick Seurin. EWCDM: an efficient, beyond-birthday secure, nonce-misuse resistant MAC. In *Advances in Cryptology – CRYPTO ’16, Proceedings*, pages 121–149, 2016.
- [50] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-Damgård Revisited: How to Construct a Hash Function. In *Advances in Cryptology – CRYPTO ’05, Proceedings*, pages 430–448, 2005.
- [51] Jean-Sébastien Coron, Jacques Patarin, and Yannick Seurin. The random oracle model and the ideal cipher model are equivalent. In *Advances in Cryptology – CRYPTO ’08, Proceedings*, pages 1–20, 2008.
- [52] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Information Security and Cryptography. Springer, 2002.
- [53] Joan Daemen and Vincent Rijmen. Understanding two-round differentials in AES. In *Security and Cryptography for Networks – SCN ’06, Proceedings*, pages 78–94, 2006.
- [54] Wei Dai, Viet Tung Hoang, and Stefano Tessaro. Information-theoretic indistinguishability via the chi-squared method. In *Advances in Cryptology – CRYPTO ’17, Proceedings, Part III*, pages 497–523, 2017.
- [55] Ivan Damgård. A design principle for hash functions. In *Advances in Cryptology – CRYPTO ’89, Proceedings*, pages 416–427, 1989.

- [56] Nilanjan Datta and Mridul Nandi. ELM_E: A misuse resistant parallel authenticated encryption. In *Information Security and Privacy – ACISP '14, Proceedings*, pages 306–321, 2014.
- [57] Nilanjan Datta, Avijit Dutta, Mridul Nandi, and Goutam Paul. Double-block hash-then-sum: A paradigm for constructing secure prf. *IACR Trans. Symmetric Cryptol.*, 2018(3):36–92, 2018.
- [58] Nilanjan Datta, Avijit Dutta, Mridul Nandi, and Kan Yasuda. Encrypt or decrypt? to make a single-key beyond birthday secure nonce-based MAC. In *Advances in Cryptology – CRYPTO '18, Proceedings, Part I*, pages 631–661, 2018.
- [59] R.D. Dean. *Formal Aspects of Mobile Code Security*. PhD thesis, Princeton University, 1999.
- [60] Whitfield Diffie and Martin E. Hellman. New Directions in Cryptography. *IEEE Trans. Information Theory*, 22(6):644–654, 1976.
- [61] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  fer. Ascon. Specification document v1.2, CAESAR Submission, 2016. Online: <https://competitions.cr.yp.to/round3/asconv12.pdf> (Accessed: 02 September, 2019).
- [62] Yevgeniy Dodis, Rosario Gennaro, Johan H  stad, Hugo Krawczyk, and Tal Rabin. Randomness extraction and key derivation using the CBC, cascade and HMAC modes. In *Advances in Cryptology – CRYPTO '04, Proceedings*, pages 494–510, 2004.
- [63] Avijit Dutta, Ashwin Jha, and Mridul Nandi. A new look at counters: Don't run like marathon in a hundred meter race. *IEEE Trans. Computers*, 66(11):1851–1864, 2017.
- [64] Avijit Dutta, Mridul Nandi, and Suprita Talnikar. Beyond birthday bound secure MAC in faulty nonce model. In *Advances in Cryptology – EUROCRYPT '19, Proceedings, Part I*, pages 437–466, 2019.
- [65] William F. Ehrsam, Carl H. W. Meyer, John L. Smith, and Walter L. Tuchman. Message Verification and Transmission Error Detection by Block Chaining. Patent 4074066, USPTO, 1976.
- [66] Peter Elias. Minimum times and memories needed to compute the values of a function. *J. Comput. Syst. Sci.*, 9:196–212, 1974.
- [67] Peter Elias. Universal codeword sets and representations of the integers. *IEEE Trans. Information Theory*, 21(2):194–203, 1975.

- [68] Robert M. Fano. The transmission of information. Technical Report 65, Research Laboratory of Electronics at MIT, 1949. Online: <https://hcs64.com/files/fano-tr65-ocr-only.pdf> (Accessed: 02 May, 2020).
- [69] William Feller. *An Introduction to Probability Theory and Its Applications*. Wiley, 1970.
- [70] Ewan Fleischmann, Christian Forler, and Stefan Lucks. Mcoe: A family of almost foolproof on-line authenticated encryption schemes. In *Fast Software Encryption – FSE ’12, Revised Selected Papers*, pages 196–215, 2012.
- [71] Christian Forler, Eik List, Stefan Lucks, and Jakob Wenzel. POEx: A beyond-birthday-bound-secure on-line cipher. ArcticCrypt ’16, 2016. Online: https://www.researchgate.net/publication/299565944_POEx_A_Beyond-Birthday-Bound-Secure_On-Line_Cipher (Accessed: 02 May, 2020).
- [72] Christian Forler, Eik List, Stefan Lucks, and Jakob Wenzel. Poex: A beyond-birthday-bound-secure on-line cipher. *Cryptogr. Commun.*, 10(1):177–193, 2018.
- [73] Aviezri S. Fraenkel and Shmuel T. Klein. Robust universal complete codes for transmission and compression. *Discrete Applied Mathematics*, 64:31–55, 1996.
- [74] Peter Gazi, Krzysztof Pietrzak, and Michal Rybár. The exact security of PMAC. *IACR Trans. Symmetric Cryptol.*, 2016(2):145–161, 2016.
- [75] Edgar N. Gilbert, F. Jessie MacWilliams, and Neil J. A. Sloane. Codes which detect deception. *Bell System Technical Journal*, 53:405–424, 1974.
- [76] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *Annual Symposium on Foundations of Computer Science – FOCS ’84, Proceedings*, pages 464–479, 1984.
- [77] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. On the cryptographic applications of random functions. In *Advances in Cryptology – CRYPTO ’84, Proceedings*, pages 276–288, 1984.
- [78] Shafi Goldwasser and Mihir Bellare. Lecture Notes on Cryptography. Online: <https://cseweb.ucsd.edu/~mihir/papers/gb.pdf>, 2008. (Accessed: 02 May, 2020).
- [79] Shafi Goldwasser and Silvio Micali. Probabilistic encryption and how to play mental poker keeping secret all partial information. In *ACM Symposium on Theory of Computing – STOC ’82, Proceedings*, pages 365–377, 1982.

- [80] Sergey Gorbunov and Charles Rackoff. On the security of cipher block chaining message authentication code. Online: <https://cs.uwaterloo.ca/~sgorbuno/publications/securityOfCBC.pdf>, 2016. (Accessed: 02 September, 2019).
- [81] Robert Granger, Philipp Jovanovic, Bart Mennink, and Samuel Neves. Improved masking for tweakable blockciphers with applications to authenticated encryption. In *Advances in Cryptology – EUROCRYPT ’16, Proceedings, Part I*, pages 263–293, 2016.
- [82] Shay Gueron. Intel® Advanced Encryption Standard (AES) new instructions set. White paper, Intel Corporation, 2010. Online: <https://www.intel.com/content/dam/doc/white-paper/advanced-encryption-standard-new-instructions-set-paper.pdf> (Accessed: 02 May, 2020).
- [83] Shay Gueron and Michael E. Kounavis. Intel® carry-less multiplication instruction and its usage for computing the GCM mode. White paper, Intel Corporation, 2011. Online: <https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/carry-less-multiplication-instruction-in-gcm-mode-paper.pdf> (Accessed: 02 May, 2020).
- [84] Shay Gueron and Yehuda Lindell. GCM-SIV: full nonce misuse-resistant authenticated encryption at under one cycle per byte. In *ACM Conference on Computer and Communications Security - CCS ’15, Proceedings*, pages 109–119, 2015.
- [85] Chun Guo and Lei Wang. Revisiting key-alternating feistel ciphers for shorter keys and multi-user security. *IACR Cryptology ePrint Archive*, 2018(816), 2018. Online: <https://eprint.iacr.org/2018/816> (Accessed: 02 May, 2020).
- [86] Jian Guo, Jérémy Jean, Nicky Mouha, and Ivica Nikolic. More rounds, less security? *IACR Cryptology ePrint Archive*, 2015(484), 2015. Online: <https://eprint.iacr.org/2015/484> (Accessed: 02 May, 2020).
- [87] Shai Halevi and Hugo Krawczyk. MMH: software message authentication in the gbit/second rates. In *Fast Software Encryption – FSE ’97, Proceedings*, pages 172–189, 1997.
- [88] Shai Halevi and Phillip Rogaway. A tweakable enciphering mode. In *Advances in Cryptology – CRYPTO ’03, Proceedings*, pages 482–499, 2003.
- [89] Shai Halevi and Phillip Rogaway. A parallelizable enciphering mode. In *Topics in Cryptology – CT-RSA ’04, Proceedings*, pages 292–304, 2004.

- [90] Viet Tung Hoang and Stefano Tessaro. Key-alternating ciphers and key-length extension: Exact bounds and multi-user security. In *Advances in Cryptology – CRYPTO '16, Proceedings, Part I*, pages 3–32, 2016.
- [91] Viet Tung Hoang and Stefano Tessaro. The multi-user security of double encryption. In *Advances in Cryptology – EUROCRYPT '17, Proceedings, Part II*, pages 381–411, 2017.
- [92] David A. Huffman. A method for the construction of minimum-redundancy codes. *IRE, Proceedings*, 40:1098–1101, 1952.
- [93] K Iggoe and J Solinas. AES Galois Counter Mode for the Secure Shell Transport Layer Protocol. RFC 5647, Internet Engineering Task Force (IETF), 2009. Online: <https://tools.ietf.org/html/rfc5647> (Accessed: 02 May, 2020).
- [94] Intel[®] SSE4 (AES) Programming Reference. Intel Corporation, 2007.
- [95] ISO/IEC. Information Technology – Security Techniques – Message Authentication Codes (MACs) – Part 1: Mechanisms Using A Block Cipher. International Standard ISO/IEC 9797-1:2011, International Organization for Standardization, 2011.
- [96] ISO/IEC. Information technology – Digital compression and coding of continuous-tone still images: JPEG File Interchange Format (JFIF). International Standard ISO/IEC 10918-5:2013, International Organization for Standardization, 2013.
- [97] Tetsu Iwata and Kaoru Kurosawa. OMAC: One-Key CBC MAC. In *Fast Software Encryption - FSE '03, Revised Papers*, pages 129–153, 2003.
- [98] Tetsu Iwata and Kaoru Kurosawa. Stronger Security Bounds for OMAC, TMAC, and XCBC. In *Progress in Cryptology – INDOCRYPT '03, Proceedings*, pages 402–415, 2003.
- [99] Tetsu Iwata and Kazuhiko Minematsu. Stronger security variants of GCM-SIV. *IACR Trans. Symmetric Cryptol.*, 2016(1):134–157, 2016.
- [100] Goce Jakimoski and K. P. Subbalakshmi. On efficient message authentication via block cipher design techniques. In *Advances in Cryptology – ASIACRYPT '07, Proceedings*, pages 232–248, 2007.
- [101] Éliane Jaulmes and Reynald Lercier. FRMAC, a proc. fast randomized message authentication code. *IACR Cryptology ePrint Archive*, 2004(166), 2004. Online: <https://eprint.iacr.org/2004/166.pdf> (Accessed: 02 May, 2020).

- [102] Éliane Jaulmes, Antoine Joux, and Frédéric Valette. On the Security of Randomized CBC-MAC Beyond the Birthday Paradox Limit: A New Construction. In *Fast Software Encryption – FSE ’02, Proceedings*, pages 237–251, 2002.
- [103] Jérémy Jean, Ivica Nikolic, and Thomas Peyrin. Tweaks and keys for block ciphers: The TWEAKEY framework. In *Advances in Cryptology – ASIACRYPT ’14, Proceedings, Part II*, pages 274–288, 2014.
- [104] Ashwin Jha and Mridul Nandi. Revisiting Structure Graphs: Applications to CBC-MAC and EMAC. *J. Mathematical Cryptol.*, 10(3–4):157–180, 2016.
- [105] Ashwin Jha and Mridul Nandi. On rate-1 and beyond-the-birthday bound secure online ciphers using tweakable block ciphers. *Cryptography and Communications*, 10(5):731–753, 2018.
- [106] Ashwin Jha and Mridul Nandi. A survey on applications of H-technique: Revisiting security analysis of PRP and PRF. *IACR Cryptology ePrint Archive*, 2018 (1130), 2018. Online: <https://eprint.iacr.org/2018/1130> (Accessed: 02 May, 2020).
- [107] Ashwin Jha and Mridul Nandi. Tight security of cascaded LRW2. *J. Cryptol.*, 2020.
- [108] Ashwin Jha, Eik List, Kazuhiko Minematsu, Sweta Mishra, and Mridul Nandi. XHX - A framework for optimally secure tweakable block ciphers from classical block ciphers and universal hashing. In *Progress in Cryptology – LATINCRYPT ’17, Revised Selected Papers*, pages 207–227, 2017.
- [109] Ashwin Jha, Avradip Mandal, and Mridul Nandi. On the exact security of message authentication using pseudorandom functions. *IACR Trans. Symmetric Cryptol.*, 2017(1):427–448, 2017.
- [110] Ashwin Jha, Cuauhtemoc Mancillas-López, Mridul Nandi, and Sourav Sen Gupta. On random read access in OCB. *IEEE Trans. Information Theory*, 65(12):8325–8344, 2019.
- [111] I. Johansson and M. Westerlund. Support for reduced-size real-time transport control protocol (RTCP): Opportunities and consequences. RFC 5506, Internet Engineering Task Force (IETF), 2009. Online: <https://tools.ietf.org/html/rfc5506> (Accessed: 02 May, 2020).
- [112] Charanjit S. Jutla. PRF domain extension using dags. In *Theory of Cryptography, – TCC ’06, Proceedings*, pages 561–580, 2006.

- [113] David Kahn. *The Codebreakers: The Comprehensive History of Secret Communication from Ancient Times to the Internet*. Scribner, 1996.
- [114] Liam Keliher. Refined analysis of bounds related to linear and differential cryptanalysis for the AES. In *Advanced Encryption Standard – AES '04, Revised Selected and Invited Papers*, pages 42–57, 2004.
- [115] Liam Keliher and Jiayuan Sui. Exact maximum expected differential and linear probability for two-round Advanced Encryption Standard. *IET Information Security*, 1(2):53–57, 2007.
- [116] John Kelsey and Bruce Schneier. Second preimages on n -bit hash functions for much less than 2^n work. In *Advances in Cryptology – EUROCRYPT '05, Proceedings*, pages 474–490, 2005.
- [117] Hugo Krawczyk. LFSR-based hashing and authentication. In *Advances in Cryptology – CRYPTO '94, Proceedings*, pages 129–139, 1994.
- [118] Ted Krovetz. Message authentication on 64-bit architectures. In *Selected Areas in Cryptography – SAC '06, Revised Selected Papers*, pages 327–341, 2006.
- [119] Ted Krovetz and Phillip Rogaway. The software performance of authenticated-encryption modes. In *Fast Software Encryption – FSE '11, Revised Selected Papers*, pages 306–327, 2011.
- [120] Ted Krovetz and Phillip Rogaway. The OCB authenticated-encryption algorithm. RFC 7253, Internet Engineering Task Force (IETF), 2014. Online: <https://tools.ietf.org/html/rfc7253> (Accessed: 02 May, 2020).
- [121] Kaoru Kurosawa and Tetsu Iwata. TMAC: Two-key cbc mac. In *Topics in Cryptology - CT-RSA '03, Proceedings*, pages 33–49, 2003.
- [122] Rodolphe Lampe and Yannick Seurin. Tweakable blockciphers with asymptotically optimal security. In *Fast Software Encryption – FSE '13, Revised Selected Papers*, pages 133–151, 2013.
- [123] Will Landecker, Thomas Shrimpton, and R. Seth Terashima. Tweakable blockciphers with beyond birthday-bound security. In *Advances in Cryptology – CRYPTO '12, Proceedings*, pages 14–30, 2012.
- [124] J. Lennox. Encryption of header extensions in the secure real-time transport protocol (SRTP). RFC 6904, Internet Engineering Task Force (IETF), 2013. Online: <https://tools.ietf.org/html/rfc6904> (Accessed: 02 May, 2020).

- [125] Moses Liskov, Ronald L. Rivest, and David A. Wagner. Tweakable block ciphers. In *Advances in Cryptology – CRYPTO '02, Proceedings*, pages 31–46, 2002.
- [126] Atul Luykx, Bart Preneel, Elmar Tischhauser, and Kan Yasuda. A MAC mode for lightweight block ciphers. In *Fast Software Encryption – FSE '16, Revised Selected Papers*, pages 43–59, 2016.
- [127] Mitsuru Matsui and Atsuhiro Yamagishi. A new method for known plaintext attack of FEAL cipher. In *Advances in Cryptology – EUROCRYPT '92, Proceedings*, pages 81–91, 1992.
- [128] Ueli M. Maurer. Indistinguishability of Random Systems. In *Advances in Cryptology – EUROCRYPT '02, Proceedings*, pages 110–132, 2002.
- [129] D McGrew and D Bailey. AES-CCM Cipher Suites for Transport Layer Security (TLS). RFC 6655, Internet Engineering Task Force (IETF), 2012. Online: <https://tools.ietf.org/html/rfc6655> (Accessed: 02 May, 2020).
- [130] David A. McGrew and John Viega. The security and performance of the galois/-counter mode (GCM) of operation. In *Progress in Cryptology – INDOCRYPT '04, Proceedings*, pages 343–355, 2004.
- [131] Bart Mennink. Insurability of the standard versus ideal model gap for tweakable blockcipher security. In *Advances in Cryptology – CRYPTO '17, Proceedings, Part II*, pages 708–732, 2017.
- [132] Bart Mennink. Towards tight security of cascaded LRW2. In *Theory of Cryptography – TCC '18, Proceedings, Part II*, pages 192–222, 2018.
- [133] Bart Mennink and Samuel Neves. Encrypted davies-meyer and its dual: Towards optimal security using mirror theory. In *Advances in Cryptology – CRYPTO '17, Proceedings, Part III*, pages 556–583, 2017.
- [134] Ralph C. Merkle. One way hash functions and DES. In *Advances in Cryptology – CRYPTO '89, Proceedings*, pages 428–446, 1989.
- [135] Kazuhiko Minematsu. Improved security analysis of XEX and LRW modes. In *Selected Areas in Cryptography – SAC '06, Revised Selected Papers*, pages 96–113, 2006.
- [136] Kazuhiko Minematsu. How to thwart birthday attacks against macs via small randomness. In *Fast Software Encryption – FSE '10, Proceedings*, pages 230–249, 2010.

- [137] Kazuhiko Minematsu. A short universal hash function from bit rotation, and applications to blockcipher modes. In *Provable Security – ProvSec '13, Proceedings*, pages 221–238, 2013.
- [138] Kazuhiko Minematsu. Parallelizable rate-1 authenticated encryption from pseudorandom functions. In *Advances in Cryptology – EUROCRYPT '14, Proceedings*, pages 275–292, 2014.
- [139] Kazuhiko Minematsu and Tetsu Iwata. Tweak-length extension for tweakable blockciphers. In *Cryptography and Coding – IMACC '15, Proceedings*, pages 77–93, 2015.
- [140] Kazuhiko Minematsu and Toshiyasu Matsushima. New Bounds for PMAC, TMAC, and XCBC. In *Fast Software Encryption – FSE '07, Revised Selected Papers*, pages 434–451, 2007.
- [141] Kazuhiko Minematsu and Yukiyasu Tsunoo. Provably secure MACs from differentially-uniform permutations and AES-based implementations. In *Fast Software Encryption – FSE '06, Revised Selected Papers*, pages 226–241, 2006.
- [142] Valérie Nachev, Jacques Patarin, and Emmanuel Volte. *Feistel Ciphers - Security Proofs and Cryptanalysis*. Springer, 2017.
- [143] Yusuke Naito. Blockcipher-based macs: Beyond the birthday bound without message length. In *Advances in Cryptology – ASIACRYPT '17, Proceedings, Part III*, pages 446–470, 2017.
- [144] Mridul Nandi. A simple and unified method of proving indistinguishability. In *Progress in Cryptology - INDOCRYPT '06, Proceedings*, pages 317–334, 2006.
- [145] Mridul Nandi. A simple security analysis of hash-cbc and a new efficient one-key online cipher. *IACR Cryptology ePrint Archive*, 2007(158), 2007. Online: <https://eprint.iacr.org/2007/158> (Accessed: 02 May, 2020).
- [146] Mridul Nandi. Two new efficient cca-secure online ciphers: MHCBC and MCBC. In *Progress in Cryptology – INDOCRYPT '08, Proceedings*, pages 350–362, 2008.
- [147] Mridul Nandi. Fast and Secure CBC-type MAC Algorithms. In *Fast Software Encryption – FSE '09, Revised Selected Papers*, pages 375–393, 2009.
- [148] Mridul Nandi. Improved Security Analysis for OMAC as a Pseudorandom Function. *J. Mathematical Cryptol.*, 3(2):133–148, 2009.
- [149] Mridul Nandi. A Unified Method for Improving PRF Bounds for a Class of Blockcipher Based MACs. In *Fast Software Encryption – FSE '10, Revised Selected Papers*, pages 212–229, 2010.

- [150] Mridul Nandi. On the minimum number of multiplications necessary for universal hash functions. In *Fast Software Encryption – FSE '14, Revised Selected Papers*, pages 489–508, 2014.
- [151] Mridul Nandi and Avradip Mandal. Improved security analysis of PMAC. *J. Mathematical Cryptol.*, 2(2):149–162, 2008.
- [152] NIST. Announcing the ADVANCED ENCRYPTION STANDARD (AES). FIPS 197, National Institute of Standards and Technology, U. S. Department of Commerce, 2001. Online: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf> (Accessed: 02 May, 2020).
- [153] Morris Dworkin (NIST). Recommendation for block cipher modes of operation – methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, U. S. Department of Commerce, 2001. Online: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf> (Accessed: 02 May, 2020).
- [154] Morris Dworkin (NIST). Recommendation for Block Cipher Modes of Operation: The CCM Mode for Authentication and Confidentiality. NIST Special Publication 800-38C, National Institute of Standards and Technology, U. S. Department of Commerce, 2001. Online: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38c.pdf> (Accessed: 02 May, 2020).
- [155] Morris Dworkin (NIST). Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication. NIST Special Publication 800-38B, National Institute of Standards and Technology, U. S. Department of Commerce, 2005. Online: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38b.pdf> (Accessed: 02 May, 2020).
- [156] NIST LwC Standardization Team. Submission requirements and evaluation criteria for the lightweight cryptography standardization process. <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/final-lwc-submission-requirements-august2018.pdf>, 2018. (Accessed: 02 September, 2019).
- [157] Sangwoo Park, Soo Hak Sung, Sangjin Lee, and Jongin Lim. Improving the upper bound on the maximum differential and the maximum linear hull probability for SPN structures and AES. In *Fast Software Encryption – FSE '03, Revised Selected Papers*, pages 247–260, 2003.

- [158] Jacques Patarin. *Etude des Générateurs de Permutations Pseudo-aléatoires Basés sur le Schéma du DES*. PhD thesis, Université de Paris, 1991.
- [159] Jacques Patarin. The “coefficients H” technique. In *Selected Areas in Cryptography - SAC '08, Revised Selected Papers*, pages 328–345, 2008.
- [160] Jacques Patarin. Introduction to mirror theory: Analysis of systems of linear equalities and linear non equalities for cryptography. *IACR Cryptology ePrint Archive*, 2010(287), 2010. Online: <https://eprint.iacr.org/2010/287> (Accessed: 02 May, 2020).
- [161] Jacques Patarin. Mirror theory and cryptography. *Appl. Algebra Eng. Commun. Comput.*, 28(4):321–338, 2017.
- [162] Erez Petrank and Charles Rackoff. CBC MAC for Real-Time Data Sources. *J. Cryptol.*, 13(3):315–338, 2000.
- [163] Krzysztof Pietrzak. A Tight Bound for EMAC. In *Automata, Languages and Programming - ICALP '06, Proceedings, Part II*, pages 168–179, 2006.
- [164] *ZIP File Format Specification*. PKWARE Inc., 2014.
- [165] J. Postel. User datagram protocol. RFC 768, Internet Engineering Task Force (IETF), 1980. Online: <https://tools.ietf.org/html/rfc768> (Accessed: 02 May, 2020).
- [166] Bart Preneel. Davies–Meyer. In *Encyclopedia of Cryptography and Security*. Springer-Verlag, 2011.
- [167] Bart Preneel and Paul C. van Oorschot. On the security of iterated message authentication codes. *IEEE Trans. Information Theory*, 45(1):188–199, 1999.
- [168] Gordon Procter. A note on the CLRW2 tweakable block cipher construction. *IACR Cryptology ePrint Archive*, 2014(111), 2014. Online: <https://eprint.iacr.org/2014/111> (Accessed: 02 May, 2020).
- [169] Michael O. Rabin and Shmuel Winograd. Fast evaluation of polynomials by rational preparation. *Communications on Pure and Applied Mathematics*, 25(4):433–458, 1972.
- [170] Phillip Rogaway. Bucket hashing and its application to fast message authentication. *J. Cryptol.*, 12(2):91–115, 1999.
- [171] Phillip Rogaway. Efficient instantiations of tweakable blockciphers and refinements to modes OCB and PMAC. In *Advances in Cryptology – ASIACRYPT '04, Proceedings*, pages 16–31, 2004.

- [172] Phillip Rogaway and Thomas Shrimpton. Deterministic authenticated-encryption: A provable-security treatment of the key-wrap problem. In *Advances in Cryptology – EUROCRYPT '06, Proceedings*, pages 373–390, 2006.
- [173] Phillip Rogaway and Haibin Zhang. Online ciphers from tweakable blockciphers. In *Topics in Cryptology - CT-RSA '11, Proceedings*, pages 237–249, 2011.
- [174] Phillip Rogaway, Mihir Bellare, John Black, and Ted Krovetz. OCB: a block-cipher mode of operation for efficient authenticated encryption. In *ACM Conference on Computer and Communications Security – CCS '01, Proceedings*, pages 196–205, 2001.
- [175] David Salomon. *Variable-length Codes for Data Compression*. Springer-Verlag New York, Inc., 2007.
- [176] Palash Sarkar. Improving upon the TET mode of operation. In *Information Security and Cryptology – ICISC '07, Proceedings*, pages 180–192, 2007.
- [177] Palash Sarkar. A new multi-linear universal hash family. *Des. Codes Cryptography*, 69(3):351–367, 2013.
- [178] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson. RTP: A transport protocol for real-time applications. RFC 3550, Internet Engineering Task Force (IETF), 2003. Online: <https://tools.ietf.org/html/rfc3550> (Accessed: 02 May, 2020).
- [179] Claude Elwood Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27:379–423, 1948.
- [180] Claude Elwood Shannon. Communication theory of secrecy systems. *The Bell System Technical Journal*, 28:656–715, 1949.
- [181] C Shao, H Deng, R Pazhyannur, F Bari, R Zhang, and S Matsushima. IEEE 802.11 Medium Access Control (MAC) Profile for Control and Provisioning of Wireless Access Points (CAPWAP). RFC 7494, Internet Engineering Task Force (IETF), 2015. Online: <https://tools.ietf.org/html/rfc7494> (Accessed: 02 May, 2020).
- [182] Victor Shoup. On fast and provably secure message authentication based on universal hashing. In *Advances in Cryptology – CRYPTO '96, Proceedings*, pages 313–328, 1996.
- [183] Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. *IACR Cryptology ePrint Archive*, 2004(332), 2004. Online: <https://eprint.iacr.org/2004/332> (Accessed: 02 May, 2020).

- [184] Peng Wang, Dengguo Feng, and Wenling Wu. HCTR: A variable-input-length enciphering mode. In *Information Security and Cryptology – CISC '05, Proceedings*, pages 175–188, 2005.
- [185] Mark N. Wegman and Larry Carter. New hash functions and their use in authentication and set equality. *J. Comput. Syst. Sci.*, 22(3):265–279, 1981.
- [186] D. Whiting, R. Housley, and N. Ferguson. Counter with CBC-MAC (CCM). RFC 3610, Internet Engineering Task Force (IETF), 2003. Online: <https://tools.ietf.org/html/rfc3610> (Accessed: 02 September, 2019).
- [187] S. Wigert. Sur l'ordre de grandeur du nombre des diviseurs d'un entier. *Ark. Mat. Astron. Fys.*, 3(18), 1907.
- [188] Shmuel Winograd. A new algorithm for inner product. *IEEE Trans. Computers*, 17(7):693–694, 1968.
- [189] Kan Yasuda. The sum of CBC MACs is a secure prf. In Josef Pieprzyk, editor, *Topics in Cryptology – CT-RSA '10, Proceedings*, pages 366–381, 2010.
- [190] Kan Yasuda. A new variant of PMAC: beyond the birthday bound. In *Advances in Cryptology – CRYPTO '11, Proceedings*, pages 596–609, 2011.
- [191] Kan Yasuda. PMAC with parity: Minimizing the query-length influence. In *Topics in Cryptology – CT-RSA '12, Proceedings*, pages 203–214, 2012.